Dragon Tools V9.9.1

Vollständiges Anwender-, Referenz- und Entwicklerhandbuch

Stand: 13.10.2026. Gültig für DragonTools V9.9.1. Dieses Handbuch erklärt Bedienung, Regeln und technische Fachmodule. Die aktuellen Funktionen sind in den zugehörigen Kapiteln beschrieben: Untertitelwahl, Laufzeitprüfung, Reparaturarchiv, Renamer und Serienbestand, Zielordner und geprüftes Verschieben. Das Anwender-, Referenz- und Entwicklerhandbuch bleibt eigenständig aufgebaut; die HTML-Hilfe erklärt die Bedienung direkt im Programm.

Dieses Handbuch ersetzt die alte V8-Dokumentation. Es erklärt die Bedienung, die Regeln, die Worker-Pipelines, die externe Tool-Kette und die wichtigsten Fehlerfälle so, dass man bei Problemen gezielt nachschlagen kann.

Bereich	Aktueller Stand
Startdatei	DragonToolsV9.py
Build-Ziel	DragonToolsV9.9.1.exe im PyInstaller-onedir-Aufbau mit Daten-Unterordner
GUI	PyQt6 mit Tabs für Konverter, ISO, Merge, MP4-Remux, Audio-Muxer und Untertitel
Schwerpunkt V9	Stabilität, DV/HDR-Korrektheit, Reparaturpfade, Regeln, Parallelbetrieb, Post-Processing, TMDB/TheTVDB, Renamer-Regeln und Fuzzy-Bewertung, SQLite-Mediathek Schema 6, Watch-Folder, Jellyfin-API, Fix Queue, Audio-Spracherkennung, PGS/VobSub-OCR, automatische VMAF-Zielsuche, Windows-Benachrichtigungen und experimentelles SDR→HDR-Enhancement.

Inhaltsverzeichnis

Kapitel	Inhalt	Seiten
1.	Wegweiser durch Dragon Tools	S. 9
1.1	Schnellstart	S. 9
1.2	Die Entscheidungskette	S. 9
1.3	Statussymbole und Sprache	S. 10
2.	Projektaufbau und Architektur	S. 11
2.1	Projektstatistik des aktuellen Stands	S. 11
2.2	Schichtenmodell	S. 12
2.3	Start im Entwicklungsmodus und als EXE	S. 12
2.4	Einstellungen und Regeln	S. 13

Kapitel	Inhalt	Seiten
3.	Oberfläche, Tabs und tägliche Bedienung	S. 14
3.1	Hauptregister	S. 14
3.2	Queue und Kontextfunktionen	S. 15
3.2.1	Schnellschalter im Konverter	S. 15
3.3	Steuerung während des Laufs	S. 16
3.3.1	Worker Anzahl während des Laufs ändern	S. 17
3.4	Menüs und Shortcuts	S. 18
4.	Konvertierungsworkflow	S. 19
4.1	Ablauf pro Datei	S. 19
4.2	Pipeline-Auswahl	S. 19
4.3	Per-Datei-Profile und manuelle Encoder-Overrides	S. 20
4.4	Strip-Only	S. 21
5.	Encoder, Video und Bildlogik	S. 22
5.1	Zielcodecs	S. 22
5.2	Encoderwahl	S. 22
5.3	Qualitätsregler	S. 23
5.4	H.265 10-bit in V9	S. 23
5.5	Skalierung, Auto-Crop und IMAX	S. 23
6.	HDR, Dolby Vision und HDR10+	S. 25
6.1	Erkennung	S. 25
6.2	Standardpfad bei HDR und entfernten Metadaten	S. 25
6.3	DV-Pipeline	S. 25
6.4	HDR10+-Pipeline	S. 26
6.5	SDR→HDR mit ComfyUI/HDRTVDM	S. 27
6.6	HDR-Erkennung, Datei-Overrides und Strip-Only	S. 28
6.7	DV + HDR10+ Sonderkombination	S. 28
6.8	Typische Fehlerbilder	S. 28
6.9	DV-P5-Remux über zentralen Prozesspfad	S. 30

Kapitel	Inhalt	Seiten
7.	Audio-Regeln	S. 30
7.1	Sprachen und Fallback	S. 30
7.2	Passthrough, Transkodierung und Kanalregeln	S. 30
7.3	5.1-Downmix	S. 31
7.4	Zusatz-Stereo-Downmix	S. 31
7.5	Entscheidungsbeispiele	S. 31
7.6	Audio-Dynamik, DRC und Lautheitsnormalisierung	S. 33
8.	Untertitel-Regeln	S. 33
8.1	Sprachliste und Mengenbegrenzung	S. 33
8.1.1	Automatische Standardspur	S. 34
8.2	Forced-Burn-In	S. 34
8.2.1	Bitmap-Untertitel und Videozeitachse	S. 34
8.3	Welche Untertitel bleiben erhalten?	S. 34
8.4	Forced-Subtitle-Plausibilitätsprüfung	S. 35
8.5	Untertitel automatisch zuordnen und einfügen	S. 35
8.6	Zusätzliche Untertiteldateien und OCR	S. 36
9.	Preflight, Zielordner und Verschieben	S. 36
9.1	Zielpfade nach Codec und Medientyp	S. 36
9.2	Zielordner-Preflight	S. 36
9.2.1	Bestätigte Renamer-Zuordnungen	S. 37
9.3	Konflikte beim Verschieben	S. 37
9.3.1	Kopieren und Integritätsnachweise	S. 37
9.4	Verschiebebericht	S. 37
9.5	SxxExx-Ersetzung beim Verschieben	S. 38
9.6	Persistente Ersetzungs-Erinnerungen	S. 38

Kapitel	Inhalt	Seiten
9.7	Preflight und persistente Mediathek-Pfade	S. 38
10.	Zusatzwerkzeuge	S. 39
10.1	Externe Tools und Fähigkeiten	S. 39
10.2	Jellyfin-NFO und Trickplay	S. 41
10.3	Qualitätstester	S. 41
10.4	Film-/Serien-Umbenennung und Online-Metadaten	S. 42
10.4.1	Dateinamen im Hintergrund ändern	S. 43
10.4.2	Staffel und Serie prüfen	S. 43
10.5	Quellbildprüfung	S. 43
10.6	Audio-Video-Matcher	S. 44
10.7	Entscheidungsmodell des Serien-Renamers	S. 44
10.8	Renamer-Verhalten und Funktionsumfang	S. 45
10.9	Release-Normalisierung und technische Tags	S. 46
10.10	Renamer-Regeln, Dateizeichen und Aliasregeln	S. 47
10.11	Providerkandidaten, Jahr und Fuzzy-Bewertung	S. 48
10.12	Renamer-Oberfläche und inkrementeller Resolver	S. 49
10.13	Trickplay: Hardware-Decoding mit Kompatibilitätsstrategie	S. 49
10.14	Jellyfin-API: Pfadvalidierung und gezielte Metadatenanalyse	S. 49
10.15	Audio-Spracherkennung und NFO-/Track-Synchronisierung	S. 50
11.	Validierung, Reparatur und Fehlerberichte	S. 50
11.1	Output-Validierung	S. 50
11.1.1	Video- und Audiodauer getrennt prüfen	S. 51
11.2	Automatische Laufzeitreparatur	S. 51

Kapitel	Inhalt	Seiten
11.3	Größenregeln	S. 51
11.4	Fehlerberichte	S. 52
11.5	Cleanup und Rollback	S. 52
11.5.1	Reparaturarchiv nach erfolgreichem Abschluss	S. 53
12.	Logging, Backup, Build und Release	S. 53
12.1	Normaler Log	S. 53
12.2	Abschlussbericht	S. 53
12.3	Backup und Restore	S. 53
12.4	Build	S. 53
12.5	Backup-Restore und Legacy-Secrets	S. 55
13.	Fehlerfälle und Nachschlagehilfe	S. 55
13.1	Welche Datei ist für welchen Fehler zuständig?	S. 56
14.	Glossar	S. 57
14.1	DRC	S. 58
14.2	Loudnorm	S. 58
14.3	SSIM / VMAF	S. 58
15.	Programm- und Modulreferenz	S. 58
15.1	Einstiegspunkt	S. 58
15.2	Core	S. 59
15.3	GUI	S. 109
15.4	Regeln	S. 168
15.5	Untertitel	S. 177
15.6	Worker	S. 181
15.7	Paket	S. 248
15.8	Tests und QA-Module	S. 249
15.9	Modulreferenz – Renamer und Mediathek	S. 267
15.10	Modulreferenz - Mediathek-Scan, Move-Konflikte und Erinnerungen	S. 269
15.11	Historische Modulreferenz zentraler Fachmodule	S. 271
15.12	Ergänzungen vom 06.09.2026	S. 272

Kapitel	Inhalt	Seiten
15.13	Aktuelle Fachmodule für Timeline, Jellyfin und Sprachreparatur	S. 273
16.	Vollständige GUI- und Bedienreferenz	S. 274
16.1	Hauptregister und Lazy Loading	S. 274
16.2	Converter – sichtbare Bedienelemente	S. 275
16.3	Queue-Kontextmenü	S. 277
16.4	Spezialregister – Bedienelemente	S. 278
16.5	Menüs und Tastenkürzel	S. 278
16.6	Persistenz von GUI-Bereichen, Tabs und Fenstern	S. 279
16.6.1	Live Worker Auswahl und Schnellschalter	S. 280
16.7	Über-Dialog als aktuelle Programmzusammenfassung	S. 280
17.	Einstellungen, Defaultwerte und technische Auswirkungen	S. 280
17.1	Encoder-Grundlagen	S. 280
17.2	NVIDIA NVENC	S. 281
17.3	Intel QSV, AMD AMF und CPU-x265	S. 282
17.4	Audio-Regeln – Standardkonfiguration	S. 283
17.5	Untertitel-Regeln – Standardkonfiguration	S. 284
17.6	Auto-Crop, IMAX, Validierung, Parallelisierung und Postprocessing	S. 285
17.7	Timeouts	S. 286
17.8	Zentrale Settings-Helfer	S. 287
17.9	QSV-Altlast und Profil-Schema 3	S. 287
17.10	Zuverlässiger Standardwerte-Reset und CPU-CRF 22	S. 288
18.	Worker-System von DragonTools	S. 288
18.1	Normaler Converter	S. 290
18.2	Spezialworker	S. 291

Kapitel	Inhalt	Seiten
18.3	Kritische Worker-Fehler und Parallelbetrieb	S. 291
19.	Online-Metadaten-System	S. 291
19.1	Providerwahl und Priorität	S. 292
19.2	Zugangsdaten und Sichtbarkeit	S. 292
19.3	Datenfluss	S. 293
20.	Konfiguration, Datenhaltung und Dateistrukturen	S. 293
20.1	QSettings und zentrale Schlüssel	S. 293
20.2	Regeldateien	S. 299
20.3	Relevante Ordner-/Dateistrukturen	S. 299
20.4	Eigene SQLite-Mediathek und Jellyfin-Leseimport	S. 300
20.5	Mediathek-Suche und Qualitätsklassen	S. 300
20.6	Abweichungssuche: Mehrheits- und Gleichstandslogik	S. 301
20.7	SQLite-Indizes, Performance und SQL-Sicherheit	S. 302
20.8	Ziel des Mediathek-Speicherpfad-Scans	S. 302
20.9	Datenbankzustand: exists_flag und active	S. 302
20.10	Mediathek-Suche, Duplikate und Bereinigung	S. 303
20.11	Fehlertoleranter Mediathek-Scan	S. 303
21.	Logging, Fehlerberichte, Recovery und Diagnose	S. 303
21.1	Änderungshistorie: JSON-Struktur und Seitennavigation	S. 304
21.2	Eindeutige Job- und Move-Journale	S. 304
21.3	Später entscheiden und vollständige Move-Wiederaufnahme	S. 305
21.4	Crash-Recovery beim Verschieben	S. 305
22.	Externe Bibliotheken, Programme, Build und Veröffentlichung	S. 305

Kapitel	Inhalt	Seiten
22.1	Python-Abhängigkeiten	S. 305
22.2	Externe Programme	S. 306
22.3	Build-Dateien im bereitgestellten Paket	S. 306
22.4	Aktuelle Renamer-Module und Simulator-Erweiterung	S. 307
23.	Qualitätssicherung und Testabdeckung	S. 307
23.1	Qualitätssicherung und Abnahmestand	S. 317
23.2	Regressionsschutz	S. 318
23.3	Release-Prüfung und Regressionstests	S. 319
23.4	Dokumentations- und Teststand	S. 319
23.5	Aktueller technischer Reviewstand	S. 319
23.6	Architekturgrenzen und Modulzuständigkeiten	S. 321
24.	Erweiterte Fehlerdiagnose – symptomorientierte Referenz	S. 321
25.	Historische Modulübersicht vom 06 September 2026	S. 322
25.1	Einstiegspunkt - 1 Datei(en)	S. 322
25.2	Core - 78 Datei(en)	S. 322
25.3	GUI - 120 Datei(en)	S. 327
25.4	Rules - 8 Datei(en)	S. 333
25.5	Subtitle - 6 Datei(en)	S. 334
25.6	Worker - 111 Datei(en)	S. 334
25.7	Tests - 144 Datei(en)	S. 340
25.8	Sonstige Paketdateien - 1 Datei(en)	S. 362
26.	Interne und externe Abhängigkeiten	S. 362
27.	Externe Programme	S. 367
28.	Pythonbibliotheken	S. 368
29.	Workerübersicht	S. 369
30.	Einstellungen und Defaultwerte	S. 371

Kapitel	Inhalt	Seiten
31.	Zentrale Datenflüsse	S. 375
32.	Dateistrukturen	S. 376
33.	Fehler und Troubleshooting	S. 376
34.	Glossar	S. 378
35.	Aktuelles Quellinventar	S. 380
35.1	Einstiegspunkt	S. 380
35.2	GUI	S. 380
35.3	Core	S. 385
35.4	Worker	S. 392
35.5	Rules	S. 398
35.6	Subtitle	S. 399
35.7	Dragon HDR10+ Generator (Source)	S. 399
35.8	Tests (DragonTools + Generator)	S. 400
35.9	Sonstige	S. 408

1 Wegweiser durch Dragon Tools

Dragon Tools ist ein Arbeitsprogramm für reale Medien-Batches: Dateien analysieren, passende Pipeline wählen, Audio- und Untertitelregeln anwenden, Video encoden oder remuxen, die Ausgabe prüfen und am Ende ersetzen, archivieren oder verschieben. V9 ist vor allem darauf ausgelegt, dass Fehler nicht still passieren.

> Arbeitsprinzip: Merksatz: Die App arbeitet nicht nur einen FFmpeg-Befehl ab. Sie trifft vor jeder Datei eine Regelentscheidung und prüft danach, ob die Ausgabedatei fachlich plausibel ist.

1.1 Schnellstart

1. Dateien oder Ordner in den passenden Codec-Tab ziehen oder über Dateien/Ordner hinzufügen.
2. Encoder, Skalierung, Auto-Crop, Original überschreiben und Verschieben nach Bedarf setzen.
3. Bei DV/HDR10+ entscheiden, ob Metadaten erhalten bleiben sollen.
4. Optional pro Datei über das Kontextmenü ein eigenes Encoderprofil oder Track-Override setzen.
5. Konvertierung starten; der Zielordner-Preflight fasst Serien zusammen und fragt Film/Anime/TV-Ziele ab.
6. Während des Laufs wartende Dateien sortieren und die Worker-Anzahl neben Pause anpassen. Laufende Dateien bleiben beim Reduzieren aktiv; Abbruch ist sofort oder nach Datei möglich.
7. Nach dem Lauf prüft Dragon Tools Ausgabe, Laufzeit, Größe und Zielpfade und zeigt den Abschlussstatus.
8. Bei Fehlern den erzeugten Fehlerbericht im Log-Monatsordner unter ErrorReports öffnen.

1.2 Die Entscheidungskette

Stufe	Was passiert	Warum wichtig

Stufe	Was passiert	Warum wichtig
Analyse	MediaInfo und ffprobe lesen Container, Streams, HDR/DV/HDR10+, Audio, Untertitel, Dauer, Frames und Framerate.	Ohne saubere Analyse wären Regeln und Pipelines nur Vermutung.
Regeln	Audio-, Untertitel-, Serien- und Zielpfadregeln werden auf die konkrete Datei angewendet.	Die Datei bekommt einen konkreten Plan statt globaler Pauschalentscheidung.
Pipeline	Standard, DV, HDR10+ oder DV+HDR10+ wird gewählt; Strip-only und DV-Remux sind Sonderwege.	Metadaten und Containeranforderungen unterscheiden sich stark.
Encoding/Remux	FFmpeg, dovi_tool, hdr10plus_tool, mkvmerge, MP4Box oder MakeMKV arbeiten je nach Aufgabe.	Externe Tools erledigen die eigentliche Medienarbeit.
Validierung	Ausgabe wird mit ffprobe geprüft; Laufzeitfehler können automatisch remuxt oder per Timestamp-Reparatur korrigiert werden.	Defekte Ausgaben sollen nicht automatisch Originale ersetzen.
Abschluss	Ersetzen, Archivieren, optionale NFO-/Trickplay-Nacharbeit, Verschieben, Sidecars und Run Summary werden finalisiert.	Der Endzustand bleibt nachvollziehbar; fertige Dateien werden erst nach abgeschlossener Nacharbeit endgültig als OK gemeldet.

1.3 Statussymbole und Sprache

- 🧩 steht für eine erfolgreich ersetzte Datei, deren NFO-/Trickplay-Nacharbeit noch läuft. Die nächste Konvertierung darf bereits starten; als endgültig fertig gilt die Datei erst nach dem folgenden ✅.
- ✅ bleibt der finale OK-Zustand. Der Abschlussbericht, das Verschieben nach dem Lauf und die Sidecar-Listen warten dadurch auf fertige Untertitel-, NFO- und Trickplay-Dateien.

Die normalen Logs dürfen bewusst lesbar bleiben: ✅ für erfolgreiche Schritte, ℹ️ für Hinweise, ⚠️ für Warnungen und ❌ für Fehler. V9 nutzt außerdem echte deutsche Umlaute in sichtbaren Texten. Technische Dateien wie Fehlerberichte bleiben UTF-8-Textdateien, damit sie auch außerhalb der App lesbar sind.

2 Projektaufbau und Architektur

2.1 Projektstatistik des aktuellen Stands

Bereich	Dateien	Gesamt-zeilen	Codezeilen	Einordnung
Einstiegspunkt	1	502	379	Start, Splash, PyInstaller-Pfade
GUI	253	36.278	31.332	Tabs, Dialoge, Menüs, Queue, Einstellungen und schlanke GUI-Fassaden
Core	310	44.096	37.815	Modelle, Analyse, SQLite-Mediathek, Pfade, Logging, Metadaten und Release-Logik
Worker	310	42.352	36.680	Konvertierung, DV/HDR, Reparatur, ISO, Merge, Move, Postprocessing, OCR und Quality-Target
Rules	32	4.681	3.949	Audio, Untertitel, Serien, Renamer und Pipeline-Auswahl
Subtitle	12	1.440	1.265	Extraktion, Injektion, Konvertierung, Flags und OCR-nahe Untertitelpfade
Dragon HDR10+ Generator (Source)	14	1.075	894	Eigenständige CLI/EXE-Analyse für PQ/BT.2020 → ST-2094-40 Profile A
Tests (DragonTools + Generator)	393	81.923	67.053	3.424 Testfunktionen in 387 Testdateien; Regression und Release
Gesamt	1.330	213.236	180.133	Vollständiger Quellstand einschließlich Einstiegspunkt und Generator-Unterprojekt
Produktiv ohne Tests	937	131.313	113.080	Einstiegspunkt, Programm-/Paketcode und Generator-Source außerhalb der Testpakete
Sonstige	5	889	766	Paket-Initialisierung, Snapshot-Helfer und optionale ComfyUI-Nodes

Historische Abnahme 9.8.2: Stand 13.09.2026: Der vollständige Quellstand ist neu vermessen und entlang klarer Fachgrenzen organisiert. Release-Validierung, Timestamp-Reparatur, Strip-Only, Audio-/Video-Time-Mapping, Qualitätsservices, Streamargumente, finaler DV-Mux, Conversion-Fortschritt und ISO-Eingabeverarbeitung besitzen getrennte Zuständigkeiten. Die aktuellen Bereiche wurden mit gezielten Regressionstests, Architekturgrenzen, Paket-Smoke und compileall geprüft. Der vollständige lokale Testlauf ergab 1.269 bestandene und 2 mangels externer Werkzeuge und Testmedien übersprungene reale DV/HDR-Tests.

Aktueller Quellstand 9.9.1 vom 07.10.2026: 1.297 Python-Dateien/Programme, 208.072 Gesamtzeilen und 175.787 Codezeilen (nichtleer, keine reinen Kommentarzeilen). Testpakete: 375 Python-Dateien, davon 369 test_*.py mit 3.287 statisch erkannten Testfunktionen. Produktivcode ohne Tests: 922 Dateien, 129.255 Gesamtzeilen und 111.314 Codezeilen. Patch-29-Abnahme: 5.175 bestanden (5.152 Gesamtsuite + 23 native DV/HDR-Tests). Parametrisierte Testfälle werden getrennt von Testfunktionen gezählt.

Patch-BV-Review: Changelog-Schema und Release-Validierung sind zusätzlich fail-closed abgesichert. In der Linux-Prüfumgebung fehlt PyQt6; deshalb bleibt ein echter Windows-/Qt-Eventloop-Smoke für die interaktive Drag-&-Drop-Oberfläche ein separates Release-Gate.

2.2 Schichtenmodell

Stand 9.9.1: Mediathek-Schema, Migration, Scan und Suche erhalten die plattformgerechte Pfadidentität. Normalisierte Pfadschlüssel verhindern doppelte Datensätze für dieselbe Windows-Datei. Stream-Snapshots werden zusammen mit dem Mediendatensatz aktualisiert; NFO-Import und Fix Queue prüfen Datenherkunft und Quelländerungen. Bestehende Datenbanken werden vor darauf aufbauenden Abfragen migrationssicher ergänzt.

Die Architektur trennt öffentliche Fassaden von den eigentlichen Fachservices. Große Einstiegs- und Worker-Module koordinieren vor allem Abläufe; I/O, Persistenz, Toolaufrufe, Validierung und reine Fachlogik liegen in getrennten Modulen. Bestehende Importpfade bleiben als Kompatibilitätsfassaden erhalten, während interne Consumer die zuständigen Fachmodule direkt verwenden.

Schicht	Aufgabe	Typische Dateien
Einstieg	Bereitet sys.path, PATH und Splash vor und startet die Qt-Anwendung.	DragonToolsV9.py
GUI	Zeigt Tabs, Dialoge, Queue, Einstellungen, Regeln und Abschlussberichte.	main_window.py, convert_widget_layout.py, settings_dialog.py
Core	Enthält wiederverwendbare Logik ohne GUI-Zwang: Pfade, Modelle, Analyse, Backups, Reports.	models.py, paths.py, media_analyzer.py, error_report.py
Rules	Berechnet Entscheidungen aus Regeln: Sprache, Audio, Untertitel, Serien, Pipeline.	audio_rules.py, subtitle_rules.py, move_rules.py
Worker	Führt lange externe Prozesse im Hintergrund aus und kapselt Reparatur-, Replace- und Cleanup-Logik.	converter_thread.py, workflow_services.py, duration_repair_service.py
Subtitle	Kleine Fachmodule für Untertitel-Extraktion, -Injektion, -Konvertierung und Track-Flags.	extractor.py, injector.py, converter.py, tagger.py

2.3 Start im Entwicklungsmodus und als EXE

DragonToolsV9.py unterscheidet beim Start zwischen Python-Start und PyInstaller-Frozen-Modus. Im Frozen-Modus wird der EXE-Ordner als Basis verwendet; bei PyInstaller mit --contents-directory Daten liegen Programmdateien und Ressourcen unterhalb von Daten. Deshalb werden Daten\Programme, Programme, third_party und mehrere Tool-Unterordner in PATH aufgenommen.

- Ressourcen wie Icon, Splash und Hilfe werden zuerst im Daten-Unterordner gesucht und danach in klassischen Entwicklungsordnern.
- Externe Tools können gebündelt sein, manuell in den Einstellungen hinterlegt werden oder aus PATH kommen.
- Die zentrale Toolsuche in dragontools/core/paths.py verhindert, dass GUI und Worker unterschiedliche Pfade verwenden.

2.4 Einstellungen und Regeln

Stand 9.9.1: Einstellungen werden vor dem Speichern vollständig validiert; versteckte Dialogbereiche werden nicht unbeabsichtigt mitgeschrieben. Nicht entschlüsselbare DPAPI-Secrets bleiben erhalten. Backup-Restore prüft Format, Typen und Secret-Verfügbarkeit vor Änderungen und schreibt Dateien mit atomarem Austausch. Fehlgeschlagene Profilpersistenz veröffentlicht keinen neuen In-Memory-Stand. Timeout-Einstellungen behalten vorhandene Sekundenwerte.

Benutzereinstellungen laufen über QSettings mit zentralen Schlüsseln aus dragontools/core/settings.py. Regeldateien haben Default-JSONs im Projekt und können vom Benutzer überschrieben werden. Audio-Regeln verwenden Schema 4, Untertitel-Regeln Schema 7, Move Schema 1, Renamer Schema 3, Profile Schema 3. Migration und GUI-Speicherung verwenden atomare JSON-Schreibvorgänge. Ist eine Benutzer-Regel- oder Profil-Datei syntaktisch oder strukturell beschädigt, wird sie vor einem Fallback als *.corrupt_<Zeitstempel>.json gesichert.

3 Oberfläche, Tabs und tägliche Bedienung

3.1 Hauptregister

Stand 9.9.1: Mindestens ein Haupt-Tab bleibt sichtbar. Beim Start wird die vollständige Tab-Liste vor der Sichtbarkeitsprüfung angelegt; sind alle Tabs gespeichert ausgeblendet, wird bevorzugt der Standardcodec-Tab wieder geöffnet und gespeichert. Geladene versteckte Tabs erhalten neue Einstellungen. MediaInfo- und Quellbild-Worker nehmen am globalen Shutdown teil; spätere Startcallbacks sind an den Fenster-Lebenszyklus gebunden. Nicht geladene entfernte Tab-Widgets werden freigegeben.

Tab	Zweck	Wichtige Hinweise
H.265 / DV / HDR10+	Hauptpfad für moderne HEVC-Konvertierung, DV/HDR10+-Erhalt und Standard-H.265.	H.265 erzwingt in V9 echtes 10-bit-Pixelformat bei allen Encodern.
H.264	Kompatibilitätspfad für ältere Geräte.	Bleibt bewusst ohne 10-bit-Zwang; H.264 ist meist der 8-bit-Kompatibilitätscodec.
AV1	Sehr moderne, effiziente Ausgaben.	Standard-AV1 nutzt die normale Encoderwahl. AV1-Dolby-Vision Profile 10 ist als CPU/SVT-AV1-Betapfad verfügbar; AV1-HDR10+ als CPU/libaom-av1-Betapfad. Bei Quelle mit DV+HDR10+ und beiden Erhaltungsoptionen hat DV Priorität.
ISO	Analysiert ISO/IMG, Disc-Root sowie direkt gewählte BDMV- oder VIDEO_TS-Ordner mit MakeMKV; FFmpeg-Fallback bleibt als Notfallmethode erhalten.	makemkvcon muss gefunden werden und MakeMKV muss gültig lizenziert oder per Test-/Beta-Lizenz aktiviert sein. Der FFmpeg-Fallback kann bei komplexen Blu-rays Spuren/Playlist-Struktur schlechter erkennen.
Merge	Dateien ohne Reencode zusammenführen, wenn Container und Streams kompatibel sind.	Prüft Kompatibilität vor dem eigentlichen Merge.
MP4-Remux	Kompatible Dateien nach MP4 remuxen.	DV und HDR10+ werden bewusst nicht durch den einfachen MP4-Remux geschickt.
Audio Muxer	Audiospuren in Videodateien einfügen oder austauschen.	Nutzt Audio-Regeln und Fortschrittslogik ähnlich wie der Konverter.
Untertitel	Untertitel extrahieren, injizieren, konvertieren und Flags setzen.	MKV nutzt MKVToolNix, MP4/MOV nutzt FFmpeg mit mov_text für Text-Untertitel.

Die Hauptregister bündeln die täglichen Arbeitsbereiche: Konverter, ISO, Merge, MP4-Remux, Audio-Muxer, Untertitel, Qualitätstester, Film-/Serien-Umbenennung und Audio-Video-Matcher. Einstellungen und Regeln sind über die

Menüleiste erreichbar; Online-Metadaten haben wegen API-Key, Read-Token, TheTVDB-PIN und Bearer-Token einen eigenen Menüpunkt.

3.2 Queue und Kontextfunktionen

Die Queue ist die Arbeitsliste des jeweiligen Konverter-Tabs. Dateien können per Drag & Drop, Dateidialog oder Ordnerdialog hinzugefügt werden. Während eine Konvertierung läuft, dürfen wartende Dateien umsortiert werden; die aktuelle Datei bleibt geschützt.

- Die Schnellschalter stehen nebeneinander rechts von den Queue-Pfeilen; bei wenig Fensterbreite lässt sich die Zeile seitlich scrollen. Alle Schalter speichern in den vorhandenen globalen Einstellungen beziehungsweise Untertitelregeln und werden zwischen geöffneten Konverter-Tabs synchronisiert. Laufende Dateien behalten ihre übernommenen Optionen. Untertitelregeln und Encoderoptionen werden für einen neuen Konverterlauf übernommen; NFO und Trickplay lesen den Zustand beim Start des jeweiligen Datei-Workers. Watch-Folder wird unmittelbar aktualisiert. Nur die Worker-Anzahl ist ausdrücklich während eines Laufs veränderbar.
- Doppelstart-Schutz: Ein synchroner Start-Lock wird bereits vor Preflight, Worker-Erzeugung und QThread.start() reserviert. Zusätzlich dedupliziert Dragon Tools die initiale Parallelqueue über normalisierte Pfade und prüft vor jeder Child-Worker-Zuordnung erneut, ob dieselbe Quelle bereits einem Worker gehört. Damit ist auch das kurze Fenster vor QThread.isRunning() gegen einen zweiten Start derselben Datei geschützt.
- Entfernen löscht nur die Datei aus der Queue, nicht von der Festplatte.
- Das separate Warteschlangenfenster zeigt dieselbe Reihenfolge und erlaubt denselben Blick auf wartende Dateien.
- Per-Datei-Einstellungen können ein gespeichertes Encoderprofil oder einen manuellen Encoder-/Skalierungs-Override verwenden. Der manuelle Override kann CPU, NVIDIA NVENC, Intel QSV oder AMD AMF, Qualitätswert, Preset, Skalierung und backend-spezifische Optionen nur für die betroffene Datei überschreiben.
- Bei Mehrfachauswahl können dieselben Encoder-/Skalierungswerte gleichzeitig auf mehrere markierte wartende Dateien angewendet werden. Audio-, Untertitel-, IMAX-, DV/HDR- und Remux-Overrides der Dateien bleiben dabei unverändert.
- Codec-Profil-Assistent: vorbereitete Profile wie Anime klein, Film ausgewogen oder CPU effizient setzen Startwerte, bleiben aber als Expertenwerte editierbar.
- Der Regeltest zeigt Audio-/Untertitel-/Pipeline-Entscheidungen ohne Encoding.
- Die Medieninfo aus dem Kontextmenü zeigt Streams, HDR/DV/HDR10+, Dauer, Frameanzahl und rationale Framerate.

3.2.1 Schnellschalter im Konverter

Die folgenden zehn Optionen sind direkt aus dem Konverter erreichbar. Die übrigen Einstellungen zu Sprache, Ablage, Qualität, HDR und Nachbearbeitung bleiben in den jeweiligen Fachdialogen konfigurierbar.

Schalter	Wirkung
Forced einbrennen	Automatischen Forced-Burn-In gemäß Untertitelregeln ein- oder ausschalten. Manuelle Datei-Overrides bleiben wirksam; Strip-Only brennt nichts ein.
Untertitel extern	Die ausgewählten Untertitel zusätzlich im ursprünglichen Format neben der Ausgabe speichern. Die MP4-Ablage folgt weiterhin den MP4-Regeln.
PGS → SRT	Ausgewählte PGS/SUP-Spuren zusätzlich mit FFmpeg und Tesseract per OCR als SRT exportieren. Die Original-PGS-Spur bleibt gemäß Regelwerk erhalten; OCR-Text kontrollieren.

Schalter	Wirkung
Text → SRT	Ausgewählte Textuntertitel zusätzlich als SRT speichern. Formatierung und Positionierung können bei der Umwandlung verloren gehen.
Auto-Qualität	Für geeignete SDR-Videos den Qualitätswert anhand des eingestellten VMAF-Ziels ermitteln. Zusätzliche Testdurchläufe benötigen Zeit; HDR, Dolby Vision und HLG bleiben beim festen Qualitätswert.
HDR+ Generator	Die optionale HDR10+-Erzeugung für geeignete Ausgaben aktivieren.
SDR → HDR	Das konfigurierte SDR-HDR-Enhancement aktivieren.
NFO	Jellyfin-NFO nach erfolgreicher Konvertierung gemäß den vorhandenen Metadaten- und Konfliktregeln erstellen.
Trickplay	Jellyfin-Vorschaubilder zum Spulen gemäß den vorhandenen Einstellungen erzeugen. Die Nachbearbeitung benötigt zusätzliche Rechenzeit.
Watch-Folder	Die konfigurierte Ordnerüberwachung global aktivieren oder deaktivieren.

3.2.2 Watch-Folder und manueller Sofortscan

Stand 9.9.1: Watch-Intake und Live-Queue ordnen Dateien und Profile eindeutig zu und verhindern Doppelstarts. Bereits manuell eingereihte Dateien werden nicht nachträglich als Watch-Aufträge übernommen. Geänderte Worker-Limits starten nur zulässige wartende Jobs; bei Pause und Abbruch kommen keine neuen hinzu. Journale und Wiederaufnahme behalten den richtigen Auftrag und den tatsächlichen Bearbeitungszustand. Bei mehreren aktiven Workern öffnet ein Rechtsklick auf die laufende Videodatei „Worker pausieren“ beziehungsweise „Worker fortsetzen“. Nur der zugehörige Worker wird angehalten; andere Worker laufen weiter. Die pausierte Datei belegt ihren Worker-Platz weiter. Eine globale Pause hat Vorrang, und alte Menüaktionen können keinen neuen Auftrag steuern.

Automatische Watch-Folder übernehmen Dateien nach der konfigurierten Stabilitätszeit in die normale Converter-Queue. Persistente Dateisignaturen werden erst nach terminalem Erfolg bestätigt; Fehler und übersprungene Dateien bleiben erneut versuchbar.

Der Button „Watchfolder durchsuchen“ startet unabhängig vom globalen Automatikschalter einen manuellen Sofortscan über alle aktivierten Regeln. Bereits bestätigte Signaturen und bereits in der Queue vorhandene Pfade werden nicht doppelt übernommen. Während eines laufenden Batches gehen neue Treffer in die bestehende Live-Queue; im Leerlauf werden sie nur eingereiht und starten keinen Batch.

Wichtig: Der manuelle Sofortscan setzt nur für diesen Lauf die Stabilitätswartezeit auf 0. Er ist deshalb für vollständig geschriebene/kopierte Dateien gedacht. Ein bereits laufender Watch-Scan merkt genau einen manuellen Folgescan vor.

3.3 Steuerung während des Laufs

Stand 9.9.1: Der Start-Lock bleibt beim Übergang von Konvertierung zu Verschieben aktiv; auch Move-Only besitzt einen Doppelstartschutz. Aktive Ergebnis- und Recovery-Zustände bleiben beim Bearbeiten der Queue erhalten. Fehlgeschlagene Workerstarts lösen reservierte Zustände kontrolliert auf. Logging- und Benachrichtigungsfehler dürfen ein verifiziertes Medienergebnis nicht in einen falschen Abschlusszustand versetzen. Pause steuert auch den aktiven Move-Worker.

Funktion	Wirkung	Endzustand
Pause/Fortsetzen	Der laufende Prozess wird angehalten oder fortgesetzt.	Queue bleibt erhalten; Fortschritt läuft danach weiter.

Funktion	Wirkung	Endzustand
Sofort abbrechen	Der aktuelle Prozess wird beendet.	Dragon Tools fragt, ob bereits erfolgreiche Ausgaben noch verschoben werden sollen.
Nach Datei abbrechen	Die aktuelle Datei wird fertig verarbeitet, danach stoppt der Batch.	Auch hier kann V9 erfolgreiche Ausgaben noch verschieben.
Herunterfahren	Nach einem erfolgreichen unbeaufsichtigten Lauf startet der Countdown.	Der Abschlussdialog wird bei aktivem Shutdown nicht blockierend geöffnet.
Nur verschieben	Kein Encoding, nur Zielordner-Preflight und MoveThread.	Verschiebebericht wird trotzdem erstellt.

Während eines Laufs bleiben wartende Queue-Einträge bearbeitbar. Die aktuell laufende Datei ist geschützt; bereits fertige Dateien können jedoch vorzeitig verschoben werden, ohne den Encoderjob zu stoppen.

- Pause hält die nächste Datei zurück, beendet aber nicht den laufenden Toolprozess.
- Sofort abbrechen stoppt den aktuellen Lauf direkt. Abbruch nach Datei wartet bis zum Ende der aktuellen Datei und kann, solange er nur vorgemerkt ist, über denselben Button wieder zurückgenommen werden.
- Zwischenverschieben / Bereits fertige Daten verschieben startet einen separaten MoveThread. Eingabefelder bleiben dabei nutzbar; nur ein parallel laufender Verschiebevorgang ist gesperrt.
- Bei Abbruch fragt Dragon Tools, ob bereits erfolgreich konvertierte Dateien noch verschoben werden sollen.

3.3.1 Worker Anzahl während des Laufs ändern

Neben Pause steht die Auswahl Worker mit Werten von 1 bis 8. Sie legt die maximale Zahl gleichzeitig konvertierter Dateien im aktuellen Konverter-Tab fest. 1 bedeutet Verarbeitung nacheinander. Die Anzeige daneben nennt die aktuell aktiven Konvertierungen.

Erhöhen von 1 auf 2 oder 3: Freie Plätze werden sofort mit wartenden Dateien in der aktuellen Queue-Reihenfolge belegt. Bereits laufende Dateien werden nicht neu gestartet; ohne wartende Dateien entstehen keine zusätzlichen Worker.

Reduzieren von 2 auf 1: Beide aktiven Dateien laufen weiter. Sobald die erste fertig ist, startet keine Ersatzdatei. Erst wenn wieder ein Platz unterhalb des neuen Limits frei ist, beginnt die nächste Datei. Beim Wechsel von 3 auf 1 gilt dieselbe Regel.

Während Pause oder eines vorgemerkten Abbruchs starten durch eine Erhöhung keine neuen Dateien. Fortsetzen oder das Zurücknehmen von Abbruch nach Datei nutzt anschließend das gewählte Limit. Die Live-Auswahl ersetzt keinen Abbruch.

Das Limit betrifft die Konvertierung beziehungsweise das Encoding. DV/HDR-Metadaten, Final-Mux und weitere Nachbearbeitung können nach der Freigabe eines Encode-Platzes zusätzlich weiterlaufen. Ein kleineres Worker-Limit beendet diese Arbeiten nicht.

Die Auswahl wird als CPU- beziehungsweise GPU-Vorgabe gespeichert. Während eines Laufs ist der Encoder dieses Laufs maßgeblich; ohne aktiven Lauf der gewählte Encoder. Andere bereits laufende Konverter-Tabs behalten ihr eigenes Limit. Auch ein mit einer Datei und einem Worker gestarteter Lauf nutzt den Koordinator, damit später hinzugefügte Dateien parallel starten können. Beim separaten DV-Remux und während Verschieben ist die Auswahl deaktiviert.

3.4 Menüs und Shortcuts

Shortcut	Funktion	Bereich
F1	Hilfe öffnen	Hilfe
F2	PDF-Handbuch öffnen	Hilfe
F9	Werkzeuge prüfen	Werkzeuge
F11	Legacy-Changelog V8 öffnen	Hilfe
F12	Changelog V9 öffnen	Hilfe
Ctrl+Q	App beenden	Datei
Ctrl+W	Aktuellen Tab schließen	Ansicht
Ctrl+Shift+Q	Warteschlange öffnen	Zoom-Fenster
Ctrl+Shift+T	Letzten Tab wieder öffnen	Ansicht
Ctrl+Shift+A	Alle Tabs wieder öffnen	Ansicht
Ctrl+R	Standardwerte wiederherstellen	Datei
Delete	Markierte Datei aus Queue entfernen	Konverter

Die Menüs sind nach Arbeitslogik getrennt: Datei für Queue und Beenden, Einstellungen für Pfade/Timeouts/IMAX/Logbereinigung, Regeln für Audio und Untertitel, Online-Metadaten für TMDB/TheTVDB, Werkzeuge für Diagnose und Systemtest, Zoom-Fenster für große Listen und Hilfe für Help, Handbuch und Changelogs.

4 Konvertierungsworkflow

Der normale Konverter läuft in V9 über eine Workflow-Engine. Dadurch sind Analyse, Planung, Verarbeitung, Validierung, Ersetzen und Cleanup getrennte Schritte. Genau diese Trennung ist wichtig, damit Fehlerberichte sagen können, an welcher Stelle eine Datei gescheitert ist.

4.1 Ablauf pro Datei

9. Analyse: MediaAnalysisService ruft MediaAnalyzer auf und sammelt Stream- und HDR-Daten.
10. Planung: Effective Encoder Settings, PipelineDecisionService und EncodePlanService bauen den konkreten Plan.
11. Verarbeitung: StandardPipelineRunner, DVProcessingPipeline oder HDRPlusConversionHelper erzeugen die Ausgabedatei.
12. Validierung: OutputVerifier prüft Container, Streams, Audio und Dauer.
13. Reparatur: Bei MKV-Laufzeitfehlern läuft maximal ein normaler Remux und danach nur bei sicherem Muster eine Timestamp-Reparatur.
14. Replace/Archive: ReplaceService ersetzt Originale nur bei gültiger Ausgabe; Größenregelverletzungen landen im Archiv.
15. Cleanup: Temporäre Dateien und Sidecars werden bereinigt oder an den finalen Ausgabestem verschoben.
16. Result: WorkerResultService und ConversionResultService schreiben Status, Abschlussbericht und optionale Wiederaufnahme.

Seit V9.3 ist die optionale NFO-/Trickplay-Erstellung als asynchrone Nacharbeit vom eigentlichen Encoding entkoppelt. Nach erfolgreicher Validierung und Ersetzung wird die Datei zunächst mit 🧩 markiert. Danach kann die nächste Queue-Datei bereits starten, während im Hintergrund NFO und Trickplay erzeugt werden.

Wichtig für Verschieben und Abschlussbericht: Der Worker wartet am Batch-Ende auf alle offenen Nacharbeitsaufträge. Dadurch werden Sidecars, NFO-Dateien und Trickplay-Ordner erst vollständig registriert und anschließend gemeinsam verschoben oder im Bericht aufgeführt.

4.2 Pipeline-Auswahl

Stand 9.9.1: Preflight, Planung und Laufzeit verwenden dieselben effektiven Container- und Dateioptionen. Audio-/Untertitel-Titel, Sprache sowie Default-/Forced-Auswahl werden nach dem Mux zurückgelesen. MKV-Audio-Forced und MP4-Untertitel-Forced werden passend zum Container gesetzt; eine Audio-Forced-Auswahl wird in MP4 nicht als Untertitelrolle ausgegeben. MP4-Metadaten werden am eigenen Kandidaten vor der rein lesenden Verifikation abgeschlossen. Quell-Streamindizes bleiben von Output-Spur-IDs getrennt.

Pipeline	Auslöser	Container	Was erhalten bleibt
standard	Normale Quellen oder DV/HDR10+ deaktiviert.	meist MKV	Video wird neu encodet; HDR10-Farbtags bleiben bei HDR-Basis erhalten.
dv	Dolby Vision erkannt und DV erhalten aktiv.	MP4; optional MKV	DV-RPU wird extrahiert, ggf. Profil 7 zu 8.1 konvertiert und wieder injiziert.
hdrplus	HDR10+ erkannt, HDR10+ erhalten aktiv, kein DV-Sonderfall.	MKV	HDR10+-Metadaten werden extrahiert und in den finalen Bitstream injiziert.

Pipeline	Auslöser	Container	Was erhalten bleibt
dv + hdr10+	Datei enthält DV und HDR10+, beide Erhalte-Optionen aktiv.	MP4; optional MKV	HDR10+ wird zusätzlich durch den DV-Pfad getragen und nach der RPU-Injektion geprüft.
strip-only	Sonderoption Strip-Only aktiv.	Zielcontainer	Streams werden kopiert; kein Burn-In und kein Reencode.
dv-remux	Button DV-Remux → MP4/MKV; Zielcontainer folgt der Dolby-Vision-Einstellung.	MP4; optional MKV	Video bleibt unangetastet; Audio folgt den zentralen Regeln und dem Zielcontainer. Bei MP4 gilt die globale Sidecar-Policy: aktiv = Untertitel extern, deaktiviert = Text intern als mov_text und Bitmap-Formate extern. MKV speichert ausgewählte Untertitel intern.
av1_dv	AV1-Ziel, Dolby Vision erkannt und DV erhalten aktiv.	MP4; optional MKV	CPU/SVT-AV1 schreibt Dolby Vision Profile 10; finale DV-Prüfung ist fail-closed.
av1_hdrplus	AV1-Ziel, HDR10+ erkannt, HDR10+ erhalten aktiv und DV nicht priorisiert.	MKV; optional MP4	CPU/libaom-av1 erhält HDR10+ als T.35/OBU; deutlich langsamer als SVT-AV1; finale HDR10+-Prüfung.
av1_dv Priorität	AV1-Ziel, Quelle enthält DV + HDR10+, beide Erhaltungsoptionen aktiv.	MP4; optional MKV	Nur Dolby Vision wird erhalten; HDR10+ wird bewusst deaktiviert und als INFO im Kurzlog gemeldet.

4.3 Per-Datei-Profile und manuelle Encoder-Overrides

Stand 9.9.1: Die im Preflight ausgewählte Serienfassung und der geplante Zielordner bleiben nach dem Schließen des Dialogs verbindlich. Titel, Jahr und Medienbereich gehören zur Auswahl; Änderungen verwerfen alte Auflösungen. Dateioptionen, Profile und Warteschlangeneinträge werden tief aufgenommen. Ungültige CRF-/Trackwerte werden vor dem Start abgewiesen; spätere GUI-Änderungen verändern laufende Aufträge nicht.

Stand 9.9.1: Preflight, Regelvorschau, Medieninfo und Worker verwenden die wirksamen Einstellungen pro Datei. Globale Werte, zugewiesenes Encoderprofil und direkte Datei-Overrides werden in derselben Priorität zusammengeführt. DV-/HDR10+-Erhalt, Generatoranforderung und Container dürfen dadurch nicht auseinanderlaufen. Ungültige Container werden sichtbar abgelehnt. Verspätete Metadatenantworten überschreiben keine neuere Benutzereingabe.

Globale Tab-Einstellungen gelten zunächst für alle Dateien. Ein gespeichertes Per-Datei-Profil kann diese Werte gezielt übersteuern. Zusätzlich steht ein manueller Encoder-/Skalierungs-Override zur Verfügung, der pro Datei den Encoder-Backend (CPU/NVENC/QSV/AMF), Qualitätswert, Preset, Skalierung und backend-spezifische Optionen setzen kann. Die Priorität lautet global < gespeichertes Encoderprofil < manueller Datei-Override. Der Zielcodec bleibt an den aktuellen Codec-Tab gebunden; Overrides mit falschem Zielcodec werden nicht angewendet. Über die Mehrfachauswahl der Queue kann derselbe manuelle Encoder-/Skalierungs-Override auf mehrere wartende Dateien verteilt werden, ohne deren Audio-, Untertitel-, IMAX-, DV/HDR- oder Remux-Einstellungen zu verändern.

4.4 Strip-Only

Strip-Only ist intern in Runtime-, Audio-, Untertitel- und Sidecar-Services aufgeteilt. Der öffentliche Strip-Only-Pfad bleibt unverändert; die einzelnen Entscheidungen sind getrennt testbar und verwenden denselben zentralen Prozess-Lifecycle wie die übrigen Toolaufrufe.

Strip-Only ist kein Qualitätsmodus, sondern ein Copy-Modus. Er entfernt oder übernimmt Streams nach Plan, encodet Video aber nicht neu. Untertitel-Burn-In wird in diesem Modus nicht ausgeführt, weil Burn-In immer ein neues Videobild erzeugen würde.

5 Encoder, Video und Bildlogik

5.1 Zielcodecs

Codec	Geeignet für	V9-Verhalten
H.265 / HEVC	Moderner Standard für Serien, Filme, HDR, DV und UHD.	Bei allen Encodern echtes 10-bit für H.265: CPU yuv420p10le, GPU p010le, Profil main10.
H.264 / AVC	Maximale Kompatibilität mit älteren Playern.	Bleibt 8-bit-orientiert; kein Main10-Zwang, weil H.264-10-bit viele Geräte ausschließt.
AV1	Sehr moderne Geräte und hohe Kompression.	Standardpfad je nach Encoder CPU/NVENC/QSV/AMF. AV1-DV10 nutzt derzeit CPU/SVT-AV1; AV1-HDR10+ CPU/libaom-av1. Bei gleichzeitigem DV+HDR10+ und beiden Erhalt-Haken hat Dolby Vision Priorität.

5.2 Encoderwahl

Encoder	Bedeutung	Typische Einstellung
CPU	Software-Encoding über libx264, libx265, libsvtav1 oder für AV1-HDR10+ libaom-av1.	SVT-AV1 ist der normale CPU-AV1-Pfad und wird auch für AV1-DV10 genutzt; libaom-av1 ist nur der deutlich langsamere AV1-HDR10+-Spezialpfad.
Auto	GPU-Erkennung wählt bevorzugten Hardwareencoder.	Praktisch für Standardläufe, wenn mehrere GPUs vorhanden sind.
NVIDIA NVENC	Sehr schneller Hardwareencoder.	CQ 23, Preset p6, B-Frames 4, Lookahead 32, Spatial/Temporal AQ aktiv.
Intel QSV	Intel Quick Sync.	Q 23, Preset medium/slow, Lookahead bei Bedarf.
AMD AMF	AMD Hardwareencoder.	QP/CRF 23, balanced oder quality.

5.3 Qualitätsregler

Regler	Bedeutung	Faustregel
CRF / CQ / Q / QP	Qualitätsziel; kleinere Werte bedeuten bessere Qualität und größere Dateien.	18-28 für H.264/H.265, AV1 oft höher.
Preset	Geschwindigkeit gegen Kompression.	Langsamer spart oft Dateigröße, kostet aber Zeit.
B-Frames	Zusätzliche Referenzbilder für bessere Kompression.	4 bei NVENC, 8 bei x265 sind typische Ausgangswerte.
Lookahead	Encoder betrachtet kommende Frames für bessere Entscheidungen.	32 bei NVENC, 40 bei QSV/x265 sind gute Startwerte.
AQ	Adaptive Quantisierung verteilt Bits auf komplexe Bildbereiche.	Aktiv lassen; zu hohe Stärke kann Artefakte erzeugen.
Tune	Spezialprofil für Materialarten wie grain oder animation.	Nur setzen, wenn das Material dazu passt.
Lookahead-Level (NVENC)	Feinsteuerung der NVENC-Lookahead-Qualität; Auto übergibt keinen zusätzlichen Parameter.	Nur bei bewusster Testsituation setzen; vorher F9-Systemtest prüfen.
Multipass (NVENC)	Steuert NVENC-Multipass-Modi wie qres oder fullres.	Auto/Default belassen, wenn keine gezielte Qualitätsmessung läuft.

5.4 H.265 10-bit in V9

NVENC-Erweiterung V9.4: Lookahead-Level und Multipass sind als Expertenoptionen ergänzt. Auto bleibt absichtlich der Standard; dann werden keine zusätzlichen FFmpeg-Parameter gesetzt. Nur eine bewusste Auswahl schreibt -lookahead_level beziehungsweise -multipass in den finalen FFmpeg-Befehl. Der F9-Systemtest erkennt zusätzlich, ob der gebündelte FFmpeg/NVENC-Build diese Optionen überhaupt anbietet.

V9 korrigiert den Unterschied zwischen Signal und echter Bildtiefe. Früher konnte eine Datei als Main10 wirken, obwohl das Pixel-Format noch 8-bit war. Jetzt setzt Dragon Tools bei H.265 den passenden Pixel-Format-Zwang: libx265 nutzt yuv420p10le, NVENC/QSV/AMF nutzen p010le, AMF zusätzlich -bitdepth 10.

> Wichtig: 10-bit macht SDR nicht automatisch zu HDR. Es kann Banding verringern, erhöht aber je nach Quelle und Encoder leicht den Aufwand. H.264 bleibt bewusst ohne 10-bit-Zwang, weil viele Player H.264 High10 nicht unterstützen.

5.5 Skalierung, Auto-Crop und IMAX

Stand 9.9.1: Unbekannte Pipelinebezeichnungen werden als Fehler beendet. Auto-Crop, IMAX und Frame-Probes übernehmen keine Ergebnisse eines fehlgeschlagenen Tools. Eine nicht mehr vorhandene Bilduntertitel-Auswahl brennt

keine andere Spur ersatzweise ein. Windows-Pfadvarianten führen zum selben Datei-Override; Encoderregler und gespeicherte Optionen werden vor dem Einsatz auf gültige Werte geprüft.

- Skalierung original lässt die Auflösung unverändert; 4K, 1080p, 720p und 480p erzeugen feste Zielhöhen mit gerader Breite.
- Auto-Crop erkennt schwarze Balken und setzt einen crop-Filter. Bei DV wird der Crop zusätzlich in Level-5-Metadaten der RPU berücksichtigt.
- IMAX Auto-Erkennung prüft wechselnde Seitenverhältnisse und deaktiviert Auto-Crop für Dateien, bei denen variable Bildhöhe plausibel ist.
- Bei fraglichen Quellen ist ein Blick ins Log wichtig: Dort steht, ob kein Crop, ein fixer Crop oder IMAX-Schutz gewählt wurde.

Die IMAX-Auto-Erkennung ist global einstellbar. Standardmäßig wird alle 90 Sekunden ein kurzer Abschnitt geprüft; auffällige Wechsel der Bildhöhe schützen die Datei vor einem falschen dauerhaften Crop.

- Quellbildprüfung: Vor dem Encoding kann Dragon Tools defektes Bildmaterial prüfen. Standard: alle 10 % der Laufzeit, jeweils 2 Sekunden mit 2 fps.
- Blockierlogik: Ab 80 % auffälligen Prüfpunkten und mindestens 4 Treffern wird die Datei als verdächtig markiert.
- Bedienung: Über das Rechtsklickmenü kann die Quellbildprüfung manuell ausgeführt, eine Warnung übergangen oder die Datei aus der Queue entfernt werden.

6 HDR, Dolby Vision und HDR10+

6.1 Erkennung

Stand 9.9.1: Analyse und Pipelinewahl beziehen sich auf die primäre Videospur. FFmpeg-Streamindex und Matroska-Tracknummer bleiben getrennte Kennungen. HDR10+ oder Dolby Vision auf einer zweiten Videospur schaltet die primäre Pipeline nicht um. BT.2020-Primaries allein beweisen kein HDR. Unplausible Einheiten, fehlende Probewerte und widersprüchliche DV-Profile werden konservativ ausgewertet; der Medienvertrag prüft auch das erwartete DV-Profil.

MediaInfo ist die primäre Analysequelle für HDR, HDR10+ und Dolby Vision. HDR10+ wird nicht nur über den Text „HDR10+“ erkannt, sondern auch über ST-2094-40-Bezeichnungen wie „SMPTE ST 2094 App 4“. ffprobe ergänzt Streamdaten und kann HDR10+ zusätzlich über Frame-Side-Data „HDR Dynamic Metadata SMPTE2094-40 (HDR10+)“ nachweisen. Für Dolby Vision werden MediaInfo-Profilangaben, DV-Codec-Tags (dvhe/dvh1/dva1/dvav/dav1) und der strukturierte ffprobe-DOVI-Configuration-Record zusammengeführt. Bei einem Profilkonflikt wird gewarnt; der strukturierte DOVI-Profilwert wird für die Profilnummer bevorzugt. Erkennung und Pipeline-Unterstützung bleiben bewusst getrennt.

6.2 Standardpfad bei HDR und entfernten Metadaten

Wenn Dolby Vision oder HDR10+ nicht erhalten werden, läuft die Datei durch den Standardpfad. Für HDR10-Basisquellen bleiben BT.2020/PQ-Farbtags erhalten. DV Profil 7 kann dabei als HDR10-Basis sinnvoll weiterverarbeitet werden. DV Profil 5 braucht eine eigene Farbkonvertierung, weil es keinen normalen HDR10-Fallback besitzt.

6.3 DV-Pipeline

Stand 9.9.1: DV-Erhalt verlangt finale RPU-Evidenz im tatsächlichen Bitstream und eine verlässliche Frame-Parität; Containersignalisierung allein genügt nicht. Bei der Quell-RPU-Extraktion in STEP 3/7 löst ausschließlich die dovi_tool-Signatur Invalid RPU last byte einen einmaligen Neuplanungsversuch aus. Nur Dolby Vision wird für diese Datei deaktiviert. HDR10+-Policy, Audio, Untertitel und Encoderprofil bleiben erhalten. Dieselbe Signatur bei Injection oder Verifikation sowie andere DV-Fehler bleiben harte Fehler.

Der 9.9.1-DV-Pfad trennt Metadaten und Bildpfad konsequent. Bei Matroska wird die RPU direkt aus dem Originalcontainer gelesen/normalisiert; der Encoder decodiert ebenfalls direkt aus der Original-MKV. Ein roher HEVC-Stream entsteht erst nach dem Encode für die spätere RPU-/HDR10+-Injection.

1. RPU direkt aus der Matroska-Quelle extrahieren; Profil 5 nutzt dovi_tool Mode 3, Profil 7/8 Mode 2, damit der Metadatenpfad auf P8.1 normalisiert ist.
2. Video mit den effektiven Encoder-/Filtereinstellungen direkt aus der Original-MKV nach encoded.hevc encodieren. source.hevc und p8.hevc sind keine regulären Bildquellen mehr.
3. Bei DV Profil 5 den Base Layer im produktiven Bildpfad mit libplacebo von ICtCp nach BT.2020/PQ umsetzen; Profil 7/8 verwenden denselben normalen Filter-/Crop-Pfad wie der jeweilige Encode.
4. Auto-Crop auf gerade 4:2:0-Geometrie anwenden; RPU-Level-5 nur diagnostisch abgleichen und anschließend auf die tatsächlich gecroppte Zielgeometrie anpassen.

5. Verlässliche Encoder-Framezahl gegen die normalisierte RPU prüfen. Bei einem geeigneten Mismatch darf der konservative Teilreparaturpfad versuchen, nur einen bewiesenen GOP-/IRAP-Bereich aus der Original-MKV neu zu encodieren.
6. RPU in encoded.hevc injizieren; im DV+HDR10+-Kombipfad zusätzlich HDR10+-Metadaten injizieren. Rückextraktion/Frame-Parität und semantische Metadatenprüfungen bleiben verpflichtend.
7. Finalen Zielcontainer bauen: MP4 mit MP4Box (-inter 500) oder MKV mit mkvmerge. Audio, Untertitel und Sidecars folgen weiterhin den zentralen Regeln.

Die Audioaufbereitung aus Original oder Trackplan bleibt eine unterstützende Teilstufe. Ein temporärer source.hevc-Hilfsstream ist nur noch für nicht direkt unterstützte Container oder besondere Metadaten-Fallbacks zulässig; er darf nicht zur normalen Bildquelle werden. Fehlgeschlagene oder nicht eindeutig verifizierbare Recovery-Kandidaten werden nicht übernommen.

6.3.1 Frame-Mismatch-Teilreparatur

Der Recovery-Pfad ist bewusst fail-closed. Er ist nur für DV5/DV7/DV8 mit fehlenden Bildern und unter engen Grenzen freigegeben. Bild-Fingerprints müssen die zeitliche Ausrichtung vor und nach dem Defekt beweisen; der Reparaturbereich wird auf sichere HEVC-GOP-/IRAP-Grenzen erweitert. Der Ersatzbereich wird direkt aus der Original-MKV mit derselben Filter-/Encoderkonfiguration neu encodiert.

Ein Kandidat wird nur übernommen, wenn er vollständig decodierbar ist, exakt der RPU-Bildzahl entspricht und die Übergänge an beiden Seiten zur Quelltimeline passen. Zeitlich bildverändernde Filter, unsichere Fingerprints, ungeeignete Encoder, zu große Fehlbereiche oder jede Validierungsabweichung brechen die Reparatur ab und lassen den bestehenden Diagnose-/Archivpfad greifen.

6.4 HDR10+-Pipeline

Stand 9.9.1: Der HDR10+-Postprozess erzeugt, injiziert und verifiziert Metadaten im fertigen Output. Planner und Generator prüfen denselben PQ-/BT.2020-Quellvertrag. JSON-Struktur, Frameanzahl, Generatorausgabe und injizierter Bitstream müssen zusammenpassen. Bei einem Pflichtfehler bleibt der Job fehlgeschlagen; verwertbare Video-, JSON- und Bitstream-Kandidaten bleiben erhalten. AV1 übernimmt bildabhängige HDR10+-Metadaten nach Bildänderungen nicht ungeprüft.

Der HDR10+-Pfad arbeitet in fünf Hauptschritten: (1) Bei Matroska-Quellen liest hdr10plus_tool die dynamischen Metadaten direkt aus der Quelldatei; nur für nicht direkt unterstützte Container bleibt innerhalb dieses Schritts ein Annex-B-Fallback. (2) FFmpeg encodiert das Video direkt nach encoded.hevc und bereitet Audio/interne Untertitel im selben Lauf nur als kleinen streams.mkv-Donor vor. (3) hdr10plus_tool injiziert die Metadaten in den neuen HEVC-Bitstream. (4) MKV wird final mit mkvmerge, MP4 mit MP4Box -inter 500 gemuxt. (5) Die dynamischen HDR10+-Metadaten werden im fertigen Ziel semantisch gegen die Quelle verifiziert. Die früheren großen Zwischenstufen source.hevc und encoded.mkv -> encoded.hevc entfallen bei normalen MKV-Quellen.

Die interne Verantwortung ist im Abschlussstand getrennt: HDRPlusToolRunner führt externe Tools mit expliziten zulässigen Returncodes aus, HDRPlusMuxService baut MKV/MP4, und HDR10PlusBitstreamService übernimmt Extract/Inject/semantische Verifikation. mkvmerge-Returncode 1 gilt als Warnung und nicht als Fehler. Ein vorhandener Audio-Donor, dessen ffprobe-Analyse fehlschlägt oder keine Audiospur bestätigt, stoppt den MP4-Mux fail-closed, damit kein stiller Audioverlust als Erfolg durchläuft.

6.4.1 Dragon HDR10+ Generator 0.2.0

Der eigenständige Generator ist eine separate EXE/CLI, gehört aber zum DragonTools-Projekt. Er akzeptiert vorhandene PQ/ST2084-Videoquellen mit BT.2020, decodiert zeitlich jeden Frame auf eine räumlich reduzierte 16-Bit-RGB-Analysefläche, linearisiert PQ nach SMPTE ST 2084, ermittelt MaxSCL, AverageMaxRGB und Luminanzverteilungen, erkennt Szenen und schreibt ein hdr10plus_tool-kompatibles ST-2094-40-Profile-A-JSON. Profil A ist bewusst gewählt:

Dragon Tools erzeugt gemessene dynamische Szenenstatistik, aber keine erfundenen Profile-B-Knee-/Bezier-Tone-Mapping-Kurven. Der Generatorbereich ist direkt über „Einstellungen → ✨ Dragon HDR10+ Generator“ erreichbar; der ausführbare EXE-Pfad bleibt zentral unter „Einstellungen → Werkzeugpfade“ gepflegt.

Direkter PowerShell-Aufruf:

```
HDRPlusGenerator.exe analyze `
  --input "D:\Videos\Film.mkv" `
  --output "D:\Videos\Film_hdr10plus.json"
```

Optionale Parameter sind --analysis-width (Standard 256), --scene-threshold (0.32), --min-scene-frames (6) sowie explizite --ffmpeg- und --ffprobe-Pfade. Eine höhere Analysebreite erhöht die räumliche Messauflösung; zeitlich wird unabhängig davon weiterhin jeder Frame analysiert. Die Generator-JSON ist noch keine fertige Medienausgabe: Injection, finaler Container-Mux und semantische Endverifikation bleiben in Dragon Tools und verwenden hdr10plus_tool, mkvmerge bzw. MP4Box.

6.4.2 HDR10+ nach SDR→HDR oder verlustfreiem Remux

Ist die HDR10+-Erzeugung global oder im Datei-Override aktiviert, wird bei SDR→HDR zuerst der fertige PQ/BT.2020-10-Bit-HEVC-Output erzeugt. Erst danach analysiert der Dragon-HDR10+-Generator genau diesen finalen Bildstrom; die ursprüngliche SDR-Quelle wird nicht zur HDR10+-Messung verwendet. Bei einer vorhandenen HDR10-HEVC-Datei ohne HDR10+ kann derselbe Postprozess nach Strip-Only/Remux laufen. Das Video wird dabei nicht erneut encodiert: HEVC wird extrahiert, analysiert, mit HDR10+-SEI versehen, neu gemuxt und anschließend erneut geprüft.

Für ComfyUI/HDRTVDM wurde auf einer RTX 4080 SUPER 16 GB bei 1080p ein Praxiswert von ungefähr 4:1 Konvertierungsdauer zu Filmdauer gemessen. Beobachtet wurden etwa 85–90 % GPU-Auslastung und rund 4,3–4,4 GB VRAM. Dieser Wert dient nur als Orientierung; Auflösung, Quelle, Modell, Batchgröße und Hardware können die Laufzeit deutlich verändern.

AV1-HDR10+ ist ein eigener Beta-Pfad und verwendet derzeit CPU/libaom-av1 mit 10-Bit sowie HDR10+-T.35/OBU-Metadaten und finaler Verifikation. Der Pfad ist deutlich langsamer als der normale SVT-AV1-Pfad und deshalb als Spezialoption zu verstehen. AV1-Dolby-Vision verwendet dagegen CPU/SVT-AV1 und schreibt Dolby Vision Profile 10 nativ über den aktuellen FFmpeg-Encoderwrapper.

6.5 SDR→HDR mit ComfyUI/HDRTVDM

Stand 9.9.1: ComfyUI-Output und Manifest müssen zum aktuellen Auftrag gehören; ein alter erfolgreicher Output bestätigt keinen neuen Job. Nach einem Fehler der History/API wird ein bereits gesendeter Auftrag kontrolliert abgebrochen. Auch der letzte Encoder- oder Mux-Schritt erhält die wirksamen Einstellungen des jeweiligen Datei-Overrides.

Dragon Tools kann eindeutig als SDR BT.709 erkannte CFR-Quellen optional per ComfyUI/HDRTVDM in einen 10-Bit-PQ/BT.2020-Arbeitsstrom überführen. Der Voll-Datei-Worker verarbeitet Frames über die lokale ComfyUI-API und speist die Ausgabe ohne vollständige PNG-/TIFF-Zwischensequenz direkt in den vorhandenen Encoder.

- Direkter Zugriff: Einstellungen → 🌈 SDR → HDR / ComfyUI.
- ComfyUI-Autostart: Ist die API nicht erreichbar, kann Dragon Tools die konfigurierte Startdatei starten, bis zur eingestellten Frist wiederholt prüfen und weiterarbeiten, sobald die API verfügbar ist.
- Per-Datei-Override: SDR→HDR kann global aus bleiben und nur für einzelne Quellen aktiviert werden; umgekehrt kann eine einzelne Quelle bei global aktiver Funktion ausgeschlossen werden.
- Fail-safe: Fehlen BT.709-Colorimetry, API, Modell, Bridge-Nodes oder ein gültiger Workflow, bleibt die Datei im normalen SDR-Pfad. Ein bereits gestarteter und später fehlgeschlagener HDRTVDM-Job ist dagegen ein echter Auftragsfehler.

Praxiswert: Ein realer 1080p-Test mit params_3DM.pth auf einer RTX 4080 SUPER 16 GB lag bei ungefähr 4:1 Konvertierungsdauer zu Videodauer, etwa 85–90 % GPU-Auslastung und rund 4,3–4,4 GB dediziertem VRAM. Bei Animationen oder stark wechselnder Beleuchtung sollten kurze Referenzszenen vor einem Serienlauf geprüft werden.

6.6 HDR-Erkennung, Datei-Overrides und Strip-Only

Die HDR-Policy kann pro Datei die globalen Einstellungen gezielt übersteuern. Das gilt für DV/HDR10+-Erhalt, SDR→HDR und die Erzeugung neuer HDR10+-Metadaten. Dadurch lassen sich Spezialpfade auf einzelne Quellen begrenzen, ohne ein globales Profil umzuschalten.

- HDR10+-Erkennung: MediaInfo HDR10+/HDR10plus, SMPTE ST 2094 App 4 bzw. ST 2094-40 sowie ffprobe-Frame-Side-Data werden zusammengeführt.
- Dolby-Vision-Erkennung: MediaInfo-Profilfelder, DV-Codec-Tags und ffprobe-DOVI-Configuration-Record liefern Profil/Level; Konflikte werden protokolliert statt still überschrieben.
- Strip-Only auf Mehrfachauswahl: Die Aktion wirkt auf alle markierten wartenden Dateien. Bei gemischter Auswahl wird Strip-Only für alle aktiviert; sind alle bereits aktiv, wird es für alle deaktiviert. Laufende/abgeschlossene Dateien werden einzeln abgelehnt.
- HDR10+-Remux ohne Re-Encode: Für vorhandenes HDR10-HEVC wird die Videospur extrahiert, ST-2094-40 in den HEVC-Bitstream injiziert und anschließend mit den vorhandenen Audio-/Untertitel-/Kapitelspuren neu gemuxt. Die codierten Bildframes bleiben unverändert.

6.7 DV + HDR10+ Sonderkombination

Für H.265/HEVC gilt: Wenn eine Quelle sowohl Dolby Vision als auch HDR10+ enthält und beide Erhalte-Haken aktiv sind, läuft sie über den DV-Pfad. Zusätzlich werden HDR10+-Metadaten aus der Quelle extrahiert, vor der finalen DV-RPU-Injektion in den Encode geschrieben und danach erneut geprüft. Der finale Container richtet sich nach der DV-Container-Einstellung: MP4 wird mit MP4Box, MKV mit mkvmerge gebaut. Anschließend prüft MediaInfo zuerst direkt auf Dolby Vision und HDR10+. Bestätigt MediaInfo alle laut Medienvertrag benötigten Merkmale, ist die Finalverifikation abgeschlossen. Fehlt ein Merkmal oder ist das Ergebnis unklar, wird nur dieses Merkmal über den bestehenden Bitstream-Fallback geprüft.

Für AV1 gilt bewusst eine andere Policy: Es gibt derzeit keinen DV+HDR10+-Kombipfad. Sind Dolby Vision und HDR10+ in der Quelle vorhanden und beide Erhaltungsoptionen aktiv, hat Dolby Vision Priorität. Dragon Tools wählt AV1-DV10 über CPU/SVT-AV1, setzt den effektiven HDR10+-Erhalt auf aus und schreibt dazu eine INFO in den Kurzlog. Es erfolgt keine Archivierung und keine Warnung. Ein expliziter per-Datei-Override auf AV1-HDR10+ bleibt möglich.

6.8 Typische Fehlerbilder

Symptom	Wahrscheinliche Ursache	Nachschlagen
Farben wirken flach oder falsch	DV Profil 5 ohne richtige Farbkonvertierung oder HDR-Tags fehlen.	Log: Quelle, HDR/DV-Status, Pipeline, Farbraum-Argumente.
DV nicht mehr erkannt	DV-Erhalt deaktiviert, Tool fehlt oder RPU-Injection fehlgeschlagen.	Fehlerbericht: DV-Schritte, dovi_tool-Ausgabe, MP4Box-Mux.
HDR10+ fehlt	HDR10+-Pfad nicht gewählt oder Verify nach Injection fehlgeschlagen.	Fehlerbericht und Toolprüfung F9.

Symptom	Wahrscheinliche Ursache	Nachschlagen
MP4 hat keine eingebetteten Untertitel	Die globale Option „MP4-Sidecars aktivieren“ ist eingeschaltet oder die ausgewählte Spur ist PGS/SUP bzw. VobSub und deshalb nicht als mov_text kompatibel.	Untertitel-Regeln: MP4-Sidecars aktivieren; bei internem Textmodus Codec/Format der Spur prüfen.

6.9 DV-P5-Remux über zentralen Prozesspfad

Stand 9.9.1: DV-Remux verwendet die ausgewählte Videospur und die zum jeweiligen Tool passende Trackkennung. Finale Verifikation prüft die echte RPU sowie erwartete Track- und Default-Flags. MOV_TEXT wird für MKV als SRT geplant. Unbekannte Container werden abgelehnt. Ein rechtzeitig erkannter Abbruch verhindert den finalen Commit; bei einem Verifikationsfehler bleibt ein brauchbarer Kandidat für Diagnose und Wiederaufnahme erhalten.

- DV-P5-Remux-Hilfsschritte laufen über den zentralen Prozesspfad mit Timeout, Abbruchsignal und sauberer Fehlerausgabe.
- Nackte subprocess.run()-Aufrufe ohne Timeout wurden für diesen Pfad entfernt.
- Fehlerausgaben werden mit gekürztem stdout/stderr-Kontext in Log und Fehleranalyse übernommen.

7 Audio-Regeln

Blu-ray-/M2TS-Audiomapping V9.4: Dragon Tools übernimmt den globalen FFmpeg-Streamindex aus ffprobe auch dann, wenn MediaInfo zusätzliche Komfortdaten liefert. Dadurch zeigen Analyse, GUI, Regelentscheidung und FFmpeg-Map wieder auf dieselbe Spur. Das behebt typische Fälle, in denen japanische Blu-rays wegen abweichender M2TS-Streamreihenfolge die falsche Audiospur gewählt hätten.

Audio wird in V9 nicht über starre Indexnummern ausgewählt, sondern über Sprache, Kommentar-/Deskriptivmarker, Codec, Kanalanzahl, Bitrate und Zusatzregeln. Die Regeln bleiben getrennt von den Untertitelregeln.

7.1 Sprachen und Fallback

Einstellung	Wirkung
Sprach-Priorität	Reihenfolge der bevorzugten Audiosprachen, z. B. Deutsch vor Englisch. Fehlende Sprachen werden übersprungen.
Max. Sprachen übernehmen	Begrenzt, wie viele unterschiedliche Sprachen in die Ausgabe kommen.
Max. Tonspuren je Sprache	Begrenzt mehrere Tracks derselben Sprache, z. B. Hauptton und Kommentar.
Fallback bei keinem Treffer	Alle Audiospuren, erste Audiospur oder keine Audiospur übernehmen.
Kommentarspuren überspringen	Tracks mit typischen Kommentar-Markern in Titel/Codecinfo werden ignoriert.
Audiodeskription überspringen	Deskriptive Audiospuren werden nicht als normale Hauptspuren behandelt.

7.2 Passthrough, Transkodierung und Kanalregeln

Stand 9.9.1: Audioplan, FFmpeg-Mapping, Tracktitel und Default-Flags beschreiben dieselben ausgewählten Streams. Ein Abbruch während der Abschlussprüfung verhindert den Audio-Mux-Commit. Kanal-, Sprach- und Codecentscheidungen werden konservativ normalisiert. Bereits geprüfte Quellen werden bei einem fehlgeschlagenen Staging- oder Installationsschritt nicht überschrieben.

Regelstufe	Default	Wirkung

Regelstufe	Default	Wirkung
Mono	AAC bis 128 kbps	Mono wird nicht mehr aus alten Stereo-Regeln abgeleitet; Ziel ist AAC.
Stereo	AAC 192-256 kbps kopieren, Neukodierung auf 192 kbps	Stereo-AAC/AC3/E-AC3 wird im eingestellten Kopierbereich kopiert. Darunter oder darüber wird auf Ziel-Codec und Ziel-Bitrate transkodiert.
3-6 Kanäle	E-AC3 5.1 bis 640 kbps	Kann 5.1 erhalten, transkodieren oder nach neuer Downmix-Regel direkt auf Stereo gehen.
7-8 Kanäle	E-AC3, Zielkanäle einstellbar	Kann auf 5.1 reduziert oder in Zielkanälen neu kodiert werden.
Immer transkodieren	TrueHD, DTS, FLAC, PCM u. a.	Verlustfreie oder problematische Codecs werden in kompatible Zielcodecs umgewandelt.

7.3 5.1-Downmix

Modus	Wirkung	Sinnvoll wenn
Nie downmixen	3-6 Kanäle bleiben mehrkanalig oder werden mehrkanalig transkodiert.	Surround erhalten werden soll.
Immer downmixen	Jede 3-6-Kanal-Spur wird direkt Stereo.	Zielgeräte oder Serienworkflow konsequent Stereo sollen.
Nur bis Bitrate	Downmix nur, wenn die Quellbitrate bis zur Grenze liegt.	z. B. E-AC3 5.1 256 kbps praktisch als schwaches Surround vorliegt.

Flexible Codec-Wahl: Wenn bei aktivem 5.1-Downmix als 5.1-Zielcodec copy gewählt ist, nutzt Dragon Tools den konfigurierten Stereo-Zielcodec. Das ist absichtlich flexibel, damit später nicht überall AAC fest im Code geändert werden muss.

7.4 Zusatz-Stereo-Downmix

Der Zusatz-Stereo-Downmix erzeugt neben einer übernommenen oder transkodierten Mehrkanalspur eine zusätzliche Stereo-Spur. Das ist ein anderer Fall als der direkte 5.1-Downmix: Die Mehrkanalspur bleibt erhalten, und Stereo kommt als Fallback hinzu. Codec und Bitrate sind im Audio-Regeldialog wählbar.

7.5 Entscheidungsbeispiele

Quelle	Regel	Ergebnis

Quelle	Regel	Ergebnis
AAC 2.0 196 kbps	Stereo AAC 192-256 kbps kopieren, Neukodierung auf 192 kbps passt.	Stream copy.
AAC 2.0 500 kbps	Stereo über 256 kbps.	AAC 2.0 256 kbps, wenn Zielcodec AAC ist.
E-AC3 5.1 256 kbps	5.1-Downmix nur bis 256 aktiv.	AAC/E-AC3/AC3 Stereo je nach Stereo-Zielcodec.
TrueHD 7.1	Immer transkodieren und 7.1-Regel.	E-AC3 mit Zielkanälen, z. B. 5.1/640 kbps.
Kommentarspur Deutsch	Kommentarspuren überspringen aktiv.	Wird nicht als Hauptaudio übernommen.

7.6 Audio-Dynamik, DRC und Lautheitsnormalisierung

DRC / Nachtmodus nutzt die Dynamic-Range-Control-Metadaten von AC3- und E-AC3-Quellen. Der Wert wird als FFmpeg-Eingabeoption -drc_scale vor dem Input gesetzt und wirkt deshalb nur, wenn die Quelle diese Metadaten besitzt und die Spur dekodiert wird.

Der DRC-Wert ist von 0,0 bis 4,0 in 0,1-Schritten einstellbar. 0,0 entspricht voller Dynamik, 1,0 normaler AC3/E-AC3-DRC, 2,6 bis 3,0 einem deutlichen Nachtmodus und 3,1 bis 4,0 einem extremen Bereich, der nicht als Standard empfohlen ist.

Wenn DRC aktiv ist und eine passende AC3/E-AC3-Quelle gewählt wurde, verhindert Dragon Tools Stream-Copy automatisch. Der Zielcodec folgt weiterhin den Audio-Regeln oder dem Datei-Override; DRC erzwingt also nicht blind E-AC3, sondern nur das notwendige Dekodieren und Neukodieren.

Die allgemeine Lautheitsnormalisierung nutzt FFmpeg loudnorm und ist codecübergreifend. Sie besitzt einen Zielwert in LUFS, standardmäßig -18 LUFS, und erzwingt ebenfalls eine Neukodierung, weil Lautheit nur über einen Audiofilter geändert werden kann.

Beide Funktionen sind getrennt aktivierbar. In den globalen Audio-Regeln werden die Defaults festgelegt; im Rechtsklick-Dialog einer Datei kann DRC oder Loudnorm gezielt auf Regeln übernehmen, aktivieren oder deaktivieren gesetzt werden.

Diagnose: Wenn ein erwarteter DRC-Effekt ausbleibt, zuerst Quellcodec, Kopierstatus und FFmpeg-Befehl prüfen. Bei AAC, DTS, TrueHD oder FLAC gibt es keine AC3/E-AC3-DRC-Metadaten; dort kann nur die allgemeine Loudnorm-Verarbeitung greifen.

8 Untertitel-Regeln

Untertitel haben eine eigene Sprach-Priorität und eigene Keep-/Burn-In-Regeln. Das ist wichtig, weil eine Sprache für Audio nicht automatisch dieselbe Entscheidung für Untertitel bedeuten muss.

8.1 Sprachliste und Mengenbegrenzung

Einstellung	Bedeutung
Max. Untertitel-Sprachen behalten	Begrenzt, wie viele Sprachen aus der Prioritätsliste übernommen werden.
Max. Untertitel je Sprache	Begrenzt mehrere SRT/ASS/PGS-Spuren derselben Sprache.
Fallback	Keine zusätzlichen Untertitel, ersten passenden oder alle passenden übernehmen.
Bevorzugte Formate	Sortiert z. B. subrip vor ass vor PGS/SUP.
Forced bevorzugen	Forced-Kandidaten werden vor regulären Untertiteln einsortiert.

8.1.1 Automatische Standardspur

Steht dieselbe Sprache an erster Stelle der Audio- und Untertitelregeln, hinterlegt DragonTools einen passenden ausgewählten Untertitel als Standard, wenn die Hauptaudiospur eine andere bekannte Sprache verwendet. Ein vollständiger Untertitel hat Vorrang vor einer Forced-Spur. Die Forced-Kennzeichnung bleibt erhalten.

Diese Entscheidung gilt gemeinsam für Konvertierung, Strip-Only und Remux. Exportierte Sidecars erhalten die Kennung .default. Datei-Overrides bleiben maßgeblich; eine unbekannte Audiosprache führt zu keiner geratenen Sprachentscheidung.

8.2 Forced-Burn-In

Forced-Untertitel können automatisch eingebrannt werden. Die Burn-In-Sprache ist bewusst getrennt von der Keep-Liste, weil man z. B. nur deutsche Forced-Untertitel ins Bild brennen, aber zusätzlich andere Untertitel als auswählbare Spuren behalten kann.

- Bei mehreren gleich plausiblen Forced-Kandidaten kann Dragon Tools nachfragen oder den Auto-Burn-In vermeiden.
- Wenn keine passende Audiosprache vorhanden ist, kann Burn-In blockiert werden.
- Burn-In verändert das Videobild und funktioniert deshalb nicht im Strip-Only-Modus.
- PGS/SUP kann in MKV kopiert oder als Sidecar exportiert werden; MP4 kann nicht alle Bilduntertitel sinnvoll intern tragen.

8.2.1 Bitmap-Untertitel und Videozeitachse

Beim Ende einer eingebrannten PGS-Untertitelspur läuft das Hauptvideo auf seiner eigenen Zeitachse weiter. Der Overlay-Filter verwendet eof_action=pass und repeatlast=0. Das letzte Untertitelbild wird dadurch nicht wiederholt und es entsteht kein zusätzliches Videobild mit einem weit entfernten Zeitstempel. Bildrate, B-Frame-Einstellungen sowie der zeitliche Abstand zwischen Video und Audio bleiben erhalten.

8.3 Welche Untertitel bleiben erhalten?

Für MP4 gilt eine globale Ablageoption für alle Pipelines einschließlich Standard-Encoding, Strip/Remux, HDR10+, AV1, Dolby Vision und DV-Remux. Ist „MP4-Sidecars aktivieren“ eingeschaltet, werden alle ausgewählten Untertitel extern gespeichert. Ist die Option aus, werden Textformate wie SRT, ASS/SSA und WebVTT intern als mov_text/tx3g gemuxt; PGS/SUP sowie VobSub/DVD-Sub bleiben aus Kompatibilitätsgründen externe Sidecars. MKV speichert die ausgewählten Untertitel intern. Bei ASS/SSA gehen beim mov_text-Umbau Styles, Positionierungen und Effekte verloren.

Option	Wirkung
Forced-Untertitel zusätzlich behalten	Forced-Spuren bleiben als auswählbare Spur/Sidecar erhalten, auch wenn ein Burn-In erfolgt.
Untertitel der Sprachliste behalten	Normale Untertitel werden anhand der Prioritätsliste übernommen.
Normale Untertitel behalten	Wenn aus, bleiben nur Forced-Kandidaten.
Nur behalten, wenn kein Forced-Burn-In erfolgt	Verhindert doppelte Untertitel bei Dateien, in denen Forced bereits eingebrannt wurde.

Option	Wirkung
MP4-Sidecars aktivieren	Globale MP4-Policy für Standard-Encoding, Strip/Remux, HDR10+, AV1, Dolby Vision und DV-Remux. Aktiv: alle ausgewählten Untertitel extern. Aus: Text intern als mov_text/tx3g; PGS/SUP und VobSub/DVD-Sub extern. MKV speichert intern.

8.4 Forced-Subtitle-Plausibilitätsprüfung

Wird eine als forced markierte Spur als Full-Sub verdächtig, brennt Dragon Tools sie nicht ein. Stattdessen bleibt sie erhalten oder wird als Sidecar gesichert; die maximale Untertitelanzahl wird dafür um diese Schutzspur erweitert. So geht die Spur nicht verloren, aber ein falsch markierter Volluntertitel zerstört nicht das Bild.

Forced-Spuren werden vor einem automatischen Burn-In auf Plausibilität geprüft. Grundlage ist die Anzahl neuer Untertitelereignisse pro Minute. 0 bis 3 Events pro Minute gelten als unauffällig, 3 bis 5 erzeugen eine Warnung, darüber wird die Spur als wahrscheinlich vollständiger Untertitel behandelt.

8.5 Untertitel automatisch zuordnen und einfügen

Im Register Untertitel und dort Einfügen die Ziel-Videos hinzufügen. Das Feld Untertitel-Datei darf leer bleiben: Dragon Tools sucht für jedes Video ausschließlich in dessen eigenem Ordner nach passenden Untertiteldateien.

Deutsch ist die Standardsprache. Die Suche erkennt die Sprachkennungen .de und .deu sowie den Alias .ger; für Englisch .en und .eng. Die Kennung muss als abgegrenztes Element im Dateinamen stehen. Unterstützt werden .srt, .ass, .ssa und .sup.

Zuerst wird ein übereinstimmender Videostamm vor der Sprachkennung bevorzugt. Beispiel: Serie.S01E02.mkv und Serie.S01E02.deu.srt. Bei unterschiedlichen Release-Zusätzen dürfen Serienname, Staffel und Folge übereinstimmen, zum Beispiel Serie.S01E02.1080p.mkv und Serie.S01E02.de.srt. Folgenkennungen wie S01E02 und 1x02 sind möglich. Die Suche greift nicht auf Unterordner zu.

Fehlt eine passende Datei oder passen mehrere Kandidaten gleich gut, wird dieses Video mit einem Hinweis im Log übersprungen. Es wird keine beliebige Datei gewählt. Eine explizit ausgewählte Untertiteldatei wird dagegen für alle ausgewählten Videos verwendet.

Sprache lässt sich zwischen Deutsch und Englisch wählen. Der Muxer erhält deu beziehungsweise eng; MKV-Werkzeuge können Deutsch beim Auslesen auch als gleichwertiges ger anzeigen. Titelname ist frei editierbar: Standard Deutsch, die Vorgabe Deutsch GPT oder ein beliebiger eigener Titel wie Custom. Ein noch unveränderter Standardtitel folgt dem Sprachwechsel; ein eigener Titel bleibt erhalten. Sprache und Titel sind unabhängige Angaben.

Forced setzt die Kennzeichnung der eingefügten Spur. Das ist von Forced einbrennen im Konverter unabhängig: Einfügen fügt eine auswählbare Spur hinzu und brennt keinen Text ins Videobild.

Die Ausgabe erhält einen freien Namen mit _sub neben dem Video; das Original und vorhandene Ausgabedateien werden nicht überschrieben. MKV verwendet bevorzugt mkvmerge und erhält vorhandene Spuren. Der FFmpeg-Fallback setzt Metadaten auf den neu angehängten Untertitelindex.

MP4/MOV unterstützt in diesem Einfügepfad nur Textuntertitel aus SRT oder ASS/SSA. Video und Audio werden kopiert, der neue Untertitel intern als mov_text gespeichert; bisherige interne Untertitelspuren werden in diesem Pfad nicht mit übernommen. Der Titel wird zusätzlich als MP4-Spurname beziehungsweise handler_name gesetzt. Für SUP/PGS ist hier MKV erforderlich.

8.6 Zusätzliche Untertiteldateien und OCR

Stand 9.9.1: Untertitel behalten eindeutige Streamzuordnung, Sprache und Flags. Die MP4-Policy unterscheidet interne Textspuren von externen Bilduntertiteln; MOV_TEXT wird bei MKV-Zielen konvertiert. Ein fehlgeschlagener PGS-/VobSub-OCR-Lauf entfernt keine originale Bilduntertitelspur. Pflicht-Sidecars müssen vollständig erzeugt sein, bevor ein Auftrag als erfolgreich veröffentlicht wird. Lange Hilfsprozesse verwenden denselben Abbruch- und Timeoutvertrag wie der Job.

Untertitel extern exportiert ausgewählte Spuren zusätzlich in ihrem ursprünglichen Format. Text zu SRT erzeugt eine weitere Textdatei, während PGS zu SRT eine optische Texterkennung mit FFmpeg und Tesseract benötigt. Die normale Sprach- und Spurauswahl bleibt maßgeblich; der Schalter exportiert nicht pauschal alle vorhandenen Spuren.

Die PGS-SRT-Erzeugung im Konverter ist Best-Effort: OCR-Fehler erzeugen Warnungen und die Original-PGS-Spur bleibt gemäß ihrer Ablageregel erhalten. Dieser Weg ist von der Mediathek-Fix-Queue zu unterscheiden, die zunächst eine .srt.pending-Datei und einen Review-Bericht erstellt und erst nach manueller Bestätigung eine SRT schreibt. OCR-Text, Zeitmarken und Sonderzeichen sollten in beiden Fällen kontrolliert werden.

9 Preflight, Zielordner und Verschieben

9.1 Zielpfade nach Codec und Medientyp

In den Einstellungen können TV-, Anime- und Film-Ziele pro Codec getrennt werden. Dadurch kann H.265 in andere Ordner laufen als AV1 oder H.264. Leere Felder dienen als Fallback und verhindern, dass jede Kategorie zwingend gepflegt werden muss.

9.2 Zielordner-Preflight

Vor dem Start fasst Dragon Tools Serien gruppiert zusammen. Für jede Gruppe wird Typ und Serienname bestätigt. Danach gelten diese Entscheidungen für die passenden Folgen automatisch. Bestehende Serienordner und Specials-Ordner werden bevorzugt erkannt.

Medientyp	Zielentscheidung
Anime	Serienname, Staffel und ggf. vorhandener Anime-Serienordner bestimmen den Zielpfad.
TV	Analog zu Anime, aber über TV-Zielpfade.
Film	Filmziel und optional relative Unterordner werden aus Titel/Jahr und Zielregeln bestimmt.
Move-only	Nutzt denselben Preflight, führt aber keine Konvertierung aus.

Bei Serien bleibt der erkannte Serienname manuell korrigierbar. Schreibweisen wie ReZERO / Re:ZERO werden für Ordnernamen normalisiert, damit ein nicht erlaubtes oder ungewolltes Zeichen keinen vorhandenen Ordner verdeckt. Der Preflight darf Vorschläge liefern, aber keine Zielentscheidung unsichtbar erzwingen.

V9.4 ergänzt die Preflight-Logik um eine zurückhaltende Online-Metadatensuche. Zuerst wird weiterhin geprüft, ob auf der Zielplatte bereits ein passender Serien- oder Filmordner existiert. Nur wenn daraus kein belastbares Jahr ableitbar ist, fragt Dragon Tools den konfigurierten Provider ab: Filme bleiben bei TMDB, Serien können TMDB oder TheTVDB nutzen. Die vorgeschlagene Jahreszahl bleibt im Preflight immer manuell änderbar.

9.2.1 Bestätigte Renamer-Zuordnungen

Nach erfolgreicher Serienumbenennung kann die Zielordner-Vorabprüfung die bestätigte Metadatenquelle, Serien-ID und das Serienjahr aus dem Renamer übernehmen. Die Zuordnung gilt innerhalb derselben Programmsitzung und nur für unverändert passende Einträge. Sie erspart eine erneute Online-Suche.

Bestehende Serienordner behalten Vorrang. Fehlende, widersprüchliche oder manuell umbenannte Zuordnungen verwenden die normale Suche. Auf Netzwerkpfaden werden nur passende Namen auf ihren Ordnertyp geprüft; für unpassende Seriennamen entfallen diese Dateizugriffe.

9.3 Konflikte beim Verschieben

Stand 9.9.1: Move prüft Zielkonflikte, Sidecars, Journale und Pfadidentität an der Transaktionsgrenze. Der im Preflight bestätigte Zielpfad bleibt maßgeblich. Datenträgerübergreifende Transfers werden vollständig gestaged und verifiziert, bevor die Quelle entfernt wird. Abbruch, fehlgeschlagener Rollback und ausstehendes Cleanup bleiben im Journal sichtbar; Recovery darf keinen unvollständigen Transfer als abgeschlossen behandeln.

Modus	Verhalten
Überspringen	Zieldatei bleibt erhalten; neue Datei wird nicht verschoben, Warnung im Log.
Zieldatei zuerst löschen	Vorhandene Datei wird gelöscht, danach wird die neue verschoben.
Direkt ersetzen	Neue Datei ersetzt die vorhandene atomar.
Umbenennen	Neue Datei bekommt Suffixe wie _01, _02, damit nichts überschrieben wird.

Das Konfliktverhalten wird zentral ausgewertet. Für Videodateien zählt gleicher Stammname unabhängig von der Endung als Konflikt: Eine neue MKV erkennt also auch eine vorhandene MP4, AVI oder andere Videodatei mit demselben Namen.

- Skip behält vorhandene Ziele bei und verschiebt die neue Datei nicht darüber.
- Overwrite ersetzt vorhandene Zielvideos, wobei auch endungsübergreifende Konflikte berücksichtigt werden.
- Rename erzeugt eindeutige Namen mit Zählsuffix, wenn der Zielname bereits belegt ist.
- Sidecars wie NFO und Trickplay nutzen eigene Konfliktregeln, damit ein Videokonflikt nicht unbeabsichtigt Cover, NFO oder Trickplay löscht.

9.3.1 Kopieren und Integritätsnachweise

Beim Verschieben auf ein anderes Laufwerk oder ein NAS wird die vollständige Kopie mit der Quelle verglichen. Dieser Lesevergleich liefert zugleich die Inhaltsnachweise für das Verschiebejournal. Dadurch entfallen zusätzliche vollständige Lesevorgänge der Quelle und der Kopie.

Vor dem Löschen der Quelle werden Quelle und installiertes Ziel nach dem Journal-Schreibvorgang erneut frisch geprüft. Nachträglich veränderte Dateien erlauben keine Löschung der Quelle. Abbruch, Rückabwicklung und Wiederherstellung bleiben abgesichert. Anschließend wird die Mediathek-Datenbank aktualisiert.

9.4 Verschiebebericht

Der normale Log fasst am Ende die Move-Ergebnisse nach Zielordnern zusammen. Pro Preflight-Ziel ist sichtbar, wie viele Dateien dorthin gingen und ob eine bestehende Datei gelöscht oder ersetzt werden musste.

9.5 SxxExx-Ersetzung beim Verschieben

Vor dieser Erweiterung konnte eine Datei mit gleichem Inhalt, aber anderem Titel oder anderer Dateiendung neben der alten Folge liegen bleiben. V9.5 betrachtet deshalb bei Serienfolgen die erkannte Identität aus Serienname, Staffel und Episode. Stimmen diese Werte im Zielordner überein, gilt die alte Datei als fachlicher Konflikt, auch wenn sich Episodentitel, Release-Gruppe oder Container unterscheiden.

17. MoveThread bestimmt den finalen Zielordner über die Move-Regeln und den Preflight-Entscheid.
18. move_conflicts extrahiert aus Quelle und Zielkandidaten eine EpisodeIdentity.
19. Bei gleicher Serien-/Staffel-/Folgenkennung werden vorhandene Zielvideos als Konflikt markiert.
20. Multi-Episoden werden als vollständiges Episoden-Tupel verglichen. `S01E03E04` ist deshalb nicht identisch mit `S01E03`.
21. Die vorhandene Zieldatei wird im Rahmen des Verschiebevorgangs ersetzt; der Log nennt alte Datei, neue Datei, erkannte Folge, Uhrzeit und Grund.
22. record_moved_file schreibt die neue Datei aktiv in die SQLite-Mediathek und setzt alte bzw. ersetzte Pfade auf inaktiv.

Diese Logik ist absichtlich im Verschiebepfad verankert und nicht nur in der Suche. Dadurch bleibt das Verhalten auch dann konsistent, wenn der Dateititel unvollständig, anders übersetzt oder technisch bereinigt wurde.

9.6 Persistente Ersetzungs-Erinnerungen

Automatische Ersetzung ist praktisch, soll aber nicht unsichtbar bleiben. Wenn Dragon Tools eine vorhandene Folge wegen gleicher SxxExx-Identität ersetzt und Titel oder Container abweichen, wird zusätzlich eine Erinnerung gespeichert. Diese Erinnerung ist unabhängig vom Abschlussbericht und liegt als JSON-Datei unter `Documents\DragonTools\Erinnerungen\episode_replacements.json`.

Feld	Inhalt
id	Stabile Erinnerung im Format #YYYYMMDD-HHMMSS-001.
datetime	Zeitpunkt der erkannten Ersetzung.
series / season / episode	Fachliche Serienkennung, soweit erkannt.
old_path / new_path	Vorherige Datei und neue Datei.
reason	Warum die Erinnerung angelegt wurde.

Ohne Shutdown zeigt Dragon Tools diese Erinnerungen vor dem Abschlussbericht in einem auffälligen Dialog. Bei aktivem Herunterfahren wird kein blockierender Dialog geöffnet; die Erinnerung erscheint beim nächsten Programmstart. Damit bleibt der automatische Shutdown zuverlässig, ohne dass die Ersetzung verloren geht.

- `Bestätigen` löscht die angezeigten Erinnerungen. Die tatsächlich entfernten Erinnerungs-IDs werden danach im normalen Log als bestätigte Löschung protokolliert.
- `Erneut vorlegen` behält sie für später.
- Schließen über das Fenster-X verhält sich wie `Erneut vorlegen`, damit nichts versehentlich verschwindet.

9.7 Preflight und persistente Mediathek-Pfade

- Die SQLite-Mediathek speichert Pfad-Mappings persistent. Beim Preflight werden gespeicherte Mappings, aktuelle Mappings und aktuelle Speicherpfade gemeinsam betrachtet.

- Zeigt ein alter Datenbankpfad auf einen nicht mehr gültigen lokalen Zielpfad, versucht Dragon Tools eine eindeutige Umlegung auf den aktuellen Speicherpfad. Ist das nicht möglich oder nicht erreichbar, erscheint die Warnung bereits im Preflight statt erst im Move-Schritt.
- Dadurch bleibt die Datenbank als schnelle Bestandsquelle nutzbar, ohne veraltete lokale Pfade still als verschiebbares Ziel zu behandeln.

10 Zusatzwerkzeuge

Werkzeug	Was es macht	Wichtig bei Fehlern
ISO	Analysiert ISO/IMG, Disc-Root sowie direkt gewählte BDMV- oder VIDEO_TS-Ordner mit MakeMKV; FFmpeg-Fallback bleibt als Notfallmethode erhalten.	makemkvcon muss gefunden werden und MakeMKV muss gültig lizenziert oder per Test-/Beta-Lizenz aktiviert sein. Der FFmpeg-Fallback kann bei komplexen Blu-rays Spuren/Playlist-Struktur schlechter erkennen.
Merge	Prüft mehrere Dateien und führt sie ohne Reencode zusammen, wenn Streamsignaturen passen.	Bei abweichender Framerate, Codecstruktur oder Audio-/Subtitle-Signatur wird nicht blind gemergt.
MP4-Remux	Remuxt kompatibles Video/Audio nach MP4.	DV/HDR10+ werden abgelehnt, damit Metadaten nicht verloren gehen.
Audio Muxer	Fügt externe Audiospuren in passende Videodateien ein.	Audioentscheidungen und Fortschritt werden geloggt; Abbruch läuft über Worker-Schnittstelle.
Untertitel	Extrahiert, injiziert, konvertiert und setzt Flags.	MKVToolNix für MKV, FFmpeg für MP4/MOV; SUP/PGS in MP4 ist eingeschränkt.
DV-Remux	Dolby-Vision-Dateien ohne Video-Reencode nach MP4 oder MKV remuxen; der Zielcontainer folgt der Dolby-Vision-Einstellung.	Video bleibt unverändert; Audio wird kompatibel gemacht; Untertitel werden exportiert.
Medieninfo	Zeigt Analyse einer Datei aus dem Kontextmenü.	Nützlich für DV/HDR, Audio, Untertitel, Frames und Framerate.

10.1 Externe Tools und Fähigkeiten

Stand 9.9.1: ISO-Import, Merge und normaler MP4-Remux verwenden geprüfte Toolresultate und getrennte Staging-Ausgaben. Der normale MP4-Copy-Pfad lehnt dynamisches HDR ab, wenn dessen Erhalt nicht nachgewiesen werden kann. Genau eine geplante Videospur wird übernommen. Audiozeitversatz bleibt erhalten; Abbruch vor der Installation schützt das Original und kontrolliert zugehörige Sidecars. Der MP4-Remux verwendet die bereits vorliegende Medienanalyse und vermeidet zusätzliche vollständige Eingangsprüfungen vor dem Start. Die abschließende Ausgabeprüfung bleibt erhalten.

MakeMKV-Lizenzhinweis: Für ISO- und Disc-Verarbeitung muss MakeMKV separat freigeschaltet sein. Dragon Tools erkennt ISO/IMG, Disc-Root sowie direkt gewählte BDMV- oder VIDEO_TS-Ordner; für MakeMKV wird bei BDMV/VIDEO_TS automatisch der übergeordnete Disc-Root verwendet. Wenn MakeMKV nicht nutzbar oder nicht lizenziert ist, bleibt der FFmpeg-Fallback als Vorbeck-/Notfallmethode erhalten, ist aber bei komplexen Blu-ray-Strukturen deutlich fehleranfälliger als MakeMKV.

Tool	Genutzt für	F9 prüft
ffmpeg / ffprobe	Encoding, Remux, Analyse, Timestamp-Reparatur, Untertitelkonvertierung, Qualitätstests, Trickplay und FFmpeg-Fallback bei Disc-Quellen.	Version, Encoder, Filter, libplacebo/zscale, Burn-In-Fähigkeit, NVENC Lookahead-Level und Multipass.
MKVToolNix	mkvmerge, mkvextract, mkvinfo, mkvpropedit für MKV, Remux und Untertitel.	Pfade, Version und verfügbare Komponenten.
MakeMKV	ISO/Disc-Ausgabe.	makemkvcon64.exe oder makemkvcon.exe; Lizenzstatus/Testaktivierung kann erst beim Extraktionslauf relevant werden.
MediaInfo	Erweiterte Analyse, HDR/DV/HDR10+, Frames/Framerate.	MediaInfo.exe und JSON-Ausgabe.
dovi_tool	DV RPU extrahieren, konvertieren, editieren und injizieren.	Version und Grundfähigkeit.
hdr10plus_tool	HDR10+-Metadaten extrahieren/injizieren.	Version und JSON/Bitstream-Fähigkeit.
MP4Box	Finales DV-MP4-Muxing.	GPAC/MP4Box-Version.
HandBrake / RMTS	Externe Hilfsprogramme aus Menü/Ansicht.	Ob ausführbare Dateien gefunden werden.
Dragon HDR10+ Generator	PQ/BT.2020 frameweise analysieren und ST-2094-40-Profile-A-JSON erzeugen.	Eigene Version/CLI-Vertrag; separate EXE, im DragonTools-Projekt enthalten.

System prüfen / Werkzeuge (F9) liest nicht nur Pfade, sondern auch Versionen und erkannte Fähigkeiten aus. Erweiterter Systemtest: Er erzeugt Mini-Testdateien und prüft ffmpeg, ffprobe, MediaInfo, mkvmerge, MP4Box sowie die CLI-Reaktion von dovi_tool und hdr10plus_tool praktisch.

Diagnosepaket: Über Werkzeuge kann ein ZIP mit Manifest, maskierten Einstellungen, Tooldiagnose, erweitertem Systemtest, ausgewählten Logs, Backups und Job-Journalen erstellt werden.

Werkzeugpfade lassen sich direkt im noch geöffneten Einstellungsfenster prüfen. „Eingaben prüfen“ verwendet die aktuell sichtbaren Feldwerte, ohne sie vorher in QSettings oder den globalen Toolpfad-Zustand zu schreiben; das Fenster muss dafür nicht geschlossen werden.

F9 prüft zusätzlich mkvpropedit und Tesseract sowie faster-whisper/CTranslate2 als Runtime-Komponente. faster-whisper ist kein externes whisper.exe: Für vollständig lokale Nutzung kann stattdessen ein CTranslate2-Modellordner mit mindestens model.bin und config.json hinterlegt werden. Ist ein lokaler Ordner konfiguriert, ist er verbindlich; ein ungültiger Pfad fällt nicht still auf einen Download zurück.

10.2 Jellyfin-NFO und Trickplay

Stand 9.9.1: Vorbereitete NFO-Dateien werden erst nach erfolgreicher Konvertierung und Verifikation committed. Nachbearbeitung, Trickplay und Jellyfin-Refresh bleiben an den tatsächlichen Ergebnis- und Zielpfad gebunden. Ein fehlgeschlagener Pflichtschritt darf weder einen zweiten asynchronen Auftrag auslösen noch einen zuvor fehlgeschlagenen Medienjob nachträglich als Erfolg melden.

Dragon Tools kann nach einer erfolgreichen Konvertierung optionale Jellyfin-Sidecars erzeugen. Diese Nacharbeit läuft seit V9.3 im Hintergrund, damit der nächste Encode nicht auf JPEG-Sprites oder Online-Metadaten warten muss.

- NFO: Für Filme wird je nach Einstellung movie.nfo oder eine dateibezogene NFO geschrieben; Serienfolgen erhalten eine Folgen-NFO neben der Ausgabedatei. Die Daten stammen aus der konfigurierten Online-Metadatenquelle. Bei Filmen ist das TMDB, bei Serien je nach Einstellung TMDB oder TheTVDB; IDs werden passend als tmdbid/imdbid/tvdbid beziehungsweise uniqueid-Felder ausgegeben.
- Trickplay: Der Generator erstellt einen Ordner Filmname.trickplay und darin Jellyfin-kompatible Sprite-Unterordner im Format Breite - SpaltenxZeilen. Die Erzeugung läuft transaktional über eine Partial-Struktur; erst nach erfolgreichem Abschluss wird der neue Bestand committed. Vorhandene Trickplay-Daten können je nach Konfliktregel beibehalten, gesichert oder ersetzt werden.
- Quellmodus: Trickplay kann aus der finalen Ausgabedatei oder aus dem Originalmaterial erzeugt werden. Die finale Datei ist am saubersten für exakt passende Ausgabeinformationen; das Originalmaterial kann sinnvoll sein, wenn Trickplay möglichst früh parallel zum Encoding entstehen soll.
- Konflikte: NFO und Trickplay besitzen eigene Konfliktregeln. Vorhandene Sidecars können beibehalten, überschrieben oder vorher gesichert werden. Beim späteren Verschieben werden diese Sidecars getrennt im Verschiebebericht ausgewiesen.
- Fehlerfall und Async-Abschluss: Wenn NFO oder Trickplay fachlich scheitern, bleibt die eigentliche Konvertierung erfolgreich und der Postprocessing-Bericht weist den Teilfehler separat aus. Ein bereits an den Executor übergebener Async-Job gilt eindeutig als gestartet und darf nach einem Loggingfehler nicht synchron ein zweites Mal anlaufen. Native PyQt-Signale werden an der zentralen Logging-Grenze über .emit(...) angesprochen; dadurch ist der frühere Fehler "TypeError: native Qt signal is not callable" im Trickplay-/Postprocessing-Pfad behoben.

10.3 Qualitätstester

Die Qualitätswerkzeuge verwenden einen gemeinsamen Qt-freien Prozess- und Metrik-Layer. Vergleich und Testlauf bleiben getrennte Worker-Fassaden; Prozessausführung, Kennzahlen und Ergebnisaufbereitung sind unabhängig testbar.

Der Qualitätstester ist ein eigener Arbeitsbereich für kurze, reproduzierbare Encodervergleiche. Er verändert keine normale Converter-Queue, sondern erzeugt kleine Testausgaben in einem separaten QualityTests-Ordner.

Zeitbereiche können automatisch über die Laufzeit verteilt oder manuell angegeben werden. Unterstützt werden Prozentpositionen und Zeitstempel, zum Beispiel 10%+20, 00:12:30+30 oder 01:05:00+20.

Neue Testläufe werden über einen Assistenten angelegt: zuerst wird H.264, H.265 oder AV1 gewählt, danach CPU, NVENC, QSV oder AMF und anschließend die dazu passenden Detailoptionen. Vorhandene Testläufe können per

Doppelklick wieder geöffnet werden. Das freie Feld für zusätzliche FFmpeg-Argumente bleibt erhalten, damit Spezialparameter weiter möglich sind.

Nach jedem Testsegment liest Dragon Tools die Ausgabedatei mit ffprobe aus und protokolliert Größe, Bitrate, Codec, Profil, Pixelformat und Laufzeit. Zusätzlich werden SSIM und VMAF gemessen, sofern der vorhandene FFmpeg-Build die jeweiligen Filter unterstützt.

Bewertung: SSIM/VMAF sind technische Hilfen, ersetzen aber keine Sichtprüfung. Gute Kandidaten sind Einstellungen, die bei ähnlicher Sichtqualität deutlich kleinere Dateien erzeugen oder bei gleicher Größe sichtbar weniger Artefakte zeigen.

Fehlerfall: Wenn SSIM oder VMAF nicht verfügbar sind, scheitert der Testlauf nicht. Dragon Tools markiert die Messung als nicht verfügbar und lässt Größe, Bitrate und ffprobe-Daten trotzdem auswertbar.

V9.4 nutzt dafür bevorzugt OpenCV und NumPy: Frames werden auf Struktur, Kanten, Helligkeit und robuste Signaturen reduziert. Wenn OpenCV fehlt, arbeitet der interne Fallback weiter, aber weniger fein. Der Worker protokolliert die verwendete Bildanalyse, erkannte gemeinsame Bereiche, Offsets, Warnungen und Schnittentscheidungen.

Fall A beschreibt einen konstanten Offset über den gesamten gemeinsamen Hauptinhalt. Zusätzliches Schwarzbild oder Outro nur vor dem ersten bzw. nach dem letzten gemeinsamen Bereich ist kein Fall C. Fall C liegt erst vor, wenn innerhalb des gemeinsamen Inhalts ein abweichender Bereich sitzt und danach wieder gemeinsamer Inhalt folgt, sodass sich der Offset neu bildet.

Der Audio-Video-Matcher ist ein eigener Worker-Tab für Fälle, in denen eine deutsche Tonspur an ein anderes Zielvideo angepasst werden soll. Das Zielvideo definiert die finale Videolänge; längere deutsche Tonbereiche am Ende werden auf die Zielvideolänge zugeschnitten, fehlende deutsche Entsprechungen im Zielvideo werden als Warnung gemeldet.

Regel-/Profil-Simulator: Für Queue- und Einzeldateiprüfungen zeigt Dragon Tools jetzt neben Pipeline, Profil, HDR/DV, Audio, Untertiteln und Ziel auch die berechnete Endauflösung. Die Vorschau verwendet dieselbe Downscale-only-Logik wie der echte Encode-Pfad. Wenn Auto-Crop aktiv ist und der konkrete Crop erst während der Verarbeitung ermittelt werden kann, wird die Auflösung ausdrücklich als „vor Auto-Crop“ gekennzeichnet, statt eine noch unbekannte endgültige Crop-Auflösung vorzutäuschen. Der angezeigte Ausgabecontainer wird aus denselben Einstellungen wie die reale Pipeline gelesen: Standard/HDR10+ und Dolby Vision besitzen getrennte Containerwerte. Deshalb zeigen Regeltester, Batch-Preflight und der DragonTools-Reiter der Medieninfo z. B. DV → MKV oder Standard → MP4 korrekt an, wenn diese Werte gewählt wurden.

10.4 Film-/Serien-Umbenennung und Online-Metadaten

Stand 9.9.1: Ein optional manuell hinterlegtes TheTVDB-Bearer-Token wird zunächst verwendet. Fehlt es oder wird es als nicht autorisiert abgelehnt, fordert DragonTools mit gültigem API-Key und optionalem Subscriber-PIN ein neues Token an, speichert es über die Secret-/DPAPI-Verwaltung und wiederholt den fehlgeschlagenen Request genau einmal. Parallele Clients teilen einen Refresh-Lock und können ein bereits erneuertes Token übernehmen. Speicherfehler werden protokolliert; ein gültiges neues Token bleibt für die Sitzung nutzbar. Transiente Providerfehler werden nicht als dauerhafter Kein-Treffer-Cache gespeichert; Provider-IDs und Titelidentität werden vor automatischer Übernahme geprüft.

Der Film-/Serien-Renamer ist ein eigenständiger Arbeitsbereich. Er bereinigt Release-Namen, unterscheidet Film- und Serienmuster, sucht Vorschläge über die getrennt konfigurierten Online-Metadatenprovider und lässt die endgültige Entscheidung nachvollziehbar beim Benutzer. Die Ergebnistabelle zeigt neben Vorschlag und Sicherheit jetzt auch den tatsächlich gewählten Provider (TMDB oder TheTVDB). Eine falsch erkannte Datei kann gezielt als Serie oder als Film neu gesucht werden; der Suchbegriff lässt sich manuell bearbeiten und bei Bedarf können alle Provider-Rohkandidaten ohne Fuzzy-Untergrenze angezeigt werden. Enthält der lokale Metadatencache für eine Serienfolge nur einen Platzhalter wie Folge 10, startet der Renamer eine frische Providerabfrage und nutzt einen inzwischen vorhandenen echten Episodentitel.

Serienfolgen werden über bereinigten Serientitel und flexible Staffel-/Episodenmuster aufgelöst. Neben SxxExx und 1x05 werden auch umgedrehte Release-Schreibweisen wie E05S06 korrekt als Staffel 6, Episode 5 erkannt. EP01/EPISODE01 ohne Staffelangabe wird als Serie erkannt, aber die Staffel wird niemals geraten: Vor der Metadatensuche verlangt Dragon Tools eine explizite Staffelwahl; mehrere EPxx-Dateien derselben Serie im selben Ordner können gemeinsam zugeordnet werden. Die automatische Bewertung arbeitet in konfigurierbaren Stufen (standardmäßig 60 % -> 45 % -> 30 %). Ein generischer Episodentitel aus dem persistenten Cache gilt nicht als endgültige Metadatenquelle: Werte wie "Folge 10", "Episode 10" oder reine SxxExx-Platzhalter können eine frische Providerabfrage erzwingen. Bei TheTVDB kann eine vorhandene Episode ohne brauchbaren Titel zusätzlich über die konfigurierte Fallback-Sprache ergänzt werden.

Metadaten-Browser: Der Renamer kann Serien oder Filme auch bei vollständig leerer Dateiliste direkt bei TMDB und TheTVDB suchen. Die Treffer bleiben provider-explizit mit Titel, Jahr und Provider-ID sichtbar. Nach Auswahl einer Serie werden Staffeln einschließlich S00/Specials und die Episoden geladen; lokale Videodateien oder komplette Ordner können anschließend per Drag & Drop hinzugefügt werden.

- Serien-Mapping: Eine Datei kann 1 bis 4 aufeinanderfolgende Episoden enthalten. Die Zieldatei verwendet die vorhandene Schreibweise S01E03E04 bzw. S01E03E04E05E06. Für Mehrfachfolgen kann Dragon Tools alle Episodentitel, nur den ersten oder keinen Episodentitel verwenden. Automatische Zuordnung verteilt noch freie Dateien ab einer markierten Episode; vorhandene Filmzuordnungen werden dabei nicht still überschrieben.
- Film-Mapping: Ein Film-Treffer wird mit „Film auswählen“ ausdrücklich aktiviert und anschließend genau einer lokalen Datei zugeordnet. Ein später nur markierter anderer Suchtreffer ändert die aktive Zuordnung nicht unbemerkt.
- Async-Sicherheit: Während TMDB/TheTVDB-Abfragen laufen, sind Suchtyp, Treffer-/Staffelauswahl, Zuordnung und Import gesperrt. Beim Serien- oder Staffelwechsel werden alte Episodenzeilen vor dem Request sofort entfernt. Verspätete Suchergebnisse werden zusätzlich nur angenommen, wenn sie zum gestarteten Medientyp passen. Nicht fortlaufende Provider-Episoden erzeugen eine sichtbare Warnung statt einer unbehandelten GUI-Exception.
- „Zuordnung in Renamer übernehmen“ benennt noch keine Datei um. Der Browser erzeugt normale MovieRenameProposal-/SeriesRenameProposal-Datensätze; danach greifen weiterhin Zielkonfliktprüfung, editierbarer Zielname, manuelle Annahme und der bestehende Renamer-Commitpfad.

10.4.1 Dateinamen im Hintergrund ändern

Das Ausführen der Umbenennung läuft im Hintergrund und zeigt den Fortschritt. Der Renamer kontrolliert die Dateiendung und prüft, ob der gewünschte Name frei ist. Er liest den Videoinhalt für die Namensänderung nicht vollständig ein. Begleitdateien werden mit umbenannt; vorhandene Zieldateien werden nicht überschrieben.

10.4.2 Staffel und Serie prüfen

Die Schaltflächen Staffel prüfen und Serie prüfen stehen neben Metadaten-Browser. Zur Auswahl erscheinen erkannte Serien und Staffeln mit einer bestätigten Provider-Zuordnung. Die gewählte Quelle wird aktuell abgefragt; die Folgenzählung stammt aus der Renamer-Liste.

Die Ergebnistabelle unterscheidet vollständige Staffeln, fehlende Staffeln und fehlende einzelne Folgen. Specials erscheinen als Staffel 0. Mehrfachfolgen zählen einzeln, doppelte Dateien nicht mehrfach. Die Tabelle ist als CSV exportierbar. Fehlende Zugangsdaten, Abfragefehler und widersprüchliche Daten werden als nicht prüfbar angezeigt.

10.5 Quellbildprüfung

Stand 9.9.1: Die Quellbildprüfung läuft im Hintergrund und lässt sich bei Tool-Aufrufen abbrechen. Qualitätssuche und Auswertung prüfen tatsächlich erzeugte Dateien und die maßgebliche Ausgabegeometrie; temporäre Ergebnisse gehören zum jeweiligen Auftrag. Der Matcher unterscheidet Offset, Drift und Schnittbereiche. Zusätzliche Zielbereiche ohne passende deutsche Audioentsprechung werden nicht unsicher automatisch zugeordnet.

Die Quellbildprüfung sucht vor dem Encoding nach klar defektem Ausgangsmaterial. Sie bewertet leere Bilder, einfarbige Flächen, extremes Pixelrauschen und andere unplausible Zustände an prozentual verteilten Prüfpositionen. Mehrere Positionen werden gebündelt: bei 10-%-Intervallen sinkt der Normalfall von 9 auf 3 FFmpeg-Starts, bei 5 % von 19 auf 5. Schlägt eine Gruppe technisch fehl, wird nur diese Gruppe über den bewährten Einzelprobe-Pfad erneut geprüft; die eigentliche Bildheuristik bleibt unverändert.

Die Prüfung ist ein Schutz vor offensichtlich kaputten Quellen, keine Qualitätsbewertung. Auffällige Dateien werden blockiert und können per Rechtsklick bewusst freigegeben oder entfernt werden. Wenn bis zum Ende eines Laufs keine Entscheidung getroffen wird, behandelt Dragon Tools sie wie übersprungene bzw. abgebrochene Dateien.

10.6 Audio-Video-Matcher

Das Audio-/Video-Time-Mapping ist in Modell-Fit und Randbereichsbewertung getrennt. Der Qt-Thread koordiniert Analyse, Rendering und Ergebnisübergabe; mathematische Zuordnung und Validierung liegen in eigenständigen Services.

Der Audio-Video-Matcher ist ein eigener Worker-Tab für Fälle, in denen eine deutsche Tonspur an ein anderes Zielvideo angepasst werden soll. Das Zielvideo definiert die finale Videolänge; längere deutsche Tonbereiche am Ende werden auf die Zielvideolänge zugeschnitten, fehlende deutsche Entsprechungen im Zielvideo werden als Warnung gemeldet.

Fall A beschreibt einen konstanten Offset über den gesamten gemeinsamen Hauptinhalt. Zusätzliches Schwarzbild oder Outro nur vor dem ersten bzw. nach dem letzten gemeinsamen Bereich ist kein Fall C. Fall C liegt erst vor, wenn innerhalb des gemeinsamen Inhalts ein abweichender Bereich sitzt und danach wieder gemeinsamer Inhalt folgt, sodass sich der Offset neu bildet.

V9.4 nutzt dafür bevorzugt OpenCV und NumPy: Frames werden auf Struktur, Kanten, Helligkeit und robuste Signaturen reduziert. Wenn OpenCV fehlt, arbeitet der interne Fallback weiter, aber weniger fein. Der Worker protokolliert die verwendete Bildanalyse, erkannte gemeinsame Bereiche, Offsets, Warnungen und Schnittentscheidungen.

10.7 Entscheidungsmodell des Serien-Renamers

Der Renamer trennt Erkennung, Provider-Suche, Kandidatenbewertung und Annahme. Ein Suchfallback darf den ursprünglichen Quelltitel nicht überschreiben. Die Provider liefern Kandidaten einmalig; anschließend prüft Dragon Tools die konfigurierten Mindestgrenzen in Reihenfolge. Nur bei generischen Cache-Titeln wie Folge XX wird eine gezielte frische Episode-Abfrage ausgelöst. Dadurch werden zusätzliche API-Anfragen für 60/45/30-%-Stufen vermieden und dennoch lange offizielle Titel oder gekürzte Release-Namen gefunden.

```
Dateiname
  ↓
Release-Tags entfernen
  ↓
Originalen Titel + Staffel/Episode erkennen (SxxExx, ExxSxx, x; EPxx -> Staffel abfragen)
  ↓
TMDB/TheTVDB einmal abfragen
  ↓
Kandidaten gegen Originaltitel bewerten
  ↓
Stufe 1: normale Mindestgrenze (Standard 60 %)
  ↓ falls leer
Stufe 2: Fallback 1 (Standard 45 %)
  ↓ falls leer
Stufe 3: Fallback 2 (Standard 30 %)
  ↓
Provider + echte Sicherheit sichtbar anzeigen
  ↓
manuell prüfen / sicher akzeptieren / Zielname erzeugen
```

Datenebene	Bedeutung	Darf den Score definieren?

Datenebene	Bedeutung	Darf den Score definieren?
Erkannter Titel	Titel aus der lokalen Quelldatei nach Release-Bereinigung	Ja – Referenz für die Textähnlichkeit
Suchbegriff	Originaltitel, Alias oder Fuzzy-/Prefix-Variante	Nein – dient nur zur Kandidatensuche
Online-Treffer	Titel/Jahr/Provider des tatsächlich gefundenen Kandidaten	Wird gegen den erkannten Titel verglichen
SxxExx	Erkannte Staffel/Episode	Validiert die Existenz; kein positiver 100-%-Bonus
Match-Sicherheit	Titelähnlichkeit bzw. explizite Aliasregel	Bestimmt Anzeige/Prüfung/Auto-Akzeptanz

10.8 Renamer-Verhalten und Funktionsumfang

Stand 9.9.1: Namensparser, Staffelkorrektur, Episodenmapping und Vorschlagsanzeige verwenden denselben Datensatz. Ungültige Episodenwerte und mehrdeutige Zuordnungen werden nicht still ausgeführt. Benutzerauswahl und manuelle Korrekturen bleiben erhalten; die endgültige Dateiumbenennung erfolgt erst nach bestätigtem Vorschlag.

Nr.	Änderung	Technische Wirkung
1	Release-Normalisierung	Technische Tags werden aus dem Serientitel entfernt; Watson-Beispiel wird korrekt als S01E01 erkannt.
2	Technische Tags	DED, DL, EAC3, DD51, 18p, AZHD, AVC/HEVC, x264/x265 und weitere Scene-Tags ergänzt.
3	Breites Dropdown	Popup wird dynamisch an den längsten Text angepasst; Tabellenbreite bleibt kompakt.
4	TMDB + TheTVDB	Beide Quellen bleiben sichtbar; Präferenz verdrängt den zweiten Provider nicht.
5	Limit je Provider	Bis zu 6 je Provider; bei zwei Providern maximal 12.
6	Mehrere Serienkandidaten	Mehrere Providerergebnisse werden geprüft; SxxExx muss tatsächlich existieren.
7	Score-first Sortierung	Match-Qualität vor Providerpräferenz; Provider nur Tie-Breaker.
8	Serienjahr	Providerjahr wird im Kandidaten mitgeführt und bei passenden Bewertungswegen berücksichtigt.

Nr.	Änderung	Technische Wirkung
9	Inkrementelle Autosuche	Neue Zeilen werden automatisch aufgelöst; vorhandene Ergebnisse bleiben erhalten.
10	Preflight-Normalisierung	Bindestriche, Slash-/Backslash- und Apostrophvarianten sowie Endzeichen werden toleranter behandelt.
11	Renamer-Regeln	Eigener Menüpunkt und Tab im globalen Regelfenster.
12	Editierbare Dateizeichen	Früher hart codierte Zielnamensregeln liegen in renamer_rules.
13	Aliasregeln	Benutzer kann Quelle → Online-Suchname definieren, ohne den Quelltitel zu ersetzen.
14	Fuzzy-Suche	Optionale Prefix-/Fallback-Suchvarianten und Retry ohne Jahr.
15	Treffergrenzen	Standard: Anzeige ab 60 %, manuelle Prüfung unter 72 %, sicher ab 78 %.
16	Fuzzy-Score korrigiert	Bewertung bleibt gegen den erkannten Originaltitel; Suchvariante erzeugt kein falsches 100 %.
17	SxxExx ohne Bonus	Episode validiert Plausibilität, erhöht ähnliche Titel aber nicht auf 100 %.
18	Alias-Markierung	Explizite Regel wird als „Aliasregel“ kenntlich gemacht.
19	Manuelle Auswahl	Dropdown-Auswahl verwendet ebenfalls die konfigurierbare Prüfgrenze.

10.9 Release-Normalisierung und technische Tags

`parse_series_release_name()` trennt Serieninformationen und technische Segmente. Zusätzlich zur heuristischen Gruppenposition besitzt das Renamer-Regelschema eine explizite Releasegruppen-Liste. Exakt konfigurierte Gruppen wie STARS oder NIMA4K werden nur am Anfang oder Ende des Release-Namens entfernt; ein gleichlautendes Wort innerhalb eines echten Titels bleibt unangetastet. Dadurch werden Gruppenpräfixe wie STARS.Show.S01E01 nicht als Titelbestandteil interpretiert, ohne legitime Titel zu beschädigen.

Tag/Gruppe	Interpretation bzw. Bereinigung

Tag/Gruppe	Interpretation bzw. Bereinigung
DED / Dubbed	deutscher Dub
DL	Dual Language
EAC3	E-AC3
DD51	Dolby Digital 5.1
18p	ältere 1080p-Release-Kurzform
AZHD	AmazonHD
AVC / x264	H.264
HEVC / x265	H.265/HEVC
WEB-DL / BluRay / AMZN	Source-/Distributionsangaben
HDR / HDR10+ / DoVi	Dynamikumfang-/Dolby-Vision-Tags

```
tvs-watson-eac3-51-ded-dl-18p-azhd-avc-s01e01
→ Serie: Watson
→ Staffel: 1
→ Episode: 1
```

10.10 Renamer-Regeln, Dateizeichen und Aliasregeln

`renamer_rules.py` ist das zentrale Regelschema (Schema 3). Die GUI `rules_renamer_tab.py` editiert dieselben Daten, die der Renamer zur Laufzeit abfragt. Neben Zeichenersetzungen, Aliasregeln und den Review-/Auto-Accept-Schwellen enthält das Schema jetzt eine eigene Liste konfigurierbarer Releasegruppen sowie die normale Mindestgrenze und zwei getrennt einstellbare automatische Fallback-Grenzen. Standard ist 0,60 / 0,45 / 0,30.

Zeichen	Standardersetzung
*	X
"	'
'	`
/	-
\	-
#	leer
?	leer
<	leer
>	leer
:	leer
|	.

Matching-Regel	Standard	Wirkung
minimum_candidate_score	0,60	darunter Kandidat nicht anzeigen
manual_review_below	0,72	darunter als manuell zu prüfen behandeln

Zeichen	Standardersetzung	
auto_accept_from	0,78	ab hier für „sicher akzeptieren“ geeignet
fuzzy_fallback	aktiv	zusätzliche Suchvarianten zulassen
retry_without_year	aktiv	bei Bedarf ohne Jahr erneut suchen
fuzzy_prefix_min_words	3	untere Wortgrenze für Prefix-Fallbacks
fallback_candidate_scores[0]	0,45	automatischer zweiter Versuch, wenn 60 % keine Treffer liefert
fallback_candidate_scores[1]	0,30	letzter automatischer Fallback; Treffer bleiben prüfbedürftig

Aliasregeln werden durch `apply_title_exception()` auf einen normalisierten Titelvergleich angewendet. Beispiel: Der erkannte Titel „Daemons of the Shadow Realm“ kann gezielt nach „Das Band der Unterwelt“ suchen. Die Regel ändert den Suchbegriff, nicht den lokalen Originaltitel.

- Die Releasegruppen-Liste wird durch `strip_configured_release_groups()` ausgewertet. Trennzeichen und Klammern am Namensrand werden toleriert; Duplikate werden beim Speichern/Migrieren case-insensitiv entfernt. Beispiele: `STARS.Show.S01E01`, `[STARS] Show S01E01` und `Show.S01E01-STARS` werden bereinigt, `A STARS Story S01E01` dagegen nicht.
- EPxx-Sonderfall: Fehlt die Staffel vollständig, setzt der Parser `season_missing=True`. Die GUI zeigt „Staffel fehlt“ und fragt mit `QInputDialog` nach der Staffel, bevor eine Providerabfrage startet. Die gewählte Staffel gilt gruppiert für dieselbe erkannte Serie im selben Ordner; 0 ist für Specials erlaubt.

10.11 Providerkandidaten, Jahr und Fuzzy-Bewertung

`CompositeMetadataClient.resolve_episode_candidates(limit=6)` interpretiert das Limit je Provider. Bei zwei aktiven Providern können bis zu zwölf Kandidaten in die Renamer-Pipeline gelangen. Die Providerclients liefern mehrere Serienkandidaten; für jeden wird geprüft, ob die konkrete Staffel/Episode existiert. Die interne Provider-Vorauswahl ist bewusst locker genug, damit die konfigurierte niedrigste Fuzzy-Stufe (standardmäßig 30 %) Kandidaten nicht schon vor der eigentlichen Renamer-Bewertung verliert. Wenn ein Kandidat nur einen Cache-Platzhalter wie Folge XX enthält, kann der Client denselben Kandidaten frisch vom Provider laden und den Cache aktualisieren.

- `_sort_candidates()` sortiert zuerst absteigend nach Score. Die konfigurierte Provider-Reihenfolge wird nur als Tie-Breaker verwendet.
- `_series_score()` berechnet die angezeigte Seriensicherheit aus echter Titelähnlichkeit. Richtige Staffel/Episode liefern keinen positiven Bonus; echte S/E-Abweichungen können weiterhin einen Sicherheitsabzug verursachen.
- Fuzzy- und Prefix-Suchbegriffe dienen nur dazu, Kandidaten zu finden. Der gefundene Titel wird anschließend gegen den ursprünglich erkannten Serientitel bewertet.
- Ein expliziter Alias kann bewusst als Regelvertrauen gekennzeichnet werden; damit ist klar, dass 100 % aus der Benutzerzuordnung und nicht aus Textähnlichkeit stammen.
- Das Providerjahr bleibt am Kandidaten erhalten. Bei titel-/jahresbasierten Bewertungswegen kann ein passendes Jahr gleichnamige Serien besser trennen.
- Automatische Mindestgrenzen werden in Reihenfolge geprüft; die erste Stufe mit Treffern gewinnt. Standard: 60 %, 45 %, 30 %.
- Treffer unterhalb der normalen Mindestgrenze werden als Fallback angezeigt und nicht von „Sichere akzeptieren“ als normaler Treffer behandelt.

- Bei der niedrigsten Stufe verhindert eine Mehrdeutigkeitsprüfung, dass zwei ähnlich schwache Treffer automatisch als eindeutig gelten.

10.12 Renamer-Oberfläche und inkrementeller Resolver

`movie_renamer_widget.py` startet nach dem Einfügen neuer Dateien verzögert eine Auflösung nur für Zeilen, die noch keinen Vorschlag besitzen und im bereit-Zustand sind. Der Resolver unterstützt zusätzlich manuelle Film- und Seriensuche unabhängig vom zuvor erkannten Typ, einen editierbaren Suchbegriff sowie „Alle Treffer anzeigen“. Die Provider-Herkunft wird in einer eigenen Tabellenspalte geführt.

- Der spezielle Kandidaten-ComboBox-Typ misst den längsten sichtbaren Eintrag beim Öffnen.
- Das Popup darf breiter als die Tabellenzelle werden, wird aber auf die verfügbare Bildschirmbreite begrenzt.
- Beim manuellen Wechsel eines Kandidaten werden Zielname, Score, Provider-/Alias-Hinweis und Reviewstatus sofort neu gesetzt.
- Die manuelle Statusentscheidung verwendet `manual_review_below()` aus denselben Renamer-Regeln wie der automatische Ablauf.
- Die 13 Renamer-Aktionen sind auf drei logische Reihen verteilt, damit ihre addierten Button-Mindestbreiten nicht mehr die Mindestbreite des gesamten Hauptfensters erzwingen.
- Alle Renamer-Spalten verwenden interaktive Breiten. Über „Spalten“ können sie einzeln ein-/ausgeblendet werden; Breite, Reihenfolge und Sichtbarkeit werden per QSettings persistent gespeichert.
- Die Standardansicht blendet OK, Typ und Hinweise aus und priorisiert Quelldatei, Vorschlag/Match und Provider. „Standardansicht“ und „Alle Spalten einblenden“ stellen definierte Ansichten wieder her.
- EPxx-Zeilen bleiben bis zur Staffelwahl im Status „Staffel fehlt“. Eine abgebrochene Nachfrage startet keine automatische Titelerkennung mit erfundener Staffel; die Zeile kann später erneut als Serie gesucht und zugeordnet werden.

10.13 Trickplay: Hardware-Decoding mit Kompatibilitätsstrategie

- Der Trickplay-Pfad unterscheidet jetzt explizite Strategien. Bei CUDA wird zunächst GPU-Decoding/NVDEC mit kompatiblen CPU-Filtern für fps, scale, tile und JPEG versucht.
- Schlägt dieser Weg fehl, startet ein CUDA-Kompatibilitätsretry mit explizitem hwdownload/format=nv12 vor den CPU-Filtern. Erst danach folgt der CPU-Fallback.
- Für QSV/DXVA/D3D11VA wird analog ein Hardware-Decoding-Pfad und danach ein CPU-Fallback verwendet. Das Log benennt den tatsächlich gestarteten und erfolgreichen Pfad, statt pauschal nur „Hardwarepfad fehlgeschlagen“ zu melden.

10.14 Jellyfin-API: Pfadvalidierung und gezielte Metadatenanalyse

Nach erfolgreichen Moves oder Renames kann Dragon Tools die betroffenen Jellyfin-Pfade gezielt melden. Vor der Meldung liest der API-Client /Library/PhysicalPaths aus und validiert jeden Zielpfad gegen die von Jellyfin tatsächlich gesehenen Mediathekswurzeln. Ein lokaler Windows-/UNC-Pfad darf dadurch nicht mehr als scheinbar erfolgreiche Pfadmeldung durchgehen.

Die Refresh-Logik kombiniert die aktuellen lokalen Zielpfade aus den Einstellungen mit den externen Jellyfin-Pfaden aus den path_mappings der eigenen Mediathek-Datenbank. Bei neuen Dateien wird zusätzlich der enthaltene Film-/Staffelordner einmalig als Modified gemeldet; Jellyfin kann den bekannten Parent aktualisieren und die neue Datei anschließend über seinen normalen Discovery-/Metadatenpfad aufnehmen. Der Verbindungstest zeigt die vom Server gemeldeten Roots.

Ein optionaler Full-Scan-Fallback bleibt erhalten. Jellyfins Linux-inotify-Limit ist davon unabhängig eine Host-/NAS-Konfiguration und wird von Dragon Tools nicht verändert.

10.15 Audio-Spracherkennung und NFO-/Track-Synchronisierung

Die Mediathek-Fix-Queue kann unbekannte interne Audio-Sprachen mit faster-whisper aus mehreren kurzen Samples bestimmen. Sichere Treffer werden nicht nur in der Datenbank vorgemerkt: Bei MKV setzt mkvpropedit den Legacy-Sprachtag und language-ietf auf einer isolierten Kopie. Eine vorhandene NFO mit fileinfo/streamdetails wird auf derselben Audio-/Untertitel-Typordinalposition mit dem ISO-639-2-Tag synchronisiert.

MKV- und NFO-Kandidaten werden vor dem Commit gemeinsam vorbereitet. Ist eine vorhandene NFO beschädigt oder die Spur nicht eindeutig zuordenbar, wird der Fix fail-closed verworfen, damit Datei und NFO nicht auseinanderlaufen. Nach erfolgreichem Commit analysiert Dragon Tools das Medium erneut und übernimmt den tatsächlich geschriebenen Streamtag in die SQLite-Mediathek. MP4 wird weiterhin nicht riskant in-place geändert.

Interne Streams speichern für diese Reparaturen den echten Container-/ffprobe-Index statt eines typenlokalen Indexes. Dadurch adressieren FFmpeg, MKVToolNix und Fix Queue dieselbe reale Spur.

11 Validierung, Reparatur und Fehlerberichte

11.1 Output-Validierung

Stand 9.9.1: Scheitert ein Pflichtschritt nach erfolgreicher Videoverifikation, bleibt der brauchbare Kandidat erhalten und der Auftrag im Fehlerstatus. Das gilt auch beim späten Abbruch. Gesperrte Sidecar-Rollbacks behalten Journal, Staging und Backups und lassen sich nach Freigabe wiederholen. Vorbereitete NFO-Dateien behalten einen Besitzer bis zur sicheren Bereinigung. Beim Verschieben wird der Identitätsnachweis vor dem Hardlink gespeichert; ohne passenden gespeicherten Nachweis bleiben Quelle und Ziel bei Recovery erhalten.

Stand 9.9.1: Der finale Soll-/Ist-Vergleich prüft Video, dynamische HDR-Metadaten, Audio, Untertitel, Dauer und geplanten Container. Reparaturkandidaten werden getrennt erzeugt und erneut verifiziert. Unsichere Timestamp- oder Frame-Ergebnisse werden nicht als Erfolg installiert. Ein Pflichtfehler nach dem Encode schützt verwertbare Kandidaten vor generischem Cleanup und sperrt Auto-Move sowie den Erfolgsstatus.

Nach jeder erzeugten Datei prüft OutputVerifier die allgemeinen Ausgabeverträge mit ffprobe: Existenz, Mindestgröße, Container-Endung, lesbare Formatdaten, mindestens ein Videostream, Audio wenn die Quelle Audio hatte, und eine plausible Laufzeit. Für den DV-Pfad kommt davor eine spezialisierte Finalverifikation der fertig gemuxten MP4 hinzu: MediaInfo ist die primäre Beweisquelle für Dolby Vision und – bei aktiviertem Kombipfad – HDR10+. Nur bei fehlendem oder unklarem MediaInfo-Nachweis wird selektiv der strengere Bitstream-Fallback ausgeführt.

Prüfung	Regel
Datei existiert	Ausgabepfad muss vorhanden sein.
Mindestgröße	Datei muss mindestens 1024 Byte groß sein.
Container	Endung muss zum geplanten Container passen.
Video	Mindestens ein Videostream muss vorhanden sein.
Audio	Wenn Quelle Audio hatte, muss Ausgabe mindestens eine Audiospur haben.
Dauer	Muss > 0 sein und im Toleranzfenster zur Quelldauer liegen: nicht unter 90 %, nicht über max(Quelle + 60 s, Quelle * 1,25).

Die Output-Validierung ist von der Quellbildprüfung getrennt. Quelleffekte wie schwarze Bilder, einfarbige Flächen oder starkes Pixelrauschen werden vor dem Encode bewertet; fertige Ausgaben werden danach über Streams, Größe und

Laufzeit geprüft. Positive MediaInfo-Ergebnisse für DV/HDR10+ vermeiden unnötige Post-Mux-Toolläufe. Negative oder unklare Ergebnisse werden nicht blind akzeptiert, sondern über dovi_tool bzw. hdr10plus_tool am aus der finalen MP4 extrahierten HEVC-Bitstream gegengeprüft.

11.1.1 Video- und Audiodauer getrennt prüfen

Die Videodauer wird eigenständig mit der Quelle verglichen. Eine ausgewählte Audiospur darf länger als das Video sein, wenn ihre Ausgabedauer zur entsprechenden Quellspur passt und alle geplanten Spuren vorhanden sind. Ein nachvollziehbarer Audio-Nachlauf löst auch bei Strip-Only keine unnötige Timestamp-Reparatur aus. Verkürzte oder weit entfernte Videotimestamps bleiben Fehler.

Bei Matroska wird die tatsächliche Schreibanwendung aus dem Container berücksichtigt. FFmpeg-Laufzeit-Tags und MKVToolNix-Statistik-Tags können Startversätze unterschiedlich darstellen. Ein kopierter ENCODER-Tag reicht nicht aus, um die aktuelle Schreibanwendung zu bestimmen.

11.2 Automatische Laufzeitreparatur

Die Timestamp-Reparatur trennt Kandidatenerzeugung, Validierung und Archivierung. Erst ein vollständig validierter Kandidat darf das Original ersetzen; nicht prüfbare Ergebnisse bleiben fail-closed.

23. Nur wenn die Validierung wegen unplausibler Laufzeit scheitert und die Ausgabe eine MKV ist, startet die Reparatur.
24. Zuerst wird genau ein normaler MKVToolNix-Remux mit mkvmerge versucht. Streams werden nicht neu kodiert.
25. Wenn der Remux die Dauer korrigiert, gilt die Konvertierung als erfolgreich.
26. Wenn der Remux nicht hilft, analysiert Dragon Tools Container-, Video-, Audio-, Untertitel-, Kapitel-, Frame- und Framerate-Daten.
27. Für ein eindeutig erkanntes CFR-Muster prüft DragonTools zunächst eine MKVToolNix-Reparatur mit --default-duration anhand der verlässlichen Bildrate und Dauer der Quelle. Falls diese keinen gültigen Kandidaten liefert, stehen verlustfreie FFmpeg-setts- und +genpts/+igndts-Reparaturwege bereit. Eine nur vermutete Framerate oder widersprüchliche Framezahl wird nicht erzwungen.
28. Jeder Reparaturkandidat wird vollständig geprüft. Streamverlust, Widerspruch, unplausible Laufzeit oder Vertragsverletzung verhindern Ersetzen, Postprocessing und Verschieben. Verworfene Kandidaten werden zunächst unter Archiv\Timestamp_Reparatur gesichert. Nach erfolgreicher Übernahme einer gültigen Ausgabe werden die unveränderten Reparaturkopien desselben Auftrags bereinigt.

Sicherheitsgrenze: Es gibt keine zweite FFmpeg-Konvertierung. Die Reparatur arbeitet nur mit Remux bzw. Timestamp-Neutaktung ohne Video-Neuencoding.

29. Bei echter VFR wird keine blinde CFR-Reparatur erzwungen. Ist die Originaldatei noch vorhanden, kann Dragon Tools die echten PTS jedes Videoframes auslesen und als MKVToolNix-Timestamp-v2-Zeitbasis auf den Reparaturkandidaten übertragen.
30. Dieser Original-Timeline-Fallback ist nur zulässig, wenn Original und Ausgabe exakt dieselbe Videoframeanzahl besitzen und die Original-Timeline selbst plausibel ist. Bei Audio werden Startzeit und Audio-/Videodauer zusätzlich vor und nach der Reparatur geprüft. Jede Unsicherheit führt weiterhin zum sicheren Abbruch statt zu einer geratenen Zeitbasis.

11.3 Größenregeln

Die Speichereinstellungen können größere Ausgaben begrenzen oder zu kleine Ausgaben blockieren. Bei einem Verstoß ersetzt Dragon Tools das Original nicht. Die erzeugte Ausgabe wird in Archiv verschoben und der Batch-Abschluss markiert die Datei nicht als OK.

11.4 Fehlerberichte

Für fehlgeschlagene Konvertierungen schreibt Dragon Tools einen separaten Textbericht im Log-Monatsordner unter ErrorReports. Der Bericht enthält Workflow, Quelle, Pfade, Encoderprofil, Per-Datei-Override, Validierung, Tool-Ausgabe und Traceback. Bei Laufzeitreparaturen enthält er zusätzlich Dauer nach FFmpeg, nach Remux, nach Timestamp-Fix, Timing-Prüfung und den FFmpeg-Befehl.

Abschnitt im Fehlerbericht	Wofür nutzen
Workflow	Erkennen, ob standard, dv, hdrplus, strip-only oder ein Sonderpfad betroffen war.
Quelle	Prüfen, welche Streams, HDR-Daten und Analysewarnungen vorlagen.
Ausgabe / Pfade	Sehen, wo temporäre, finale oder archivierte Dateien lagen.
Encoder / Profil	Nachvollziehen, welche echten Qualitäts- und Encoderwerte aktiv waren.
Output-Validierung	Sofort erkennen, ob Dauer, Audio, Video, Container oder Größe schuld waren.
Letzte Tool-Ausgabe	Die letzten externen Fehlermeldungen ohne kompletten Logballast sehen.

11.5 Cleanup und Rollback

CleanupService entfernt bei fehlgeschlagenen Läufen nur temporäre Outputs, Burn-In-Dateien und Sidecars, deren Ownership für den aktuellen Job explizit registriert ist. Ein pauschales Löschen über output_stem.* im Benutzer-/Zielverzeichnis findet nicht mehr statt; dadurch überleben bereits vor dem Lauf vorhandene gleichnamige SRT/ASS/SUP/SUB/IDX/VTT-Dateien. Der zentrale Output-Commit verwendet PathSwapTransaction und ReplaceJournal: Bei Same-Path-Overwrite wird das Original gesichert und bei Commitfehlern zurückgerollt; bei Containerwechsel wird die Quellbereinigung journalisiert. Converter, DV-Remux, MP4-Remux und Audio-Mux verwenden denselben Commitmechanismus. Fehlerhafte, aber absichtlich aufbewahrte Ausgaben bleiben wegen keep_failed_output erhalten.

11.5.1 Reparaturarchiv nach erfolgreichem Abschluss

Verworfene Timestamp-Reparaturversuche bleiben zunächst zur Diagnose erhalten. Sobald derselbe Auftrag eine gültige Ausgabe erfolgreich verifiziert und übernommen hat, entfernt DragonTools seine unveränderten Reparaturkopien.

Bei einem fehlgeschlagenen oder abgebrochenen Auftrag bleiben sie erhalten. Ältere Archivdateien und zwischenzeitlich geänderte Dateien werden nicht automatisch gelöscht. Die Bereinigung verwendet die registrierten Dateien dieses Auftrags und sucht nicht pauschal nach gleichnamigen Archivinhalten.

12 Logging, Backup, Build und Release

12.1 Normaler Log

Stand 9.9.1: Timeout und Sofortabbruch beenden den zum Auftrag gehörenden Prozessbaum. Leere Logpfade, Symlinks und Junctions werden vor destruktiven Hilfsoperationen abgewiesen. Parallele Logs und Berichte erhalten getrennte Dateinamen; Diagnosepakete maskieren Zugangsdaten und prüfen die Herkunft der eingelesenen Dateien. Ein nicht verfügbares Tool oder ein fehlgeschlagener Prozessstart wird als Fehler gemeldet.

Seit V9.7 ist der sichtbare Standardlog ein kompakter Kurzlog für Menschen. Pro Datei stehen dort vor allem Quelle (Name, Auflösung, Codec/HDR-Typ), verwendete Audio- und Untertitelspuren einschließlich Forced-Burn-In, gewählte Pipeline und Zielauflösung/Container sowie Ergebnis, Dauer und Größenersparnis. Warnungen, Fehler und gezielte INFO-Entscheidungen wie die AV1-Priorität von Dolby Vision vor HDR10+ bleiben sichtbar. Der bisherige vollständige technische Log bleibt unverändert erhalten, läuft jedoch im Hintergrund und wird als *_lang.txt gespeichert. FFmpeg-/Tool-Kommandos, Detailausgaben, temporäre Pfade und interne Entscheidungen bleiben damit für Diagnose vollständig verfügbar, ohne das sichtbare Logfenster zu überladen.

12.2 Abschlussbericht

Der Abschlussbericht zeigt OK, Fehler, Übersprungen, Größenbilanz, Archivzähler, Sidecars und Fehlerberichte. Fehlerdateien können direkt wieder in die Queue gelegt werden. Bei aktivem Herunterfahren öffnet der Bericht nicht blockierend.

12.3 Backup und Restore

Herunterfahren läuft über eine gekapselte Systemfunktion. Unter Windows wird bevorzugt System32\shutdown.exe genutzt und ein Fehler mit stdout/stderr im Log dokumentiert, damit ein fehlgeschlagener Shutdown nicht wie ein stiller Absturz wirkt.

Über das Einstellungsmenü können Einstellungen, Toolpfade, UI-Zustand, Regeln und eigene Encoderprofile als ZIP exportiert werden. Beim Restore wird vorher ein Sicherheitsbackup erzeugt, und ZIP-Mitglieder werden geprüft, damit kein fremder Pfad geschrieben wird.

12.4 Build

Stand 9.9.1: Version 9.9.1 ist zentral definiert. Öffentliche Quellarchive prüfen private Pfade und Secret-Literale in Text, Python, DOCX und PDF; ihre Archivinstallation schützt vorhandene Dateien. Build- und CI-Anforderungen verwenden denselben Laufzeitvertrag. Ein Windows-Smoke prüft echte Qt-Widgets, JPEG und OpenCV. Die Source-Abnahme ist keine vollständige Hardware- oder Tool-Bundle-Abnahme; dokumentierte optionale Komponenten benötigen gesonderte Nachweise.

Die Release-Prüfung ist nach Source-, Build- und App-Vertrag getrennt. Dadurch lassen sich fehlende Quelldateien, fehlerhafte Buildartefakte und unvollständige App-Pakete getrennt diagnostizieren.

Seit V9.7 enthält das Projekt den kanonischen PyInstaller-CLI-Builder `build_v9.bat`. Die konkrete Buildbezeichnung wird nicht mehr hart codiert, sondern direkt aus `dragontools/core/version.py` gelesen. `build_v9_angepasst.bat` bleibt nur als Kompatibilitäts-Wrapper erhalten. Das Source-Archiv kann große Third-Party-Binaries und Bildressourcen bewusst auslassen; der Builder prüft alle lokal benötigten Artefakte vor PyInstaller fail-fast.

Build-Bestandteil	Ziel im Bundle
DragonToolsV9.py	Startdatei der EXE.
dragontools	Python-Paket mit GUI, Core, Worker, Rules und Subtitle.
help.html	Hilfe aus dem Hilfe-Menü.
Handbuch\Handbuch.pdf	PDF-Handbuch aus der gepflegten DOCX-Grundlage; wird über Hilfe/F2 geöffnet.
Aenderungshistorie	V9-, Legacy-V8- und Legacy-V7-Changelogs.
third_party	FFmpeg, MKVToolNix, MakeMKV, GPAC/MP4Box, MediaInfo, dovi_tool, hdr10plus_tool, HandBrake, RMTS.

Für die frühe EXE-Startphase nutzt der Build einen separaten PyInstaller-Boot-Splash aus Bilder\splash_pyinstaller.png. Sobald der Qt-Splash aktiv ist, wird der PyInstaller-Splash geschlossen. Runtime-, optionale Bildanalyse-, Test- und Build-Abhängigkeiten sind getrennt in requirements-runtime.txt, requirements-optional.txt, requirements-test.txt und requirements-build.txt dokumentiert.

- Release-/Build-Prüfung im Quellmodus wertet `release_manifest.json` aus, prüft die zentrale App-Version, die PyInstaller-CLI-Strategie, Build-Skript, Requirements, pytest-Marker, CI-/Integrationstestvertrag, Regelschemata, Python-Paket, Bytecode-Freiheit und Datenschutzmarker. Das Manifest kennzeichnet explizit, wenn große externe Build-Artefakte nicht Bestandteil des Source-ZIPs sind.
- Release-/Build-Prüfung in der fertigen App prüft nur, was im Bundle relevant ist: DragonToolsV9.9.1.exe, Datenordner, Handbuch, Hilfe, Changelogs, Regelschemata, Python-Paket und Programme/Tools.
- Eine versionierte .spec-Datei ist für den aktuellen V9-Build nicht erforderlich, weil PyInstaller vollständig über die CLI konfiguriert wird. Der Builder bereinigt vor dem Source-Preflight Bytecode-/Testcaches und validiert nach PyInstaller zusätzlich das erzeugte App-Bundle mit derselben zentralen Release-Prüfung.

- Der Builder prüft faster-whisper/CTranslate2 als paketierte Python-Runtime. Für Offline-Betrieb kann ein lokaler CTranslate2-Modellordner verwendet werden; ohne lokalen Ordner bleibt der normale Modellcache/-Downloadpfad erhalten. Auf dem Benutzer-PC wird faster-whisper nicht per pip nachinstalliert.

12.5 Backup-Restore und Legacy-Secrets

- Legacy-V1-Backups können noch Klartext-Secrets enthalten. Die GUI weist darauf hin und fragt, ob alte Zugangsdaten übernommen oder lokale Secrets behalten werden.
- Der Restore arbeitet gestaged: Einstellungen und Dateien werden vorbereitet, validiert und erst danach ersetzt.
- Bei einem Fehler während des Restores wird der vorherige Zustand wiederhergestellt, damit keine halben Einstellungen oder Regeldateien zurückbleiben.
- Für verschlüsselte Backups inklusive API-Keys/Tokens wird das Python-Paket cryptography benötigt. Das Build-Skript prüft diese Abhängigkeit vor dem PyInstaller-Lauf.

13 Fehlerfälle und Nachschlagehilfe

Problem	Prüfen	Wahrscheinliche Lösung
Tool wird nicht gefunden	F9-Werkzeugprüfung, Einstellungen/Werkzeugpfade, Bundle-Aufbau Daten\Programme.	Pfad manuell setzen oder Build-Add-Data/Add-Binary prüfen.
AVI/alter Container bricht ab	MediaInfo/ffprobe-Analyse, Containerwarnung, ffmpeg-Lesbarkeit.	Erst remuxen oder Quelle reparieren; alte Container können fehlerhafte Zeitstempel haben.
Ausgabe hat falsche Laufzeit	Output-Validierung, ErrorReport Timing-Prüfung.	V9 versucht automatisch MKV-Remux und bei sicherem Muster Timestamp-Reparatur.
Ausgabe größer als Quelle	Speichereinstellungen und Größenregel-Log.	Original wird nicht ersetzt; Ausgabe liegt in Archiv. CQ/CRF oder Audio-Regel anpassen.
Audio fehlt	Audio-Regeln, Fallback, Quelle hatte Audio, ErrorReport Output-Validierung.	Sprachliste/Fallback prüfen; bei Quelle mit Audio darf Ausgabe nicht ohne Audio durchgehen.
Falsche Audiosprache	Audio-Sprachpriorität und max. Sprachen/Tracks.	Prioritätsliste neu sortieren oder Per-Datei-Override verwenden.
Forced-Untertitel fehlt	Untertitelregeln, Burn-In-Sprache, ask_if_ambiguous, Datei-Override.	Ambige Kandidaten manuell wählen oder Sprach-/Forced-Flags prüfen.

Problem	Prüfen	Wahrscheinliche Lösung
DV/HDR10+ fehlt	Quelle erkannt?, Preserve-Haken, F9 dovi_tool/hdr10plus_tool, Pipeline im Log.	Toolpfade und Erhalte-Optionen prüfen; bei Kombiquelle DV+HDR10+ Log beachten.
Farben falsch	HDR-Farbtags, DV Profil, Standardpfad-Meldung.	DV5 braucht eigene Konvertierung; HDR10-Basis muss BT.2020/PQ behalten.
Shutdown läuft nicht	Herunterfahren-Haken vor Start, Abschlussstatus, erneut eingereihte Fehlerdateien.	Shutdown wird bei Retry/Abbruch bewusst verhindert; Haken wird nicht mehr beim Encoderladen zurückgesetzt.
Queue lässt sich nicht ändern	Ob Datei aktuell läuft oder Move-only aktiv ist.	Nur wartende Dateien sind während Encoding sortierbar; MoveThread sperrt bewusst.

13.1 Welche Datei ist für welchen Fehler zuständig?

Fehlerart	Erste Dateien zum Prüfen
Toolpfade	dragontools/core/paths.py, dragontools/core/tool_diagnostics.py, settings_dialog.py
Pipeline-Auswahl	dragontools/rules/pipeline_selector.py, worker/pipeline_decision_service.py
Encoderargumente	dragontools/worker/encoder_args.py, encode_plan_service.py, standard_pipeline_runner.py
Audio	dragontools/rules/audio_rules.py, audio_plan.py, worker/converter_stream_args.py
Untertitel	dragontools/rules/subtitle_rules.py, subtitle_sidecar_service.py, subtitle/*.py
Dauer/Reparatur	output_verifier.py, duration_repair_service.py, duration_timing_analyzer.py
Ersetzen/Archiv	replace_service.py, output_size_policy.py, archive_service.py, cleanup_service.py
Preflight/Move	move_rules.py, preflight_dialog.py, move_preflight_controller.py, move_thread.py
Abschlussdialog	conversion_result_service.py, run_summary.py, run_summary_dialog.py

14 Glossar

Begriff	Bedeutung
Burn-In	Untertitel wird direkt ins Videobild gerendert und ist danach nicht mehr abschaltbar.
CRF/CQ/QP	Qualitätsziel des Encoders; kleinere Werte bedeuten größere Dateien und bessere Qualität.
DV RPU	Dolby-Vision-Metadatenteil, der Helligkeits- und Mappinginformationen enthält.
HDR10+	Dynamische HDR-Metadaten, die separat aus HEVC/AV1 extrahiert und wieder injiziert werden können.
Main10	HEVC-Profil für 10-bit-Video. In V9 wird zusätzlich das echte Pixel-Format gesetzt.
Passthrough	Stream wird ohne Neuencoding kopiert.
Remux	Streams werden in einen neuen Container gepackt, ohne sie neu zu kodieren.
Sidecar	Begleitdatei neben der Videodatei, z. B. externe Untertitel.
Strip-only	Copy-Modus ohne Reencode; Regeln können Streams entfernen/übernehmen, aber kein Burn-In erzeugen.
Timestamp-Reparatur	Verlustfreie Neutaktung eines defekten Videostreams, wenn die Frame-/Framerate-Dauer sicher rekonstruierbar ist.

14.1 DRC

Dynamic Range Control bei AC3/E-AC3; reduziert nutzbare Dynamik nach Quellmetadaten und wird in Dragon Tools über -drc_scale gesteuert.

14.2 Loudnorm

FFmpeg-Filter zur allgemeinen Lautheitsnormalisierung auf einen Zielwert in LUFS.

14.3 SSIM / VMAF

Objektive Qualitätsmetriken für Testsegmente. Dragon Tools nutzt sie im Qualitätstester, wenn der vorhandene FFmpeg-Build die Filter bereitstellt.

15 Programm- und Modulreferenz

Dieses Kapitel enthält die ausführliche statische Modulreferenz des historisch dokumentierten Kernbestands. Der historische V9.7-Referenzstand umfasste 579 Python-Dateien. Die bis zum 09.09.2026 ausgelagerten Fachmodule für Prozesse, Move-Batches, Pipeline- und Audioplan-Policy, Online-Metadaten, Sidecars, Transfers, Medienanalyse sowie Soll-/Ist-Prüfung sind thematisch integriert. Maßgeblich für den aktuellen Umfang sind die zusammengefasste Inventur und der Quellbaum; die lange Einzeldateiliste in Kapitel 25 bleibt eine historische Referenzaufnahme. Der eigenständige Dragon-HDR10+-Generator wird als separates Unterprojekt unter dragon_hdr10plus_generator/ gepflegt; seine Source- und Testdateien sind in der aktuellen Projektstatistik enthalten, obwohl die ausführliche historische Einzeldateiliste des Kernpakets weiterhin nur den damaligen dragontools-Bestand abbildet.

> *Lesart: „direkt verwendet von“ bezeichnet statisch erkennbare Importbeziehungen. Dynamische Lazy-Imports, Factory-Aufrufe und indirekte Signalketten können darüber hinaus zusätzliche Laufzeitbeziehungen erzeugen.*

15.1 Einstiegspunkt

1 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.1.1 DragonToolsV9.py

Merkmal	Wert
Kategorie	Einstiegspunkt
Umfang	447 Quellzeilen
Bedeutung	Kritisch: Prozess-Einstiegspunkt.

15.1.1.1 Aufgabe des Moduls

Startet die Anwendung, setzt Bundle-/Toolpfade, zeigt zuerst den Boot- und danach den Qt-Splash und lädt anschließend die GUI. Lazy Loading bedeutet hier spätes Importieren beim Programmstart, nicht Live-Kompilierung oder Nachladen geänderter Python-Dateien in der gebauten EXE.

> *DragonToolsV9.py – Einstiegspunkt Dragon Tools V9 Eigenständig, kein Legacy-Code. Lazy Loading: dragontools/-Module werden erst bei Bedarf importiert.*

15.1.1.2 Erwartete Informationen / Eingaben

- Programmstart, Kommandozeilen-/Frozen-Umgebung, Ressourcenpfade
- Wichtige Parameter im Code: `app`, `text`, `icon`

15.1.1.3 Rückgabe / Ausgabe

- initialisierte Anwendung und MainWindow
- Explizite Rückgabetypen im Code: `None`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/crash_guard.py, dragontools/core/settings.py, dragontools/core/timeout_settings.py, dragontools/gui/main_window.py
Externe Pythonbibliotheken	PyQt6, pyi_splash
Externe Programme/Tools	dovi_tool, ffmpeg, handbrake, hdr10plus_tool, mediainfo
Direkt verwendet von	Kein direkter statischer Importnutzer; möglich sind Einstiegspunkt, Lazy-Import oder reine Paketfunktion.

15.1.1.4 Wichtige Klassen und Methoden

DragonSsplashScreen [object] – Qt-nativer Splash-Screen mit Feuer/Eis-Thema. Rendert über dem Bild: • Dunkel-transparentes Panel mit Farbverläufen

```
set_progress(value) -> None
finish(main_window) -> None
```

15.1.1.5 Wichtige Funktionen

```
main()
```

15.1.1.6 Datenverarbeitung und Kommunikation

Prozessstart → Pfad-/Frozen-Erkennung → Qt-Anwendung/Splash → MainWindow → Eventloop.

15.1.1.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2 Core

76 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.2.1 dragontools/core/__init__.py

Merkmal	Wert
Kategorie	Core
Umfang	2 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (2 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.2.1.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls

15.2.1.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_crash_guard_cleanup.py, dragontools/tests/test_preflight_dialog_performance.py, dragontools/tests/test_system_shutdown.py, dragontools/tests/test_tool_paths.py

15.2.1.4 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.2 dragontools/core/audio_video_matcher.py

Merkmal	Wert
Kategorie	Core
Umfang	1241 Quellzeilen
Bedeutung	Zentraler Baustein (1241 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.2.1 Aufgabe des Moduls

Enthält die fachliche Vergleichslogik des Audio-Video-Matchers. Das Modul liest technische Videodaten, erzeugt Bildsignaturen, vergleicht gemeinsame Inhaltsbereiche und leitet daraus konstante oder wechselnde Offsets ab.

15.2.2.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`, `low`, `high`, `text`, `cmd`, `timeout_s`, `frame`, `width`, `height`, `box`, `gw`, `gh`, `time_s`, `gray`, `grid`

15.2.2.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `bool`, `str`, `float`, `list[CutRegion]`, `bytes`, `tuple[int, int, int, int]`, `list[int]`, `FrameSignature`, `tuple[int, int]`, `FrameSignature | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/process_runner.py
Externe Pythonbibliotheken	cv2, numpy
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/audio_video_matcher_widget.py, dragontools/tests/test_audio_video_matcher.py, dragontools/worker/audio_video_match_thread.py

15.2.2.4 Wichtige Klassen und Methoden

AudioVideoMatcherSettings [object]

VideoInfo [object]

FrameSignature [object]

MatchPoint [object]

CutRegion [object]

```
duration_s() -> float
```

CutMatchResult [object]

TimeMappingResult [object]

```
needs_cut_regions() -> bool
```

AudioSyncSegment [object]

```
target_duration_s() -> float
source_duration_s() -> float
```

AudioSyncPlan [object]

FrameExtractor [object]

```
extract_window_signatures(path, start_s, duration_s) -> list[FrameSignature]
```

15.2.2.5 Wichtige Funktionen

```
opencv_available() -> bool
image_analysis_backend_label() -> str
format_seconds(value) -> str
parse_timecode(text) -> float
parse_cut_regions(text) -> list[CutRegion]
frame_similarity(a, b) -> float
select_landmark_times(duration_s) -> list[float]
classify_time_mapping(points) -> TimeMappingResult
preferred_german_audio_stream(info) -> AudioStream | None
atempo_chain(factor) -> list[str]
```

15.2.2.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.2.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.3 dragontools/core/audit_log.py

Merkmal	Wert
Kategorie	Core
Umfang	115 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (115 Zeilen, direkt von 6 Projektmodulen importiert).

15.2.3.1 Aufgabe des Moduls

Schreibt ein Änderungsprotokoll für relevante Einstellungen.

15.2.3.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `settings`, `action`, `details`, `keys`, `before`, `after`, `section`, `key`, `value`

15.2.3.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `Path | None`, `dict[str, Any]`, `tuple[int, str]`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/config_migration.py, dragontools/gui/encoder_profile_service.py, dragontools/gui/profile_manager_dialog.py, dragontools/gui/rules_dialog_storage.py, dragontools/gui/settings_dialog.py, dragontools/tests/test_audit_log.py

15.2.3.4 Wichtige Funktionen

```
settings_change_log_dir(settings) -> Path
append_audit_event(action, details) -> Path | None
```

Schreibt einen kurzen nachvollziehbaren Eintrag ins Änderungsprotokoll.

```
snapshot_qsettings(settings, keys) -> dict[str, Any]
summarize_changes(before, after) -> tuple[int, str]
log_qsettings_changes(section, before, after) -> Path | None
```

15.2.3.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.3.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.4 dragontools/core/batch_preflight.py

Merkmal	Wert
Kategorie	Core
Umfang	864 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (864 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.4.1 Aufgabe des Moduls

Berechnet Preflight-Informationen ohne Encoding.

15.2.4.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `fallback`, `channels`, `preview`, `entry`, `streams`, `codec`, `base_dir`, `stem`, `tag`, `container`, `path`, `directory`, `fs_info`, `exc`, `path_value`

15.2.4.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `Any`, `Path`, `dict[str, Any]`, `Path | None`, `tuple[bool, str | None]`, `int | None`, `tuple[dict[str, Any], list[str], bool]`, `list[str]`, `tuple[list[str], bool]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/move_conflicts.py, dragontools/core/rules_preview.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/batch_preflight_dialog.py, dragontools/gui/convert_widget.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/tests/test_batch_preflight.py

15.2.4.4 Wichtige Funktionen

`build_batch_preflight_rows(files) -> list[dict[str, Any]]`

Erstellt die Tabellenzeilen für den Batch-Preflight. Der Default-Preview-Builder wird absichtlich lazy importiert, damit dieser reine Kern auch ohne PyQt6 getestet werden kann.

`split_problem_rows(rows) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]`

`default_batch_preflight_report_path(documents_dir) -> Path`

`format_batch_preflight_report(rows) -> str`

15.2.4.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.4.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.5 dragontools/core/codec_profile_assistant.py

Merkmal	Wert
Kategorie	Core
Umfang	362 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (362 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.5.1 Aufgabe des Moduls

Definiert vorbereitete Encoderprofile wie Anime klein, Film ausgewogen oder CPU effizient.

Empfohlene Startprofile für den Codec-Profil-Assistenten.

15.2.5.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `codec`, `key`

15.2.5.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `CodecProfileSuggestion`, `list[CodecProfileSuggestion]`, `dict[str, Any] | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/profile_manager.py, dragontools/gui/encoder_profile_service.py, dragontools/tests/test_codec_profile_assistant.py, dragontools/tests/test_encoder_settings_shutdown.py

15.2.5.4 Wichtige Klassen und Methoden

CodecProfileSuggestion [object]

15.2.5.5 Wichtige Funktionen

```
assistant_profiles_for_codec(codec) -> list[CodecProfileSuggestion]
profile_for_assistant_key(codec, key) -> dict[str, Any] | None
```

15.2.5.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.5.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.6 dragontools/core/config_migration.py

Merkmal	Wert

Merkmal	Wert
Kategorie	Core
Umfang	294 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (294 Zeilen, direkt von 7 Projektmodulen importiert).

15.2.6.1 Aufgabe des Moduls

Migriert ältere Regel- und Profil-Schemata auf den aktuellen Stand.

15.2.6.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `data`, `config_name`, `path`, `from_version`, `to_version`, `messages`, `value`, `default`, `name`, `fallback`, `encoder`, `codec`, `crf`, `key`, `profile`, `raw`

15.2.6.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `int`, `dict[str, Any]`, `None`, `MigrationResult`, `str`, `tuple[dict[str, Any], list[str]]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audit_log.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/profile_manager.py, dragontools/gui/rules_dialog_storage.py, dragontools/rules/audio_rules.py, dragontools/rules/move_rules.py, dragontools/rules/rule_loader.py, dragontools/rules/subtitle_rules.py, dragontools/tests/test_config_migration.py

15.2.6.4 Wichtige Klassen und Methoden

MigrationResult [object]

15.2.6.5 Wichtige Funktionen

```
schema_version(data) -> int
current_schema_version(config_name) -> int
sanitize_config_for_persistence(data) -> dict[str, Any]
```

Entfernt nur flüchtige interne Top-Level-Felder vor dem Speichern.

```
write_json_atomic(path, data) -> None
log_config_migration(config_name, from_version, to_version, messages) -> None
finish_migration(config_name, data) -> MigrationResult
migrate_encoder_profile(key, profile) -> tuple[dict[str, Any], list[str]]
migrate_profile_collection(raw) -> MigrationResult
```

15.2.6.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.6.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.7 dragontools/core/crash_guard.py

Merkmal	Wert
Kategorie	Core
Umfang	310 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (310 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.7.1 Aufgabe des Moduls

Speichert Laufaktivität und bereinigt leere Crashreport-Platzhalter.

15.2.7.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `app_version`, `stage`, `reason`, `exc_type`, `exc_value`, `exc_tb`, `args`, `state`, `path_value`, `command`, `value`, `pid`

15.2.7.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path | None`, `None`, `str`, `Path`, `dict[str, Any]`, `list[str] | str`, `int | None`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	DragonToolsV9.py, dragontools/worker/converter_progress.py, dragontools/worker/converter_thread.py, dragontools/worker/tool_runner.py

15.2.7.4 Wichtige Funktionen

`install_crash_guard(app_version) -> Path | None`

Installiert globale Diagnose-Hooks fuer unerwartete Programmabbrueche.

`mark_activity(stage) -> None`

Speichert den letzten bekannten Arbeitsschritt atomar.

`clear_activity() -> None`

Entfernt den aktiven Marker nach sauberem Abschluss.

`write_manual_crash_note(reason) -> str`

Schreibt einen sofortigen Crash-/Exceptionbericht.

15.2.7.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.7.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, PermissionError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.8 dragontools/core/diagnostic_package.py

Merkmal	Wert
Kategorie	Core
Umfang	193 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (193 Zeilen, direkte Importnutzer: 2).

15.2.8.1 Aufgabe des Moduls

Erstellt Diagnosepakete aus Logs, Einstellungen und relevanten Laufdaten.

15.2.8.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `documents_dir`, `target_path`, `created_at`, `log_root`, `folder`, `limit`, `files`, `zf`, `path`, `arcname`

15.2.8.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `str`, `dict[str, str]`, `list[Path]`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/log_cleanup.py, dragontools/core/logger.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/core/settings_backup.py, dragontools/core/tool_diagnostics.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mkvextract, mkvmerge, mp4box
Direkt verwendet von	dragontools/gui/main_window_actions.py, dragontools/tests/test_diagnostic_package.py

15.2.8.4 Wichtige Funktionen

`default_diagnostic_package_path(documents_dir) -> Path`

`create_diagnostic_package(target_path) -> Path`

Erstellt ein lokales Diagnosepaket für Support und Fehlersuche.

15.2.8.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / ffprobe / handbrake / hdr10plus_tool / makemkvcon / mediainfo / mkvextract / mkvmerge / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.8.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.9 dragontools/core/disk_space.py

Merkmal	Wert
Kategorie	Core
Umfang	147 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (147 Zeilen, direkte Importnutzer: 2).

15.2.9.1 Aufgabe des Moduls

Prüft freien Speicher vor kritischen Dateioperationen.

15.2.9.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `files`, `result`, `path`, `value`

15.2.9.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `DiskSpaceCheckResult`, `str`, `Path`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/tests/test_disk_space.py

15.2.9.4 Wichtige Klassen und Methoden

DiskSpaceIssue [object]

DiskSpaceCheckResult [object]

```
has_critical() -> bool
has_warning() -> bool
```

15.2.9.5 Wichtige Funktionen

```
check_conversion_disk_space(files) -> DiskSpaceCheckResult
```

Schätzt den freien Platz für parallel laufende Konvertierungen. Dragon Tools schreibt Standard-Ausgaben und Overwrite-Tempdateien zunächst neben die Quelle. Deshalb wird pro Zielordner/Volume mit den größten

```
format_disk_space_issues(result) -> str
```

15.2.9.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.9.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.10 dragontools/core/encoder_profile_override.py

Merkmal	Wert
Kategorie	Core
Umfang	123 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (123 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.10.1 Aufgabe des Moduls

Zentrale Normalisierung und Priorisierung wirksamer Encoderwerte für globale Einstellungen, gespeicherte Encoderprofile und manuelle per-Datei-Encoder-/Skalierungs-Overrides. Das Modul hält die Worker-seitige Auswertung Qt-unabhängig.

15.2.10.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: globale Encoderwerte, Zielcodec, gespeichertes Encoderprofil, manueller `encoder_override`, Backend (CPU/NVENC/QSV/AMF), Qualität, Preset, Skalierung und backend-spezifische Optionen.

15.2.10.3 Rückgabe / Ausgabe

- Normalisierte Override-Dictionaries sowie die final wirksamen Encoderwerte für die nachgelagerte Pipeline- und Encode-Planung.
- Explizite Rückgabetypen im Code: `str`, `bool`, `int | None`, `dict[str, Any] | None`, `dict[str, Any]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget_queue_actions.py, dragontools/tests/test_encoder_profile_override.py, dragontools/worker/workflow_services.py

15.2.10.4 Wichtige Funktionen

`profile_to_override(key, profile) -> dict[str, Any] | None`

Erzeugt ein kompaktes per-Datei-Profil für den Override-Speicher.

`normalize_profile_override(profile) -> dict[str, Any] | None`

Validiert ein gespeichertes Profil-Override für die Worker-Nutzung.

`normalize_encoder_override(override, default_codec) -> dict[str, Any] | None`

Validiert und normalisiert den manuellen per-Datei-Encoder-/Skalierungs-Override einschließlich Backend, Qualität, Preset, Skalierung und Optionen.

`effective_encoder_settings() -> dict[str, Any]`

Berechnet die wirksamen Encoderwerte mit der Priorität global < gespeichertes Encoderprofil < manueller Datei-Override.

15.2.10.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.10.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.11 dragontools/core/error_report.py

Merkmal	Wert
Kategorie	Core
Umfang	327 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (327 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.11.1 Aufgabe des Moduls

Schreibt detaillierte Fehlerberichte pro fehlgeschlagener Datei.

15.2.11.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `title`, `lines`, `ctx`, `text`, `stream`, `analysis`, `primary`, `key`, `value`

15.2.11.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `list[str]`, `Any`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_error_report.py, dragontools/worker/converter_thread.py, dragontools/worker/workflow_services.py

15.2.11.4 Wichtige Funktionen

`write_conversion_error_report() -> str`

Schreibt einen Diagnosebericht für eine fehlgeschlagene Datei. Der Bericht ist absichtlich Plain-Text, damit er auch ohne DragonTools lesbar bleibt und für Support/Fehlersuche einfach weitergegeben werden kann.

`build_error_report_preview(path) -> str`

Liest einen kurzen Vorschautext für GUI/Tests, ohne bei Fehlern zu werfen.

`current_traceback_text() -> str`

15.2.11.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.11.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.12 dragontools/core/german_title_variants.py

Merkmal	Wert
Kategorie	Core
Umfang	59 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (59 Zeilen, direkte Importnutzer: 2).

15.2.12.1 Aufgabe des Moduls

Implementiert den Programmbaustein „german title variants“ innerhalb der Schicht Core.

15.2.12.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `replacement`, `matched`

15.2.12.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `tuple[str, ...]`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/movie_renamer.py, dragontools/core/online_metadata.py

15.2.12.4 Wichtige Funktionen

`german_umlaut_search_variants(value) -> tuple[str, ...]`

Erzeugt vorsichtige TMDB-Suchvarianten fuer deutsche ASCII-Umlaute. Release-Namen schreiben deutsche Titel oft als ``Muenchen`` oder ``Kinderfluesterer``. TMDB fuehrt dieselben Titel meist mit echten

`fold_german_umlauts(value) -> str`

Macht echte Umlaute und ASCII-Umschreibungen vergleichbar.

15.2.12.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.12.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.13 dragontools/core/jellyfin_nfo.py

Merkmal	Wert
Kategorie	Core
Umfang	273 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (273 Zeilen, direkte Importnutzer: 2).

15.2.13.1 Aufgabe des Moduls

Schreibt Jellyfin-kompatible Movie- und Episode-NFO-Dateien inklusive TMDB-, IMDb- und TheTVDB-IDs, soweit der Metadatenprovider diese liefert.

15.2.13.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `suggestion`, `video_path`, `ffprobe_path`, `parent`, `stream`, `format_data`, `root`, `include_fileinfo`, `tag`, `value`, `values`, `actors`, `id_type`, `elem`, `level`

15.2.13.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `ET.Element | None`, `None`, `str`, `int | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/online_metadata.py, dragontools/core/process_runner.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_jellyfin_nfo.py, dragontools/worker/postprocess_service.py

15.2.13.4 Wichtige Funktionen

```
write_movie_nfo(path, suggestion) -> Path
write_episode_nfo(path, suggestion) -> Path
build_fileinfo(video_path, ffprobe_path) -> ET.Element | None
```

15.2.13.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.13.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.14 dragontools/core/job_journal.py

Merkmal	Wert
Kategorie	Core

Merkmal	Wert
Umfang	352 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (352 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.14.1 Aufgabe des Moduls

Sichert laufende Jobs, damit nach einem Absturz Restdateien wieder angeboten werden können.

15.2.14.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `root`, `data`, `status`, `paths`, `path`

15.2.14.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `dict[str, Any] | None`, `dict[str, Any]`, `Path | None`, `str`, `list[str]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/job_resume_dialog.py, dragontools/gui/main_window.py, dragontools/tests/test_job_journal.py

15.2.14.4 Wichtige Klassen und Methoden

JobJournal [object]

```
start() -> 'JobJournal'
start_file(input_path) -> None
finish_file(input_path) -> None
finish_run() -> None
```

15.2.14.5 Wichtige Funktionen

```
job_journal_dir(root) -> Path
active_job_journal_path(root) -> Path
read_active_job_journal(root) -> dict[str, Any] | None
build_resume_plan(data) -> dict[str, Any]
```

Ermittelt aus einem aktiven Journal die Dateien für eine Wiederaufnahme.

```
archive_active_job_journal() -> Path | None
```

Archiviert das aktive Journal und entfernt active_run.json.

```
format_unfinished_job_summary(data) -> str
```

15.2.14.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.14.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.15 dragontools/core/lang_codes.py

Merkmal	Wert
Kategorie	Core
Umfang	304 Quellzeilen
Bedeutung	Zentraler Baustein (304 Zeilen, direkt von 10 Projektmodulen importiert).

15.2.15.1 Aufgabe des Moduls

dragontools/core/lang_codes.py Technischer Kontext aus dem Quellcode: dragontools/core/lang_codes.py Gemeinsame Sprachcode- und Subtitle-Codec-Tabellen. Vorher waren diese Daten dreifach dupliziert: worker/dv_remux_thread.py LANG_CODES dict (Modulebene) historisch worker/dv_conversion.py (entfernt) lang_map dict (2x lokal in Methoden) Oeffentliche API LANG_MAP : dict[str, str] ISO-639-1/2-Codes -> lesbare Sprachbezeichnung.

> *Gemeinsame Sprachcode- und Subtitle-Codec-Tabellen. Vorher waren diese Daten dreifach dupliziert: - worker/dv_remux_thread.py LANG_CODES dict (Modulebene)*

15.2.15.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `code`, `value`, `stream_language`, `wanted_language`, `values`, `codec`

15.2.15.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `set[str]`, `bool`, `list[str]`, `'tuple[str, list[str]] | None'`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/rules_language_editor.py, dragontools/gui/rules_subtitle_tab.py, dragontools/gui/subtitle_widget.py, dragontools/rules/audio_rules.py, dragontools/rules/subtitle_rules.py, dragontools/tests/test_subtitle_sidecar_service.py, dragontools/worker/dv_audio_mux_service.py, dragontools/worker/dv_remux_components.py, dragontools/worker/mp4_remux_thread.py, dragontools/worker/subtitle_sidecar_service.py

15.2.15.4 Wichtige Funktionen

`lang_display(code) -> str`

Gibt den lesbaren Sprachnamen für *code* zurück. Unbekannte Codes werden unverändert zurückgegeben (kein KeyError).

`lang_iso_tag(code) -> str`

Konvertiert einen ISO-639-2-Code in ISO-639-1 (2-stellig), falls bekannt. Bekannte 2-stellige Codes werden unverändert zurückgegeben. Unbekannte Codes werden unverändert zurückgegeben (kein Fehler).

`canonical_lang(code) -> str`

Normalisiert Spracheingaben auf einen bevorzugten ISO-639-1-Code.

`language_aliases(code) -> set[str]`

Alle bekannten Schreibweisen für eine Sprache.

`language_matches(stream_language, wanted_language) -> bool`

Prüft ISO-639-1/2- und Namensvarianten einer Sprache.

`normalize_language_priority(values) -> list[str]`

Macht aus Codes/Namen eine eindeutige Prioritätsliste.

`sub_codec_to_ext_and_args(codec) -> 'tuple[str, list[str]] | None'`

Gibt (Dateiendung, ffmpeg-Codec-Argumente) für *codec* zurück. Gibt None zurück wenn der Codec nicht unterstützt wird (statt einer leeren Liste oder eines Exceptions). Der Aufrufer soll dann die Spur

15.2.15.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.15.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.16 dragontools/core/log_cleanup.py

Merkmal	Wert
Kategorie	Core
Umfang	133 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (133 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.16.1 Aufgabe des Moduls

Bereinigt normale Logs, VerboseLogs, ErrorReports und CrashReports nach Benutzerauswahl.

15.2.16.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `base_root`, `folder`, `path`

15.2.16.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `LogCleanupResult`, `None`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/core/diagnostic_package.py, dragontools/gui/settings_dialog.py, dragontools/tests/test_log_cleanup.py

15.2.16.4 Wichtige Klassen und Methoden

LogCleanupResult [object]

15.2.16.5 Wichtige Funktionen

`cleanup_logs(base_root) -> LogCleanupResult`

Löscht DragonTools-Logdateien unterhalb des konfigurierten Log-Basisordners. cutoff_date=None löscht alle Logdateien. Bei gesetztem Datum werden Dateien gelöscht, deren Änderungszeit am oder vor diesem Datum liegt.

15.2.16.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.16.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.17 dragontools/core/logger.py

Merkmal	Wert
Kategorie	Core
Umfang	717 Quellzeilen
Bedeutung	Zentraler Baustein (717 Zeilen, direkt von 13 Projektmodulen importiert).

15.2.17.1 Aufgabe des Moduls

Erstellt Hauptlog und formatierte Logmeldungen. Technischer Kontext aus dem Quellcode: DragonLogger V9 – Formatiertes Logging. Schreibt in Datei und sendet an GUI.

15.2.17.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `base_root`, `settings`, `current_log_file`, `b`, `s`, `log_dir`, `verbose_logger`, `enabled`, `log_enabled`, `gui_callback`

15.2.17.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `tuple[bool, Path]`, `'DragonLogger'`, `str`, `None`, `VerboseLogger`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/move_report.py, dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/core/crash_guard.py, dragontools/core/diagnostic_package.py, dragontools/core/error_report.py, dragontools/core/paths.py, dragontools/gui/convert_widget.py, dragontools/tests/test_verbose_cleanup.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/iso_thread.py, dragontools/worker/merge_thread.py, dragontools/worker/move_thread.py, dragontools/worker/mp4_remux_thread.py, dragontools/worker/parallel_converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_LOG_ROOT, SET_KEY_VERBOSE_LOG_ENABLED, SET_KEY_VERBOSE_LOG_ROOT

15.2.17.4 Wichtige Klassen und Methoden

VerboseLogger [object] – Separater Logger für Debug-/Trace-Meldungen. Schreibt in eigenen Ordner (keine Jahres-/Monatsunterordner). Max. 10 .txt-Dateien, älteste wird automatisch gelöscht.

```
write(msg) -> None
discard() -> bool
log_dir() -> Path | None
```

DragonLogger [object] – Zentraler Logger mit V7-kompatiblem Ausgabeformat. Schreibt formatierte Log-Dateien und sendet an die GUI.

```
set_gui_callback(cb) -> None
has_logging_failures() -> bool
failure_summary() -> str | None
drain_failures() -> tuple[int, int]
header(gpus, chosen_encoder, total_files, codec, crf, preset) -> None
move_header(total_files) -> None
file_start(idx, total, path, codec, crf, preset, encoder, q_label, encoder_options) -> None
crop(w, h, cw, ch) -> None
scale(orig_w, orig_h, scale_str) -> None
audio(track_num, codec, action, target_codec, channels, bitrate_k) -> None
subtitle(msg) -> None
pipeline(name, container, has_dv, has_hdrplus) -> None
```

15.2.17.5 Wichtige Funktionen

`make_log_dir(base_root) -> Path`

Erstellt und gibt zurück: <base_root>/Logging/<YYYY>/<MM-Monat>/ Keine locale-Abhängigkeit: Monatsnamen kommen aus _DE_MONTHS. Wird von allen Workern genutzt – einheitliche Pfadstruktur.

`log_base_from_settings(settings) -> Path`

Liest den konfigurierten Logging-Basisordner aus QSettings.

`log_settings_from_qsettings(settings) -> tuple[bool, Path]`

Liest die globalen Logging-Einstellungen mit Legacy-Fallbacks.

`resolve_log_month_dir(current_log_file) -> Path`

Liefert den echten aktuellen Log-Monatsordner. Priorität: 1) Parent von `current_log_file`, falls vorhanden

`create_worker_logger() -> 'DragonLogger'`
Erzeugt Worker-Logger aus der globalen Logging-Konfiguration.
`make_verbose_log_dir(base_root) -> Path`
Erstellt und gibt zurück: <base_root>/VerboseLog/ Keine Jahres-/Monatsunterordner. Direkt .txt-Dateien.
`verbose_log_dir_from_settings(settings) -> Path`
Liest den Verbose-Log-Pfad aus QSettings.
`create_verbose_logger(settings) -> VerboseLogger`
Erzeugt einen VerboseLogger aus den globalen Einstellungen.
`discard_verbose_after_clean_run(verbose_logger) -> bool`
Entfernt den Verbose-Log nur nach einem sauberen fehlerfreien Lauf.

15.2.17.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.17.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.18 dragontools/core/media_analyzer.py

Merkmal	Wert
Kategorie	Core
Umfang	739 Quellzeilen
Bedeutung	Zentraler Baustein (739 Zeilen, direkt von 13 Projektmodulen importiert).

15.2.18.1 Aufgabe des Moduls

Baut MediaInfo-Objekte aus MediaInfo und ffprobe.

15.2.18.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `tools`, `mi_json`, `fp_json`, `stream_type`, `video_track`, `warnings`, `stream`, `track`, `fallback`, `mi_videos`, `fp_videos`, `analysis_warnings`, `mi_audios`, `fp_audios`, `value`

15.2.18.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `tuple[dict, list[str], bool]`, `tuple[dict, list[str]]`, `list[dict]`, `dict`, `str | None`, `tuple[bool, bool, str | None]`, `int | None`, `bool`, `int`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_hdr_detection.py, dragontools/core/media_metadata.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/process_runner.py, dragontools/core/type_utils.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.

Abhängigkeit	Details
Externe Programme/Tools	ffmpeg, ffprobe, mediainfo
Direkt verwendet von	dragontools/core/audio_video_matcher.py, dragontools/core/rules_preview.py, dragontools/gui/convert_widget_override_dialog.py, dragontools/gui/media_info_dialog.py, dragontools/tests/test_media_analyzer_pix_fmt.py, dragontools/worker/audio_mux_thread.py, dragontools/worker/audio_video_match_thread.py, dragontools/worker/dv_processing_pipeline.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/media_analysis_service.py, dragontools/worker/merge_thread.py, dragontools/worker/mp4_remux_thread.py, dragontools/worker/quality_test_thread.py

15.2.18.4 Wichtige Funktionen

```
analyze_media(path, tools) -> MediaInfo
```

15.2.18.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg / ffprobe / mediainfo → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.18.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (ValueError, AttributeError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.19 dragontools/core/media_hdr_detection.py

Merkmal	Wert
Kategorie	Core
Umfang	286 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (286 Zeilen, direkte Importnutzer: 1).

15.2.19.1 Aufgabe des Moduls

Normalisiert HDR-, DV- und HDR10+-Erkennung.

15.2.19.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `video_track`, `mi`, `stream`, `codec`, `bit_depth`, `pix_fmt`, `is_hdr`, `has_hdr10plus`, `dv_profile`, `warnings`

15.2.19.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `dict`, `str`, `tuple[bool, bool, str | None]`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_metadata.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffprobe, mediainfo
Direkt verwendet von	dragontools/core/media_analyzer.py

15.2.19.4 Wichtige Funktionen

`parse_dolby_vision_profile(video_track) -> dict`

Parse Dolby-Vision profile details from a MediaInfo video track.

`choose_dovi_convert_mode(mi) -> str`

Choose the dovi_tool -m mode from the detected Dolby-Vision profile.

`detect_hdr_from_mediainfo_track(video_track) -> tuple[bool, bool, str | None]`

Detect HDR, HDR10+ and Dolby Vision from structured MediaInfo fields.

`detect_hdr_from_ffprobe_stream(stream) -> tuple[bool, bool, str | None]`

Detect HDR, HDR10+ and Dolby Vision from structured ffprobe fields.

`validate_hdr_flags(codec, bit_depth, pix_fmt, is_hdr, has_hdr10plus, dv_profile, warnings) -> tuple[bool, bool, str | None]`

Apply hard guards to remove impossible HDR/DV false positives.

15.2.19.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffprobe / mediainfo → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.19.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.20 dragontools/core/media_metadata.py

Merkmal	Wert
Kategorie	Core
Umfang	219 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (219 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.20.1 Aufgabe des Moduls

Parst Bitdepth, Framerate, Dauer, Sprach- und Farbraumdetails.

15.2.20.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `raw`, `language`, `title`, `lang`, `codec`, `track`, `value`, `mi_track`, `fp_stream`

15.2.20.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `bool`, `object | None`, `int | None`, `float | None`, `Fraction | None`, `str | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe, mediainfo
Direkt verwendet von	dragontools/core/media_analyzer.py, dragontools/core/media_hdr_detection.py, dragontools/core/mediainfo_details.py

15.2.20.4 Wichtige Funktionen

`normalize_color_range_for_zscale(raw) -> str`

Konvertiert MediaInfo/ffprobe Farbraum-Range zu zscale-Format. MediaInfo: 'Full' oder 'Limited' ffprobe: 'pc' (full) oder 'tv' (limited)

`normalize_color_range_label(raw) -> str`

Lesbarer Label für die Range (für Log/UI): 'Full' oder 'Limited'.

`is_german(language, title) -> bool`

`normalize_video_codec(codec) -> str`

Normalisiert Codec-Namen auf kanonische Kurzform ('h264', 'hevc', 'av1', ...).

`build_pix_fmt_from_mediainfo(track) -> str | None`

Baut ein ffmpeg-kompatibles pix_fmt aus MediaInfo JSON-Feldern. Genutzt werden nur Speicherformat-Felder: ColorSpace, ChromaSubsampling und BitDepth. Farbwiedergabe-Metadaten wie transfer_characteristics,

`parse_bit_depth(mi_track, fp_stream) -> int | None`

Ermittelt Bit-Tiefe: MediaInfo -> ffprobe bits_per_raw_sample -> pix_fmt-Ableitung.

15.2.20.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg / ffprobe / mediainfo → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.20.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.21 dragontools/core/mediainfo_details.py

Merkmal	Wert
Kategorie	Core
Umfang	395 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (395 Zeilen, direkte Importnutzer: 2).

15.2.21.1 Aufgabe des Moduls

Bereitet ausführliche MediaInfo-/ffprobe-Daten für den Rechtsklick-Dialog auf.

15.2.21.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `file_path`, `media_info`, `warning`, `track`, `fallback_index`, `payload`, `tracks`, `kind`, `value`, `fallback`, `bytes_value`, `seconds_value`, `frame_rate`, `numerator`, `denominator`

15.2.21.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `tuple[dict[str, Any], str | None]`, `MediaInfoDisplayDetails`, `list[tuple[str, str]]`, `dict[str, str]`, `list[dict[str, Any]]`, `dict[str, Any]`, `int | None`, `float | None`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_metadata.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/process_runner.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	mediainfo
Direkt verwendet von	dragontools/gui/media_info_dialog.py, dragontools/tests/test_mediainfo_details.py

15.2.21.4 Wichtige Klassen und Methoden

MediaInfoDisplayDetails [object]

15.2.21.5 Wichtige Funktionen

`load_mediainfo_json(file_path) -> tuple[dict[str, Any], str | None]`

Lädt die vollständige MediaInfo-JSON-Ausgabe für die Detailansicht.

`build_mediainfo_display_details(file_path) -> MediaInfoDisplayDetails`

15.2.21.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → mediainfo → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.21.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, OSError, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.22 dragontools/core/models.py

Merkmal	Wert
Kategorie	Core
Umfang	364 Quellzeilen
Bedeutung	Zentraler Baustein (364 Zeilen, direkt von 28 Projektmodulen importiert).

15.2.22.1 Aufgabe des Moduls

Datamodelle für Jobs, MediaInfo, Streams, Pipeline und Overrides.

15.2.22.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`, `file_override`

15.2.22.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte

- Explizite Rückgabetypen im Code: `float | None`, `dict[str, Any]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/type_utils.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, hdr10plus_tool, mp4box
Direkt verwendet von	dragontools/core/audio_video_matcher.py, dragontools/core/media_analyzer.py, dragontools/core/mediainfo_details.py, dragontools/core/rules_preview.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/gui/rules_audio_tab.py, dragontools/rules/audio_plan.py, dragontools/rules/audio_rules.py, dragontools/rules/pipeline_selector.py, dragontools/rules/subtitle_rules.py, dragontools/tests/test_audio_processing.py, dragontools/tests/test_audio_video_matcher.py, dragontools/tests/test_auxiliary_workers.py, dragontools/tests/test_encoder_profile_override.py, dragontools/tests/test_extended.py, dragontools/tests/test_hdr10_color_standard.py, dragontools/tests/test_language_priority_rules.py, dragontools/tests/test_models.py

15.2.22.4 Wichtige Klassen und Methoden

JobStatus [str, Enum]

TargetCodec [str, Enum] – Ziel-Videocodec für Encoding-Jobs. str-Enum: Werte serialisieren als Plain-String ("h265", "h264", "av1"), damit QSettings, JSON-Profile und ConversionJob.codec rueckwaertskompatibel

Pipeline [str, Enum] – Konvertierungs-Pipeline-Key. Steuert, welcher Verarbeitungspfad gewählt wird: STANDARD – normaler ffmpeg-Encode ohne HDR-Spezialbehandlung

SubtitleOverride [object] – Legacy-Brücke für ältere Burn-Mode-Semantik.

AudioStream [object]

SubtitleStream [object]

VideoStream [object]

MediaInfo [object]

```
primary_video() -> VideoStream | None
has_dv() -> bool
has_hdrplus() -> bool
dv_profile_label() -> str
```

ConversionJob [object]

```
input_name() -> str
```

ConversionResult [object] – Fasst alle Ausgabepfade einer einzelnen Konvertierung zusammen. Felder: input_path – Original-Eingabedatei.

15.2.22.5 Wichtige Funktionen

`normalize_override_dict(file_override) -> dict[str, Any]`

Kanonisiert alte und neue per-Datei-Overrides auf die aktuelle Hauptsemantik: audio_mode/audio_tracks, subtitle_mode/subtitle_tracks, imax, processing_mode. Legacy-Felder bleiben nur als Eingabe und werden unter _legacy erhalten.

15.2.22.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / hdr10plus_tool / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.22.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.23 dragontools/core/move_conflicts.py

Merkmal	Wert
Kategorie	Core
Umfang	91 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (91 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.23.1 Aufgabe des Moduls

Erkennt und formatiert Zielkonflikte beim Verschieben, besonders bei Videodateien mit gleichem Namen, aber anderer Endung.

15.2.23.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `a`, `b`, `dst_p`, `src_p`, `conflicts`

15.2.23.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `bool`, `list[Path]`, `Path`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/batch_preflight.py, dragontools/tests/test_move_conflicts.py, dragontools/worker/move_thread.py

15.2.23.4 Wichtige Funktionen

```
norm_path(path) -> str
same_path(a, b) -> bool
find_target_conflicts(dst_p, src_p) -> list[Path]
```

Findet Zielkonflikte für den Verschiebevorgang. Für Videodateien zählt gleicher Stammname unabhängig von der Endung: Film.mkv kollidiert also mit Film.mp4, Film.avi usw. Für andere Dateien

`resolve_rename_path(dst_p, src_p) -> Path`

Gibt einen freien Zielpfad mit Suffix _01, _02, ... zurück.

`format_conflict_names(conflicts) -> str`

15.2.23.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.23.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.24 dragontools/core/move_report.py

Merkmal	Wert
Kategorie	Core
Umfang	241 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (241 Zeilen, direkte Importnutzer: 2).

15.2.24.1 Aufgabe des Moduls

Baut gruppierte Verschiebeberichte.

15.2.24.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `move_log`, `entry`, `bool_key`, `count_key`, `value`

15.2.24.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `list[dict[str, Any]]`, `list[str]`, `dict[str, Any] | None`, `int`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/logger.py, dragontools/tests/test_move_report.py

15.2.24.4 Wichtige Funktionen

`build_move_target_summary(move_log) -> list[dict[str, Any]]`

Verdichtet Move-Einträge auf Zielordner-Ebene. Neue MoveThread-Versionen liefern Dicts. Alte Tuple-Einträge werden weiterhin verstanden, damit bestehende Worker kompatibel bleiben.

`format_move_target_summary_lines(move_log) -> list[str]`

15.2.24.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.24.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.25 dragontools/core/movie_renamer.py

Merkmal	Wert
Kategorie	Core
Umfang	787 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (787 Zeilen, direkt von 2 Projektmodulen importiert).

15.2.25.1 Aufgabe des Moduls

Bereinigt Film- und Serienrelease-Namen, extrahiert Titel/Jahr beziehungsweise SxxEyy und bewertet Online-Metadaten-Treffer.

15.2.25.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `path`, `title`, `year`, `suffix`, `series`, `season`, `episode`, `episode_title`, `source_path`, `target_name`, `patterns`, `parsed`, `raw`, `query_title`, `query_year`

15.2.25.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `ParsedMovieReleaseName`, `ParsedSeriesReleaseName | None`, `MovieRenameProposal`, `SeriesRenameProposal`, `RenameProposal`, `str`, `Path`, `tuple[str, ...]`, `Iterable[Any]`, `MovieRenameCandidate | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/german_title_variants.py, dragontools/core/paths.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/movie_renamer_widget.py, dragontools/tests/test_movie_renamer.py

15.2.25.4 Wichtige Klassen und Methoden

ParsedMovieReleaseName [object]

MovieRenameCandidate [object]

```
display_title() -> str
choice_label() -> str
```

MovieRenameProposal [object]

```
can_auto_accept() -> bool
```

ParsedSeriesReleaseName [object]

```
query_title() -> str
```

SeriesRenameCandidate [object]

```
display_title() -> str
choice_label() -> str
```

SeriesRenameProposal [object]

```
can_auto_accept() -> bool
```

15.2.25.5 Wichtige Funktionen

```
parse_movie_release_name(value) -> ParsedMovieReleaseName
parse_series_release_name(value) -> ParsedSeriesReleaseName | None
build_movie_rename_proposal(path) -> MovieRenameProposal
build_series_rename_proposal(path) -> SeriesRenameProposal
build_rename_proposal(path) -> RenameProposal
build_target_filename(title, year, suffix) -> str
build_series_target_filename(series, season, episode, episode_title, suffix) -> str
sanitize_filename_part(value) -> str
rename_movie_file(source_path, target_name) -> Path
```

15.2.25.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.25.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.26 dragontools/core/online_metadata.py

Merkmal	Wert
Kategorie	Core
Umfang	1883 Quellzeilen
Bedeutung	Zentraler Baustein (1883 Zeilen, direkt von 7 Projektmodulen importiert).

15.2.26.1 Aufgabe des Moduls

Kapselt Online-Metadaten über TMDB und TheTVDB. Filme nutzen TMDB; Serien können je nach Einstellung TMDB oder TheTVDB verwenden.

15.2.26.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`, `cfg`, `provider`, `settings`, `media_type`, `root`, `cache_dir`, `data`, `record`, `language`, `config`, `path`, `suggestion`, `items`, `credits`

15.2.26.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `bool`, `OnlineMetadataConfig`, `None`, `ParsedMovieQuery`, `ParsedSeriesQuery`, `Path`, `int`, `list[dict[str, Any]]`, `dict[str, Any] | None`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/german_title_variants.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/jellyfin_nfo.py, dragontools/gui/main_window_actions.py, dragontools/gui/movie_renamer_widget.py, dragontools/gui/preflight_dialog.py, dragontools/tests/test_jellyfin_nfo.py, dragontools/tests/test_online_metadata.py, dragontools/worker/postprocess_service.py
Verwendete Settings-Konstanten	SET_KEY_METADATA_CACHE_DAYS, SET_KEY_METADATA_CACHE_ENABLED, SET_KEY_METADATA_FALLBACK_LANGUAGE, SET_KEY_METADATA_LANGUAGE, SET_KEY_METADATA_MOVIE_PREFERRED_PROVIDER, SET_KEY_METADATA_MOVIE_PROVIDER, SET_KEY_METADATA_SERIES_PREFERRED_PROVIDER, SET_KEY_METADATA_SERIES_PROVIDER, SET_KEY_METADATA_TMDB_API_KEY, SET_KEY_METADATA_TMDB_ENABLED, SET_KEY_METADATA_TMDB_READ_TOKEN, SET_KEY_METADATA_TVDB_API_KEY, SET_KEY_METADATA_TVDB_BEARER_TOKEN, SET_KEY_METADATA_TVDB_ENABLED, SET_KEY_METADATA_TVDB_PIN

15.2.26.4 Wichtige Klassen und Methoden

OnlineMetadataError [RuntimeError]

OnlineMetadataAuthError [OnlineMetadataError]

OnlineMetadataConfig [object]

```
has_tmdb_credentials() -> bool
has_tvdb_credentials() -> bool
provider_for(media_type) -> str
preferred_provider_for(media_type) -> str
provider_chain_for(media_type) -> tuple[str, ...]
```

ParsedMovieQuery [object]

ParsedSeriesQuery [object]

MovieMetadataSuggestion [object]

```
has_collection() -> bool
```

```
movie_folder_name() -> str
```

SeriesMetadataSuggestion [object]

```
folder_name() -> str
```

EpisodeMetadataSuggestion [object]

TmdbClient [object]

```
test_connection() -> dict[str, Any]
search_movies(query) -> list[dict[str, Any]]
search_tv(query) -> list[dict[str, Any]]
movie_details(movie_id) -> dict[str, Any]
tv_details(tv_id) -> dict[str, Any]
tv_episode_details(tv_id, season, episode) -> dict[str, Any]
collection_details(collection_id) -> dict[str, Any]
resolve_movie(query) -> MovieMetadataSuggestion | None
resolve_movie_file(path) -> MovieMetadataSuggestion | None
resolve_episode_candidates(path) -> tuple[EpisodeMetadataSuggestion, ...]
resolve_series(query) -> SeriesMetadataSuggestion | None
resolve_series_name(value) -> SeriesMetadataSuggestion | None
```

TheTvdbClient [object]

```
test_connection() -> dict[str, Any]
search_movies(query) -> list[dict[str, Any]]
search_series(query) -> list[dict[str, Any]]
movie_details(movie_id) -> dict[str, Any]
series_details(series_id) -> dict[str, Any]
series_episodes(series_id) -> list[dict[str, Any]]
resolve_series(query) -> SeriesMetadataSuggestion | None
resolve_movie(query) -> MovieMetadataSuggestion | None
resolve_movie_file(path) -> MovieMetadataSuggestion | None
resolve_series_name(value) -> SeriesMetadataSuggestion | None
resolve_episode_file(path) -> EpisodeMetadataSuggestion | None
resolve_episode_candidates(path) -> tuple[EpisodeMetadataSuggestion, ...]
```

15.2.26.5 Wichtige Funktionen

```
metadata_series_folder_title(value) -> str
```

Bereinigt TMDB-Serientitel fuer lokale Windows-Ordner. TMDB liefert einige Anime-/Serientitel mit Doppelpunkt, z.B. "Re:ZERO - Starting Life in Another World". Ein Doppelpunkt ist unter

```
config_from_settings(settings) -> OnlineMetadataConfig
metadata_provider_configured(cfg, media_type) -> bool
parse_movie_query(value) -> ParsedMovieQuery
parse_series_query(value) -> ParsedSeriesQuery
clean_tmdb_collection_name(value) -> str
```

Entfernt reine Provider-Suffixe aus TMDB-Collection-Namen. TMDB lokalisiert Collections oft als "Titel Filmreihe" oder "Title Collection". Für DragonTools-Zielordner ist der reine Reihenname sinnvoller.

```
default_metadata_cache_dir(root, provider) -> Path
clear_default_metadata_cache(root) -> int
client_from_settings(settings)
client_from_config(config, media_type)
client_from_settings_for(settings, media_type)
suggest_movie_metadata_for_file(path, settings) -> MovieMetadataSuggestion | None
suggest_series_metadata_for_name(value, settings) -> SeriesMetadataSuggestion | None
suggest_episode_metadata_for_file(path, settings) -> EpisodeMetadataSuggestion | None
format_movie_suggestion(suggestion) -> str
format_series_suggestion(suggestion) -> str
```

15.2.26.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.26.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, HTTPError, OSError, OnlineMetadataAuthError, OnlineMetadataError, TimeoutError, URLError, json.JSONDecodeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.27 dragontools/core/parallel_settings.py

Merkmal	Wert
Kategorie	Core
Umfang	96 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (96 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.27.1 Aufgabe des Moduls

Bestimmt und begrenzt die parallele Bearbeitung je Encoder.

15.2.27.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`, `encoder`, `settings`, `key`, `files`, `jobs`

15.2.27.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `int`, `bool`, `None`, `list[list[str]]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/settings_dialog.py, dragontools/tests/test_parallel_settings.py, dragontools/worker/parallel_converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_PARALLEL_CPU_JOBS, SET_KEY_PARALLEL_DEFAULTS_MIGRATED, SET_KEY_PARALLEL_GPU_JOBS

15.2.27.4 Wichtige Funktionen

`clamp_parallel_jobs(value, default) -> int`

`is_gpu_encoder(encoder) -> bool`

`migrate_parallel_defaults(settings) -> None`

Migriert alte konservative Parallel-Defaults einmalig auf CPU 1 / GPU 2.

`parallel_jobs_for_encoder(settings, encoder) -> int`

Liest die Parallelitätsgrenze passend zum gewählten Encoder.

`split_files_for_parallel_workers(files, jobs) -> list[list[str]]`
Verteilt Dateien rundlaufend auf Worker, damit frühe Queue-Dateien sofort starten.

15.2.27.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.27.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.28 dragontools/core/path_safety.py

Merkmal	Wert
Kategorie	Core
Umfang	65 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (65 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.28.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _resolved, _is_root, is_safe_subpath, safe_unlink.

15.2.28.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path_like`, `path`, `base_dir`, `target_path`

15.2.28.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_path_safety.py, dragontools/worker/cleanup_service.py, dragontools/worker/dv_failure_recovery.py

15.2.28.4 Wichtige Funktionen

`is_safe_subpath(base_dir, target_path) -> bool`
`safe_unlink(base_dir, target_path) -> bool`
`safe_rmtree(base_dir, target_path) -> bool`

15.2.28.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.28.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.29 dragontools/core/paths.py

Merkmal	Wert
Kategorie	Core
Umfang	612 Quellzeilen
Bedeutung	Zentraler Baustein (612 Zeilen, direkt von 39 Projektmodulen importiert).

15.2.29.1 Aufgabe des Moduls

Zentrale Pfad- und Toolsuche inklusive PyInstaller-Datenordnern. Technischer Kontext aus dem Quellcode: Zentrale Pfad- und Tool-Verwaltung für DragonTools V9. Einmalig instantiiert, von allen Modulen genutzt. Ersetzt die verstreuten ffmpeg_path(), find_tool(), resource_path() aus dem Altcode.

15.2.29.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `max_len`, `root`, `codec`, `media_type`, `settings_key`, `root_path`, `patterns`, `rel`, `extra`, `executable_names`, `name`, `provider`

15.2.29.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `bool`, `Path`, `dict[str, str]`, `dict[str, dict[str, Path]]`, `list[Path]`, `None`, `str | None`, `ToolPaths`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mkvextract, mkvmerge, mkvpropedit, mp4box

Abhängigkeit	Details
Direkt verwendet von	dragontools/core/audio_video_matcher.py, dragontools/core/diagnostic_package.py, dragontools/core/job_journal.py, dragontools/core/media_analyzer.py, dragontools/core/mediainfo_details.py, dragontools/core/move_conflicts.py, dragontools/core/movie_renamer.py, dragontools/core/online_metadata.py, dragontools/core/path_safety.py, dragontools/core/rules_preview.py, dragontools/gui/audio_video_matcher_widget.py, dragontools/gui/changelog_dialog.py, dragontools/gui/conversion_controller.py, dragontools/gui/convert_widget.py, dragontools/gui/convert_widget_custom_widgets.py, dragontools/gui/convert_widget_file_queue.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/gui/drop_path_extractor.py
Verwendete Settings-Konstanten	SET_KEY_PATH_ANIME, SET_KEY_PATH_AV1_ANIME, SET_KEY_PATH_AV1_FILME, SET_KEY_PATH_AV1_TV, SET_KEY_PATH_FILME, SET_KEY_PATH_H264_ANIME, SET_KEY_PATH_H264_FILME, SET_KEY_PATH_H264_TV, SET_KEY_PATH_H265_ANIME, SET_KEY_PATH_H265_FILME, SET_KEY_PATH_H265_TV, SET_KEY_PATH_TV

15.2.29.4 Wichtige Klassen und Methoden

ToolPathSettingsProvider [ABC] – Interface für den Zugriff auf benutzerdefinierte Tool-Einstellungen. Entkoppelt core.paths von Qt: Die Implementierung (QtToolPathSettingsProvider in gui/tool_path_settings.py) liest QSettings; Tests oder CLI-Nutzung können

```
get_custom_dirs() -> list[Path]
find_in_settings(tool_key) -> str | None
```

ToolPaths [object] – Zugriff auf alle externen Tool-Pfade. Nimmt beim Init einen optionalen ToolPathSettingsProvider entgegen. Ohne Provider (z.B. in Tests oder CLI) werden nur PATH und Bundle-

```
find_in_settings(tool_key) -> str | None
ffmpeg() -> str
ffprobe() -> str
mkvmerge() -> str
makemkvcon() -> str
mkvextract() -> str
mkvinfo() -> str
mkvpropedit() -> str
mediainfo() -> str
dovi_tool() -> str
hdr10plus_tool() -> str
mp4box() -> str
```

AppPaths [object] – Verwaltet die Anwendungspfade für Logs und Ausgaben.

```
log_root(codec) -> Path
output_root(codec, media_type) -> Path
```

15.2.29.5 Wichtige Funktionen

```
strip_long_path_prefix(path) -> str
normalize_user_path(path) -> str
to_long_path(path) -> str
path_compare_key(path) -> str
display_path(path, max_len) -> str
display_name(path) -> str
is_video_file(path) -> bool
app_documents_dir(root) -> Path
```

Zentraler DragonTools-Dokumentordner. Standard: ``%USERPROFILE%\Documents\DragonTools``. Tests können über ``root`` einen temporären Basisordner übergeben.

```
default_output_base(root) -> Path
default_target_path(codec, media_type, root) -> Path
```

Standard-Zielordner für einen Codec und Medientyp. Beispiele: ``Dokumente/DragonTools/Ausgabe/H265/TV``

15.2.29.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / ffprobe / handbrake / hdr10plus_tool / makemkvcon / mediainfo / mkvextract / mkvmerge / mkvpropedit / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.29.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.30 dragontools/core/preflight_report.py

Merkmal	Wert
Kategorie	Core
Umfang	245 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (245 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.30.1 Aufgabe des Moduls

Speichert optional einen Preflight-Bericht vor dem Start.

Speicherbarer Preflight-Bericht für Start- und Verschiebeprüfungen.

15.2.30.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `documents_dir`, `value`, `fallback`, `path`, `max_len`, `rows`, `lines`, `row`, `warnings`, `reasons`, `files`

15.2.30.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `str`, `dict[str, dict[str, Any]]`, `None`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/move_preflight_controller.py, dragontools/tests/test_incremental_move_queue_edit.py, dragontools/tests/test_preflight_report.py

15.2.30.4 Wichtige Funktionen

`default_preflight_report_dir(documents_dir) -> Path`

Standardordner für lokal gespeicherte Preflight-Berichte.

`default_preflight_report_path() -> Path`

`build_preflight_report(files) -> str`

Erzeugt den Textinhalt eines speicherbaren Preflight-Berichts.

`write_preflight_report(files) -> Path`

Schreibt den Preflight-Bericht und gibt den Dateipfad zurück.

15.2.30.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.30.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.31 dragontools/core/process_runner.py

Merkmal	Wert
Kategorie	Core
Umfang	96 Quellzeilen
Bedeutung	Zentraler Baustein (96 Zeilen, direkt von 9 Projektmodulen importiert).

15.2.31.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie run_analysis_tool.

15.2.31.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `cmd`

15.2.31.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `dict`, `subprocess.CompletedProcess[str]`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, mediainfo, mkvpropedit, mp4box
Direkt verwendet von	dragontools/core/audio_video_matcher.py, dragontools/core/jellyfin_nfo.py, dragontools/core/media_analyzer.py, dragontools/core/mediainfo_details.py, dragontools/core/tool_diagnostics.py, dragontools/gui/subtitle_widget.py, dragontools/subtitle/extractor.py, dragontools/subtitle/injector.py, dragontools/subtitle/tagger.py

15.2.31.4 Wichtige Funktionen

`subprocess_no_window_kwargs() -> dict`

subprocess-Kwargs, damit Windows keine kurzen Konsolenfenster öffnet.

`run_analysis_tool(cmd) -> subprocess.CompletedProcess[str]`

Führt ein Analyse-Tool aus und gibt das vollständige Ergebnis zurück. Parameters ----------

15.2.31.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / ffprobe / mediainfo / mkvpropedit / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.31.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, RuntimeError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.32 dragontools/core/profile_manager.py

Merkmal	Wert
Kategorie	Core
Umfang	212 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (212 Zeilen, direkt von 4 Projektmodulen importiert).

15.2.32.1 Aufgabe des Moduls

dragontools/core/profile_manager.py – Default + User Profile

15.2.32.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `fallback`, `codec`, `path`

15.2.32.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `None`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/codec_profile_assistant.py, dragontools/core/config_migration.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/gui/main_window_actions.py, dragontools/tests/test_config_migration.py, dragontools/tests/test_gui_dialog_services.py

15.2.32.4 Wichtige Klassen und Methoden

ProfileManager [object]

```
save() -> None
data() -> dict[str, Any]
defaults() -> dict[str, Any]
get(key) -> dict[str, Any]
set(key, value) -> bool
profile_label(key, value) -> str
profile_context(value) -> tuple[str, str]
profile_display_name(key, value) -> str
```

15.2.32.5 Wichtige Funktionen

```
_extend_defaults_with_assistant_profiles() -> None
_normalise_profile_label(value) -> str
_slugify_profile_label(value) -> str
_profile_encoder(value) -> str
_profile_codec(value, fallback) -> str
_codec_display(codec) -> str
```

15.2.32.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.32.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.33 dragontools/core/quality_tester.py

Merkmal	Wert
Kategorie	Core
Umfang	175 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (175 Zeilen, direkte Importnutzer: 2).

15.2.33.1 Aufgabe des Moduls

Definiert Datenmodelle und Zeitbereichslogik für den Qualitätstester.

15.2.33.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`, `seconds`, `text`, `duration_s`, `input_path`, `run`, `segment`, `data`

15.2.33.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `float`, `str`, `list[QualitySegment]`, `list[str]`, `QualityTestRun`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_quality_tester.py, dragontools/worker/quality_test_thread.py

15.2.33.4 Wichtige Klassen und Methoden

QualitySegment [object]

QualityTestRun [object]

QualityMetricResult [object]

15.2.33.5 Wichtige Funktionen

`seconds_to_label(seconds) -> str`

`parse_time_value(text, duration_s) -> float`

`parse_quality_segments(text) -> list[QualitySegment]`

Parst manuelle Bereiche wie ``10%+20`` oder ``00:05:00+30``.

`automatic_quality_segments() -> list[QualitySegment]`

`quality_output_name(input_path, run, segment) -> str`

`parse_extra_args(text) -> list[str]`

Kleine Shell-ähnliche Zerlegung für zusätzliche FFmpeg-Argumente.

`quality_run_from_dict(data) -> QualityTestRun`

15.2.33.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.33.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.34 dragontools/core/release_validation.py

Merkmal	Wert
Kategorie	Core
Umfang	306 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (306 Zeilen, direkte Importnutzer: 2).

15.2.34.1 Aufgabe des Moduls

Prüft Build-/Release-Ordner auf fehlende Startdateien, Handbuch, Hilfe, Changelogs und Schemas.

15.2.34.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `root`, `title`, `expected_version`, `dist_root`, `project_root`, `app_root`, `checks`

15.2.34.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `bool`, `tuple[Path, Path]`, `ReleaseCheck`, `dict`, `Iterable[Path]`, `list[ReleaseCheck]`, `Path | None`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/main_window_actions.py, dragontools/tests/test_release_validation.py

15.2.34.4 Wichtige Klassen und Methoden

ReleaseCheck [object]

15.2.34.5 Wichtige Funktionen

```
validate_release(project_root) -> list[ReleaseCheck]
validate_app_bundle(app_root) -> list[ReleaseCheck]
format_release_checks(checks) -> str
```

15.2.34.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.34.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, SyntaxError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.35 dragontools/core/rules_preview.py

Merkmal	Wert
Kategorie	Core
Umfang	276 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (276 Zeilen, direkte Importnutzer: 2).

15.2.35.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _lang, _build_audio_preview, _build_subtitle_preview, _value.

15.2.35.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `mi`, `ov`, `container`, `subtitle_rules`, `pipeline`, `file_path`

15.2.35.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `dict[str, Any]`, `Any`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/rules/audio_plan.py, dragontools/rules/audio_rules.py, dragontools/rules/move_rules.py, dragontools/rules/pipeline_selector.py, dragontools/rules/subtitle_rules.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/core/batch_preflight.py, dragontools/gui/media_info_dialog.py
Verwendete Settings-Konstanten	SET_KEY_PRESERVE_DV, SET_KEY_PRESERVE_HDRPLUS

15.2.35.4 Wichtige Funktionen

`build_rules_preview(file_path) -> dict[str, Any]`

Baut eine reine Anzeige-/Preview-Struktur aus den vorhandenen Analyse- und Regelbausteinen. Keine ffmpeg-Args, keine Worker, keine Ausführungslogik.

15.2.35.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → ffmpeg → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.35.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.36 dragontools/core/run_summary.py

Merkmal	Wert
Kategorie	Core
Umfang	196 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (196 Zeilen, direkte Importnutzer: 2).

15.2.36.1 Aufgabe des Moduls

Normalisiert Abschlusszeilen und Statuswerte.

15.2.36.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `status`, `path`, `value`, `sidecars`, `items`, `rows`, `totals`, `entry`, `result_entries`

15.2.36.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `str`, `int`, `dict[str, int]`, `dict[str, Any]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_result_service.py, dragontools/tests/test_run_summary.py

15.2.36.4 Wichtige Funktionen

```
normalize_result_row(entry) -> dict[str, Any]
build_run_summary(result_entries) -> dict[str, Any]
```

15.2.36.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.36.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.37 dragontools/core/settings.py

Merkmal	Wert
Kategorie	Core
Umfang	281 Quellzeilen
Bedeutung	Zentraler Baustein (281 Zeilen, direkt von 51 Projektmodulen importiert).

15.2.37.1 Aufgabe des Moduls

Zentrale Settings-Konstanten für DragonTools V9. Technischer Kontext aus dem Quellcode: Zentrale Settings-Konstanten für DragonTools V9. Alle QSettings-Keys an einem einzigen Ort – nie mehr in mehreren Dateien.

15.2.37.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls

15.2.37.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mp4box
Direkt verwendet von	DragonToolsV9.py, dragontools/core/audit_log.py, dragontools/core/diagnostic_package.py, dragontools/core/logger.py, dragontools/core/online_metadata.py, dragontools/core/parallel_settings.py, dragontools/core/paths.py, dragontools/core/preflight_report.py, dragontools/core/release_validation.py, dragontools/core/rules_preview.py, dragontools/core/settings_backup.py, dragontools/core/timeout_settings.py, dragontools/gui/audio_video_matcher_widget.py, dragontools/gui/changelog_dialog.py, dragontools/gui/conversion_controller.py, dragontools/gui/convert_widget.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/convert_widget_override_dialog.py

Abhängigkeit	Details
Verwendete Settings-Konstanten	SET_KEY_ACTIVE_ALL_ANIME, SET_KEY_ACTIVE_ALL_FILME, SET_KEY_ACTIVE_ALL_TV, SET_KEY_ALLOW_GROWTH, SET_KEY_AUTOCROP_ENABLED, SET_KEY_AUTOCROP_MODE, SET_KEY_AUTOCROP_PROBE_DURATION, SET_KEY_AUTOCROP_PROBE_INTERVAL, SET_KEY_AUTOCROP_PROBE_START, SET_KEY_AV1_LOG_ENABLED, SET_KEY_AV1_LOG_ROOT, SET_KEY_H264_LOG_ENABLED, SET_KEY_H264_LOG_ROOT, SET_KEY_H265_LOG_ENABLED, SET_KEY_H265_LOG_ROOT, SET_KEY_IMAX_DETECT, SET_KEY_IMAX_MIN_HITS, SET_KEY_IMAX_MIN_VARIANCE_PERCENT, SET_KEY_IMAX_PROBE_DURATION, SET_KEY_IMAX_PROBE_INTERVAL

15.2.37.4 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / handbrake / hdr10plus_tool / makemkvcon / mediainfo / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.37.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.2.38 dragontools/core/settings_backup.py

Merkmal	Wert
Kategorie	Core
Umfang	182 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (182 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.38.1 Aufgabe des Moduls

Exportiert und importiert Einstellungen, Regeln und Profile.

15.2.38.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `documents_dir`, `value`, `key`, `settings`, `root`, `target_path`, `member_name`, `archive_path`

15.2.38.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `Path`, `Any`, `bool`, `dict[str, Any]`, `list[tuple[Path, str]]`, `Path | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/diagnostic_package.py, dragontools/gui/main_window_actions.py, dragontools/tests/test_settings_backup.py

15.2.38.4 Wichtige Funktionen

```
dragon_documents_dir() -> Path
default_backup_dir(documents_dir) -> Path
default_backup_path(documents_dir) -> Path
settings_to_dict(settings) -> dict[str, Any]
export_backup(target_path) -> Path
restore_backup(archive_path) -> dict[str, Any]
```

15.2.38.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.38.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.39 dragontools/core/system_shutdown.py

Merkmal	Wert
Kategorie	Core
Umfang	58 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (58 Zeilen, direkte Importnutzer: 2).

15.2.39.1 Aufgabe des Moduls

Kapselt die Shutdown-/Neustart-Logik nach erfolgreichen Läufen.

15.2.39.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `exc`

15.2.39.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `bool`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.

Abhängigkeit	Details
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/worker/move_thread.py

15.2.39.4 Wichtige Funktionen

`schedule_system_shutdown() -> bool`

Schedule system shutdown and return True if the OS accepted the request.

15.2.39.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.39.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.40 dragontools/core/timeout_settings.py

Merkmal	Wert
Kategorie	Core
Umfang	423 Quellzeilen
Bedeutung	Zentraler Baustein (423 Zeilen, direkt von 10 Projektmodulen importiert).

15.2.40.1 Aufgabe des Moduls

dragontools/core/timeout_settings.py Technischer Kontext aus dem Quellcode: dragontools/core/timeout_settings.py Zentrale Timeout-Definitionen für DragonTools. Alle user-konfigurierbaren Timeouts an einem Ort. Zeiteinheit intern: Sekunden | Anzeige im Dialog: Minuten

15.2.40.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `qs`, `td`, `key`, `seconds`, `enabled`, `values`

15.2.40.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `int`, `bool`, `int | None`, `dict[str, int]`, `dict[str, bool]`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, hdr10plus_tool, mediainfo, mkvextract, mkvmerge, mkvpropedit, mp4box

Abhängigkeit	Details
Direkt verwendet von	DragonToolsV9.py, dragontools/gui/timeout_settings_dialog.py, dragontools/subtitle/extractor.py, dragontools/subtitle/injector.py, dragontools/subtitle/tagger.py, dragontools/tests/test_gui_dialog_services.py, dragontools/worker/converter_progress.py, dragontools/worker/duration_repair_service.py, dragontools/worker/dv_pipeline_timeouts.py, dragontools/worker/hdrplus_conversion.py

15.2.40.4 Wichtige Klassen und Methoden

TimeoutDef [NamedTuple]

15.2.40.5 Wichtige Funktionen

`get_timeout_value(key) -> int`

Gibt den konfigurierten Timeout-Wert in **Sekunden** zurück. Liest aus QSettings; fällt auf den Standardwert zurück wenn nicht gesetzt. Kann aus Worker-Threads aufgerufen werden (QSettings ist thread-safe

`is_timeout_enabled(key) -> bool`

Gibt zurück, ob ein Timeout aktiv angewendet werden soll.

`get_timeout(key) -> int | None`

Gibt den aktiven Timeout in **Sekunden** zurück. Ist der Timeout deaktiviert, wird None geliefert. subprocess.run/wait behandeln None als unbegrenzt.

`get_all_timeouts() -> dict[str, int]`

Gibt alle Timeout-Werte (Sekunden) als Dict zurück.

`get_all_timeout_enabled() -> dict[str, bool]`

Gibt alle Timeout-Aktivstatuswerte als Dict zurück.

`save_timeout(key, seconds) -> None`

Speichert einen einzelnen Timeout-Wert (Sekunden) in QSettings.

`save_timeout_enabled(key, enabled) -> None`

Speichert den Aktivstatus eines Timeouts in QSettings.

`save_all_timeouts(values, enabled) -> None`

Speichert mehrere Timeout-Werte auf einmal.

`reset_all_timeouts() -> None`

Setzt alle Timeouts auf ihre Standardwerte zurück.

`get_default(key) -> int`

Gibt den Standardwert in Sekunden zurück.

15.2.40.6 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / ffprobe / hdr10plus_tool / mediainfo / mkvextract / mkvmerge / mkvpropedit / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.40.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.41 dragontools/core/tool_diagnostics.py

Merkmal	Wert
Kategorie	Core

Merkmal	Wert
Umfang	387 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (387 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.41.1 Aufgabe des Moduls

Prüft Toolversionen und Fähigkeiten für F9.

15.2.41.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `path`, `args`, `cmd`, `timeout`, `text`, `tool_name`, `tool_paths`, `key`, `name`, `ok`, `detail`, `rows`

15.2.41.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `bool`, `tuple[int, str]`, `str`, `list[str]`, `dict[str, Any]`, `list[dict[str, Any]]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/process_runner.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mkvextract, mkvmerge, mp4box
Direkt verwendet von	dragontools/core/diagnostic_package.py, dragontools/gui/main_window_actions.py, dragontools/tests/test_tool_diagnostics.py

15.2.41.4 Wichtige Funktionen

`probe_tool(tool_name, path) -> dict[str, Any]`

`build_tool_diagnostics(tool_paths) -> list[dict[str, Any]]`

`run_extended_system_test(tool_paths) -> list[dict[str, Any]]`

Praktischer Mini-Systemtest mit temporären Testdateien. Der Test führt keine produktive Konvertierung aus. Er erzeugt eine 1-Sekunden-Testdatei und nutzt sie für einfache ffprobe/mkvmerge/MP4Box-

`format_tool_diagnostics(rows) -> str`

`format_extended_system_test(rows) -> str`

15.2.41.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → dovi_tool / ffmpeg / ffprobe / handbrake / hdr10plus_tool / makemkvcon / mediainfo / mkvextract / mkvmerge / mp4box → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.41.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (IndexError, TypeError, ValueError), Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.42 dragontools/core/type_utils.py

Merkmal	Wert

Merkmal	Wert
Kategorie	Core
Umfang	48 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (48 Zeilen, direkt von 3 Projektmodulen importiert).

15.2.42.1 Aufgabe des Moduls

Gemeinsame, zustandslose Typkonvertierungs-Hilfsfunktionen für DragonTools. Technischer Kontext aus dem Quellcode: Gemeinsame, zustandslose Typkonvertierungs-Hilfsfunktionen für DragonTools. Kein Import von PyQt6, kein Import aus anderen dragontools-Modulen – diese Datei darf von überall im Paket importiert werden, ohne Zirkularitäts- oder Abhängigkeitsprobleme zu verursachen. Hintergrund: _safe_int und _safe_float waren in core/models.py und core/media_analyzer.py iden…

15.2.42.2 Erwartete Informationen / Eingaben

- fachliche Eingabedaten des aufrufenden Moduls
- Wichtige Parameter im Code: `value`, `default`

15.2.42.3 Rückgabe / Ausgabe

- fachliche Datenobjekte, Dictionaries/Listen, Pfade oder Statuswerte
- Explizite Rückgabetypen im Code: `int | None`, `float`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/media_analyzer.py, dragontools/core/models.py, dragontools/tests/test_extended.py

15.2.42.4 Wichtige Funktionen

`_safe_int(value, default) -> int | None`

Konvertiert *value* sicher nach int. Gibt *default* zurück wenn: - value ist None, leer oder "N/A"

`_safe_float(value, default) -> float`

Konvertiert *value* sicher nach float. Gibt *default* zurück wenn die Konvertierung fehlschlaegt.

15.2.42.5 Datenverarbeitung und Kommunikation

Aufruferdaten → Normalisierung/Validierung → fachliche Verarbeitung → strukturiertes Ergebnis, Pfad, Bericht oder Konfigurationswert.

15.2.42.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.2.43 dragontools/core/gpu_detection.py

15.2.43.1 Aufgabe des Moduls

Qt-unabhängige GPU-Erkennung für die automatische Encoderwahl. Unter Windows werden Display-Adapter direkt aus der Registry gelesen; auf Linux dient lspci als Fallback.

15.2.43.2 Erwartete Informationen / Eingaben

Betriebssystem und lokal erkannte Display-Adapter. Die Erkennung benötigt unter Windows keinen wmic-Aufruf und öffnet kein Konsolenfenster.

15.2.43.3 Rückgabe / Ausgabe

GpuInfo-Objekte mit Name, Hersteller, Encoderfamilie und iGPU-Markierung sowie best_encoder() und eine lesbare Zusammenfassung für die GUI.

15.2.43.4 Wichtige Klassen und Funktionen

GpuInfo; detect_gpus(); best_encoder(); gpu_summary().

15.2.43.5 Datenverarbeitung und Kommunikation

Registry/lspci -> Herstellererkennung -> GpuInfo-Liste -> Priorisierung dGPU NVIDIA > AMD > Intel > iGPU > CPU -> GUI/Encoderwahl.

15.2.43.6 Fehlerbehandlung und Diagnose

Fehler der optionalen Hardwareerkennung führen sicher zu einer leeren GPU-Liste bzw. CPU-Fallback; produktive Encoder werden zusätzlich über FFmpeg/Tooldiagnose geprüft.

15.3 GUI

120 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.3.1 dragontools/gui/__init__.py

Merkmal	Wert
Kategorie	GUI
Umfang	2 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (2 Zeilen, direkte Importnutzer: 0).

15.3.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.3.1.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.1.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	Kein direkter statischer Importnutzer; möglich sind Einstiegspunkt, Lazy-Import oder reine Paketfunktion.

15.3.1.4 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.2 dragontools/gui/audio_muxer_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	281 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (281 Zeilen, direkte Importnutzer: 1).

15.3.2.1 Aufgabe des Moduls

Created on Thu Apr 16 01:49:23 2026 Technischer Kontext aus dem Quellcode: Created on Thu Apr 16 01:49:23 2026 @author: DragonTools Team

15.3.2.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.2.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/audio_mux_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.2.4 Wichtige Klassen und Methoden

_DropListWidget [QListWidget]

`dragEnterEvent(event)`
`dragMoveEvent(event)`

`dropEvent(event)`

AudioMuxerWidget [QWidget]

`__init__(parent)`

15.3.2.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.2.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.3 dragontools/gui/audio_video_matcher_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	366 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (366 Zeilen, direkte Importnutzer: 1).

15.3.3.1 Aufgabe des Moduls

Stellt den sichtbaren Tab für Quellvideo, Zielvideo, deutsche Audiospur, Prüfoptionen und Ergebnisanzeige bereit.

15.3.3.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.3.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audio_video_matcher.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/info_button.py, dragontools/worker/audio_video_match_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.3.4 Wichtige Klassen und Methoden

AudioVideoMatcherWidget [QWidget]

`__init__(parent) -> None`

15.3.3.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.3.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.4 dragontools/gui/batch_preflight_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	254 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (254 Zeilen, direkte Importnutzer: 1).

15.3.4.1 Aufgabe des Moduls

Enthält BatchPreflightDialog.

15.3.4.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `rows`, `parent`

15.3.4.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/batch_preflight.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget_queue_actions.py

15.3.4.4 Wichtige Klassen und Methoden

BatchPreflightDialog [QDialog]

```
action() -> str
accepted_files() -> list[str]
skipped_files() -> list[str]
reject() -> None
```

15.3.4.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.4.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError, IndexError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.5 dragontools/gui/changelog_dialog.py

Merkmal	Wert
Kategorie	GUI

Merkmal	Wert
Umfang	234 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (234 Zeilen, direkte Importnutzer: 1).

15.3.5.1 Aufgabe des Moduls

dragontools/gui/changelog_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/changelog_dialog.py Changelog-Dialog wie im Screenshot: Tab-Buttons oben in zwei Reihen, farblich hervorgehoben, Zurück/Weiter-Navigation.

15.3.5.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `version_key`, `text`, `parent`

15.3.5.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.5.4 Wichtige Klassen und Methoden

ChangelogDialog [QDialog]

```
__init__(parent, version_key)
```

15.3.5.5 Wichtige Funktionen

```
_find_changelog(version_key)
_changelog_title(version_key) -> str
_parse_sections(text)
```

15.3.5.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.5.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.6 dragontools/gui/conversion_controller.py

Merkmal	Wert
Kategorie	GUI

Merkmal	Wert
Umfang	898 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (898 Zeilen, direkt von 2 Projektmodulen importiert).

15.3.6.1 Aufgabe des Moduls

dragontools/gui/conversion_controller.py Technischer Kontext aus dem Quellcode: dragontools/gui/conversion_controller.py Worker-Steuerung: Start, Pause, Abort sowie Fortschrittsanzeige. Die Logik arbeitet über explizite Dependency Injection statt über ein owner-basiertes Widget-Muster.

15.3.6.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.6.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/disk_space.py, dragontools/core/job_journal.py, dragontools/core/parallel_settings.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/conversion_result_service.py, dragontools/gui/conversion_session_state.py, dragontools/gui/convert_widget_custom_widgets.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/move_preflight_controller.py, dragontools/gui/preflight_dialog.py, dragontools/gui/ui_helpers.py, dragontools/rules/rule_loader.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/parallel_converter_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/tests/test_conversion_controller_start.py

15.3.6.4 Wichtige Klassen und Methoden

ConversionController [object] – Steuert den Conversion-Worker-Lifecycle und die Fortschrittsanzeige. Alle externen Abhängigkeiten werden über den Konstruktor injiziert – kein Zugriff auf ein »owner«-Widget.

```
active_worker()
get_start_files() -> list[str]
start_convert() -> None
start_move_only() -> None
start_dv_remux() -> None
```

```
toggle_pause() -> None
is_file_active(path) -> bool
terminate_current_ffmpeg(path) -> bool
abort() -> None
on_total_progress(pct) -> None
on_file_progress(path, pct, eta_s) -> None
running_job_diagnostics() -> str
```

15.3.6.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.6.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.7 dragontools/gui/conversion_result_service.py

Merkmal	Wert
Kategorie	GUI
Umfang	429 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (429 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.7.1 Aufgabe des Moduls

dragontools/gui/conversion_result_service.py Technischer Kontext aus dem Quellcode: dragontools/gui/conversion_result_service.py Ergebnisverarbeitung: pro-Datei-Callbacks und Run-Abschluss. Vollständig vom owner-basierten Muster entkoppelt; alle Abhängigkeiten werden per Dependency Injection übergeben.

15.3.7.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.7.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/run_summary.py, dragontools/gui/conversion_session_state.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/run_summary_dialog.py, dragontools/gui/ui_helpers.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/convert_widget.py, dragontools/tests/test_conversion_result_service.py

15.3.7.4 Wichtige Klassen und Methoden

ConversionResultService [object] – Verarbeitet Worker-Ergebnisse und koordiniert den Run-Abschluss. Verantwortlichkeiten: - on_file_result – Slot für worker.file_result (Status pro Datei)

```
set_file_progress_handler(handler) -> None
on_file_progress(path, pct, eta_s) -> None
on_file_result(input_path, output_path, status) -> None
on_finished() -> None
finalize_run(finished_thread, move_log, did_shutdown, move_ok, move_errors) -> None
log_run_summary(finished_thread, move_log, move_ok, move_errors) -> None
```

15.3.7.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.7.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.8 dragontools/gui/conversion_session_state.py

Merkmal	Wert
Kategorie	GUI
Umfang	161 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (161 Zeilen, direkt von 6 Projektmodulen importiert).

15.3.8.1 Aufgabe des Moduls

dragontools/gui/conversion_session_state.py Technischer Kontext aus dem Quellcode: dragontools/gui/conversion_session_state.py Zentraler Laufzeitzustand einer Konvertierungs-Session. Alle transienten Zustandsfelder, die früher direkt auf ConvertWidget lagen, sind hier gebündelt. Keine PyQt6-Abhängigkeiten.

15.3.8.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.8.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/conversion_result_service.py, dragontools/gui/convert_widget.py, dragontools/gui/move_preflight_controller.py, dragontools/tests/test_conversion_result_service.py, dragontools/tests/test_incremental_move_queue_edit.py

15.3.8.4 Wichtige Klassen und Methoden

ConversionSessionState [object] – Kapselt den gesamten veränderlichen Zustand eines Konvertierungs-Laufs. Instanz wird einmalig in ConvertWidget erzeugt und per Dependency Injection an alle Controller weitergereicht. Controller lesen und

```
reset_for_run(file_count) -> None
clear_all() -> None
```

15.3.8.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.8.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.9 dragontools/gui/convert_queue_window.py

Merkmal	Wert
Kategorie	GUI
Umfang	199 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (199 Zeilen, direkte Importnutzer: 1).

15.3.9.1 Aufgabe des Moduls

Enthält QueueWindowListWidget, ConvertQueueWindow.

15.3.9.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `owner`

15.3.9.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/gui/convert_widget_file_queue.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget_queue_actions.py

15.3.9.4 Wichtige Klassen und Methoden

QueueWindowListWidget [FileListWidget]

```
dragEnterEvent(e)
dragMoveEvent(e)
dropEvent(e)
```

ConvertQueueWindow [QWidget]

```
schedule_refresh_from_owner(delay_ms) -> None
refresh_from_owner() -> None
closeEvent(event)
```

15.3.9.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.9.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.10 dragontools/gui/convert_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	593 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (593 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.10.1 Aufgabe des Moduls

dragontools/gui/convert_widget.py Technischer Kontext aus dem Quellcode: dragontools/gui/convert_widget.py Hauptwidget für die Konvertierung – als schlanker Orchestrator mit klar abgegrenzten Zuständigkeiten. Einzige Verantwortlichkeiten des Widget: 1. UI aufbauen -> ConvertWidgetLayoutBuilder 2. Controller instanziieren -> ConversionController, ConversionResultService, MovePreflightController 3. Signale verbinden ->…

15.3.10.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `default_codec`, `parent`

15.3.10.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/batch_preflight.py, dragontools/core/logger.py, dragontools/core/paths.py, dragontools/core/profile_manager.py, dragontools/core/settings.py, dragontools/core/system_shutdown.py, dragontools/gui/conversion_controller.py, dragontools/gui/conversion_result_service.py, dragontools/gui/conversion_session_state.py, dragontools/gui/convert_widget_file_queue.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/convert_widget_override_dialog.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/gui/encoder_profile_service.py, dragontools/gui/encoder_settings_controller.py, dragontools/gui/encoder_settings_state.py, dragontools/gui/encoder_settings_ui.py, dragontools/core/gpu_detection.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py
Verwendete Settings-Konstanten	SET_KEY_PATH_ANIME, SET_KEY_PATH_AV1_ANIME, SET_KEY_PATH_AV1_FILME, SET_KEY_PATH_AV1_TV, SET_KEY_PATH_FILME, SET_KEY_PATH_H264_ANIME, SET_KEY_PATH_H264_FILME, SET_KEY_PATH_H264_TV, SET_KEY_PATH_H265_ANIME, SET_KEY_PATH_H265_FILME, SET_KEY_PATH_H265_TV, SET_KEY_PATH_TV

15.3.10.4 Wichtige Klassen und Methoden

ConvertWidget [ConvertWidgetQueueActionsMixin, QWidget]

```
file_list() -> FileListWidget
restore_job_files(paths) -> dict[str, int]
tv_path() -> str | None
anime_path() -> str | None
filme_path() -> str | None
running_job_diagnostics() -> str
```

15.3.10.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.10.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.11 dragontools/gui/convert_widget_custom_widgets.py

Merkmal	Wert
Kategorie	GUI
Umfang	361 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (361 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.11.1 Aufgabe des Moduls

dragontools/gui/convert_widget_custom_widgets.py Technischer Kontext aus dem Quellcode: dragontools/gui/convert_widget_custom_widgets.py Ausgelagerte Custom-Widgets für das ConvertWidget: BannerLabel : Proportionales Banner-Bild DragonProgressBar : Feuer-Gradient Progressbar mit Glow PathRow : Pfad-Zeile mit Aktiv-Checkbox / Browse-Button Dazu kommen die kleinen Helfer: _find_banner_single(): sucht die banner.png an bekannten O…

> *Ausgelagerte Custom-Widgets für das ConvertWidget: - BannerLabel : Proportionales Banner-Bild - DragonProgressBar : Feuer-Gradient Progressbar mit Glow*

15.3.11.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `s`, `parent`, `label`, `key`, `info`

15.3.11.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`, `str | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/gui/info_button.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/convert_widget_file_queue.py, dragontools/gui/convert_widget_layout.py

15.3.11.4 Wichtige Klassen und Methoden

BannerLabel [QLabel] – Zeigt ein Pixmap proportional an, ohne Verzerrung. Standardmodus: Bild komplett sichtbar einpassen. Optional: auf Höhe einpassen, falls ein schmaler Banner-Effekt gewünscht ist.

```
setBannerPixmap(pixmap) -> None
setScaleMode(mode) -> None
sourceAspectRatio() -> float
resizeEvent(event)
```

DragonProgressBar [QProgressBar] – Custom ProgressBar mit: - dunklem Track - Feuer-Gradient

```
paintEvent(event)
```

PathRow [QWidget] – Pfad-Zeile: [check] [TV:] [___Pfad___] [...] [i] Status wird NUR über QStackedWidget gesteuert: Index 0 = Pfad-LineEdit (aktiv)

```
set_text(text) -> None
text() -> str
active() -> bool
path() -> str | None
```

15.3.11.5 Wichtige Funktionen

```
_eta(s) -> str
```

Formatiert Sekunden als H:MM:SS bzw. MM:SS (oder '' bei <=0).

```
_find_banner_single() -> str | None
```

Sucht die banner.png in den bekannten Ressourcen-Ordnern.

15.3.11.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.11.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.12 dragontools/gui/convert_widget_encoder_override.py

15.3.12.1 Aufgabe des Moduls

Kapselt den manuellen per-Datei-Encoder-/Skalierungs-Override der Konverter-Queue. Der Helper baut die Dialoggruppe, validiert und persistiert die Auswahl und kann denselben Override gezielt auf mehrere markierte wartende Dateien anwenden. Queue-Edit-Guard und Logging werden als Callbacks injiziert, damit der Helper nicht auf private Methoden anderer GUI-Objekte zugreift.

15.3.12.2 Erwartete Informationen / Eingaben

- Aktueller Codec-Tab und globale Encoder-/Skalierungswerte des ConvertWidget.
- Bestehendes per-Datei-Override einschließlich gespeichertem Encoderprofil sowie optionalem manuellen `encoder_override`.
- Ein oder mehrere ausgewählte Queue-Pfade; nur wartende bzw. editierbare Dateien dürfen geändert werden.

15.3.12.3 Rückgabe / Ausgabe

- Persistiert `encoder_override` mit Zielcodec, Backend (CPU/NVENC/QSV/AMF), Qualitätswert, Preset, `scale_mode` und backend-spezifischen Optionen.
- Aktualisiert wartende Worker-Overrides, Queue-Badges und den normalen Benutzerlog, ohne Audio-, Untertitel-, IMAX-, DV/HDR- oder Remux-Overrides zu überschreiben.

15.3.12.4 Wichtige Klassen und Methoden

EncoderOverrideDialogHelper – Qt-Helfer für Aufbau, Persistenz, Zusammenfassung und Mehrfachanwendung des manuellen Encoder-/Skalierungs-Overrides.

```
build_group(...)
persist_group(...)
edit_paths(paths)
summary_text(...)
```

15.3.12.5 Datenverarbeitung und Kommunikation

Queue-Auswahl + globale Vorgaben + vorhandenes Datei-Override → Dialog/Validierung → normalisiertes `encoder_override` → FileOverride-Speicher/Worker-Update → Queue-Badge und Log. Die eigentliche Worker-Auswertung erfolgt Qt-unabhängig in `core/encoder_profile_override.py`.

15.3.12.6 Fehlerbehandlung und Diagnose

Nicht editierbare laufende oder bereits abgeschlossene Dateien werden nicht still verändert. Teilweise abgelehnte Mehrfachauswahlen werden sichtbar gemeldet. Ungültige Werte werden normalisiert bzw. verworfen; Architekturtests verhindern private Cross-Object-Aufrufe.

15.3.13 dragontools/gui/convert_widget_file_queue.py

Merkmal	Wert
Kategorie	GUI
Umfang	668 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (668 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.13.1 Aufgabe des Moduls

Enthält FileListWidget, ConvertWidgetFileQueueHelper.

15.3.13.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `p`, `owner`

15.3.13.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/gui/convert_widget_custom_widgets.py, dragontools/gui/drop_path_extractor.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_queue_window.py, dragontools/gui/convert_widget.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/convert_widget_queue_actions.py

15.3.13.4 Wichtige Klassen und Methoden

FileListWidget [QListWidget] – Dateiliste mit zwei Drag-&-Drop-Modi: 1. Dateien/Ordner von außerhalb hineinziehen -> neue Dateien hinzufügen 2. Items innerhalb der Liste verschieben -> Reihenfolge aendern (Queue-Sortierung)

```
set_edit_locked(locked) -> None
dragEnterEvent(e)
dragMoveEvent(e)
dropEvent(e)
get_paths() -> list[str]
rebuild_path_index() -> None
item_for_path(path)
remove_path(path) -> bool
clear() -> None
```

```
resizeEvent(e)
keyPressEvent(e)
```

ConvertWidgetFileQueueHelper [object]

```
collect_video_paths_from_mime_data(mime) -> list[str]
collect_video_paths_from_urls(urls) -> list[str]
on_files_dropped(paths) -> None
add_files() -> None
add_folder() -> None
sync_queue_order() -> None
remove_path(path) -> None
remove_paths(paths) -> None
remove_selected() -> None
clear() -> None
```

15.3.13.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.13.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.14 dragontools/gui/convert_widget_layout.py

Merkmal	Wert
Kategorie	GUI
Umfang	543 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (543 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.14.1 Aufgabe des Moduls

dragontools/gui/convert_widget_layout.py Technischer Kontext aus dem Quellcode: dragontools/gui/convert_widget_layout.py Layout-Aufbau für ConvertWidget. Enthält: ConvertWidgetUI – Lightweight-Dataclass mit allen UI-Widget-Referenzen. Wird nach dem Build befüllt und per DI an Controller weitergegeben. Kein QObject – reines Data-Bundle. ConvertWidgetLayoutBuilder – Erzeugt alle Widgets, fügt sie in das Scroll-Layout ein und setzt…

15.3.14.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `title`, `parent`, `widget`, `default_codec`, `settings`, `enc_settings`

15.3.14.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `ConvertWidgetUI`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/gui/convert_widget_custom_widgets.py, dragontools/gui/convert_widget_file_queue.py, dragontools/gui/info_button.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	dovi_tool, hdr10plus_tool, mp4box
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/conversion_result_service.py, dragontools/gui/convert_widget.py, dragontools/gui/move_preflight_controller.py
Verwendete Settings-Konstanten	SET_KEY_AUTOCROP_ENABLED, SET_KEY_IMAX_DETECT

15.3.14.4 Wichtige Klassen und Methoden

CollapsibleGroupBox [QWidget] – Schlanker einklappbarer Bereich für PyQt6. Vorteil gegenüber einer checkable QGroupBox: - kein störendes Checkbox-Layout im Titel

```
addLayout(layout) -> None
addWidget(widget) -> None
setCollapsed(collapsed) -> None
```

FileListBannerOverlay [QWidget]

```
__init__(parent) -> None
```

ConvertWidgetUI [object] – Lightweight-Datenklasse mit direkten Referenzen auf alle relevanten UI-Widgets von ConvertWidget. Wird nach ConvertWidgetLayoutBuilder.build() instanziiert und per

ConvertWidgetLayoutBuilder [object] – Verantwortlich für den gesamten UI-Aufbau von ConvertWidget. Nimmt das Widget als Ziel entgegen, erstellt alle Unter-Layouts und Widgets, setzt sie als Attribute auf dem Widget und gibt abschließend

```
build() -> ConvertWidgetUI
```

15.3.14.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.14.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.15 dragontools/gui/convert_widget_override_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	729 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (729 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.15.1 Aufgabe des Moduls

dragontools/gui/convert_widget_override_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/convert_widget_override_dialog.py Ausgelagerte Override-Dialog-Logik für das ConvertWidget. Enthaelt: _OverrideAnalyzeThread : Hintergrund-Thread für Media-Analyse _OverrideDialog : QDialog-Subklasse mit sauberem closeEvent ConvertWidgetOverrideDialogHelper : Helper-Klasse, die vom ConvertWidget aus aufgerufen wird (edit_override). Di…

> *Ausgelagerte Override-Dialog-Logik für das ConvertWidget. Enthaelt: - _OverrideAnalyzeThread : Hintergrund-Thread für Media-Analyse*

15.3.15.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `value`, `stream`, `ov`, `layout`, `audio_streams`, `audio_track_map`, `old_audio_action`, `audio_rows`, `subtitle_streams`, `subtitle_track_map`, `old_burn_mode`, `old_burn_stream_index`, `subtitle_rows`, `path`, `tools`, `parent`

15.3.15.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`, `dict`, `None`, `list`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/settings.py, dragontools/gui/info_button.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/convert_widget.py
Verwendete Settings-Konstanten	SET_KEY_PRESERVE_DV, SET_KEY_PRESERVE_HDRPLUS

15.3.15.4 Wichtige Klassen und Methoden

_OverrideAnalyzeThread [QThread] – Führt analyze_media() im Hintergrund aus damit der Dialog nicht blockt.

```
abort() -> None
run() -> None
```

_OverrideDialog [QDialog] – Dialog, der beim Schliessen einen evtl. laufenden Loader korrekt beendet.

```
closeEvent(event) -> None
```

ConvertWidgetOverrideDialogHelper [object] – Bündelt die gesamte Dialog-Logik für die Datei-Override-Einstellungen. Benutzt das übergeordnete ConvertWidget (owner) für Logging, Tools und den laufenden Thread.

```
edit_override(path) -> None
```

15.3.15.5 Wichtige Funktionen

```
_lang_label(value) -> str
_audio_meta_text(stream) -> str
_load_override_state(ov) -> dict
```

Baut aus dem persistierten Override-Dict den Initialzustand für den Dialog.

```
_clear_layout_widgets(layout) -> None
_build_audio_rows(layout, audio_streams, audio_track_map, old_audio_action, audio_rows) -> None
```

```
_build_subtitle_rows(layout, subtitle_streams, subtitle_track_map, old_burn_mode, old_burn_stream_index,
subtitle_rows) -> None
```

15.3.15.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.15.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, RuntimeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.16 dragontools/gui/convert_widget_queue_actions.py

Merkmal	Wert
Kategorie	GUI
Umfang	655 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (655 Zeilen, direkt von 2 Projektmodulen importiert).

15.3.16.1 Aufgabe des Moduls

Enthält ConvertWidgetQueueActionsMixin.

15.3.16.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.16.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/batch_preflight.py, dragontools/core/encoder_profile_override.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/batch_preflight_dialog.py, dragontools/gui/convert_queue_window.py, dragontools/gui/convert_widget_file_queue.py, dragontools/gui/media_info_dialog.py, dragontools/gui/ui_helpers.py, dragontools/rules/move_rules.py, dragontools/worker/source_visual_check.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg, ffprobe
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/tests/test_per_file_strip_only.py

Abhängigkeit	Details
Verwendete Settings-Konstanten	SET_KEY_NFO_ENABLED, SET_KEY_TRICKPLAY_ENABLED

15.3.16.4 Wichtige Klassen und Methoden

ConvertWidgetQueueActionsMixin [object]

```
dragEnterEvent(e)
dragMoveEvent(e)
eventFilter(obj, event)
dropEvent(e)
open_queue_window() -> None
show_batch_rule_test() -> None
```

15.3.16.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.16.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.17 dragontools/gui/drop_path_extractor.py

Merkmal	Wert
Kategorie	GUI
Umfang	469 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (469 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.17.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _log_drop_message, _warn_non_video_file, _iter_video_files_in_folder, _mime_has_file_payload.

15.3.17.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `log_fn`, `message`, `path`, `folder_path`, `mime`, `method_name`, `widget`, `url`, `raw_url`, `text`, `data`, `offset`, `key`, `visible`

15.3.17.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`, `tuple[list[str], int]`, `bool`, `object`, `str`, `list[str]`, `bytes`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.

Abhängigkeit	Details
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget_file_queue.py, dragontools/gui/movie_renamer_widget.py, dragontools/gui/quality_tester_widget.py

15.3.17.4 Wichtige Funktionen

```
_log_drop_message(log_fn, message) -> None
_warn_non_video_file(log_fn, path) -> None
_iter_video_files_in_folder(folder_path) -> tuple[list[str], int]
_mime_has_file_payload(mime) -> bool
_debug_mime_data(log_fn, method_name, mime) -> None
_resolve_drop_logger(widget) -> object
```

15.3.17.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.17.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (AttributeError, OSError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.18 dragontools/gui/encoder_profile_service.py

Merkmal	Wert
Kategorie	GUI
Umfang	117 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (117 Zeilen, direkte Importnutzer: 2).

15.3.18.1 Aufgabe des Moduls

Enthält EncoderProfileService.

15.3.18.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.18.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audit_log.py, dragontools/core/codec_profile_assistant.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/tests/test_gui_dialog_services.py

15.3.18.4 Wichtige Klassen und Methoden

EncoderProfileService [object]

```
save_profile_dialog(payload) -> None
load_profile_dialog() -> dict | None
assistant_profile_dialog(codec) -> dict | None
```

15.3.18.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.18.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.19 dragontools/gui/encoder_settings_controller.py

Merkmal	Wert
Kategorie	GUI
Umfang	561 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (561 Zeilen, direkt von 2 Projektmodulen importiert).

15.3.19.1 Aufgabe des Moduls

Enthält EncoderSettingsController.

15.3.19.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.19.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/gui/encoder_settings_state.py, dragontools/gui/encoder_settings_ui.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/tests/test_encoder_settings_shutdown.py

Abhängigkeit	Details
Verwendete Settings-Konstanten	SET_KEY_AUTOCROP_MODE, SET_KEY_AUTOCROP_PROBE_DURATION, SET_KEY_AUTOCROP_PROBE_INTERVAL, SET_KEY_AUTOCROP_PROBE_START, SET_KEY_IMAX_MIN_HITS, SET_KEY_IMAX_MIN_VARIANCE_PERCENT, SET_KEY_IMAX_PROBE_DURATION, SET_KEY_IMAX_PROBE_INTERVAL

15.3.19.4 Wichtige Klassen und Methoden

EncoderSettingsController [object]

```
build_nvenc_panel()
build_qsv_panel()
build_amf_panel()
build_x265_panel()
bind_main_widgets() -> None
enc_key() -> str
refresh_enc_panel()
detect_encoders()
collect_enc_opts() -> dict
connect_encoder_settings_signals() -> None
save_encoder_settings() -> None
load_encoder_settings() -> None
```

15.3.19.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.19.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.20 dragontools/gui/encoder_settings_state.py

Merkmal	Wert
Kategorie	GUI
Umfang	24 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (24 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.20.1 Aufgabe des Moduls

Enthält EncoderSettingsState.

15.3.20.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.20.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/gui/encoder_settings_controller.py, dragontools/tests/test_encoder_settings_shutdown.py

15.3.20.4 Wichtige Klassen und Methoden

EncoderSettingsState [object]

15.3.20.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.20.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.21 dragontools/gui/encoder_settings_ui.py

Merkmal	Wert
Kategorie	GUI
Umfang	345 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (345 Zeilen, direkte Importnutzer: 2).

15.3.21.1 Aufgabe des Moduls

Enthält EncoderSettingsWidgets, EncoderSettingsUI.

15.3.21.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.21.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/gui/info_button.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg

Abhängigkeit	Details
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/gui/encoder_settings_controller.py

15.3.21.4 Wichtige Klassen und Methoden

EncoderSettingsWidgets [object]

EncoderSettingsUI [object]

```
bind_main_widgets() -> None
build_nvenc_panel()
build_qsv_panel()
build_amf_panel()
build_x265_panel()
```

15.3.21.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.21.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.22 dragontools/gui/dv_crop_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	175 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (175 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.22.1 Aufgabe des Moduls

Zeigt den Benutzerentscheid an, wenn FFmpeg-AutoCrop und Dolby-Vision-RPU-Level-5 um mehr als 20 Pixel voneinander abweichen. Der Dialog verändert die Datei nicht selbst, sondern liefert die Entscheidung an den Worker zurück.

15.3.22.2 Erwartete Informationen / Eingaben

Request-ID, Quellpfad, AutoCrop-Wert, RPU-Crop, maximale Abweichung und Worker-Referenz.

15.3.22.3 Rückgabe / Ausgabe

Eine der Entscheidungen autocrop, rpu oder disable_dv wird über provide_dv_crop_decision() an den Worker zurückgegeben.

15.3.22.4 Wichtige Funktionen

show_dv_crop_decision(worker, payload, log=None) -> None

15.3.22.5 Datenverarbeitung und Kommunikation

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.

Abhängigkeit	Details
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget.py, dragontools/gui/main_window_actions.py, dragontools/worker/converter_thread.py, dragontools/worker/parallel_converter_thread.py

Worker-Crop-Konflikt -> QMessageBox -> Benutzerwahl -> Worker-Entscheidung -> DV-Pipeline setzt physischen Crop/RPU passend fort oder plant die Datei ohne DV-Erhalt neu.

15.3.22.6 Fehlerbehandlung und Diagnose

Fehlt die Request-ID, passiert bewusst nichts. Kann die Entscheidung nicht an den Worker übergeben werden, wird der Fehler geloggt; der Dialog führt keine destruktive Dateioperation aus.

15.3.23 dragontools/gui/help_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	106 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (106 Zeilen, direkte Importnutzer: 1).

15.3.23.1 Aufgabe des Moduls

dragontools/gui/help_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/help_dialog.py WICHTIG: setSource() NICHT verwenden - das navigiert zur URL und überschreibt den Inhalt (weißes/graues leeres Fenster). Stattdessen: document().setBaseUrl() + setHtml()

15.3.23.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.23.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.23.4 Wichtige Klassen und Methoden

HelpDialog [QDialog]

```
__init__(parent)
```

15.3.23.5 Wichtige Funktionen

```
_find_help() -> str | None
```

15.3.23.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.23.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.24 dragontools/gui/info_button.py

Merkmal	Wert
Kategorie	GUI
Umfang	43 Quellzeilen
Bedeutung	Zentraler Baustein (43 Zeilen, direkt von 11 Projektmodulen importiert).

15.3.24.1 Aufgabe des Moduls

dragontools/gui/info_button.py Technischer Kontext aus dem Quellcode: dragontools/gui/info_button.py Kleines ℹ️-Icon das einen Tooltip-ähnlichen Popup zeigt. Einheitlich für alle Felder nutzbar: InfoButton("Erklärungstext")

15.3.24.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `label`, `widget`, `info`, `text`, `parent`

15.3.24.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `QWidget`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/gui/audio_video_matcher_widget.py, dragontools/gui/convert_widget_custom_widgets.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/convert_widget_override_dialog.py, dragontools/gui/encoder_settings_ui.py, dragontools/gui/quality_tester_widget.py, dragontools/gui/rules_audio_tab.py, dragontools/gui/rules_series_tab.py, dragontools/gui/rules_subtitle_tab.py, dragontools/gui/settings_dialog.py, dragontools/gui/timeout_settings_dialog.py

15.3.24.4 Wichtige Klassen und Methoden

InfoButton [QPushButton] – Kleines ℹ️-Icon. Klick oder Hover zeigt den Erklärungstext.

```
__init__(text, parent)
```

15.3.24.5 Wichtige Funktionen

```
labeled_row(label, widget, info) -> QWidget
```

Erstellt eine Zeile: [Label] [Widget] [**i**] Gibt ein QWidget zurück das direkt in ein Layout eingefügt werden kann.

15.3.24.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.24.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.25 dragontools/gui/iso_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	426 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (426 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.25.1 Aufgabe des Moduls

Enthält ISOWidget.

15.3.25.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `path`, `seconds`, `value`, `parent`

15.3.25.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/iso_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/main_window.py

15.3.25.4 Wichtige Klassen und Methoden

ISOWidget [QWidget] – DVD-/Blu-ray-ISO-Handling über MakeMKV CLI mit optionalem FFmpeg-Fallback.

```
__init__(parent) -> None
```

15.3.25.5 Wichtige Funktionen

```
_safe_display_name(path) -> str
_fmt_duration(seconds) -> str
_fmt_size(value) -> str
```

15.3.25.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.25.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.26 dragontools/gui/job_resume_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	108 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (108 Zeilen, direkte Importnutzer: 1).

15.3.26.1 Aufgabe des Moduls

Fragt beim Start nach, ob ein gesicherter Job wieder aufgenommen werden soll.

15.3.26.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `data`, `parent`

15.3.26.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/job_journal.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/gui/main_window.py

15.3.26.4 Wichtige Klassen und Methoden

JobResumeDialog [QDialog] – Dialog für die Wiederherstellung einer nicht sauber beendeten Queue.

```
action() -> str
resume_plan() -> dict
```

15.3.26.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.26.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.27 dragontools/gui/main_window.py

Merkmal	Wert
Kategorie	GUI
Umfang	492 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (492 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.27.1 Aufgabe des Moduls

dragontools/gui/main_window.py Technischer Kontext aus dem Quellcode: dragontools/gui/main_window.py Vollständiges MainWindow V9: Menü: Datei | Einstellungen | Regeln | Timeouts | Profile | Ansicht | Werkzeuge | Zoom-Fenster | Hilfe Shortcuts: F1 F9 F12 Strg+Q Strg+W Strg+Shift+T Strg+Shift+A Entf Lazy Loading: Tabs werden erst beim ersten Öffnen erzeugt Tab-Manager: welche Tabs sichtbar sind Externe Programme (HandBrake, RMT…

> *Vollständiges MainWindow V9: - Menü: Datei | Einstellungen | Regeln | Timeouts | Profile | Ansicht | Werkzeuge | Zoom-Fenster | Hilfe - Shortcuts: F1 F9 F12 Strg+Q Strg+W Strg+Shift+T Strg+Shift+A Entf*

15.3.27.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `win`

15.3.27.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/job_journal.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/audio_muxer_widget.py, dragontools/gui/audio_video_matcher_widget.py, dragontools/gui/convert_widget.py, dragontools/gui/iso_widget.py, dragontools/gui/job_resume_dialog.py, dragontools/gui/main_window_actions.py, dragontools/gui/main_window_menus.py, dragontools/gui/merge_widget.py, dragontools/gui/movie_renamer_widget.py, dragontools/gui/mp4_remux_widget.py, dragontools/gui/quality_tester_widget.py, dragontools/gui/styles.py, dragontools/gui/subtitle_widget.py, dragontools/gui/tab_manager.py, dragontools/gui/tool_path_settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	handbrake
Direkt verwendet von	DragonToolsV9.py

15.3.27.4 Wichtige Klassen und Methoden

_TabBar [QTabBar]

`mouseReleaseEvent(e)`

MainWindow [MainWindowMenuMixin, MainWindowActionsMixin, QMainWindow]

`closeEvent(e)`

15.3.27.5 Wichtige Funktionen

`bring_to_front(win) -> None`

15.3.27.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.27.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.28 dragontools/gui/main_window_actions.py

Merkmal	Wert
Kategorie	GUI
Umfang	722 Quellzeilen

Merkmal	Wert
Bedeutung	Wichtiger Fachbaustein (722 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.28.1 Aufgabe des Moduls

Enthält MainWindowActionsMixin.

15.3.28.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.28.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/diagnostic_package.py, dragontools/core/online_metadata.py, dragontools/core/paths.py, dragontools/core/profile_manager.py, dragontools/core/release_validation.py, dragontools/core/settings.py, dragontools/core/settings_backup.py, dragontools/core/tool_diagnostics.py, dragontools/gui/changelog_dialog.py, dragontools/core/gpu_detection.py, dragontools/gui/help_dialog.py, dragontools/gui/online_metadata_dialog.py, dragontools/gui/profile_manager_dialog.py, dragontools/gui/rules_dialog.py, dragontools/gui/save_settings_dialog.py, dragontools/gui/settings_dialog.py, dragontools/gui/shortcut_dialog.py, dragontools/gui/styles.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mkvextract, mkvmerge, mp4box
Direkt verwendet von	dragontools/gui/main_window.py

15.3.28.4 Wichtige Klassen und Methoden

MainWindowActionsMixin [object]

15.3.28.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.28.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (OnlineMetadataAuthError, OnlineMetadataError), Exception, OnlineMetadataAuthError, OnlineMetadataError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.29 dragontools/gui/main_window_menus.py

Merkmal	Wert
Kategorie	GUI
Umfang	235 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (235 Zeilen, direkte Importnutzer: 1).

15.3.29.1 Aufgabe des Moduls

Enthält MainWindowMenuMixin.

15.3.29.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.29.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	handbrake
Direkt verwendet von	dragontools/gui/main_window.py

15.3.29.4 Wichtige Klassen und Methoden

MainWindowMenuMixin [object]

15.3.29.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.29.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.30 dragontools/gui/media_info_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	571 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (571 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.30.1 Aufgabe des Moduls

Enthält _MediaInfoLoadThread und MediaInfoDialog. Die umfangreiche DragonTools-/Rules-Preview-Textaufbereitung liegt im aktuellen Stand in media_info_text_builder.py.

15.3.30.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `file_path`, `file_override`, `planned_target`, `subtitle_rules`, `codec`, `parent`

15.3.30.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`, `QTableWidget`, `None`, `QTableWidgetItem`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/mediainfo_details.py, dragontools/core/rules_preview.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	mediainfo
Direkt verwendet von	dragontools/gui/convert_widget_queue_actions.py

15.3.30.4 Wichtige Klassen und Methoden

_MediaInfoLoadThread [QThread]

```
run() -> None
```

MediaInfoDialog [QDialog]

```
closeEvent(event) -> None
```

15.3.30.5 Wichtige Funktionen

```
media_info_text_builder.build_media_info_text(...) -> str
```

15.3.30.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.30.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.31 dragontools/gui/merge_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	345 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (345 Zeilen, direkte Importnutzer: 1).

15.3.31.1 Aufgabe des Moduls

Enthält MergeWidget.

15.3.31.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.31.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/merge_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.31.4 Wichtige Klassen und Methoden

MergeWidget [QWidget] – Lokales Widget für konservatives, verlustfreies Merge.

```
__init__(parent) -> None
```

15.3.31.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.31.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.32 dragontools/gui/move_preflight_controller.py

Merkmal	Wert
Kategorie	GUI
Umfang	753 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (753 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.32.1 Aufgabe des Moduls

dragontools/gui/move_preflight_controller.py Technischer Kontext aus dem Quellcode: dragontools/gui/move_preflight_controller.py Preflight-Dialog und Verschieben-Steuerung. Kein owner-Zugriff; alle Abhängigkeiten werden per Dependency Injection injiziert.

15.3.32.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.32.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/preflight_report.py, dragontools/core/settings.py, dragontools/gui/conversion_session_state.py, dragontools/gui/convert_widget_layout.py, dragontools/gui/preflight_dialog.py, dragontools/gui/ui_helpers.py, dragontools/rules/move_rules.py, dragontools/worker/move_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/convert_widget.py, dragontools/tests/test_incremental_move_queue_edit.py
Verwendete Settings-Konstanten	SET_KEY_MOVE_CONFLICT, SET_KEY_SHUTDOWN_COUNTDOWN

15.3.32.4 Wichtige Klassen und Methoden

MovePreflightController [object] – Steuert Preflight-Dialog (Zielordner-Auswahl) und den MoveThread.

Verantwortlichkeiten: - run_if_needed – Preflight vor einem Run

```
run_if_needed(files) -> bool
maybe_for_new_files(added) -> None
save_report(files, planned_targets, target_paths) -> Path | None
start_move(files, finished_thread) -> None
move_finished_now() -> None
on_move_req(rid, payload) -> None
film_destination_dialog(rid, payload) -> None
```

15.3.32.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.32.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.33 dragontools/gui/movie_renamer_widget.py

Merkmal	Wert
Kategorie	GUI

Merkmal	Wert
Umfang	674 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (674 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.33.1 Aufgabe des Moduls

Bietet die Oberfläche für Film-/Serien-Umbenennung mit Vorschlagsliste, manueller Auswahl und sicherer Konfliktprüfung.

15.3.33.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`, `jobs`, `config`

15.3.33.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/movie_renamer.py, dragontools/core/online_metadata.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/drop_path_extractor.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.33.4 Wichtige Klassen und Methoden

_RenameTable [QTableWidget]

```
dragEnterEvent(event) -> None
dragMoveEvent(event) -> None
dropEvent(event) -> None
```

_MovieRenameResolveThread [QThread]

```
run() -> None
```

MovieRenamerWidget [QWidget]

```
add_paths(paths) -> None
resolve_proposals() -> None
accept_selected() -> None
accept_safe() -> None
reject_selected() -> None
execute_rename() -> None
remove_selected() -> None
clear() -> None
closeEvent(event) -> None
```

15.3.33.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.33.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OnlineMetadataAuthError, OnlineMetadataError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.34 dragontools/gui/mp4_remux_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	342 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (342 Zeilen, direkte Importnutzer: 1).

15.3.34.1 Aufgabe des Moduls

Enthält _DropListWidget, MP4RemuxWidget.

15.3.34.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.34.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/rules/rule_loader.py, dragontools/worker/mp4_remux_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.34.4 Wichtige Klassen und Methoden

_DropListWidget [QListWidget]

```
dragEnterEvent(event)
dragMoveEvent(event)
dropEvent(event)
```

MP4RemuxWidget [QWidget] – Lokales MP4-Remux-Widget ohne DV-Logik.

```
__init__(parent) -> None
```

15.3.34.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.34.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.35 dragontools/gui/online_metadata_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	366 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (366 Zeilen, direkte Importnutzer: 1).

15.3.35.1 Aufgabe des Moduls

GUI für getrennte Online-Metadatenprovider, TMDB-Zugangsdaten und TheTVDB-Zugangsdaten.

15.3.35.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.35.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py
Verwendete Settings-Konstanten	SET_KEY_METADATA_CACHE_DAYS, SET_KEY_METADATA_CACHE_ENABLED, SET_KEY_METADATA_FALLBACK_LANGUAGE, SET_KEY_METADATA_LANGUAGE, SET_KEY_METADATA_MOVIE_PREFERRED_PROVIDER, SET_KEY_METADATA_MOVIE_PROVIDER, SET_KEY_METADATA_SERIES_PREFERRED_PROVIDER, SET_KEY_METADATA_SERIES_PROVIDER, SET_KEY_METADATA_TMDB_API_KEY, SET_KEY_METADATA_TMDB_ENABLED, SET_KEY_METADATA_TMDB_READ_TOKEN, SET_KEY_METADATA_TVDB_API_KEY, SET_KEY_METADATA_TVDB_BEARER_TOKEN, SET_KEY_METADATA_TVDB_ENABLED, SET_KEY_METADATA_TVDB_PIN

15.3.35.4 Wichtige Klassen und Methoden

OnlineMetadataDialog [QDialog] – Einstellungen für optionale Online-Metadatenquellen.

```
__init__(parent) -> None
```

15.3.35.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.35.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.36 dragontools/gui/preflight_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	874 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (874 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.36.1 Aufgabe des Moduls

dragontools/gui/preflight_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/preflight_dialog.py Pre-Flight Dialog: Zeigt vor dem Start alle Zielordner gruppiert an. Gleiche Serien → eine Gruppe (einmal einstellen = alle Folgen) Filme → einzeln Alle Entscheidungen werden als planned_targets gespeichert → MoveThread läuft danach vollständig ohne Rückfragen.

> *Pre-Flight Dialog: Zeigt vor dem Start alle Zielordner gruppiert an. - Gleiche Serien → eine Gruppe (einmal einstellen = alle Folgen) - Filme → einzeln*

15.3.36.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `path`, `p`, `maxlen`, `target_dir`, `relative_subpath`, `base`, `series_name`, `existing_dir`, `series_dir`, `season`, `entries`, `tv_path`, `anime_path`, `default_type`, `parent`, `filme_path`

15.3.36.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `str`, `QFrame`, `str | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/online_metadata.py, dragontools/core/settings.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/move_preflight_controller.py, dragontools/tests/test_incremental_move_queue_edit.py, dragontools/tests/test_preflight_dialog_performance.py

Abhängigkeit	Details
Verwendete Settings-Konstanten	SET_KEY_PREFLIGHT_SAVE_REPORT, SET_KEY_SERIES_DEFAULT_TYPE

15.3.36.4 Wichtige Klassen und Methoden

SeriesGroupWidget [QWidget] – Eine Gruppe für alle Folgen der gleichen Serie. User stellt einmal TV/Anime + Serienname ein → gilt für alle Folgen.

```
metadata_lookup_job() -> tuple[str, str, Any] | None
mark_metadata_lookup_started(online_lookup) -> None
mark_metadata_lookup_failed(message) -> None
mark_existing_series_dir_not_found() -> None
apply_existing_series_dir(existing_dir, base, series_name, base_type) -> None
apply_online_metadata_suggestion(suggestion) -> None
get_planned_targets() -> dict[str, str | dict]
```

FilmWidget [QWidget] – Einzelner Film mit Einzelfilm/Filmreihe Auswahl.

```
metadata_lookup_job() -> tuple[str, str, Any] | None
mark_metadata_lookup_started() -> None
mark_metadata_lookup_failed(message) -> None
apply_online_metadata_suggestion(suggestion) -> None
get_target_path() -> str | None
validate() -> tuple[bool, str | None]
get_planned_targets() -> dict[str, str | dict]
```

PreFlightDialog [QDialog] – Pre-Flight Dialog. Gruppiert gleiche Serien → einmal einstellen. Gibt planned_targets zurück: {input_path: target_dir}

```
reject() -> None
accept() -> None
closeEvent(event) -> None
get_planned_targets() -> dict[str, str | dict]
should_save_report() -> bool
```

15.3.36.5 Wichtige Funktionen

```
_safe_stem(path) -> str
```

Dateiname für die Move-Zielfindung ohne technische Codec-Suffixe.

```
_fmt_path(p, maxlen) -> str
_sep() -> QFrame
_planned_target_entry(target_dir, relative_subpath)
_series_root_from_input(base, series_name, existing_dir) -> str | None
```

Schnelle Serienwurzel für die GUI-Vorschau ohne Netzlaufwerk-Scan.

```
_series_season_target(series_dir, season) -> str | None
```

Schneller Staffel-Zielpfad ohne bestehende Ordner zu durchsuchen.

15.3.36.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.36.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError, queue.Empty. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.37 dragontools/gui/profile_manager_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	151 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (151 Zeilen, direkte Importnutzer: 1).

15.3.37.1 Aufgabe des Moduls

dragontools/gui/profile_manager_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/profile_manager_dialog.py Profilverwaltungsdialog – ausgelagert aus main_window._open_profile_manager(). Vorher: ~100 Zeilen inline-UI-Code direkt in einer Menüaktion des MainWindow. Jetzt: eigenstaendige, wiederverwendbare Dialog-Klasse. Benutzung from .profile_manager_dialog import open_profile_manager open_profile_manager(tabs_widget, pa…

15.3.37.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `tabs`, `parent`, `pm_widget`

15.3.37.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audit_log.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.37.4 Wichtige Klassen und Methoden

ProfileManagerDialog [QDialog] – Zeigt benutzerdefinierte Profile des übergebenen ConvertWidget-Tabs an. Parameters ----------

```
__init__(pm_widget, parent)
```

15.3.37.5 Wichtige Funktionen

```
open_profile_manager(tabs, parent) -> None
```

Öffnet den Profilverwaltungsdialog für den aktuell aktiven Tab. Sucht den ersten Tab mit ``profile_manager``-Attribut. Bevorzugt den gerade sichtbaren Tab. Zeigt einen Hinweis-Dialog wenn kein passender

15.3.37.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.37.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.38 dragontools/gui/quality_tester_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	805 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (805 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.38.1 Aufgabe des Moduls

Stellt den sichtbaren Tab für Encoder-Qualitätstests bereit.

15.3.38.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`, `initial`

15.3.38.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/drop_path_extractor.py, dragontools/gui/info_button.py, dragontools/worker/quality_test_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/main_window.py

15.3.38.4 Wichtige Klassen und Methoden

_QualityFileTable [QTableWidget]

```
dragEnterEvent(event) -> None
dragMoveEvent(event) -> None
dropEvent(event) -> None
```

_QualityRunDialog [QDialog] – Assistent für einen Qualitätstestlauf mit den wichtigsten Encoder-Optionen.

```
values() -> dict
```

QualityTesterWidget [QWidget]

```
__init__(parent) -> None
```

15.3.38.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.38.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.39 dragontools/gui/rules_audio_tab.py

Merkmal	Wert
Kategorie	GUI
Umfang	542 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (542 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.39.1 Aufgabe des Moduls

Enthält _AudioTab.

15.3.39.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.39.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py, dragontools/gui/info_button.py, dragontools/gui/rules_dialog_common.py, dragontools/gui/rules_dialog_storage.py, dragontools/gui/rules_language_editor.py, dragontools/rules/__init__.py, dragontools/rules/audio_plan.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg, handbrake
Direkt verwendet von	dragontools/gui/rules_dialog.py

15.3.39.4 Wichtige Klassen und Methoden

_AudioTab [QWidget]

```
get_data() -> dict
```

15.3.39.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.39.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.40 dragontools/gui/rules_dialog.py

Merkmal	Wert
Kategorie	GUI

Merkmal	Wert
Umfang	82 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (82 Zeilen, direkte Importnutzer: 1).

15.3.40.1 Aufgabe des Moduls

dragontools/gui/rules_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/rules_dialog.py Regeln-Editor: Serien, Audio, Untertitel und Flags.

15.3.40.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.40.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/gui/rules_audio_tab.py, dragontools/gui/rules_dialog_storage.py, dragontools/gui/rules_flags_tab.py, dragontools/gui/rules_series_tab.py, dragontools/gui/rules_subtitle_tab.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.40.4 Wichtige Klassen und Methoden

RulesDialog [QDialog]

```
__init__(parent)
```

15.3.40.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.40.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.41 dragontools/gui/rules_dialog_common.py

Merkmal	Wert
Kategorie	GUI
Umfang	14 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (14 Zeilen, direkte Importnutzer: 1).

15.3.41.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _hr.

15.3.41.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.41.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `QFrame`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_audio_tab.py

15.3.41.4 Wichtige Funktionen

```
_hr() -> QFrame
```

15.3.41.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.41.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.42 dragontools/gui/rules_dialog_storage.py

Merkmal	Wert
Kategorie	GUI
Umfang	47 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (47 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.42.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _rules_dir, _load, _save.

15.3.42.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `name`, `data`

15.3.42.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `Path`, `dict`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audit_log.py, dragontools/core/config_migration.py, dragontools/rules/__init__.py, dragontools/rules/rule_loader.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_audio_tab.py, dragontools/gui/rules_dialog.py, dragontools/gui/rules_series_tab.py, dragontools/gui/rules_subtitle_tab.py

15.3.42.4 Wichtige Funktionen

```
_rules_dir() -> Path
_load(name) -> dict
_save(name, data) -> None
```

15.3.42.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.42.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.43 dragontools/gui/rules_flags_tab.py

Merkmal	Wert
Kategorie	GUI
Umfang	60 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (60 Zeilen, direkte Importnutzer: 1).

15.3.43.1 Aufgabe des Moduls

Enthält _FlagsTab.

15.3.43.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.43.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/subtitle/tagger.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	mkvpropedit
Direkt verwendet von	dragontools/gui/rules_dialog.py

15.3.43.4 Wichtige Klassen und Methoden

_FlagsTab [QWidget]

```
__init__(parent)
```

15.3.43.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.43.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.44 dragontools/gui/rules_language_editor.py

Merkmal	Wert
Kategorie	GUI
Umfang	157 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (157 Zeilen, direkte Importnutzer: 2).

15.3.44.1 Aufgabe des Moduls

Enthält _LanguagePriorityEditor.

15.3.44.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.44.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_audio_tab.py, dragontools/gui/rules_subtitle_tab.py

15.3.44.4 Wichtige Klassen und Methoden

_LanguagePriorityEditor [QWidget]

```
priority() -> list[str]
```

```
data() -> dict
```

15.3.44.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.44.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.45 dragontools/gui/rules_regex_help.py

Merkmal	Wert
Kategorie	GUI
Umfang	92 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (92 Zeilen, direkte Importnutzer: 1).

15.3.45.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _show_regex_help.

15.3.45.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.45.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_series_tab.py

15.3.45.4 Wichtige Funktionen

```
_show_regex_help(parent) -> None
```

Zeigt den Regex-Hilfe-Dialog.

15.3.45.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.45.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.46 dragontools/gui/rules_series_tab.py

Merkmal	Wert
Kategorie	GUI
Umfang	157 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (157 Zeilen, direkte Importnutzer: 1).

15.3.46.1 Aufgabe des Moduls

Enthält _SeriesTab.

15.3.46.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.46.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/gui/info_button.py, dragontools/gui/rules_dialog_storage.py, dragontools/gui/rules_regex_help.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_dialog.py

15.3.46.4 Wichtige Klassen und Methoden

_SeriesTab [QWidget] – Serien-Erkennung per Beispiele statt rohen Regex. Du gibst Dateinamen ein – die App lernt daraus und zeigt ob sie erkannt werden. Für Experten: Erweiterte Muster weiterhin editierbar.

```
get_data() -> dict
```

15.3.46.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.46.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: re.error. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.47 dragontools/gui/rules_subtitle_tab.py

Merkmal	Wert
Kategorie	GUI
Umfang	217 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (217 Zeilen, direkte Importnutzer: 1).

15.3.47.1 Aufgabe des Moduls

Enthält _SubtitleTab.

15.3.47.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.47.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/gui/info_button.py, dragontools/gui/rules_dialog_storage.py, dragontools/gui/rules_language_editor.py, dragontools/rules/subtitle_rules.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_dialog.py

15.3.47.4 Wichtige Klassen und Methoden

_SubtitleTab [QWidget]

```
get_data() -> dict
```

15.3.47.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.47.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.48 dragontools/gui/run_summary_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	161 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (161 Zeilen, direkte Importnutzer: 1).

15.3.48.1 Aufgabe des Moduls

Enthält RunSummaryDialog.

15.3.48.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `summary`, `parent`

15.3.48.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_result_service.py

15.3.48.4 Wichtige Klassen und Methoden

RunSummaryDialog [QDialog]

```
action() -> str
failed_inputs() -> list[str]
```

15.3.48.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.48.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.49 dragontools/gui/save_settings_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	15 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (15 Zeilen, direkte Importnutzer: 1).

15.3.49.1 Aufgabe des Moduls

Enthält SaveSettingsDialog.

15.3.49.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.49.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/gui/settings_dialog.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.49.4 Wichtige Klassen und Methoden

SaveSettingsDialog [SettingsDialog]

```
__init__(parent) -> None
```

15.3.49.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.49.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.50 dragontools/gui/settings_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	1415 Quellzeilen
Bedeutung	Zentraler Baustein (1415 Zeilen, direkt von 2 Projektmodulen importiert).

15.3.50.1 Aufgabe des Moduls

dragontools/gui/settings_dialog.py – mit InfoButtons

15.3.50.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `parent`

15.3.50.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audit_log.py, dragontools/core/log_cleanup.py, dragontools/core/parallel_settings.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/gui/info_button.py, dragontools/gui/timeout_settings_dialog.py

Abhängigkeit	Details
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, handbrake, hdr10plus_tool, makemkvcon, mediainfo, mkvextract, mkvmerge, mp4box
Direkt verwendet von	dragontools/gui/main_window_actions.py, dragontools/gui/save_settings_dialog.py
Verwendete Settings-Konstanten	SET_KEY_ACTIVE_ALL_ANIME, SET_KEY_ACTIVE_ALL_FILME, SET_KEY_ACTIVE_ALL_TV, SET_KEY_AUTOCROP_MODE, SET_KEY_AUTOCROP_PROBE_DURATION, SET_KEY_AUTOCROP_PROBE_INTERVAL, SET_KEY_AUTOCROP_PROBE_START, SET_KEY_IMAX_MIN_HITS, SET_KEY_IMAX_MIN_VARIANCE_PERCENT, SET_KEY_IMAX_PROBE_DURATION, SET_KEY_IMAX_PROBE_INTERVAL, SET_KEY_LOG_ENABLED, SET_KEY_LOG_ROOT, SET_KEY_MOVE_CONFLICT, SET_KEY_NFO_CONFLICT_MODE, SET_KEY_NFO_ENABLED, SET_KEY_NFO_FILEINFO_ENABLED, SET_KEY_NFO_MOVIE_TARGET_NAME, SET_KEY_NFO_ONLY_UNAMBIGUOUS, SET_KEY_OUTPUT_DURATION_MAX_EXTRA_S

15.3.50.4 Wichtige Klassen und Methoden

SettingsDialog [QDialog]

`__init__(parent)`

15.3.50.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.50.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.51 dragontools/gui/shortcut_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	46 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (46 Zeilen, direkte Importnutzer: 1).

15.3.51.1 Aufgabe des Moduls

Enthält ShortcutDialog.

15.3.51.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `entries`, `parent`

15.3.51.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py

15.3.51.4 Wichtige Klassen und Methoden

ShortcutDialog [QDialog]

```
__init__(entries, parent)
```

15.3.51.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.51.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.52 dragontools/gui/styles.py

Merkmal	Wert
Kategorie	GUI
Umfang	65 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (65 Zeilen, direkte Importnutzer: 2).

15.3.52.1 Aufgabe des Moduls

dragontools/gui/styles.py Technischer Kontext aus dem Quellcode: dragontools/gui/styles.py Qt-Stylesheets für Dragon Tools. Vorher waren STYLE_LIGHT und STYLE_DARK direkt in main_window.py definiert (~80 Zeilen CSS), ohne jeden Bezug zur Fensterlogik. Hier isoliert, damit: main_window.py schlanker wird Styling leicht gefunden und angepasst werden kann zukuenftige Theme-Erweiterungen eine klare Heimat haben Benutzung

15.3.52.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.52.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py, dragontools/gui/main_window_actions.py

15.3.52.4 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.52.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.53 dragontools/gui/subtitle_widget.py

Merkmal	Wert
Kategorie	GUI
Umfang	517 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (517 Zeilen, direkt von 1 Projektmodulen importiert).

15.3.53.1 Aufgabe des Moduls

Das Untertitel-Widget orchestriert sechs Bereiche: Extrahieren, Einfügen, SRT zu ASS, Untertitel zu TXT, TXT zu SRT und TXT zu ASS. Einfügen erlaubt eine leere Untertiteldatei, automatische Zuordnung neben den Videos, Deutsch/Englisch und einen frei editierbaren Spurtitel. Dateien und Worker liegen in fokussierten Modulen.

> *Die Zuordnung übernimmt dragontools/subtitle/sidecar_matcher.py. Mux-Metadaten werden in dragontools/subtitle/injector.py dem neu eingefügten Track zugeordnet. Die konkrete Bedienung und Containergrenzen stehen in Kapitel 8.5.*

15.3.53.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `qt_filter`, `files`, `out_dir`, `tools`, `video_files`, `sub_file`, `language`, `forced`, `mode`, `filter_exts`, `parent`

15.3.53.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `set[str]`, `None`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/core/paths.py, dragontools/core/process_runner.py, dragontools/subtitle/converter.py, dragontools/subtitle/extractor.py, dragontools/subtitle/injector.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg, ffprobe, mkvextract, mkvmerge
Direkt verwendet von	dragontools/gui/main_window.py

15.3.53.4 Wichtige Klassen und Methoden

_SubWorker [QThread]

```
cancel() -> None
```

_ExtractWorker [_SubWorker]

```
run() -> None
```

_InjectWorker [_SubWorker]

```
run() -> None
```

_ConvertWorker [_SubWorker]

```
run() -> None
```

_FileDropList [QListWidget] – QListWidget mit Drag-and-Drop-Unterstützung für lokale Dateien.

```
dragEnterEvent(event) -> None
dragMoveEvent(event) -> None
dropEvent(event) -> None
```

SubtitleWidget [QWidget]

```
__init__(parent)
```

15.3.53.5 Wichtige Funktionen

```
_parse_exts(qt_filter) -> set[str]
```

Extrahiert Dateiendungen aus einem Qt-Filterstring, z.B. '(*.mkv *.mp4)' → {'mkv','mp4'}.

15.3.53.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.53.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.54 dragontools/gui/tab_manager.py

Merkmal	Wert
Kategorie	GUI
Umfang	213 Quellzeilen

Merkmal	Wert
Bedeutung	Spezialisierter/unterstützender Baustein (213 Zeilen, direkte Importnutzer: 2).

15.3.54.1 Aufgabe des Moduls

dragontools/gui/tab_manager.py Technischer Kontext aus dem Quellcode: dragontools/gui/tab_manager.py Dialog zum Ein-/Ausblenden von Tabs und Öffnen externer Programme. Auch: Registerkarten-Zustand in QSettings persistieren.

15.3.54.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `exe_path`, `name`, `parent`

15.3.54.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`, `str | None`, `dict[str, bool]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	handbrake
Direkt verwendet von	dragontools/gui/main_window.py, dragontools/gui/main_window_actions.py

15.3.54.4 Wichtige Klassen und Methoden

TabManagerDialog [QDialog] – Zeigt welche Tabs aktiv sind und lässt sie ein-/ausblenden. Auch: externe Programme starten. Änderungen werden sofort in QSettings gespeichert.

```
__init__(parent)
```

15.3.54.5 Wichtige Funktionen

```
get_visible_tabs() -> dict[str, bool]
```

15.3.54.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.54.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.55 dragontools/gui/timeout_settings_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	289 Quellzeilen

Merkmal	Wert
Bedeutung	Wichtiger Fachbaustein (289 Zeilen, direkt von 3 Projektmodulen importiert).

15.3.55.1 Aufgabe des Moduls

dragontools/gui/timeout_settings_dialog.py Technischer Kontext aus dem Quellcode: dragontools/gui/timeout_settings_dialog.py Dialog zur individuellen Konfiguration aller Prozess-Timeouts. Zeitangaben werden in Minuten angezeigt und eingegeben; intern wird in Sekunden gespeichert.

15.3.55.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `seconds`, `minutes`, `parent`

15.3.55.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `int`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/timeout_settings.py, dragontools/gui/info_button.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window_actions.py, dragontools/gui/settings_dialog.py, dragontools/tests/test_gui_dialog_services.py

15.3.55.4 Wichtige Klassen und Methoden

TimeoutSettingsDialog [QDialog] – Dialog zur Konfiguration aller Prozess-Timeouts in Minuten.

`__init__(parent)`

15.3.55.5 Wichtige Funktionen

`_s_to_min(seconds) -> int`

Sekunden → ganze Minuten (aufgerundet auf 1 Minute mindestens).

`_min_to_s(minutes) -> int`

Minuten → Sekunden.

`_format_default(seconds) -> str`

Lesbare Standardwert-Anzeige, z.B. '240 Min (4 Std)'.

15.3.55.6 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.55.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.3.56 dragontools/gui/tool_path_settings.py

Merkmal	Wert
Kategorie	GUI
Umfang	87 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (87 Zeilen, direkte Importnutzer: 1).

15.3.56.1 Aufgabe des Moduls

Qt-Implementierung des ToolPathSettingsProvider. Technischer Kontext aus dem Quellcode: Qt-Implementierung des ToolPathSettingsProvider. Diese Datei gehoert zur GUI-Schicht und ist die einzige Stelle im Projekt, die für ToolPaths Qt (QSettings) importiert. Warum hier und nicht in core/paths.py? Das core-Paket soll keine Qt-Abhängigkeit haben, damit Worker-Code, Tests und spaetere CLI-Nutzung ohne Qt-Eventloop funktionieren. core.paths.ToolPath…

15.3.56.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen

15.3.56.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/main_window.py

15.3.56.4 Wichtige Klassen und Methoden

QtToolPathSettingsProvider [ToolPathSettingsProvider] – Liest benutzerdefinierte Tool-Verzeichnisse aus QSettings. Legt absichtlich KEINE QSettings-Instanz beim Init an, sondern erzeugt pro Aufruf eine neue. Das stellt sicher, dass nach

```
get_custom_dirs() -> list[Path]
find_in_settings(tool_key) -> str | None
```

15.3.56.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.56.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.3.57 dragontools/gui/ui_helpers.py

Merkmal	Wert
Kategorie	GUI
Umfang	32 Quellzeilen

Merkmal	Wert
Bedeutung	Wichtiger Fachbaustein (32 Zeilen, direkt von 4 Projektmodulen importiert).

15.3.57.1 Aufgabe des Moduls

dragontools/gui/ui_helpers.py Technischer Kontext aus dem Quellcode: dragontools/gui/ui_helpers.py Gemeinsame UI-Hilfsfunktionen, die von mehreren GUI-Klassen benötigt werden. Freie Funktionen statt Methoden – kein Widget-Owner-Zugriff nötig.

15.3.57.2 Erwartete Informationen / Eingaben

- Benutzereingaben und Widget-Zustände
- QSettings bzw. gespeicherte Programmoptionen
- Wichtige Parameter im Code: `file_list`, `path`, `text`

15.3.57.3 Rückgabe / Ausgabe

- GUI-Status, Dialogergebnisse und Start-/Steuersignale an Controller oder Worker
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/conversion_result_service.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/gui/move_preflight_controller.py

15.3.57.4 Wichtige Funktionen

`set_file_list_item_text(file_list, path, text) -> None`

Setzt den Anzeigetext des List-Items, dessen UserRole == path ist. Früher als private Methode in ConvertWidget, ConversionController und ConversionResultService dreifach dupliziert. Jetzt eine einzige

15.3.57.5 Datenverarbeitung und Kommunikation

Benutzereingabe → Widget/Validierung → Controller bzw. Worker-Aufruf → Signal/Ergebnis → GUI-Status, Dialog oder Log.

15.3.57.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.4 Regeln

8 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.4.1 dragontools/rules/__init__.py

Merkmal	Wert

Merkmal	Wert
Kategorie	Regeln
Umfang	2 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (2 Zeilen, direkt von 3 Projektmodulen importiert).

15.4.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.4.1.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides

15.4.1.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/rules_audio_tab.py, dragontools/gui/rules_dialog_storage.py, dragontools/tests/test_preflight_dialog_performance.py

15.4.1.4 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.4.2 dragontools/rules/audio_plan.py

Merkmal	Wert
Kategorie	Regeln
Umfang	407 Quellzeilen
Bedeutung	Zentraler Baustein (407 Zeilen, direkt von 10 Projektmodulen importiert).

15.4.2.1 Aufgabe des Moduls

Baut konkrete Trackentscheidungen für ausgewählte Audiospuren. Technischer Kontext aus dem Quellcode: dragontools/rules/audio_plan.py Zentrale, reine Entscheidungsfunktion für die Audio-Spurwahl. Zweck ===== Die gleiche fachliche Audio-Entscheidung wurde bisher dreimal parallel implementiert: core/rules_preview.py : _build_audio_preview() (Preview-Dict) worker/converter_stream_args.py : audio_args() (ffmpeg-Args)

15.4.2.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `decision`, `value`, `raw`, `default_enabled`, `default_value`, `rules`, `file_override`, `processing`, `chosen`, `plan`, `audio_streams`, `container`

15.4.2.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `bool`, `str`, `float`, `dict[str, Any]`, `tuple[list[str], float | None, list[str]]`, `list[str]`, `list[AudioTrackDecision]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py, dragontools/rules/audio_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, mediainfo, mp4box
Direkt verwendet von	dragontools/core/rules_preview.py, dragontools/gui/rules_audio_tab.py, dragontools/tests/test_audio_processing.py, dragontools/tests/test_extended.py, dragontools/worker/audio_mux_thread.py, dragontools/worker/converter_stream_args.py, dragontools/worker/converter_strip.py, dragontools/worker/dv_audio_mux_service.py, dragontools/worker/dv_remux_components.py, dragontools/worker/mp4_remux_thread.py

15.4.2.4 Wichtige Klassen und Methoden

AudioTrackDecision [object] – Eine Entscheidung pro ausgewähltem Audiostream. Felder: out_idx : 0-basierter Mux-Index in der Zielreihenfolge

15.4.2.5 Wichtige Funktionen

`needs_stereo_downmix_filter(decision) -> bool`

True wenn eine Mehrkanalspur beim Transkodieren auf Stereo reduziert wird.

`effective_audio_processing(rules, file_override) -> dict[str, Any]`

Ermittelt DRC/Loudness inklusive per-Datei-Override.

`audio_filter_chain(decision) -> str`

Gibt die kombinierte Filterchain für einen Audio-Output zurück.

`audio_input_args_for_plan(plan) -> list[str]`

Input-Argumente für ffmpeg, aktuell AC3/EAC3-DRC vor -i.

`compute_audio_track_plan(audio_streams, file_override, container, rules) -> list[AudioTrackDecision]`

Baut die Liste der Audio-Entscheidungen in Mux-Reihenfolge. Semantik entspricht 1:1 der bestehenden produktiven Logik in ``worker/converter_stream_args.py:audio_args()`` (Referenz, weil

15.4.2.6 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.2.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.4.3 dragontools/rules/audio_rules.py

Merkmal	Wert
Kategorie	Regeln
Umfang	812 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (812 Zeilen, direkt von 6 Projektmodulen importiert).

15.4.3.1 Aufgabe des Moduls

Entscheidet Audiocodec, Kanäle, Bitrate, Downmix und Transcoding. Technischer Kontext aus dem Quellcode: dragontools/rules/audio_rules.py Audio-Auswahl und Transkodierungs-Entscheidung. Alle Regeln werden aus der JSON-Konfiguration geladen – nichts ist hartcodiert. Kanal-Logik (aus channel_rules in der JSON): Mono (1 Kanal): eigene Ziel-Codec + Max-Bitrate Stereo (2 Kanäle): eigene Ziel-Codec + Max-Bitrate 5.1 (3-6 Kanäle): eigene Ziel-Codec…

15.4.3.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `value`, `default`, `minimum`, `maximum`, `codec`, `target`, `source_channels`, `name`, `rule`, `rules`, `channels`, `bitrate`, `target_channels`, `target_codec`, `max_bps`, `audio_stream`

15.4.3.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `int`, `float`, `str`, `dict[str, Any]`, `None`, `dict`, `bool`, `tuple[bool, dict[str, Any]]`, `list[AudioStream]`, `AudioStream | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/config_migration.py, dragontools/core/lang_codes.py, dragontools/core/models.py, dragontools/rules/rule_loader.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/rules_preview.py, dragontools/rules/audio_plan.py, dragontools/tests/test_audio_processing.py, dragontools/tests/test_config_migration.py, dragontools/tests/test_extended.py, dragontools/tests/test_language_priority_rules.py

15.4.3.4 Wichtige Funktionen

```
safe_int(value, default) -> int
safe_float(value, default) -> float
```

`normalize_audio_codec(codec) -> str`

`max_channels_for_audio_codec(codec) -> int`

Maximale Ziel-Kanalzahl, die DragonTools für Encoding sicher plant.

`clamp_audio_target_to_codec_cap(target, source_channels) -> dict[str, Any]`

Begrenzt Zielkanaele auf das, was der Zielcodec sauber encoden kann.

`normalize_audio_processing_config(rules) -> dict[str, Any]`

Normalisiert DRC- und Loudness-Regeln auf sichere Zahlenbereiche.

`migrate_audio_rules(rules) -> dict[str, Any]`

Return audio rules with split mono/stereo channel rules filled in. Legacy configs only had ``channel_rules.stereo`` for 1-2 channels. Missing mono rules intentionally fall back to AAC/128k so mono tracks are not

`reload_rules() -> None`

Erzwingt Neu-Laden der Regeln (nach Speichern im Editor).

`default_transcode_target(channels, bitrate, rules) -> dict[str, Any]`

Build the configured transcode target for a channel count.

`audio_requires_transcode(audio_stream, rules) -> tuple[bool, dict[str, Any]]`

Entscheidet ob transkodiert werden muss und gibt das Ziel zurück. Rückgabe: (needs_transcode, target) target = {"codec": str, "channels": int, "bitrate": int (in bps)}

`choose_audio_stream(streams, rules) -> AudioStream | None`

Wählt die beste Audio-Spur nach Sprach-Präferenz (erste Spur, Single-Track API).

`choose_audio_streams(streams, rules) -> list[AudioStream]`

Wählt Audio-Spuren nach der klickbaren Sprach-Priorität aus. max_languages begrenzt die Anzahl der Sprachgruppen, tracks_per_language die Spuren je Sprache. Alte preferred/fallback/max_tracks-Regeln werden

15.4.3.5 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.3.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.4.4 dragontools/rules/move_rules.py

Merkmal	Wert
Kategorie	Regeln
Umfang	569 Quellzeilen
Bedeutung	Zentraler Baustein (569 Zeilen, direkt von 12 Projektmodulen importiert).

15.4.4.1 Aufgabe des Moduls

Erkennt Serien, Staffeln, Filmziele und sichere Zielnamen. Technischer Kontext aus dem Quellcode: dragontools/rules/move_rules.py Serien-/Film-Erkennung aus Dateinamen. Unterstützte Formate: S01E01 Standard S01E01E02 Doppelfolge (direkt verkettet) S01E01E02E03 Dreifachfolge (Bugfix: vorher nur [1,2] erkannt) S01E01-E02 Mit Bindestrich S01E01 E02 Mit Leerzeichen

15.4.4.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `rules`, `stem`, `m`, `s`, `filename`, `value`, `fallback`, `path`, `title`, `base_dir`, `relative_subpath`, `text`, `name`, `base`, `series_name`, `series_root`

15.4.4.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `dict[str, Any]`, `str`, `dict[str, Any] | None`, `tuple[str | None, int, int] | None`, `Path`, `list[str]`, `str | None`, `list[Path]`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/config_migration.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/movie_renamer.py, dragontools/core/online_metadata.py, dragontools/core/preflight_report.py, dragontools/core/rules_preview.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/gui/move_preflight_controller.py, dragontools/gui/preflight_dialog.py, dragontools/gui/rules_series_tab.py, dragontools/tests/test_config_migration.py, dragontools/tests/test_move_rules_series_detection.py, dragontools/worker/move_thread.py, dragontools/worker/postprocess_service.py

15.4.4.4 Wichtige Funktionen

`migrate_move_rules(rules) -> dict[str, Any]`

`parse_series_match_details(filename) -> dict[str, Any] | None`

`parse_series_season_episode(filename) -> tuple[str | None, int, int] | None`

`sanitize_win_segment(value, fallback) -> str`

Bereinigt einen Pfadsegment-Namen für Windows-Kompatibilität. Ersetzt verbotene Zeichen durch '_', entfernt fuehrende/trailing Punkte und Leerzeichen, und hängt '_' an reservierte Namen an.

`move_safe_stem(path) -> str`

Bereinigt Dateinamen für die Move-Zielfindung um Codec-Suffixe.

`default_film_series_name(stem) -> str`

Leitet aus einem Film-Stem einen Vorschlagsnamen für Filmreihen ab.

`film_bucket_from_title(title) -> str`

Erster Buchstabe des Titels als Sortier-Bucket für Einzelfilme. Gibt '#' für Titel zurück, die nicht mit A-Z beginnen.

`normalize_relative_move_subpath(value) -> str`

Validiert und normalisiert einen optionalen relativen Unterpfad. Erlaubt leere Eingaben. Verhindert absolute Pfade, Laufwerksangaben, '..'-Segmente und Windows-ungültige Zeichen.

`apply_relative_move_subpath(base_dir, relative_subpath) -> Path`

Haengt einen validierten relativen Unterpfad an einen Basisordner an.

`find_series_dir_candidates(base, series_name) -> list[str]`

Liefert bestehende Serienordner-Kandidaten unterhalb des Basisordners. Matching-Logik (strikt): Der Ordnername ohne Jahres-Suffix und ohne bedeutungslose Sonderzeichen

15.4.4.5 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.4.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: OSError, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.4.5 dragontools/rules/pipeline_selector.py

Merkmal	Wert
Kategorie	Regeln
Umfang	177 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (177 Zeilen, direkt von 4 Projektmodulen importiert).

15.4.5.1 Aufgabe des Moduls

Wählt Standard-, DV- oder HDR10+-Pipeline.

15.4.5.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `media_info`

15.4.5.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `str`, `dict[str, object]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffprobe, mediainfo
Direkt verwendet von	dragontools/core/rules_preview.py, dragontools/tests/test_extended.py, dragontools/tests/test_pipeline_and_singleton.py, dragontools/worker/pipeline_decision_service.py

15.4.5.4 Wichtige Klassen und Methoden

PipelineSelector [object] – Wählt die Konvertierungs-Pipeline in Abhängigkeit von MediaInfo und der konfigurierten DV/HDR10+-Policy. Policy-Logik (Priorität: per-Datei > global > auto):

```
select(media_info) -> Pipeline
```

15.4.5.5 Wichtige Funktionen

```
resolve_pipeline_context(media_info) -> dict[str, object]
```

Zentralisiert effektive DV/HDR10+-Policy, Pipeline und Zielcontainer. Erweitertes Rueckgabe-Dict: pipeline : Pipeline (Pipeline.DV / .HDRPLUS / .STANDARD)

15.4.5.6 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.5.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.4.6 dragontools/rules/rule_loader.py

Merkmal	Wert
Kategorie	Regeln
Umfang	153 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (153 Zeilen, direkt von 7 Projektmodulen importiert).

15.4.6.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _report_visible_warning, _rules_dir, _default_rules_path, load_json_rules.

15.4.6.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `message`, `reporter`, `name`, `path`, `default`, `data`, `migrator`

15.4.6.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `None`, `Path`, `dict[str, Any]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/config_migration.py, dragontools/rules/subtitle_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/gui/mp4_remux_widget.py, dragontools/gui/rules_dialog_storage.py, dragontools/rules/audio_rules.py, dragontools/tests/test_extended.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/mp4_remux_thread.py

15.4.6.4 Wichtige Funktionen

```
load_json_rules(path, default) -> dict[str, Any]
load_named_rules(name, default) -> dict[str, Any]
load_subtitle_rules(default) -> dict[str, Any]
```

15.4.6.5 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.6.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.4.7 dragontools/rules/subtitle_rules.py

Merkmal	Wert
Kategorie	Regeln
Umfang	958 Quellzeilen
Bedeutung	Zentraler Baustein (958 Zeilen, direkt von 10 Projektmodulen importiert).

15.4.7.1 Aufgabe des Moduls

Entscheidet Untertitel-Keep, Forced, Burn-In und Sidecars.

15.4.7.2 Erwartete Informationen / Eingaben

- analysierte Stream-/Metadaten
- Regelkonfiguration und Benutzer-Overrides
- Wichtige Parameter im Code: `value`, `default`, `raw`, `rules`, `raw_values`, `preferred_language`, `values`, `streams`, `preferred_formats`, `audio_streams`, `language`, `ov`, `languages`, `stream`, `language_priority`, `media_duration_s`

15.4.7.3 Rückgabe / Ausgabe

- Entscheidungsobjekte, Streampläne oder Regelresultate
- Explizite Rückgabetypen im Code: `int`, `float`, `dict[str, Any]`, `set[str]`, `list[str]`, `list[SubtitleStream]`, `bool`, `dict[int, dict]`, `float | None`, `tuple[str, float | None, str | None]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/config_migration.py, dragontools/core/lang_codes.py, dragontools/core/models.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/core/rules_preview.py, dragontools/gui/rules_subtitle_tab.py, dragontools/rules/rule_loader.py, dragontools/tests/test_language_priority_rules.py, dragontools/worker/converter_stream_args.py, dragontools/worker/converter_strip.py, dragontools/worker/dv_remux_components.py, dragontools/worker/encode_plan_service.py, dragontools/worker/mp4_remux_thread.py, dragontools/worker/subtitle_sidecar_service.py

15.4.7.4 Wichtige Klassen und Methoden

SubtitlePlan [object]

15.4.7.5 Wichtige Funktionen

```
migrate_subtitle_rules(rules) -> dict[str, Any]
compute_subtitle_plan(subtitle_streams) -> SubtitlePlan
choose_burn_subtitle(streams, preferred_language, override, subtitle_rules, audio_streams) -> SubtitleStream | None
choose_keep_subtitles(streams) -> list[SubtitleStream]
```

15.4.7.6 Datenverarbeitung und Kommunikation

Stream-/Dateimetadaten + Regel-JSON/Override → Normalisierung → Priorisierung/Filter → Plan/Entscheidung für nachgelagerte Worker.

15.4.7.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.5 Untertitel

6 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.5.1 dragontools/subtitle/__init__.py

Merkmal	Wert
Kategorie	Untertitel
Umfang	2 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (2 Zeilen, direkte Importnutzer: 0).

15.5.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.5.1.2 Erwartete Informationen / Eingaben

- Quelldatei bzw. Untertitelspur
- Sprache/Flags/Zielpfad

15.5.1.3 Rückgabe / Ausgabe

- extrahierte/konvertierte Untertitel oder geänderter Container/Track-Header

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	Kein direkter statischer Importnutzer; möglich sind Einstiegspunkt, Lazy-Import oder reine Paketfunktion.

15.5.1.4 Datenverarbeitung und Kommunikation

Quelle + Track-/Sprachangaben → Untertitelparser/Dateisystem → Konvertierung/Extraktion/Injection/Tagging → Zieldatei oder geänderter Track.

15.5.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.5.2 dragontools/subtitle/converter.py

Merkmal	Wert
Kategorie	Untertitel
Umfang	209 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (209 Zeilen, direkt von 3 Projektmodulen importiert).

15.5.2.1 Aufgabe des Moduls

dragontools/subtitle/converter.py Technischer Kontext aus dem Quellcode: dragontools/subtitle/converter.py SRT ↔ ASS ↔ TXT Konvertierung.

15.5.2.2 Erwartete Informationen / Eingaben

- Quelldatei bzw. Untertitelspur
- Sprache/Flags/Zielpfad
- Wichtige Parameter im Code: `path`, `value`, `t`, `style`, `text`, `srt_path`, `output_path`, `txt_path`, `ass_path`, `subtitle_path`

15.5.2.3 Rückgabe / Ausgabe

- extrahierte/konvertierte Untertitel oder geänderter Container/Track-Header
- Explizite Rückgabetypen im Code: `str`, `list[tuple[str, str, str]]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/subtitle_widget.py, dragontools/tests/test_auxiliary_workers.py, dragontools/tests/test_subtitle_converter.py

15.5.2.4 Wichtige Funktionen

```
srt_to_txt(srt_path, output_path) -> str
srt_to_ass(srt_path, output_path, style) -> str
txt_to_srt(txt_path, output_path) -> str
txt_to_ass(txt_path, output_path, style) -> str
ass_to_txt(ass_path, output_path) -> str
subtitle_to_txt(subtitle_path, output_path) -> str
```

15.5.2.5 Datenverarbeitung und Kommunikation

Quelle + Track-/Sprachangaben → Untertitelparser/Dateisystem → Konvertierung/Extraktion/Injection/Tagging → Zieldatei oder geänderter Track.

15.5.2.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.5.3 dragontools/subtitle/extractor.py

Merkmal	Wert
Kategorie	Untertitel
Umfang	68 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (68 Zeilen, direkte Importnutzer: 2).

15.5.3.1 Aufgabe des Moduls

dragontools/subtitle/extractor.py Technischer Kontext aus dem Quellcode: dragontools/subtitle/extractor.py Untertitel aus MKV/MP4 extrahieren (mkvextract / ffmpeg).

15.5.3.2 Erwartete Informationen / Eingaben

- Quelldatei bzw. Untertitelspur
- Sprache/Flags/Zielpfad
- Wichtige Parameter im Code: `logger`, `tool`, `rc`, `stdout`, `stderr`, `exc`, `mkv_path`, `track_id`, `output_path`, `input_path`, `stream_index`, `ffmpeg`, `codec_args`

15.5.3.3 Rückgabe / Ausgabe

- extrahierte/konvertierte Untertitel oder geänderter Container/Track-Header
- Explizite Rückgabetypen im Code: `None`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/process_runner.py, dragontools/core/timeout_settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, mkvextract
Direkt verwendet von	dragontools/gui/subtitle_widget.py, dragontools/tests/test_auxiliary_workers.py

15.5.3.4 Wichtige Funktionen

```
extract_with_mkvextract(mkv_path, track_id, output_path, logger) -> bool
extract_with_ffmpeg(input_path, stream_index, output_path, ffmpeg, logger, codec_args) -> bool
```

15.5.3.5 Datenverarbeitung und Kommunikation

Quelle + Track-/Sprachangaben → ffmpeg / mkvextract → Konvertierung/Extraktion/Injection/Tagging → Zieldatei oder geänderter Track.

15.5.3.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.5.4 dragontools/subtitle/injector.py

Merkmal	Wert
Kategorie	Untertitel
Umfang	141 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (141 Zeilen, direkte Importnutzer: 2).

15.5.4.1 Aufgabe des Moduls

dragontools/subtitle/injector.py Technischer Kontext aus dem Quellcode: dragontools/subtitle/injector.py Untertitel in MKV/MP4 einfuegen.

15.5.4.2 Erwartete Informationen / Eingaben

- Quelldatei bzw. Untertitelspur
- Sprache/Flags/Zielpfad
- Wichtige Parameter im Code: `logger`, `tool`, `rc`, `stdout`, `stderr`, `exc`, `video_path`, `sub_path`, `output_path`, `language`, `forced`, `mkvmerge`, `ffmpeg`, `subtitle_codec`, `map_existing_subtitles`

15.5.4.3 Rückgabe / Ausgabe

- extrahierte/konvertierte Untertitel oder geänderter Container/Track-Header
- Explizite Rückgabetypen im Code: `None`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/process_runner.py, dragontools/core/timeout_settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, mkvmerge
Direkt verwendet von	dragontools/gui/subtitle_widget.py, dragontools/tests/test_auxiliary_workers.py

15.5.4.4 Wichtige Funktionen

```
inject_with_mkvmerge(video_path, sub_path, output_path, language, forced, logger, mkvmerge) -> bool
inject_with_ffmpeg(video_path, sub_path, output_path, language, ffmpeg, logger, subtitle_codec,
map_existing_subtitles) -> bool
```

15.5.4.5 Datenverarbeitung und Kommunikation

Quelle + Track-/Sprachangaben → ffmpeg / mkvmerge → Konvertierung/Extraktion/Injection/Tagging → Zieldatei oder geänderter Track.

15.5.4.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.5.5 dragontools/subtitle/tagger.py

Merkmal	Wert
Kategorie	Untertitel
Umfang	61 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (61 Zeilen, direkte Importnutzer: 1).

15.5.5.1 Aufgabe des Moduls

dragontools/subtitle/tagger.py Technischer Kontext aus dem Quellcode: dragontools/subtitle/tagger.py Sprach-Tags und Flags in MKV-Containern setzen (via mkvpropedit).

15.5.5.2 Erwartete Informationen / Eingaben

- Quelldatei bzw. Untertitelspur
- Sprache/Flags/Zielpfad
- Wichtige Parameter im Code: `logger`, `tool`, `rc`, `stdout`, `stderr`, `exc`, `mkv_path`, `track_id`, `language`, `forced`

15.5.5.3 Rückgabe / Ausgabe

- extrahierte/konvertierte Untertitel oder geänderter Container/Track-Header
- Explizite Rückgabetypen im Code: `None`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/process_runner.py, dragontools/core/timeout_settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	mkvpropedit
Direkt verwendet von	dragontools/gui/rules_flags_tab.py

15.5.5.4 Wichtige Funktionen

```
set_track_language(mkv_path, track_id, language, logger) -> bool
set_forced_flag(mkv_path, track_id, forced, logger) -> bool
```

15.5.5.5 Datenverarbeitung und Kommunikation

Quelle + Track-/Sprachangaben → mkvpropedit → Konvertierung/Extraktion/Injection/Tagging → Zieldatei oder geänderter Track.

15.5.5.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6 Worker

82 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.6.1 dragontools/worker/__init__.py

Merkmal	Wert
Kategorie	Worker
Umfang	2 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (2 Zeilen, direkte Importnutzer: 2).

15.6.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.6.1.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.1.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_converter_detection.py, dragontools/tests/test_trickplay_service.py

15.6.1.4 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.2 dragontools/worker/archive_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	153 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (153 Zeilen, direkte Importnutzer: 2).

15.6.2.1 Aufgabe des Moduls

dragontools/worker/archive_service.py Technischer Kontext aus dem Quellcode: dragontools/worker/archive_service.py Zentrale Archivierungsfunktion für Quelldateien, bevor Metadaten (HDR10+/DV) bei codecbedingter Inkompatibilitaet

verloren gehen. Regeln: Ordner "Archiv" wird direkt neben der Quelldatei angelegt. Vorhandene Dateien werden NICHT überschrieben. Bei Namenskonflikt: eindeutiger Name (_001, _002, ...). Größe der Kopie wird…

15.6.2.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `log`

15.6.2.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py, dragontools/worker/pipeline_decision_service.py

15.6.2.4 Wichtige Klassen und Methoden

ArchiveService [object] – Sichert Quelldateien in einen 'Archiv'-Unterordner. Wird vor verlustbehafteter Konvertierung (z. B. H264+HDR10+, AV1+DV) aufgerufen. Wirft RuntimeError wenn Archivierung fehlschlaegt, damit

```
archive_original(input_path, reason) -> Path
```

15.6.2.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.2.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.3 dragontools/worker/audio_mux_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	346 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (346 Zeilen, direkte Importnutzer: 1).

15.6.3.1 Aufgabe des Moduls

Worker für Audio-Muxer. Technischer Kontext aus dem Quellcode: dragontools/worker/audio_mux_thread.py Audio-Mux-Thread: Audio Mux für MKV Remux Dateien Audio: konvertieren (nach Audioregeln, aber ohne Anforderungen an Sprache und Anzahl) Video: unverändert (kein Re-Encoding!) Untertitel: werden kopiert

> *Audio-Mux-Thread: Audio Mux für MKV Remux Dateien - Audio: konvertieren (nach Audioregeln, aber ohne Anforderungen an Sprache und Anzahl) - Video: unverändert (kein Re-Encoding!)*

15.6.3.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `num`, `path`, `files`, `overwrite_original`, `parent`

15.6.3.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/rules/audio_plan.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/audio_muxer_widget.py

15.6.3.4 Wichtige Klassen und Methoden

AudioMuxThread [QThread]

```
request_abort(mode) -> None
cancel() -> None
run() -> None
```

15.6.3.5 Wichtige Funktionen

```
_fmt_size(num) -> str
_safe_name(path) -> str
```

15.6.3.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.3.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, FileNotFoundError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.4 dragontools/worker/audio_video_match_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	376 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (376 Zeilen, direkte Importnutzer: 1).

15.6.4.1 Aufgabe des Moduls

Führt den Audio-Video-Matcher im Hintergrund aus, damit die GUI bedienbar bleibt. Der Worker steuert Analyse, Synchronentscheidung, Schnitt-/Offset-Plan und die abschließende Dateierzeugung.

15.6.4.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `operation`

15.6.4.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/audio_video_matcher.py, dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/worker/base_worker.py, dragontools/worker/tool_runner.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/audio_video_matcher_widget.py

15.6.4.4 Wichtige Klassen und Methoden

AudioVideoMatchThread [QThread]

```
cancel() -> None
run() -> None
```

15.6.4.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.4.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, RuntimeError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.5 dragontools/worker/base_worker.py

Merkmal	Wert
Kategorie	Worker
Umfang	192 Quellzeilen
Bedeutung	Zentraler Baustein (192 Zeilen, direkt von 25 Projektmodulen importiert).

15.6.5.1 Aufgabe des Moduls

dragontools/worker/base_worker.py Technischer Kontext aus dem Quellcode: dragontools/worker/base_worker.py Konvention für das Worker-Interface zwischen GUI und Worker-Threads. Keine Vererbung erzwungen - die produktiven Worker sind historisch direkt von QThread abgeleitet, und ein großflächiger Umbau bringt keinen Mehrwert. Stattdessen gilt für alle Worker folgendes Namensschema, damit GUI-Code und gemeinsame Helfer (z.B. conver…

15.6.5.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `parent`

15.6.5.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `int`, `dict`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/gui/convert_widget_file_queue.py, dragontools/tests/test_parallel_converter_thread.py, dragontools/worker/audio_mux_thread.py, dragontools/worker/audio_video_match_thread.py, dragontools/worker/converter_detection.py, dragontools/worker/converter_progress.py, dragontools/worker/converter_queue_state.py, dragontools/worker/converter_stream_args.py, dragontools/worker/converter_thread.py, dragontools/worker/duration_repair_service.py, dragontools/worker/duration_timing_analyzer.py, dragontools/worker/dv_audio_mux_service.py, dragontools/worker/dv_processing_pipeline.py, dragontools/worker/dv_remux_components.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/iso_thread.py, dragontools/worker/merge_thread.py, dragontools/worker/mp4_remux_thread.py

15.6.5.4 Wichtige Klassen und Methoden

RemoveFileStatus [str, Enum] – Gemeinsamer Statusvertrag für Worker.remove_file().

BaseWorker [QThread] – Basisklasse für einfache QThread-Worker ohne Lock und ohne abort_type. Konsolidiert das identische Boilerplate aus MP4RemuxThread und MergeThread. Abort-Interface (bereitgestellt):

```
current_process() -> 'subprocess.Popen | None'
current_process(proc) -> None
request_abort(mode) -> None
cancel() -> None
```

15.6.5.5 Wichtige Funktionen

`_nw() -> int`

CREATE_NO_WINDOW für subprocess.Popen auf Windows, 0 sonst.

`_startupinfo()`

STARTUPINFO mit SW_HIDE für Windows-Toolprozesse.

`_no_window_kwargs() -> dict`

Gemeinsame subprocess-Kwargs gegen kurz aufpoppende Konsolenfenster.

`_norm_path(path) -> str`

Normalisiert einen Dateipfad für Queue-/Override-Vergleiche. Aufgeloest (resolve) und in Kleinbuchstaben – plattformunabhängig konsistent. Wird in ConverterThread und DVRemuxThread für add_file /

15.6.5.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.5.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.6 dragontools/worker/cleanup_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	187 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (187 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.6.1 Aufgabe des Moduls

Räumt temporäre Dateien und Sidecars auf.

15.6.6.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `base_dirs`

15.6.6.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/path_safety.py, dragontools/worker/output_path_service.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_cleanup_rollback.py, dragontools/worker/converter_thread.py, dragontools/worker/parallel_converter_thread.py

15.6.6.4 Wichtige Klassen und Methoden

CleanupService [object] – Kapselt Burn-in- und Overwrite-Cleanup nach einem Workflow-Lauf.

`cleanup_temp_artifacts() -> None`

`cleanup_empty_overwrite_dirs(base_dirs) -> None`

15.6.6.5 Wichtige Funktionen

```
cleanup_empty_overwrite_dirs(base_dirs) -> None
```

15.6.6.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.6.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.7 dragontools/worker/command_formatting.py

Merkmal	Wert
Kategorie	Worker
Umfang	19 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (19 Zeilen, direkte Importnutzer: 1).

15.6.7.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie is_ffmpeg_command, command_to_log_string.

15.6.7.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `cmd0`, `cmd`

15.6.7.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `bool`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.7.4 Wichtige Funktionen

```
is_ffmpeg_command(cmd0) -> bool
command_to_log_string(cmd) -> str
```

15.6.7.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.7.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.8 dragontools/worker/converter_config.py

Merkmal	Wert
Kategorie	Worker
Umfang	79 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (79 Zeilen, direkte Importnutzer: 0).

15.6.8.1 Aufgabe des Moduls

ConverterConfig – Datenhaltungsobjekt für ConverterThread. Technischer Kontext aus dem Quellcode: ConverterConfig – Datenhaltungsobjekt für ConverterThread. Problem vorher: ConverterThread.__init__ hatte 15+ Parameter und konstruierte intern ~15 Service-Objekte. Das machte den Konstruktor untestbar und machte es schwer zu erkennen, was ein ConverterThread überhaupt braucht. Loesung: Alle Konfigurationsparameter leben jetzt in einem einzigen, klar typisi…

15.6.8.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.8.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	Kein direkter statischer Importnutzer; möglich sind Einstiegspunkt, Lazy-Import oder reine Paketfunktion.

15.6.8.4 Wichtige Klassen und Methoden

ConverterConfig [object] – Alle Konfigurationsparameter für einen Konvertierungslauf. Instanzen sind bewusst mutable (kein frozen=True), damit die GUI optionale Felder nachtraeglich setzen kann bevor der Thread gestartet

15.6.8.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.8.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.9 dragontools/worker/converter_detection.py

Merkmal	Wert
Kategorie	Worker
Umfang	246 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (246 Zeilen, direkte Importnutzer: 2).

15.6.9.1 Aufgabe des Moduls

dragontools/worker/converter_detection.py Technischer Kontext aus dem Quellcode: dragontools/worker/converter_detection.py Ausgelagerte Erkennungs-Funktionen des ConverterThread: detect_crop : Auto-Crop via ffmpeg cropdetect detect_imax_auto : IMAX-Erkennung über wechselnde Aspect-Ratios

> *Ausgelagerte Erkennungs-Funktionen des ConverterThread: - detect_crop : Auto-Crop via ffmpeg cropdetect - detect_imax_auto : IMAX-Erkennung über wechselnde Aspect-Ratios*

15.6.9.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `value`, `default`, `duration_s`, `interval_s`, `raw`, `worker`

15.6.9.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `int`, `str`, `list[int]`, `tuple[int, int, int, int] | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_converter_detection.py, dragontools/worker/converter_thread.py

15.6.9.4 Wichtige Klassen und Methoden

ConverterDetectionHelper [object] – Bündelt die Erkennungs-Funktionen (Crop, IMAX).

```
detect_crop(path, w, h, mode, start_s, probe_duration_s, interval_s, duration_s) -> str | None
detect_imax_auto(path, duration_s, source_w, source_h, interval_s, probe_duration_s, min_variance_percent, min_hits) -> bool
```

15.6.9.5 Wichtige Funktionen

```
_safe_int(value, default) -> int
_sanitize_imax_interval(value) -> int
_sanitize_autocrop_mode(value) -> str
_sanitize_autocrop_start(value) -> int
_sanitize_autocrop_duration(value) -> int
_sanitize_autocrop_interval(value) -> int
```

15.6.9.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.9.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.10 dragontools/worker/converter_progress.py

Merkmal	Wert
Kategorie	Worker
Umfang	363 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (363 Zeilen, direkte Importnutzer: 1).

15.6.10.1 Aufgabe des Moduls

dragontools/worker/converter_progress.py Technischer Kontext aus dem Quellcode: dragontools/worker/converter_progress.py Ausgelagerte Prozess- und Fortschritts-Utilities des ConverterThread. Enthaelt die reinen Laufzeit-/Messfunktionen: probe_ms : Dauer-Ermittlung via ffprobe probe_frames : Anzahl Frames (Fallback wenn keine Dauer) read_prog : liest ffmpeg -progress-Output und berechnet Fortschritt/ETA run : startet Subprozes…

> *Ausgelagerte Prozess- und Fortschritts-Utilities des ConverterThread. Enthaelt die reinen Laufzeit-/Messfunktionen: - probe_ms : Dauer-Ermittlung via ffprobe*

15.6.10.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `cmd`, `worker`

15.6.10.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `int`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/crash_guard.py, dragontools/core/timeout_settings.py, dragontools/worker/base_worker.py, dragontools/worker/process_control.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.10.4 Wichtige Klassen und Methoden

ConverterProgressHelper [object] – Kapselt Fortschritts- und Prozess-Funktionen eines ConverterThread.

```
probe_ms(path) -> int | None
probe_frames(path) -> int | None
read_prog(proc, path, dur_ms, total_frames, on_activity) -> None
run(cmd) -> int
run_p(cmd, path, dur_ms) -> int
```

15.6.10.5 Wichtige Funktionen

```
_cmd_for_log(cmd) -> str
```

15.6.10.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / ffprobe → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.10.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.11 dragontools/worker/converter_queue_state.py

Merkmal	Wert
Kategorie	Worker
Umfang	118 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (118 Zeilen, direkte Importnutzer: 2).

15.6.11.1 Aufgabe des Moduls

Verwaltet aktuelle und wartende Dateien.

15.6.11.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `files`

15.6.11.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/paths.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py, dragontools/worker/dv_remux_thread.py

15.6.11.4 Wichtige Klassen und Methoden

ConverterQueueState [object] – Thread-safe Live-Queue-Zustand für ConverterThread.

```
add_file(path, log) -> bool
```

```
remove_file(path, log) -> RemoveFileStatus
is_current(path) -> bool
reorder_waiting_files(new_order) -> None
next_file(processed_count) -> tuple[str, int] | None
complete_current(path) -> None
total_after_processed(processed_count) -> int
```

15.6.11.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.11.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.12 dragontools/worker/converter_stream_args.py

Merkmal	Wert
Kategorie	Worker
Umfang	279 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (279 Zeilen, direkte Importnutzer: 1).

15.6.12.1 Aufgabe des Moduls

Erzeugt FFmpeg-Argumente für Audio, Untertitel und Filter. Technischer Kontext aus dem Quellcode: dragontools/worker/converter_stream_args.py Ausgelagerte Stream-Argument-Erzeugung für ffmpeg: audio_args : -map / -c:a / -b:a ... sub_args : Burn-In-Sub-Entscheidung + Sub-Stream-Copy base_vf_args : -vf Grundgeruest text_burn_vf_args : Burn-In via SRT-Extract + subtitles-Filter image_burn_vf_args : Burn-In…

> *Ausgelagerte Stream-Argument-Erzeugung für ffmpeg: - audio_args : -map / -c:a / -b:a ... - sub_args : Burn-In-Sub-Entscheidung + Sub-Stream-Copy*

15.6.12.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `p`, `worker`

15.6.12.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py, dragontools/rules/audio_plan.py, dragontools/rules/subtitle_rules.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.

Abhängigkeit	Details
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.12.4 Wichtige Klassen und Methoden

ConverterStreamArgsHelper [object] – Erzeugt Audio-, Subtitle- und Video-Filter-Argumente für ffmpeg.

```
audio_args(mi, ov, container)
audio_input_args(mi, ov, container)
sub_args(input_path, mi, ov)
base_vf_args(pre_filters, post_filters) -> list
text_burn_vf_args(input_path, output_path, burn_sub, pre_filters, post_filters) -> list
image_burn_vf_args(mi, burn_sub, pre_filters, post_filters) -> list
build_vf_args(input_path, output_path, mi, burn_sub_or_vf, pre_filters, post_filters) -> list
```

15.6.12.5 Ausgelagerte Laufzeitkomponenten

```
ConverterRuntimeBuilder / ConverterRunLoop / ConverterLifecycleService
```

15.6.12.6 Service-Bootstrap, Session-/Live-Queue-Orchestrierung sowie Fehlergrenze und Abschlussarbeiten des QThread-Laufs.

ConverterFileExecutor / ConverterControlService

Per-Datei-Workflow inklusive Quellbild-Preflight sowie Diagnose, Pause/Resume, Abort und gezielte FFmpeg-Prozesskontrolle.

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.13 dragontools/worker/converter_strip.py

Merkmal	Wert
Kategorie	Worker
Umfang	128 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (128 Zeilen, direkte Importnutzer: 1).

15.6.13.1 Aufgabe des Moduls

dragontools/worker/converter_strip.py Technischer Kontext aus dem Quellcode: dragontools/worker/converter_strip.py Ausgelagerte Strip-Only-Funktion des ConverterThread. Video wird immer per Stream-Copy übernommen (kein Re-Encode). Audio- und Subtitle-Auswahl laufen über die zentralen Planfunktionen (compute_audio_track_plan / compute_subtitle_plan) und respektieren File-Overrides vollständig. Audio: Video-Copy, aber Audio folgt den…

15.6.13.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `worker`

15.6.13.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/rules/audio_plan.py, dragontools/rules/subtitle_rules.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.13.4 Wichtige Klassen und Methoden

ConverterStripHelper [object] – Kapselt den Strip-Only-Modus (copy-only, kein Re-Encode).

```
strip_only(inp, out, mi, ov, container) -> bool
```

15.6.13.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.13.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.14 dragontools/worker/converter_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	967 Quellzeilen
Bedeutung	Zentraler Baustein (967 Zeilen, direkt von 4 Projektmodulen importiert).

15.6.14.1 Aufgabe des Moduls

Hauptworker für Batch-Konvertierung. Technischer Kontext aus dem Quellcode: dragontools/worker/converter_thread.py Schlanker Konvertierungs-Worker. Delegiert Spezialaufgaben an Helper: ConverterProgressHelper : Prozesse + Fortschritt/ETA ConverterDetectionHelper : Auto-Crop + IMAX-Erkennung + externe Subs ConverterStreamArgsHelper : Audio-/Sub-/Video-Filter-Argumente ConverterStripHelper : Strip-Only-Modus DVProcessingPipel…

> *Schlanke QThread-Fassade des Konverters. Im aktuellen Stand liegen Runtime-Bootstrap, Session-/Queue-Schleife, Run-Lifecycle, Per-Datei-Fehlergrenze und Prozesskontrolle in eigenen Worker-Services. Die öffentliche Worker-Oberfläche und die fachlichen DV/HDR-/Audio-/Subtitle-Pipelines bleiben kompatibel.*

15.6.14.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `ffprobe`, `settings`, `key`, `default`, `files`, `codec`, `crf`, `preset`, `scale_mode`, `overwrite_original`

15.6.14.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

- Explizite Rückgabetypen im Code: `tuple[int, int] | None`, `int`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/crash_guard.py, dragontools/core/error_report.py, dragontools/core/logger.py, dragontools/core/models.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/core/gpu_detection.py, dragontools/worker/archive_service.py, dragontools/worker/base_worker.py, dragontools/worker/cleanup_service.py, dragontools/worker/converter_detection.py, dragontools/worker/converter_progress.py, dragontools/worker/converter_queue_state.py, dragontools/worker/converter_stream_args.py, dragontools/worker/converter_strip.py, dragontools/worker/converter_thread.py, dragontools/worker/duration_repair_service.py, dragontools/worker/dv_processing_pipeline.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	dovi_tool, ffmpeg, ffprobe, mediainfo
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/tests/test_current_ffmpeg_abort.py, dragontools/worker/converter_thread.py, dragontools/worker/parallel_converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_OUTPUT_DURATION_MAX_EXTRA_S, SET_KEY_OUTPUT_DURATION_MAX_PERCENT, SET_KEY_OUTPUT_DURATION_MIN_PERCENT, SET_KEY_OUTPUT_MIN_SIZE_KB, SET_KEY_REPAIR_DURATION_REMUX_ENABLED, SET_KEY_REPAIR_DURATION_TIMESTAMP_ENABLED

15.6.14.4 Wichtige Klassen und Methoden

ConverterThread [QThread]

```
add_file(path) -> bool
remove_file(path) -> RemoveFileStatus
is_current(path) -> bool
reorder_waiting_files(new_order) -> None
update_override(path, override) -> bool
current_process() -> 'subprocess.Popen | None'
current_process(proc) -> None
diagnostic_snapshot() -> dict
pause()
resume()
request_abort(mode)
terminate_current_ffmpeg(path) -> bool
```

15.6.14.5 Wichtige Funktionen

`_quick_video_resolution(path, ffprobe) -> tuple[int, int] | None`

Fragt per ffprobe nur die Breite/Höhe des ersten Videostreams ab. Sehr schnell (kein Format-Scan, kein MediaInfo). Gibt (width, height) zurück oder None bei Fehler.

`_settings_int(settings, key, default) -> int`

`_settings_bool(settings, key, default) -> bool`

15.6.14.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → dovi_tool / ffmpeg / ffprobe / mediainfo → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.14.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.15 dragontools/worker/converter_runtime_builder.py

Laufzeitabhängiger Composition-Builder des Konverters.

15.6.15.1 Erwartete Informationen / Eingaben

- Erwartet die bereits initialisierte ConverterThread-Fassade; lädt ToolPaths und erzeugt/verknüpft die Runtime-Services erst im Worker-Thread.

15.6.15.2 Rückgabe / Ausgabe

- Schreibt die erzeugten Serviceinstanzen auf die Worker-Fassade und stellt ConversionWorkflowRunner bereit.

15.6.16 dragontools/worker/converter_run_loop.py

Session- und Live-Queue-Orchestrierung ohne Encoding-Fachlogik.

15.6.16.1 Erwartete Informationen / Eingaben

- Verarbeitet Worker-Queue, Display-Index, Gesamtfortschritt und Abort-Semantik; nutzt ffprobe nur für die schnelle Auflösungsanzeige.

15.6.16.2 Rückgabe / Ausgabe

- Start-/Datei-/Abschlussaktivitäten, Fortschrittssignale und Aufruf von ConverterThread.convert_file().

15.6.17 dragontools/worker/converter_lifecycle.py

Äußere Fehlergrenze und Abschlussarbeiten von ConverterThread.run().

15.6.17.1 Erwartete Informationen / Eingaben

- Ermittelt offene Queue-Dateien nach einem kritischen Run-Fehler, wartet Postprocessing ab und bereinigt temporäre Overwrite-Verzeichnisse.

15.6.17.2 Rückgabe / Ausgabe

- Fehlerstatus/Crashreport sowie kontrollierter Sessionabschluss ohne Änderung der Pipeline-Fachregeln.

15.6.18 dragontools/worker/converter_file_executor.py

Per-Datei-Orchestrierung und SourceVisualCheck-Preflight.

15.6.18.1 Erwartete Informationen / Eingaben

- Normalisiert File-Overrides, startet den bereits aufgebauten WorkflowRunner und hält die bestehende Conversion-Error-Report-Grenze.

15.6.18.2 Rückgabe / Ausgabe

- Boolescher Dateierfolg sowie bestehende file_result-/failure_details-Wirkung.

15.6.19 dragontools/worker/converter_control.py

Pause-, Abort-, Diagnose- und Prozesskontrolle des Converter-Workers.

15.6.19.1 Erwartete Informationen / Eingaben

- Arbeitet auf Queue-/Runtime-/Prozesszustand der Fassade und verwendet weiterhin process_control.terminate_process_tree/wait_while_paused.

15.6.19.2 Rückgabe / Ausgabe

- Diagnose-Snapshot, Pause/Resume, sofortiger bzw. nach-Datei-Abbruch und gezieltes Beenden des aktuellen FFmpeg-Prozesses.

15.6.20 dragontools/worker/converter_utils.py

Merkmal	Wert
Kategorie	Worker
Umfang	108 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (108 Zeilen, direkte Importnutzer: 2).

15.6.20.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _fs, _fd, _parse_crop, _level5_json.

15.6.20.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `b`, `s`, `crop_filter`, `src_w`, `src_h`, `cw`, `ch`, `cx`, `cy`, `path`

15.6.20.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `dict`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool
Direkt verwendet von	dragontools/worker/dv_level5_editor.py, dragontools/worker/hdrplus_conversion.py

15.6.20.4 Wichtige Funktionen

`_fs(b)`

`_fd(s)`

`_parse_crop(crop_filter)`

'crop=W:H:X:Y' → (w,h,x,y) oder None.

`_level5_json(src_w, src_h, cw, ch, cx, cy, path) -> dict`

Erstellt eine dovi_tool-editor JSON im active_area-Format, passend zu der funktionierenden manuellen Crop-JSON. Beispiel:

`_autocrop_fallback_json(path) -> None`

Minimaler Fallback für dovi_tool editor. Entspricht deiner funktionierenden config.json: {

15.6.20.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → dovi_tool → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.20.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.21 dragontools/worker/duration_repair_models.py

Merkmal	Wert
Kategorie	Worker
Umfang	175 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (175 Zeilen, direkte Importnutzer: 2).

15.6.21.1 Aufgabe des Moduls

Enthält MediaTimingInfo, TimestampProblem, TimestampRepairResult.

15.6.21.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `info`, `value`, `expected`

15.6.21.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `float | None`, `bool`, `TimestampProblem`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/workflow_engine.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/duration_repair_service.py, dragontools/worker/duration_timing_analyzer.py

15.6.21.4 Wichtige Klassen und Methoden

MediaTimingInfo [object]

`expected_video_duration_s() -> float | None`

TimestampProblem [object]

TimestampRepairResult [object]

DurationRepairOutcome [object]

15.6.21.5 Wichtige Funktionen

```
calculate_expected_duration(info) -> float | None
duration_close(value, expected) -> bool
is_extreme_mismatch(value, expected) -> bool
detect_timestamp_problem(info) -> TimestampProblem
```

Decide whether a damaged CFR video timeline can be rebuilt safely.

15.6.21.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.21.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.22 dragontools/worker/duration_repair_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	640 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (640 Zeilen, direkt von 2 Projektmodulen importiert).

15.6.22.1 Aufgabe des Moduls

Führt MKV-Remux und Timestamp-Reparatur aus.

15.6.22.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `seconds`, `archive_dir`, `filename`, `fps`, `command`, `option`

15.6.22.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `Path`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/timeout_settings.py, dragontools/worker/base_worker.py, dragontools/worker/duration_repair_models.py, dragontools/worker/duration_timing_analyzer.py, dragontools/worker/tool_runner.py, dragontools/worker/workflow_engine.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe, mkvmerge

Abhängigkeit	Details
Direkt verwendet von	dragontools/tests/test_duration_repair_service.py, dragontools/worker/converter_thread.py

15.6.22.4 Wichtige Klassen und Methoden

DurationRepairService [object] – Repairs MKV duration mismatches via remux and optional timestamp rebuild.

```
can_repair() -> bool
repair() -> DurationRepairOutcome
attempt_normal_remux() -> tuple[WorkflowVerifyResult, float | None, bool, str]
get_media_timing_info(path) -> MediaTimingInfo
repair_video_timestamps() -> TimestampRepairResult
validate_timestamp_repair() -> tuple[bool, list[str]]
```

15.6.22.5 Wichtige Funktionen

```
_fmt_duration(seconds) -> str
_unique_archive_path(archive_dir, filename) -> Path
_setts_filter_for_fps(fps) -> str
_command_arg_after(command, option) -> str
```

15.6.22.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / ffprobe / mkvmerge → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.22.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), (ValueError, IndexError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.23 dragontools/worker/duration_timing_analyzer.py

Merkmal	Wert
Kategorie	Worker
Umfang	373 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (373 Zeilen, direkte Importnutzer: 1).

15.6.23.1 Aufgabe des Moduls

Analysiert Container-, Stream-, Frame- und Framerate-Zeiten.

15.6.23.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `stream`, `value`, `info`, `fps`, `avg`, `real`, `values`

15.6.23.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `float | None`, `int | None`, `Fraction | None`, `str`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/base_worker.py, dragontools/worker/duration_repair_models.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffprobe, mediainfo
Direkt verwendet von	dragontools/worker/duration_repair_service.py

15.6.23.4 Wichtige Klassen und Methoden

MediaTimingAnalyzer [object] – Liest und normalisiert Container-, Stream-, Frame- und FPS-Zeitdaten.

```
get_media_timing_info(path) -> MediaTimingInfo
run_ffprobe_json(path) -> dict
run_mediainfo_json(path) -> dict
apply_ffprobe_timing(info, data) -> None
apply_mediainfo_timing(info, data) -> None
derive_frame_rate_from_source_duration(info, expected_duration_s) -> None
infer_frame_rate_mode(info) -> str
```

15.6.23.5 Wichtige Funktionen

```
_stream_duration(stream) -> float | None
_parse_seconds(value) -> float | None
_parse_duration_tag(value) -> float | None
_parse_mediainfo_duration(value) -> float | None
_parse_int(value) -> int | None
_parse_fraction(value) -> Fraction | None
```

15.6.23.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffprobe / mediainfo → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.23.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.24 dragontools/worker/dv_audio_mux_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	287 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (287 Zeilen, direkte Importnutzer: 1).

15.6.24.1 Aufgabe des Moduls

Bereitet Audio für DV-MP4 vor.

15.6.24.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration

- Toolpfade und Laufzeitstatus

15.6.24.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/rules/audio_plan.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffprobe, mp4box
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.24.4 Wichtige Klassen und Methoden

DVExtractedAudioTrack [object]

DVMuxAudioTrack [object]

DVAudioMuxService [object]

```
build_audio_meta(mi, ov, container) -> list[dict]
audio_track_name(meta) -> str
resolve_audio_mux_inputs() -> tuple[list[DVExtractedAudioTrack], list[DVMuxAudioTrack]]
mux_plain_mp4_without_dv() -> tuple[bool, list[DVExtractedAudioTrack]]
```

15.6.24.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffprobe / mp4box → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.24.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.25 dragontools/worker/dv_crop_reconcile.py

Merkmal	Wert
Kategorie	Worker
Umfang	13 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (13 Zeilen, direkte Importnutzer: 1).

15.6.25.1 Aufgabe des Moduls

Vergleicht den physisch erkannten FFmpeg-AutoCrop mit der Dolby-Vision-RPU-Level-5-Active-Area und entscheidet, welcher Crop sicher verwendet werden kann.

15.6.25.2 Erwartete Informationen / Eingaben

Quellauflösung, AutoCrop-Filter, exportierte Level-5-Offets bzw. RPU-Datei, dovi_tool, Prozessrunner und optional ein Benutzerentscheid bei großen Abweichungen.

15.6.25.3 Rückgabe / Ausgabe

DVCropOutcome mit gewähltem Crop, Herkunft der Entscheidung, optionalem disable_dv sowie Fehlergrund.

15.6.25.4 Wichtige Klassen und Funktionen

CropRect; DVCropComparison; DVCropOutcome; parse_crop_filter(); parse_level5_export(); compare_crops(); automatic_choice(); reconcile_dv_crop().

15.6.25.5 Fachregel

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/dv_processing_pipeline.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py

Bis einschließlich 20 Pixel Abweichung gewinnt automatisch der weniger aggressive Crop mit der größeren aktiven Bildfläche. Bei mehr als 20 Pixel entscheidet der Benutzer zwischen AutoCrop, RPU und DV deaktivieren. Mehrere wechselnde Level-5-Bereiche werden nicht auf einen einzigen globalen Crop reduziert.

15.6.25.6 Fehlerbehandlung und Diagnose

Kann Level 5 nicht exportiert werden, bleibt AutoCrop maßgeblich. Ungültige Cropwerte werden nicht erzwungen; dynamische RPU-Bereiche schützen IMAX-/Aspect-Ratio-Wechsel vor falschem Flattening.

15.6.26 dragontools/worker/dv_failure_recovery.py

Merkmal	Wert
Kategorie	Worker
Umfang	110 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (110 Zeilen, direkte Importnutzer: 1).

15.6.26.1 Aufgabe des Moduls

Enthält DVFailureRecovery.

15.6.26.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.26.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/path_safety.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.26.4 Wichtige Klassen und Methoden

DVFailureRecovery [object]

```
cleanup_tmp_sub() -> None
save_dv_crop_failure() -> Path
```

15.6.26.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.26.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.27 dragontools/worker/dv_level5_editor.py

Merkmal	Wert
Kategorie	Worker
Umfang	147 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (147 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.27.1 Aufgabe des Moduls

Passt DV-Level-5-Cropdaten an.

15.6.27.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.27.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/converter_utils.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool
Direkt verwendet von	dragontools/tests/test_dv_regressions.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_processing_pipeline.py

15.6.27.4 Wichtige Klassen und Methoden

DVLevel5Editor [object]

```
resolve_rpu_with_fallback(run_cmd) -> Path | None
```

```
edit_rpu_for_crop() -> bool
```

15.6.27.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → dovi_tool → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.27.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.28 dragontools/worker/dv_mp4box_muxer.py

Merkmal	Wert
Kategorie	Worker
Umfang	59 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (59 Zeilen, direkte Importnutzer: 1).

15.6.28.1 Aufgabe des Moduls

Finalisiert DV-MP4 mit MP4Box.

15.6.28.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.28.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.28.4 Wichtige Klassen und Methoden

DVMP4BoxMuxer [object]

```
add_audio_tracks(mp4_cmd, mux_tracks) -> None
mux_plain_mp4_without_dv(run_fn) -> bool
mux_final_output(run_fn) -> bool
```

15.6.28.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.28.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.29 dragontools/worker/dv_pipeline_timeouts.py

Merkmal	Wert
Kategorie	Worker
Umfang	40 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (40 Zeilen, direkte Importnutzer: 1).

15.6.29.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie timeout_hevc_extract, timeout_dovi_convert, timeout_rpu_extract, timeout_hevc_re_extract.

15.6.29.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.29.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `int`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/timeout_settings.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	mp4box
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.29.4 Wichtige Funktionen

```
timeout_hevc_extract() -> int
timeout_dovi_convert() -> int
timeout_rpu_extract() -> int
timeout_hevc_re_extract() -> int
timeout_dovi_editor() -> int
timeout_rpu_inject() -> int
timeout_mp4box() -> int
```

timeout_mkvmerge() -> int

```
timeout_audio() -> int
timeout_encode() -> int
```

15.6.29.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → mp4box → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.29.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.30 dragontools/worker/dv_processing_pipeline.py

Merkmal	Wert
Kategorie	Worker
Umfang	945 Quellzeilen
Bedeutung	Zentraler Baustein (945 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.30.1 Aufgabe des Moduls

DV-Encoding-Pipeline mit RPU, Crop, Audio sowie containerabhängig MP4Box oder mkvmerge.

15.6.30.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `encoder_config`, `input_path`, `output_path`, `mi`, `vf_args`, `audio_args`, `audio_input_args`, `sn`, `crop`, `ov`, `preserve_hdrplus`

15.6.30.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `list`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/models.py, dragontools/worker/base_worker.py, dragontools/worker/command_formatting.py, dragontools/worker/dv_audio_mux_service.py, dragontools/worker/dv_failure_recovery.py, dragontools/worker/dv_level5_editor.py, dragontools/worker/dv_mp4box_muxer.py, dragontools/worker/dv_pipeline_timeouts.py, dragontools/worker/dv_rpu_service.py, dragontools/worker/dv_runtime_models.py, dragontools/worker/subtitle_sidecar_service.py, dragontools/worker/dv_video_filters.py, dragontools/worker/encoder_args.py, dragontools/worker/hdr10plus_bitstream_service.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	dovi_tool, ffmpeg, mp4box
Direkt verwendet von	dragontools/tests/test_hdr10_color_standard.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_processing_pipeline.py

15.6.30.4 Wichtige Klassen und Methoden

DVProcessingPipeline [object]

```
run(input_path, output_path, mi, vf_args, audio_args, audio_input_args, sn, crop, ov, preserve_hdrplus) -> bool
```

15.6.30.5 Wichtige Funktionen

`_dv_video_encode_args(encoder_config) -> list`

DV-Video-Encoding-Argumente plus HDR10-VUI ohne Pixelformat-Doppelung.

15.6.30.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → dovi_tool / ffmpeg / MP4Box bzw. mkvmerge → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.30.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, FileNotFoundError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.31 dragontools/worker/dv_remux_components.py

Merkmal	Wert
Kategorie	Worker
Umfang	506 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (506 Zeilen, direkt von 1 Projektmodulen importiert).

15.6.31.1 Aufgabe des Moduls

Enthält DVRemuxProcessRunner, DVSubtitleExporter, DVMP4BoxPipelineRunner.

15.6.31.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `worker`, `process_runner`, `input_path`, `output_path`, `mi`, `dur_ms`, `file_override`, `name`

15.6.31.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/core/models.py, dragontools/rules/audio_plan.py, dragontools/rules/subtitle_rules.py, dragontools/worker/base_worker.py, dragontools/worker/output_size_policy.py, dragontools/worker/process_control.py, dragontools/worker/subtitle_sidecar_service.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe, mp4box, mkvmerge

Abhängigkeit	Details
Direkt verwendet von	dragontools/worker/dv_remux_thread.py

15.6.31.4 Wichtige Klassen und Methoden

DVRemuxProcessRunner [object] – Subprocess-Ausführung, Fortschritt und abortable capture für DV-Remux.

```
run_cmd(cmd, input_path, dur_ms, pct_range) -> int
probe_ms(path) -> int | None
run_abortable_capture(cmd) -> tuple[int, str, str]
```

DVSubtitleExporter [object] – Wrapper um SubtitleSidecarService für DV-Remux. Kapselt abort_check (worker.abort_requested) und leitet an SubtitleSidecarService.export_sidecars() weiter.

```
handle_export(input_path, output_path, mi, file_override) -> list[str]
extract(input_path, output_path, mi, file_override) -> list[str]
```

DVMP4BoxPipelineRunner [object] – Video-/Audio-Extraktion und finales MP4Box-Muxing.

```
build_audio_jobs(mi, file_override) -> list[dict]
extract_video(input_path, output_hevc, dur_ms) -> bool
extract_audio(input_path, audio_job, output_audio, dur_ms) -> bool
mux(video_hevc, audio_tracks, output_path) -> bool
run(input_path, output_path, mi, dur_ms, file_override, name) -> bool
```

DVOutputManager [object] – Output-Pfad, Replacement und Cleanup für DV-Remux.

```
archiviert() -> int
build_output_path(input_path) -> str
replace_output_if_needed(input_path, output_path) -> tuple[bool, str]
cleanup_temp_dir(input_path) -> None
cleanup_incomplete(input_path, output_path) -> None
```

15.6.31.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → mediainfo / mp4box → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.31.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, FileNotFoundError, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.32 dragontools/worker/dv_remux_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	375 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (375 Zeilen, direkte Importnutzer: 1).

15.6.32.1 Aufgabe des Moduls

dragontools/worker/dv_remux_thread.py Technischer Kontext aus dem Quellcode: dragontools/worker/dv_remux_thread.py DV-Remux-Thread: containerabhängiger Remux für Dolby-Vision-Dateien (MP4/MKV). Audio: normal konvertieren (nach Audioregeln) Video: unverändert (kein Re-Encoding!) Untertitel: werden im dedizierten DV-Remux optional als externe Sidecars neben der DV-Ausgabedatei exportiert. Sie werden nur dann als externe .srt/.ass-Dateien im Quellordner abgelegt, wenn die Subtitle-Regel ``dv_extract_external_subs`` = True ist; a…

DV-Remux-Thread: containerabhängiger Remux für Dolby-Vision-Dateien (MP4/MKV). - Audio: normal konvertieren (nach Audioregeln) - Video: unverändert (kein Re-Encoding!)

15.6.32.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `b`, `s`, `files`

15.6.32.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/rules/rule_loader.py, dragontools/worker/base_worker.py, dragontools/worker/converter_queue_state.py, dragontools/worker/dv_remux_components.py, dragontools/worker/process_control.py, dragontools/worker/worker_events.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg, ffprobe, mp4box, mkvmerge
Direkt verwendet von	dragontools/gui/conversion_controller.py

15.6.32.4 Wichtige Klassen und Methoden

DVRemuxThread [QThread] – Remuxt Dolby Vision ohne Video-Re-Encoding nach MP4 oder MKV; Audio folgt den zentralen Regeln, externer Untertitel-Export ist optional (dv_extract_external_subs).

```
request_abort(mode)
cancel() -> None
pause() -> None
resume() -> None
add_file(path) -> bool
remove_file(path) -> RemoveFileStatus
is_current(path) -> bool
reorder_waiting_files(new_order) -> None
```

15.6.32.5 Wichtige Funktionen

```
format_file_size(b)
format_duration(s)
```

15.6.32.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / MP4Box bzw. mkvmerge → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.32.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.33 dragontools/worker/dv_rpu_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	53 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (53 Zeilen, direkte Importnutzer: 1).

15.6.33.1 Aufgabe des Moduls

Kapselt dovi_tool für RPU-Extraktion und -Injection.

15.6.33.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.33.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.33.4 Wichtige Klassen und Methoden

DVRpuService [object]

```
extract_rpu(run_cmd) -> bool
inject_rpu(run_cmd) -> bool
```

15.6.33.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.33.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.34 dragontools/worker/dv_runtime_models.py

Merkmal	Wert
Kategorie	Worker
Umfang	19 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (19 Zeilen, direkt von 5 Projektmodulen importiert).

15.6.34.1 Aufgabe des Moduls

Enthält DVEncoderConfig, DVTempState.

15.6.34.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.34.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_hdr10_color_standard.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_processing_pipeline.py, dragontools/worker/hdrplus_conversion.py, dragontools/worker/workflow_services.py

15.6.34.4 Wichtige Klassen und Methoden

DVEncoderConfig [object]

DVTempState [object]

15.6.34.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.34.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.35 dragontools/worker/dv_mkv_muxer.py

Merkmal	Wert
Kategorie	Worker
Umfang	66 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (66 Zeilen, direkte Importnutzer: 1).

15.6.35.1 Aufgabe des Moduls

Finaler Matroska-Mux für bereits injizierte Dolby-Vision-HEVC-Bitstreams. Der Bitstream wird nicht neu encodiert.

15.6.35.2 Erwartete Informationen / Eingaben

mkvmerge-Pfad, injizierter HEVC-Bitstream, finaler Ausgabepfad und vorbereitete Audio-Tracks mit Sprache/Titel.

15.6.35.3 Rückgabe / Ausgabe

True nur bei erfolgreichem mkvmerge-Aufruf und tatsächlich erzeugter, nicht leerer MKV-Datei.

15.6.35.4 Wichtige Klasse und Methode

DVMKVMuxer; mux_final_output(run_fn, output_path, injected_hevc, mux_tracks) -> bool.

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/subtitle_sidecar_service.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.35.5 Datenverarbeitung und Kommunikation

Injizierter DV-HEVC-Bitstream + vorbereitete Audioelementarstreams -> mkvmerge -> finale MKV -> nachgelagerte MediaInfo-/Bitstream-Verifikation. Ein FFmpeg-Fallback für den finalen DV-MKV-Mux ist bewusst nicht vorgesehen.

15.6.35.6 Fehlerbehandlung und Diagnose

Fehlende oder leere Trackdateien werden übersprungen. Ein fehlgeschlagener mkvmerge-Lauf oder eine nicht erzeugte Ausgabedatei liefert False und beendet den DV-Erhalt fail-closed.

15.6.36 dragontools/worker/dv_video_filters.py

Merkmal	Wert
Kategorie	Worker
Umfang	122 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (122 Zeilen, direkte Importnutzer: 1).

15.6.36.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie build_dv5_libplacebo_vf, inject_dv_colorspace, remap_vf_to_input1.

15.6.36.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `vf_args`, `is_p5`

15.6.36.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `list`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.

Abhängigkeit	Details
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/worker/dv_processing_pipeline.py

15.6.36.4 Wichtige Funktionen

`build_dv5_libplacebo_vf(vf_args) -> list`

Build ffmpeg filter args for DV Profile 5 base-layer conversion.

`inject_dv_colorspace(vf_args, is_p5) -> list`

Inject stable DV color metadata and p010 handoff into ffmpeg filter args.

`remap_vf_to_input1(vf_args) -> list`

Move video filter references from the source input to the encoded input.

15.6.36.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.36.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (ValueError, IndexError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.37 dragontools/worker/encode_plan.py

Merkmal	Wert
Kategorie	Worker
Umfang	15 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (15 Zeilen, direkte Importnutzer: 1).

15.6.37.1 Aufgabe des Moduls

Enthält EncodePlan.

15.6.37.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.37.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/encode_plan_service.py

15.6.37.4 Wichtige Klassen und Methoden

EncodePlan [object]

15.6.37.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.37.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.38 dragontools/worker/encode_plan_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	168 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (168 Zeilen, direkte Importnutzer: 2).

15.6.38.1 Aufgabe des Moduls

Enthält EncodePlanService.

15.6.38.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `media_info`

15.6.38.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py, dragontools/rules/subtitle_rules.py, dragontools/worker/encode_plan.py, dragontools/worker/encoder_args.py, dragontools/worker/hdr10_color.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	mediainfo
Direkt verwendet von	dragontools/tests/test_hdr10_color_standard.py, dragontools/worker/converter_thread.py

15.6.38.4 Wichtige Klassen und Methoden

EncodePlanService [object] – Erzeugt den Encode-Plan für Standard-, DV- und HDR+-Pipelines.

```
prepare_encode_plan(input_path, output_path, media_info, pipeline, container, file_override) -> EncodePlan
```

15.6.38.5 Wichtige Funktionen

`_is_dv5_source(media_info) -> bool`

Gibt True zurück wenn die Quelle DV Profil 5 (ICtCp-Farbraum) ist. dv_profile_major liegt auf MediaInfo (nicht VideoStream).

15.6.38.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → mediainfo → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.38.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.39 dragontools/worker/encoder_args.py

Merkmal	Wert
Kategorie	Worker
Umfang	142 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (142 Zeilen, direkt von 6 Projektmodulen importiert).

15.6.39.1 Aufgabe des Moduls

Erzeugt FFmpeg-Videoencoderargumente inklusive H.265-10-bit.

15.6.39.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `options`, `key`, `default`, `allowed`, `encoder_key`, `codec`, `crf`, `preset`, `encoder_options`, `scale_mode`

15.6.39.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `int`, `str | None`, `list[str]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_encoder_args.py, dragontools/worker/dv_processing_pipeline.py, dragontools/worker/encode_plan_service.py, dragontools/worker/hdrplus_conversion.py, dragontools/worker/quality_test_thread.py, dragontools/worker/standard_pipeline_runner.py

15.6.39.4 Wichtige Funktionen

`_int_option(options, key, default) -> int`

```
_text_option(options, key, allowed) -> str | None
_h265_10bit_args(encoder_key) -> list[str]
_cpu_args(codec, crf, preset, encoder_options)
_nvenc_args(codec, crf, encoder_options)
_qsv_args(codec, crf, encoder_options)
```

15.6.39.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.39.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.40 dragontools/worker/hdr10_color.py

Merkmal	Wert
Kategorie	Worker
Umfang	88 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (88 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.40.1 Aufgabe des Moduls

Enthält Hilfsfunktionen wie _norm, _is_pq, _is_bt2020, _profile_major.

15.6.40.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `value`, `media_info`, `target_codec`

15.6.40.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `bool`, `int | None`, `list[str]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_hdr10_color_standard.py, dragontools/worker/encode_plan_service.py, dragontools/worker/standard_pipeline_runner.py

15.6.40.4 Wichtige Funktionen

```
source_has_hdr10_base(media_info) -> bool
```

True when the source should be treated as HDR10-compatible BT.2020/PQ.

```
should_apply_standard_hdr10_color(media_info, target_codec) -> bool
hdr10_output_args(media_info, target_codec) -> list[str]
```

15.6.40.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.40.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError). Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.41 dragontools/worker/hdr10plus_bitstream_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	95 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (95 Zeilen, direkte Importnutzer: 2).

15.6.41.1 Aufgabe des Moduls

Kapselt hdr10plus_tool Extract/Inject/Verify.

15.6.41.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.41.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	hdr10plus_tool
Direkt verwendet von	dragontools/tests/test_dv_regressions.py, dragontools/worker/dv_processing_pipeline.py

15.6.41.4 Wichtige Klassen und Methoden

HDR10PlusBitstreamService [object] – Kleine Hülle um hdr10plus_tool für HEVC-Bitstream-Metadaten.

```
extract_metadata(run_cmd) -> bool
inject_metadata(run_cmd) -> bool
verify_metadata(run_cmd) -> bool
```

15.6.41.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → hdr10plus_tool → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.41.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.42 dragontools/worker/hdrplus_conversion.py

Merkmal	Wert
Kategorie	Worker
Umfang	502 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (502 Zeilen, direkt von 1 Projektmodulen importiert).

15.6.42.1 Aufgabe des Moduls

HDR10+-Encoding-Pipeline mit Metadatenextraktion und Injection.

15.6.42.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `input_path`, `output_path`, `mi`, `vf_args`, `audio_args`, `audio_input_args`, `sn`, `crop`, `ov`

15.6.42.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/timeout_settings.py, dragontools/worker/converter_utils.py, dragontools/worker/dv_runtime_models.py, dragontools/worker/encoder_args.py, dragontools/worker/tool_runner.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, hdr10plus_tool
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.42.4 Wichtige Klassen und Methoden

HDRPlusConversionHelper [object] – Führt den HDR10+-Spezialpfad durch. Vollständig entkoppelt vom ConverterThread: alle Abhängigkeiten werden explizit als Konstruktorparameter übergeben, kein Worker-Backreference.

```
run(input_path, output_path, mi, vf_args, audio_args, audio_input_args, sn, crop, ov) -> bool
```

15.6.42.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / hdr10plus_tool → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.42.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.43 dragontools/worker/interfaces.py

Merkmal	Wert
Kategorie	Worker
Umfang	125 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (125 Zeilen, direkte Importnutzer: 2).

15.6.43.1 Aufgabe des Moduls

Enthält WorkerProtocol, PausableWorkerProtocol, AnalysisService.

15.6.43.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `input_path`, `ctx`, `override`

15.6.43.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `tuple[Any, int | None, int]`, `Any`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_dv_remux_abort_fix.py, dragontools/worker/workflow_services.py

15.6.43.4 Wichtige Klassen und Methoden

WorkerProtocol [Protocol]

```
request_abort(mode) -> None
cancel() -> None
```

PausableWorkerProtocol [WorkerProtocol, Protocol]

```
pause() -> None
resume() -> None
```

AnalysisService [Protocol]

```
analyze(input_path) -> tuple[Any, int | None, int]
```

PipelineService [Protocol]

```
select_pipeline_context(input_path, media_info) -> tuple[str, str]
```

EncodeService [Protocol]

```
prepare_encode_plan(input_path, output_path, media_info, pipeline, container, override) -> Any
```

VerifyService [Protocol]

```
verify(output_path, container) -> Any
```

ReplaceService [Protocol]

```
replace() -> str
```

CleanupService [Protocol]

```
cleanup_temp_artifacts() -> None
```

ResultService [Protocol]

```
finalize_success(ctx) -> None
finalize_success_pending_postprocess(ctx) -> None
finalize_blocked(ctx) -> None
fail(ctx, reason) -> None
```

OutputPathService [Protocol]

```
resolve_output_path(input_path, container) -> tuple[Path, str]
temp_overwrite_dir(base_dir) -> Path
```

15.6.43.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.43.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.44 dragontools/worker/iso_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	781 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (781 Zeilen, direkt von 1 Projektmodulen importiert).

15.6.44.1 Aufgabe des Moduls

MakeMKV-Worker für ISO und Disc-Ordner.

15.6.44.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `value`, `inputs`, `output_dir`, `tools`, `selected_titles`, `auto_main_title`, `handoff_to_converter`, `scan_only`, `ffmpeg_fallback`, `parent`

15.6.44.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `int`, `bool`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/paths.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	PyQt6

Abhängigkeit	Details
Externe Programme/Tools	ffmpeg, makemkvcon
Direkt verwendet von	dragontools/gui/iso_widget.py

15.6.44.4 Wichtige Klassen und Methoden

_UserAbortError [RuntimeError]

ISOThread [QThread] – Verarbeitet ISO-/Disc-Strukturen konservativ über MakeMKV CLI. FFmpeg dient nur als optionaler Fallback fuer unverschluesselte, direkt lesbare Strukturen. MakeMKV bleibt der robuste Primaerweg.

```
request_abort(mode) -> None
cancel() -> None
run() -> None
```

15.6.44.5 Wichtige Funktionen

```
_safe_name(path) -> str
_parse_duration_to_seconds(value) -> int
_parse_size_to_bytes(value) -> int
_format_size(value) -> str
_tool_exists(path) -> bool
_quote_concat_path(path) -> str
```

15.6.44.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / makemkvcon → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.44.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, _UserAbortError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.45 dragontools/worker/media_analysis_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	35 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (35 Zeilen, direkte Importnutzer: 1).

15.6.45.1 Aufgabe des Moduls

Enthält MediaAnalysisService.

15.6.45.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `input_path`

15.6.45.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `tuple[object, int | None, int]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.45.4 Wichtige Klassen und Methoden

MediaAnalysisService [object] – Kapselt Medienanalyse inkl. Dauer/Dateigroesse und Warnungslogging.

```
analyze(input_path) -> tuple[object, int | None, int]
log_analysis_warnings(media_info) -> None
```

15.6.45.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.45.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.46 dragontools/worker/merge_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	465 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (465 Zeilen, direkt von 1 Projektmodulen importiert).

15.6.46.1 Aufgabe des Moduls

Kompatibilitätsprüfung und verlustfreier Merge.

15.6.46.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `format_name`, `value`, `streams`, `files`, `output_path`, `tools`, `mode`, `parent`

15.6.46.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `float | None`, `list[dict[str, Any]]`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/worker/base_worker.py

Abhängigkeit	Details
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffprobe, mkvmerge
Direkt verwendet von	dragontools/gui/merge_widget.py

15.6.46.4 Wichtige Klassen und Methoden

_UserAbortError [RuntimeError]

MergeThread [BaseWorker] – Produktive Basis für konservatives lossless Merge.

```
run() -> None
```

15.6.46.5 Wichtige Funktionen

```
_container_from_path(path) -> str
_container_from_format_name(format_name) -> str
_parse_fps(value) -> float | None
_audio_signature(streams) -> list[dict[str, Any]]
_subtitle_signature(streams) -> list[dict[str, Any]]
```

15.6.46.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffprobe / mkvmerge → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.46.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, _UserAbortError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.47 dragontools/worker/move_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	786 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (786 Zeilen, direkt von 2 Projektmodulen importiert).

15.6.47.1 Aufgabe des Moduls

Verschiebt Dateien nach Zielregeln und fragt fehlende Ziele ab. Technischer Kontext aus dem Quellcode: dragontools/worker/move_thread.py – vollständig neu, robuste Film/Serien-Logik

15.6.47.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `p`, `path`, `a`, `b`, `dateipfade`, `tv_path`, `anime_path`, `filme_path`

15.6.47.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/move_conflicts.py, dragontools/core/settings.py, dragontools/core/system_shutdown.py, dragontools/rules/move_rules.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/move_preflight_controller.py, dragontools/tests/test_postprocess_move_sidecars.py
Verwendete Settings-Konstanten	SET_KEY_NFO_MOVIE_TARGET_NAME, SET_KEY_TRICKPLAY_CONFLICT_MODE, SET_KEY_TRICKPLAY_ONLY_MISSING

15.6.47.4 Wichtige Klassen und Methoden

MoveThread [QThread]

```
provide_decision(rid, resp)
add_planned_target(path, target) -> None
request_abort(mode) -> None
pause() -> None
resume() -> None
run()
```

15.6.47.5 Wichtige Funktionen

```
_fmt(p)
_safe_stem(path)
_similar(a, b)
```

15.6.47.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.47.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (OSError, FileExistsError), Exception, OSError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.48 dragontools/worker/mp4_remux_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	402 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (402 Zeilen, direkt von 1 Projektmodulen importiert).

15.6.48.1 Aufgabe des Moduls

Einfacher MP4-Remux mit Schutz vor DV/HDR10+.

15.6.48.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `files`, `output_dir`, `tools`, `apply_audio_rules`, `export_subtitles`, `ignore_subtitles`, `subtitle_rules`, `faststart`, `overwrite_original`, `parent`

15.6.48.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/core/logger.py, dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/rules/audio_plan.py, dragontools/rules/rule_loader.py, dragontools/rules/subtitle_rules.py, dragontools/worker/base_worker.py, dragontools/worker/subtitle_sidecar_service.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/mp4_remux_widget.py

15.6.48.4 Wichtige Klassen und Methoden

_UserAbortError [RuntimeError] – Interner Marker für benutzerinitiierte Abbrueche während ffmpeg läuft.

MP4RemuxThread [BaseWorker] – Produktive Basis für MP4-Remux ohne DV und ohne Video-Reencode.

```
run() -> None
```

15.6.48.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.48.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, _UserAbortError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.49 dragontools/worker/output_path_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	59 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (59 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.49.1 Aufgabe des Moduls

Enthält OutputPathService.

15.6.49.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `candidate`

15.6.49.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`, `Path`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_output_path_service.py, dragontools/worker/cleanup_service.py, dragontools/worker/converter_thread.py

15.6.49.4 Wichtige Klassen und Methoden

OutputPathService [object] – Bestimmt Ausgabepfade für Standard- und Overwrite-Läufe.

```
resolve_output_path(input_path, container) -> tuple[Path, str]
temp_overwrite_dir(base_dir) -> Path
```

15.6.49.5 Wichtige Funktionen

```
release_output_path_reservation(path) -> None
```

15.6.49.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.49.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.50 dragontools/worker/output_size_policy.py

Merkmal	Wert
Kategorie	Worker
Umfang	140 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (140 Zeilen, direkte Importnutzer: 2).

15.6.50.1 Aufgabe des Moduls

Archiviert Ausgaben, die Größenregeln verletzen.

15.6.50.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `logger`, `message`, `directory`, `filename`, `input_path`, `output_path`, `input_path_obj`, `output_path_obj`

15.6.50.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`, `Path`, `tuple[bool, Path | None]`, `Path | None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/dv_remux_components.py, dragontools/worker/replace_service.py
Verwendete Settings-Konstanten	SET_KEY_SAVE_ALLOW_LARGER_OUTPUT, SET_KEY_SAVE_ALLOW_LARGER_OUTPUT_PERCENT, SET_KEY_SAVE_MIN_OUTPUT_SIZE_ENABLED, SET_KEY_SAVE_MIN_OUTPUT_SIZE_PERCENT

15.6.50.4 Wichtige Funktionen

`validate_output_size_policy(input_path, output_path, logger) -> tuple[bool, Path | None]`

Prüft Größenregeln und archiviert die Ausgabe bei Regelverstoß. Returns: (True, None) – Regelkonform, normal fortfahren.

15.6.50.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.50.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.51 dragontools/worker/output_verifier.py

Merkmal	Wert
Kategorie	Worker
Umfang	133 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (133 Zeilen, direkte Importnutzer: 2).

15.6.51.1 Aufgabe des Moduls

Prüft Ausgabedateien mit ffprobe.

15.6.51.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration

- Toolpfade und Laufzeitstatus

15.6.51.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `WorkflowVerifyResult`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/base_worker.py, dragontools/worker/workflow_engine.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffprobe
Direkt verwendet von	dragontools/tests/test_output_verifier.py, dragontools/worker/converter_thread.py

15.6.51.4 Wichtige Klassen und Methoden

OutputVerifier [object] – Prüft erzeugte Ausgabedateien auf minimale Plausibilität.

```
verify(output_path, container) -> WorkflowVerifyResult
```

15.6.51.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffprobe → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.51.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.52 dragontools/worker/parallel_converter_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	538 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (538 Zeilen, direkt von 2 Projektmodulen importiert).

15.6.52.1 Aufgabe des Moduls

Koordiniert mehrere normale ConverterThread-Instanzen für parallele Bearbeitung.

15.6.52.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `files`, `codec`, `crf`, `preset`, `scale_mode`, `overwrite_original`

15.6.52.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/logger.py, dragontools/core/parallel_settings.py, dragontools/core/paths.py, dragontools/core/settings.py, dragontools/core/gpu_detection.py, dragontools/worker/base_worker.py, dragontools/worker/cleanup_service.py, dragontools/worker/converter_thread.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/gui/conversion_controller.py, dragontools/tests/test_parallel_converter_thread.py

15.6.52.4 Wichtige Klassen und Methoden

ParallelConverterThread [QObject] – Koordiniert mehrere normale ConverterThread-Instanzen als einen Lauf.

```
start() -> None
isRunning() -> bool
add_file(path) -> bool
remove_file(path) -> RemoveFileStatus
reorder_waiting_files(new_order) -> None
update_override(path, override) -> bool
pause() -> None
resume() -> None
request_abort(mode) -> None
is_current(path) -> bool
terminate_current_ffmpeg(path) -> bool
clear_abort_request() -> bool
set_parallel_jobs(jobs: int) -> int
```

Begrenzt den Wert auf 1 bis 8 und aktualisiert die Encode-Obergrenze auf dem Qt-Thread des Koordinators. Ein größerer Wert füllt freie Plätze aus der zentralen Queue, sofern keine Pause und kein Abbruch aktiv ist. Ein kleinerer Wert lässt bestehende Child-Worker auslaufen. Der Rückgabewert ist das wirksame neue Limit.

15.6.52.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.52.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.53 dragontools/worker/pipeline_decision_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	149 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (149 Zeilen, direkte Importnutzer: 1).

15.6.53.1 Aufgabe des Moduls

Enthält PipelineDecisionService.

15.6.53.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.53.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/models.py, dragontools/core/settings.py, dragontools/rules/pipeline_selector.py, dragontools/worker/archive_service.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_PRESERVE_DV, SET_KEY_PRESERVE_HDRPLUS

15.6.53.4 Wichtige Klassen und Methoden

PipelineDecisionService [object] – Kapselt Policy-Auswertung, Pipeline-Auswahl, Codec-Logging und Archivierung.

```
select_pipeline_context(input_path, media_info) -> tuple[str, str]
```

15.6.53.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.53.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.54 dragontools/worker/postprocess_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	491 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (491 Zeilen, direkt von 2 Projektmodulen importiert).

15.6.54.1 Aufgabe des Moduls

Erzeugt NFO und Trickplay nach erfolgreicher Konvertierung; seit V9.3 auch als Hintergrundauftrag.

15.6.54.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration

- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `settings`, `path`

15.6.54.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `PostProcessConfig`, `Path`, `str`, `int`, `list[str]`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/jellyfin_nfo.py, dragontools/core/online_metadata.py, dragontools/core/settings.py, dragontools/rules/move_rules.py, dragontools/worker/trickplay_service.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe
Direkt verwendet von	dragontools/tests/test_postprocess_service.py, dragontools/worker/converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_NFO_CONFLICT_MODE, SET_KEY_NFO_ENABLED, SET_KEY_NFO_FILEINFO_ENABLED, SET_KEY_NFO_MOVIE_TARGET_NAME, SET_KEY_NFO_ONLY_UNAMBIGUOUS, SET_KEY_TRICKPLAY_CONFLICT_MODE, SET_KEY_TRICKPLAY_ENABLED, SET_KEY_TRICKPLAY_HWACCEL, SET_KEY_TRICKPLAY_INTERVAL_S, SET_KEY_TRICKPLAY_JPEG_QUALITY, SET_KEY_TRICKPLAY_MAX_JOBS, SET_KEY_TRICKPLAY_ONLY_MISSING, SET_KEY_TRICKPLAY_QSCALE, SET_KEY_TRICKPLAY_SOURCE_MODE, SET_KEY_TRICKPLAY_TILE_COLUMNS, SET_KEY_TRICKPLAY_TILE_ROWS, SET_KEY_TRICKPLAY_WIDTH

15.6.54.4 Wichtige Klassen und Methoden

NfoSettings [object]

PostProcessConfig [object]

```
enabled() -> bool
```

PostProcessItem [object]

PostProcessRunResult [object]

PostProcessService [object]

```
is_enabled() -> bool
run() -> list[str]
```

```
run_result() -> PostProcessRunResult
```

AsyncPostProcessCoordinator [object] – Fuehrt NFO-/Trickplay-Nacharbeit im Hintergrund aus.

```
submit() -> bool
wait_for_all() -> None
```

15.6.54.5 Wichtige Funktionen

```
config_from_settings(settings) -> PostProcessConfig
postprocess_max_workers(settings) -> int
```

15.6.54.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / ffprobe → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.54.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, OSError, OnlineMetadataAuthError, OnlineMetadataError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.55 dragontools/worker/process_control.py

Merkmal	Wert
Kategorie	Worker
Umfang	233 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (233 Zeilen, direkt von 5 Projektmodulen importiert).

15.6.55.1 Aufgabe des Moduls

dragontools/worker/process_control.py Technischer Kontext aus dem Quellcode: dragontools/worker/process_control.py Plattformübergreifendes Suspendieren / Fortsetzen eines laufenden subprocess.Popen-Prozesses. Windows: NtSuspendProcess / NtResumeProcess via WinAPI (korrekte Prozess-API). OpenThread(pid) wäre falsch – proc.pid ist eine Prozess-ID, kein Thread-ID. Der entstehende Handle wäre ungültig; SuspendThread haette still versag…

15.6.55.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `proc`, `pid`, `worker`, `lock`

15.6.55.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.

Abhängigkeit	Details
Direkt verwendet von	dragontools/worker/converter_progress.py, dragontools/worker/converter_thread.py, dragontools/worker/dv_remux_components.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/tool_runner.py

15.6.55.4 Wichtige Funktionen

`suspend_process(proc) -> None`

Suspendiert *proc* plattformspezifisch. Fehler werden stillschweigend ignoriert, damit ein fehlschlagendes Suspend den Haupt-Worker nicht stoppt.

`resume_process(proc) -> None`

Setzt einen suspendierten *proc* fort. Fehler werden stillschweigend ignoriert.

`terminate_process_tree(worker, lock) -> bool`

Beendet den aktuell registrierten Prozess ohne den Worker-Lock zu halten.

`wait_while_paused(worker, lock) -> None`

Haelt den aufrufenden Thread an solange ``worker._paused`` True ist. Liest den laufenden Prozess einmalig und suspendiert ihn, damit die CPU während der Pause nicht belastet wird. Nach dem Aufwachen

15.6.55.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.55.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.56 dragontools/worker/quality_test_thread.py

Merkmal	Wert
Kategorie	Worker
Umfang	329 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (329 Zeilen, direkte Importnutzer: 1).

15.6.56.1 Aufgabe des Moduls

Führt die kurzen Qualitätstest-Encodes und technischen Messungen im Hintergrund aus.

15.6.56.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `files`, `output_dir`, `runs`

15.6.56.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `None`

Abhängigkeit	Details

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/media_analyzer.py, dragontools/core/paths.py, dragontools/core/quality_tester.py, dragontools/worker/base_worker.py, dragontools/worker/encoder_args.py
Externe Pythonbibliotheken	PyQt6
Externe Programme/Tools	ffprobe
Direkt verwendet von	dragontools/gui/quality_tester_widget.py

15.6.56.4 Wichtige Klassen und Methoden

QualityTestThread [QThread]

```
cancel() -> None
run() -> None
```

15.6.56.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffprobe → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.56.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.57 dragontools/worker/replace_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	182 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (182 Zeilen, direkte Importnutzer: 2).

15.6.57.1 Aufgabe des Moduls

Ersetzt Originale mit Rollback und Archivschutz.

15.6.57.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.57.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/output_size_policy.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.

Abhängigkeit	Details
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_cleanup_rollback.py, dragontools/worker/converter_thread.py

15.6.57.4 Wichtige Klassen und Methoden

ReplaceService [object] – Kapselt Overwrite-/Replace-Logik inkl. Sicherheitsprüfungen.

```
blocked_move_inputs() -> set[str]
archiviert() -> int
last_blocked_input() -> str | None
last_preserved_path() -> str | None
last_block_reason() -> str
was_blocked(input_path) -> bool
replace() -> str
```

15.6.57.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.57.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, FileNotFoundError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.58 dragontools/worker/source_visual_check.py

Merkmal	Wert
Kategorie	Worker
Umfang	400 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (400 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.58.1 Aufgabe des Moduls

Prüft Quellmaterial an prozentualen Prüfpunkten auf offensichtlich defekte Bildinhalte.

15.6.58.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `seconds`, `settings`, `key`, `default`, `frame`, `pixel_index`, `reasons`

15.6.58.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `bool`, `int`, `SourceVisualCheckSettings`, `float`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/settings.py, dragontools/worker/base_worker.py

Abhängigkeit	Details
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg, ffprobe
Direkt verwendet von	dragontools/gui/convert_widget_queue_actions.py, dragontools/tests/test_source_visual_check.py, dragontools/worker/converter_thread.py
Verwendete Settings-Konstanten	SET_KEY_SOURCE_VISUAL_CHECK_BLOCK_PERCENT, SET_KEY_SOURCE_VISUAL_CHECK_ENABLED, SET_KEY_SOURCE_VISUAL_CHECK_FPS, SET_KEY_SOURCE_VISUAL_CHECK_INTERVAL_PERCENT, SET_KEY_SOURCE_VISUAL_CHECK_MIN_HITS, SET_KEY_SOURCE_VISUAL_CHECK_SAMPLE_DURATION_S

15.6.58.4 Wichtige Klassen und Methoden

SourceVisualCheckSettings [object]

SourceVisualProbe [object]

SourceVisualCheckResult [object]

```
suspicious_count() -> int
suspicious_percent() -> float
summary_line() -> str
report_lines() -> list[str]
```

SourceVisualCheckService [object]

```
check(path, settings) -> SourceVisualCheckResult
```

15.6.58.5 Wichtige Funktionen

```
format_seconds(seconds) -> str
source_visual_settings_from_qsettings(settings) -> SourceVisualCheckSettings
most_common_reason(reasons) -> str
```

15.6.58.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg / ffprobe → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.58.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: (TypeError, ValueError), Exception, IndexError, TypeError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.59 dragontools/worker/standard_pipeline_runner.py

Merkmal	Wert
Kategorie	Worker
Umfang	138 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (138 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.59.1 Aufgabe des Moduls

Führt Standard-FFmpeg-Encoding aus.

15.6.59.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `ctx`, `file_override`

15.6.59.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `list[str]`, `bool`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/encoder_args.py, dragontools/worker/hdr10_color.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_hdr10_color_standard.py, dragontools/tests/test_per_file_strip_only.py, dragontools/worker/converter_thread.py

15.6.59.4 Wichtige Klassen und Methoden

StandardPipelineRunner [object] – Führt die Standard-Pipeline inklusive Strip-only und FFmpeg-Encode aus.

```
logger_start_params() -> tuple[str, object, str, object]
run(ctx, file_override) -> bool
```

15.6.59.5 Wichtige Funktionen

```
_clear_reencoded_video_stat_tags() -> list[str]
```

15.6.59.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.59.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.60 dragontools/worker/subtitle_sidecar_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	296 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (296 Zeilen, direkt von 5 Projektmodulen importiert).

15.6.60.1 Aufgabe des Moduls

Exportiert Untertitel-Sidecars für DV/MP4-Sonderpfade. Technischer Kontext aus dem Quellcode: dragontools/worker/subtitle_sidecar_service.py Zentraler Export-Dienst für externe Sidecar-Untertiteldateien. Beide DV-Pipelines – DVProcessingPipeline und DVRemuxThread – verwenden diese Implementierung. Damit gibt es genau eine massgebliche Logik für: Dateinamensschema Regelwerk-Auswertung (dv_extract_external_subs) Fehlerbehandlung / Rueckgabe als list[s…

15.6.60.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `language`, `base`, `lang`, `forced`, `ext`, `number`

15.6.60.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/lang_codes.py, dragontools/core/models.py, dragontools/rules/subtitle_rules.py, dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	mediainfo
Direkt verwendet von	dragontools/tests/test_dv_regressions.py, dragontools/tests/test_subtitle_sidecar_service.py, dragontools/worker/dv_remux_components.py, dragontools/worker/subtitle_sidecar_service.py, dragontools/worker/mp4_remux_thread.py

15.6.60.4 Wichtige Klassen und Methoden

SubtitleSidecarService [object] – Zentraler Export-Dienst für externe Sidecar-Untertitel. Verwendung ----------

```
export_sidecars() -> list[str]
```

15.6.60.5 Wichtige Funktionen

```
safe_lang_tag(language) -> str
```

Normalisiert einen Sprach-Tag für Dateinamen. Wandelt den Tag in Kleinbuchstaben um und entfernt alle Zeichen außer [a-z0-9]. Ergibt einen leeren String wird "und" verwendet.

```
sidecar_filename(base, lang, forced, ext, number) -> str
```

Erzeugt den vollständigen Sidecar-Pfad nach Jellyfin/VLC-Schema. Parameters ----------

15.6.60.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → mediainfo → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.60.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.61 dragontools/worker/tool_runner.py

Merkmal	Wert
Kategorie	Worker
Umfang	262 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (262 Zeilen, direkt von 5 Projektmodulen importiert).

15.6.61.1 Aufgabe des Moduls

Führt kleine Tool-Selbsttests und externe Kommandos kontrolliert aus.

15.6.61.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `cmd`, `worker`, `lock`, `proc`, `result`

15.6.61.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`, `None`, `ToolRunResult`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/crash_guard.py, dragontools/worker/base_worker.py, dragontools/worker/process_control.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_duration_repair_service.py, dragontools/tests/test_tool_runner.py, dragontools/worker/audio_video_match_thread.py, dragontools/worker/duration_repair_service.py, dragontools/worker/hdrplus_conversion.py

15.6.61.4 Wichtige Klassen und Methoden

ToolRunResult [object]

```
ok() -> bool
combined_output() -> str
tail(lines) -> str
```

15.6.61.5 Wichtige Funktionen

```
run_tool(cmd) -> ToolRunResult
```

Führt einen Toolprozess mit Timeout und optionaler Worker-Abbruchlogik aus. Im Worker-Kontext wird der gestartete Prozess als current_process registriert. Dadurch kann "Sofort abbrechen" denselben Prozessbaum beenden, den der

```
log_tool_failure(result) -> None
```

15.6.61.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.61.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, FileNotFoundError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.62 dragontools/worker/trickplay_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	280 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (280 Zeilen, direkte Importnutzer: 2).

15.6.62.1 Aufgabe des Moduls

Erstellt Jellyfin-Trickplay-Spritebilder mit FFmpeg.

15.6.62.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `video_path`, `settings`, `value`, `path`, `max_jobs`

15.6.62.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `Path`, `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/base_worker.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_trickplay_service.py, dragontools/worker/postprocess_service.py

15.6.62.4 Wichtige Klassen und Methoden

TrickplaySettings [object]

```
tile_label() -> str
```

TrickplayGenerator [object]

```
generate(video_path, settings) -> Path | None
```

_trickplay_semaphore [object]

```
__init__(max_jobs) -> None
```

15.6.62.5 Wichtige Funktionen

```
trickplay_root_for_video(video_path) -> Path
```

```
trickplay_sprite_dir_for_video(video_path, settings) -> Path
```

15.6.62.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.62.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, subprocess.TimeoutExpired. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.63 dragontools/worker/worker_events.py

Merkmal	Wert
Kategorie	Worker
Umfang	49 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (49 Zeilen, direkt von 3 Projektmodulen importiert).

15.6.63.1 Aufgabe des Moduls

Enthält WorkerEvent.

15.6.63.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `path`, `percent`, `eta`, `output_path`, `status`, `message`, `severity`

15.6.63.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `WorkerEvent`

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py, dragontools/worker/dv_remux_thread.py, dragontools/worker/worker_result_service.py

15.6.63.4 Wichtige Klassen und Methoden

WorkerEvent [object] – Strukturiertes Worker-Ereignis neben den bestehenden GUI-Signalen.

15.6.63.5 Wichtige Funktionen

```
progress_event(path, percent, eta) -> WorkerEvent
result_event(path, output_path, status) -> WorkerEvent
log_event(message, severity, path) -> WorkerEvent
```

15.6.63.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.63.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.64 dragontools/worker/worker_result_service.py

Merkmal	Wert
Kategorie	Worker
Umfang	170 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (170 Zeilen, direkte Importnutzer: 2).

15.6.64.1 Aufgabe des Moduls

Enthält WorkerConversionResultService.

15.6.64.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `value`

15.6.64.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `str`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/worker_events.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_error_report.py, dragontools/worker/converter_thread.py

15.6.64.4 Wichtige Klassen und Methoden

WorkerConversionResultService [object] – Kapselt Fortschritts-/Ergebnis-Emission und Abschluss-Statistik.

```
emit_file_progress(path, pct, eta) -> None
emit_file_result(input_path, output_path, status) -> None
finalize_success(ctx) -> None
finalize_success_pending_postprocess(ctx) -> None
record_success(ctx) -> None
finalize_blocked(ctx) -> None
fail(ctx, reason) -> None
fail_unhandled(input_path) -> None
```

15.6.64.5 Wichtige Funktionen

```
_format_size(value) -> str
```

15.6.64.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.64.7 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.65 dragontools/worker/worker_runtime_state.py

Merkmal	Wert
Kategorie	Worker
Umfang	15 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (15 Zeilen, direkte Importnutzer: 1).

15.6.65.1 Aufgabe des Moduls

Enthält WorkerRuntimeState.

15.6.65.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus

15.6.65.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/worker/converter_thread.py

15.6.65.4 Wichtige Klassen und Methoden

WorkerRuntimeState [object]

15.6.65.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.65.6 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.6.66 dragontools/worker/workflow_engine.py

Merkmal	Wert
Kategorie	Worker
Umfang	160 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (160 Zeilen, direkt von 7 Projektmodulen importiert).

15.6.66.1 Aufgabe des Moduls

Orchestriert Analyse, Plan, Prozess, Verify, Replace, Cleanup. Technischer Kontext aus dem Quellcode: Gemeinsame Workflow-Engine für Konvertierungsjobs. Ablauf: 1) analyze 2) build_plan 3) process (pipeline-spezifische Strategie) 4) verify 5) replace Verschieben gehoert nicht zu diesem Converter-Workflow. Es passiert nach

15.6.66.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `services`, `input_path`, `override`, `ctx`

15.6.66.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `bool`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/worker/workflow_services.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	dragontools/tests/test_duration_repair_service.py, dragontools/tests/test_error_report.py, dragontools/worker/converter_thread.py, dragontools/worker/duration_repair_models.py, dragontools/worker/duration_repair_service.py, dragontools/worker/output_verifier.py, dragontools/worker/workflow_services.py

15.6.66.4 Wichtige Klassen und Methoden

WorkflowVerifyResult [object]

```
ok() -> bool
```

WorkflowContext [object]

ConversionWorkflowRunner [object] – Führt den gemeinsamen Job-Workflow aus.

```
run(input_path, override) -> bool
analyze(ctx) -> None
build_plan(ctx, override) -> None
process(ctx, override) -> None
verify(ctx) -> None
```

```
replace(ctx) -> None
finalize(ctx) -> None
cleanup(ctx) -> None
```

15.6.66.5 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → interne Services bzw. Dateisystem → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.66.6 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.6.67 dragontools/worker/workflow_services.py

Merkmal	Wert
Kategorie	Worker
Umfang	630 Quellzeilen
Bedeutung	Wichtiger Fachbaustein (630 Zeilen, direkt von 5 Projektmodulen importiert).

15.6.67.1 Aufgabe des Moduls

Bündelt Workflow-Fachdienste und schreibt Fehlerberichte.

15.6.67.2 Erwartete Informationen / Eingaben

- Datei-/Ordnerpfade bzw. Queue-Einträge
- Konverter-/Worker-Konfiguration
- Toolpfade und Laufzeitstatus
- Wichtige Parameter im Code: `ctx`, `override`

15.6.67.3 Rückgabe / Ausgabe

- Fortschritts-/Logmeldungen, Ergebnisstatus und ggf. erzeugte Mediendateien/Sidecars
- Explizite Rückgabetypen im Code: `_WorkflowIOServices`, `None`

Abhängigkeit	Details
Interne Projektabhängigkeiten	dragontools/core/encoder_profile_override.py, dragontools/core/error_report.py, dragontools/core/models.py, dragontools/worker/dv_runtime_models.py, dragontools/worker/interfaces.py, dragontools/worker/workflow_engine.py
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	ffmpeg
Direkt verwendet von	dragontools/tests/test_duration_repair_service.py, dragontools/tests/test_dv_regressions.py, dragontools/tests/test_per_file_strip_only.py, dragontools/worker/converter_thread.py, dragontools/worker/workflow_engine.py

15.6.67.4 Wichtige Klassen und Methoden

WorkflowConfig [object] – Lauf-Konfiguration: unveränderliche Einstellungen für einen Konvertierungslauf. Fasst die drei knapp verwandten Job-Parameter zusammen, die bisher einzeln durch den gesamten Service-Stack gereicht wurden.

WorkflowPipelineRunners [object] – Die drei Pipeline-Ausführungsfunktionen in einer Gruppe. Fasst die freien Callable-Argumente zusammen, die bisher verstreut im WorkflowServices-Konstruktor lagen und semantisch eng zusammengehoeren.

_WorkflowIOServices [object] – Interne Gruppe der I/O-bezogenen Dienste. Kein Public API – wird als `io_services`-Parameter übergeben. Fasst vier eng zusammengehoerende Dienste zusammen, die alle mit

WorkflowServices [object] – Bündelt alle Workflow-Fachdienste hinter einer stabilen API. Parameter wurden von 18 auf 10 reduziert durch Gruppierung von: - WorkflowConfig (codec, encoder_options, strip_only)

```
analyze(ctx) -> None
build_plan(ctx, override) -> None
process(ctx, override) -> None
verify(ctx) -> None
replace(ctx) -> None
finalize(ctx) -> None
finalize_blocked(ctx) -> None
fail(ctx, reason, traceback_text) -> None
cleanup(ctx) -> None
```

15.6.67.5 Wichtige Funktionen

```
make_workflow_io_services() -> _WorkflowIOServices
```

Factory für _WorkflowIOServices – vermeidet direkten Import des privaten Typs.

15.6.67.6 Datenverarbeitung und Kommunikation

Queue/Dateipfad + Konfiguration → Worker/Service → ffmpeg → Validierung/Statusaufbereitung → Ergebnis/Sidecars/GUI-Signal.

15.6.67.7 Fehlerbehandlung und Diagnose

Explizit behandelte Ausnahmearten im Modul: Exception, ValueError. Darüber hinaus werden fachliche Fehler je nach Schicht an aufrufende Services/GUI weitergereicht oder geloggt.

15.7 Paket

1 Python-Datei(en) in dieser Kategorie im dokumentierten V9.7-Referenzstand. Die folgende Detailreferenz beschreibt den ausführlich dokumentierten Kernbestand; die vollständige aktuelle Dateiliste aller Module steht in der vollständigen Modulübersicht.

15.7.1 dragontools/__init__.py

Merkmal	Wert
Kategorie	Paket
Umfang	2 Quellzeilen
Bedeutung	Spezialisierter/unterstützender Baustein (2 Zeilen, direkte Importnutzer: 0).

15.7.1.1 Aufgabe des Moduls

Kleines Paket-/Hilfsmodul.

15.7.1.2 Erwartete Informationen / Eingaben

15.7.1.3 Rückgabe / Ausgabe

Abhängigkeit	Details
Interne Projektabhängigkeiten	Keine direkte interne Importabhängigkeit.
Externe Pythonbibliotheken	Keine; nur Python-Standardbibliothek bzw. interne Module.
Externe Programme/Tools	Kein direkter Toolname statisch im Modul erkannt.
Direkt verwendet von	Kein direkter statischer Importnutzer; möglich sind Einstiegspunkt, Lazy-Import oder reine Paketfunktion.

15.7.1.4 Datenverarbeitung und Kommunikation

Import/Initialisierung → Bereitstellung der Paket- oder Testfunktionalität.

15.7.1.5 Fehlerbehandlung und Diagnose

Im Modul ist keine breite lokale Exception-Behandlung erkennbar; Validierung bzw. Fehlerbehandlung liegt überwiegend beim aufrufenden Service/Worker oder bei den verwendeten Tool-Wrappern.

15.8 Tests und QA-Module

Historische Abnahme 9.8.3 (15.09.2026): 846 Python-Dateien, 125.770 Gesamtzeilen, 106.333 Codezeilen, 186 Python-Dateien im Testpaket, 183 test_*.py und 1.361 statisch erkennbare Testfunktionen. Der letzte vollständige Release-Basislauf vor dem abschließenden GUI-Nachpatch bestand mit 1.390 Tests ohne Fehler oder Skips; beide realen DV/HDR-Roundtrips waren darin enthalten. Der GUI-Nachpatch wurde zusätzlich mit 5/5 Architekturtests, compileall und Source-Release-Prüfung validiert. Details stehen in patch.md, PATCH_MANIFEST.json und den Release Notes.

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/__init__.py	0	Test-/QA-Modul des Projekts.	–
dragontools/tests/test_audio_processing.py	101	drc for ac3 source forces transcode and input arg; drc for aac source is documented but does not force transcode; loudnorm forces transcode and adds filter; file override can disable global audio processing	dragontools/core/models.py, dragontools/rules/audio_plan.py, dragontools/rules/audio_rules.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_audio_video_matcher.py	160	parse cut regions accepts comma decimal and timecodes; opencv signature backend when available; extra before and after source is case a not c; linear drift is case b	dragontools/core/audio_video_matcher.py, dragontools/core/models.py
dragontools/tests/test_audit_log.py	29	audit log writes event to configured log root; audit summary masks sensitive values	dragontools/core/audit_log.py, dragontools/core/settings.py
dragontools/tests/test_auxiliary_workers.py	507	iso makemkv source uses iso and file prefixes; iso makemkv source uses disc root for direct bdmv folder; iso detects direct bdmv and video ts folders; iso extracts multiple titles as single makemkv calls	dragontools/core/models.py, dragontools/subtitle/converter.py, dragontools/subtitle/extractor.py, dragontools/subtitle/injector.py
dragontools/tests/test_batch_preflight.py	335	batch preflight marks clean file as ok; batch preflight warns for ignored dv and old container; batch preflight turns analysis exception into error row; batch preflight errors when no video stream is detected	dragontools/core/batch_preflight.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_cleanup_rollback.py	172	cleanup removes failed output and sidecars; cleanup overwrite removes file but keeps temp dir until batch end; cleanup empty overwrite dirs keeps nonempty parallel dir silent; replace same path rolls original back on fai	dragontools/worker/cleanup_service.py, dragontools/worker/replace_service.py
dragontools/tests/test_codec_profile_assistant.py	58	h265 assistant profiles cover cpu and nvenc; av1 assistant profiles use numeric svt presets; profile for assistant key returns copy; anime series cpu profile uses animation and more lookahead	dragontools/core/codec_profile_assistant.py
dragontools/tests/test_config_migration.py	240	profile manager migrates legacy profile file; audio rules migration adds schema and stereo copy range; move rules migration adds missing defaults	dragontools/core/config_migration.py, dragontools/core/profile_manager.py, dragontools/rules/audio_rules.py, dragontools/rules/move_rules.py
dragontools/tests/test_conversion_controller_start.py	420	start worker ui state uses initial progress and starts worker once; parallel file progress defaults to combined display and can focus file; single initial file still uses parallel thread when limit is above one	dragontools/gui/conversion_controller.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_conversion_result_service.py	299	sofort abbruch fragt und startet move fuer erfolgreiche dateien; abbruch nach datei fragt und startet move fuer erfolgreiche dateien; abbruch ohne move wunsch finalisiert ohne shutdown; abbruch ohne fertige dateien bleib	dragontools/gui/conversion_result_service.py, dragontools/gui/conversion_session_state.py
dragontools/tests/test_converter_detection.py	103	imax probe offsets nutzen einstellbares intervall; imax auto erkennt wechselnde aktive bildflaeche; imax auto respektiert mindestanzahl verwertbarer punkte; imax auto bleibt aus bei konstanter aktiver bildflaeche	dragontools/worker/__init__.py, dragontools/worker/converter_detection.py
dragontools/tests/test_crash_guard_cleanup.py	48	clear activity removes empty fatal runtime log; unclean report removes empty previous fatal runtime log	dragontools/core/__init__.py
dragontools/tests/test_current_ffmpeg_abort.py	113	terminate current ffmpeg only kills matching ffmpeg; terminate current ffmpeg rejects other current file	dragontools/worker/converter_thread.py
dragontools/tests/test_diagnostic_package.py	65	create diagnostic package collects logs and settings	dragontools/core/diagnostic_package.py, dragontools/core/settings.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_disk_space.py	57	disk space check reports warning when reserve is low; disk space check reports critical when temp outputs do not fit	dragontools/core/disk_space.py
dragontools/tests/test_duration_repair_service.py	899	duration repair remuxes mkv and accepts fixed duration; duration repair archiviert wenn remux dauer falsch bleibt; duration repair greift nur bei reinem mkv laufzeitfehler; duration repair kann deaktiviert werden	dragontools/worker/duration_repair_service.py, dragontools/worker/tool_runner.py, dragontools/worker/workflow_engine.py, dragontools/worker/workflow_services.py
dragontools/tests/test_dv_regressions.py	1116	dv auto exportiert keep subs als sidecars; dv custom export respektiert keep auswahl; dv sidecars werden nach replace zum finalen stem verschoben; dv level5 editor nutzt keinen autocrop fallback	dragontools/worker/dv_level5_editor.py, dragontools/worker/hdr10plus_bitstream_service.py, dragontools/worker/subtitle_sidecar_service.py, dragontools/worker/workflow_services.py
dragontools/tests/test_dv_remux_abort_fix.py	105	Tests für den DVRemuxThread abort_type Bug-Fix und WorkerProtocol.	dragontools/worker/interfaces.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_encoder_args.py	205	h265 encoder forciert echtes 10bit; h264 bleibt 8bit kompatibel ohne 10bit zwang; nvenc lookahead wird ohne bframes nicht gesetzt; nvenc auto lookahead level und multipass werden nicht gesetzt	dragontools/worker/encoder_args.py
dragontools/tests/test_encoder_inactivity_timeout.py	140	ffmpeg inactivity timeout allows active progress; ffmpeg inactivity timeout aborts silent process; explicit disabled timeout does not fall back to general	–
dragontools/tests/test_encoder_profile_override.py	203	profile to override rejects wrong codec; profile to override accepts codec enum defaults; normalize override preserves encoder profile; effective encoder settings uses file profile values	dragontools/core/encoder_profile_override.py, dragontools/core/models.py
dragontools/tests/test_encoder_settings_shutdown.py	384	load encoder settings does not reset shutdown checkbox; cpu encoder panel is visible for h264 and expanded; reset defaults resets cpu encoder fields; assistant profile applies cpu profile and saves	dragontools/core/codec_profile_assistant.py, dragontools/gui/encoder_settings_controller.py, dragontools/gui/encoder_settings_state.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_error_report.py	166	error report writes context and tool output; worker result service stores failure details; worker result service blocked size policy is not success	dragontools/core/error_report.py, dragontools/worker/worker_result_service.py, dragontools/worker/workflow_engine.py
dragontools/tests/test_extended.py	971	Ergänzende Tests für dragontools – deckt Lücken auf, die im Code-Review (2026-04-24) identifiziert wurden. Bereiche:	dragontools/core/models.py, dragontools/core/type_utils.py, dragontools/rules/audio_plan.py, dragontools/rules/audio_rules.py, dragontools/rules/pipeline_selector.py
dragontools/tests/test_gui_dialog_services.py	257	timeout dialog saves minutes and enabled state; encoder profile service returns selected assistant profile; encoder profile service allows same label for different encoders; profile manager same label same encoder reuses	dragontools/core/profile_manager.py, dragontools/core/timeout_settings.py, dragontools/gui/encoder_profile_service.py, dragontools/gui/timeout_settings_dialog.py
dragontools/tests/test_hdr10_color_standard.py	309	dv7 standard encode keeps hdr10 base color tags; dv7 string profile keeps hdr10 base color tags; dv5 standard encode uses libplacebo conversion then hdr10 tags; dv cpu encode args use x265 placebo and cpu 10bit format	dragontools/core/models.py, dragontools/worker/dv_processing_pipeline.py, dragontools/worker/dv_runtime_models.py, dragontools/worker/encode_plan_service.py, dragontools/worker/hdr10_color.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_incremental_move_queue_edit.py	299	incremental move does not lock queue edit; incremental move finish keeps queue edit enabled; incremental move finish uses finished thread reference; incremental move does not start without running conversion	dragontools/core/preflight_report.py, dragontools/gui/conversion_session_state.py, dragontools/gui/move_preflight_controller.py, dragontools/gui/preflight_dialog.py
dragontools/tests/test_jellyfin_nfo.py	91	movie nfo contains clean tmdb metadata without user state; episode nfo uses episode root and numbers; episode nfo writes tvdb ids for thetvdb provider	dragontools/core/jellyfin_nfo.py, dragontools/core/online_metadata.py
dragontools/tests/test_job_journal.py	316	job journal records and archives completed run; unfinished job summary uses active journal; resume plan requeues open and running files; job journal tracks multiple parallel current files	dragontools/core/job_journal.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_language_priority_rules.py	512	audio prioritaet nimmt naechste vorhandene sprache; audio fallback uebernimmt alle wenn keine prioritaet passt; audio kommentarspur wird uebersprungen; audio max sprachen begrenzt sprachgruppen	dragontools/core/models.py, dragontools/rules/audio_rules.py, dragontools/rules/subtitle_rules.py
dragontools/tests/test_log_cleanup.py	59	log cleanup deletes old logs but keeps crash state; log cleanup respects selected categories	dragontools/core/log_cleanup.py
dragontools/tests/test_media_analyzer_pix_fmt.py	94	build video streams speichert mediainfo pix fmt im model; build video streams speichert frame infos im model; audio streams nutzen ffprobe globalindex bei bluray reihenfolge	dragontools/core/media_analyzer.py
dragontools/tests/test_mediainfo_details.py	91	mediainfo details zeigt bitraten und spuren; mediainfo details schaetzt videobitrate wenn video bitrate fehlt	dragontools/core/mediainfo_details.py
dragontools/tests/test_models.py	128	Tests für core/models.py – normalize_override_dict	dragontools/core/models.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_move_conflicts.py	144	find target conflicts matches same video stem with other extension; find target conflicts includes exact and other video extension; find target conflicts does not delete non video by same stem; find target conflicts uses	dragontools/core/move_conflicts.py
dragontools/tests/test_move_report.py	161	move report groups by target and counts conflict actions; move report format mentions deleted and replaced files; move report counts multiple removed target versions; move report groups sidecars by type and conflict acti	dragontools/core/move_report.py
dragontools/tests/test_move_rules_series_detection.py	60	series detection removes separator before episode code; series detection keeps internal hyphenated names; series detection normalizes missing space after separator; existing series dir matches tmdb colon and year suffix	dragontools/rules/move_rules.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_movie_renamer.py	359	parse movie release name strips release tags and keeps search core; build movie rename proposal prefers matching title and year; build movie rename proposal retries german umlaut variant; movie rename proposal skips prob	dragontools/core/movie_renamer.py
dragontools/tests/test_online_metadata.py	578	parse movie query extracts title and year from release name; parse movie query keeps manual dotted title without video extension; clean tmdb collection name removes provider suffixes; tmdb client resolves collection with	dragontools/core/online_metadata.py
dragontools/tests/test_output_path_service.py	51	output paths are reserved for parallel non overwrite jobs; output paths are reserved for parallel overwrite temp jobs	dragontools/worker/output_path_service.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_output_verifier.py	372	output verifier accepts video audio and plausible duration; output verifier rejects missing audio when source had audio; output verifier rejects implausibly short duration; output verifier uses configurable duration tole	dragontools/worker/output_verifier.py
dragontools/tests/test_parallel_converter_thread.py	283	parallel thread starts limited workers and aggregates progress; parallel thread delegates pause resume and abort; parallel thread add remove and reorder queue; parallel thread frees slot while postprocessing waits	dragontools/worker/base_worker.py, dragontools/worker/parallel_converter_thread.py
dragontools/tests/test_parallel_settings.py	104	parallel jobs use new safe defaults when not configured; parallel defaults migration updates old saved defaults; parallel defaults migration keeps custom values; parallel jobs use cpu and gpu limits	dragontools/core/parallel_settings.py, dragontools/core/settings.py
dragontools/tests/test_path_safety.py	98	Test-/QA-Modul des Projekts.	dragontools/core/path_safety.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_per_file_strip_only.py	148	workflow effective strip only uses per file override; standard pipeline uses strip runner for per file strip only; queue label shows processing badge from override; queue label shows pipeline and postprocess badges from	dragontools/core/settings.py, dragontools/gui/convert_widget_queue_actions.py, dragontools/worker/standard_pipeline_runner.py, dragontools/worker/workflow_services.py
dragontools/tests/test_pipeline_and_singleton.py	96	Tests für rules/pipeline_selector.py und core/paths.py (Singleton)	dragontools/core/models.py, dragontools/core/paths.py, dragontools/rules/pipeline_selector.py
dragontools/tests/test_postprocess_move_sidecars.py	183	movie nfo sidecar is renamed to movie nfo in film target; episode nfo sidecar keeps filename in series target; trickplay sidecar conflict can be backed up on move; trickplay sidecar conflict can keep existing target	dragontools/core/settings.py, dragontools/worker/move_thread.py
dragontools/tests/test_postprocess_service.py	90	prepare nfo path skip keeps existing file; prepare nfo path backup marks existing file as saved; async postprocess coordinator collects outputs	dragontools/worker/postprocess_service.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_preflight_dialog_performance.py	320	series preview path does not scan existing folders; series metadata job is payload for background lookup; series metadata job searches selected base before fallback; existing series dir can switch to fallback base	dragontools/core/__init__.py, dragontools/gui/preflight_dialog.py, dragontools/rules/__init__.py
dragontools/tests/test_preflight_report.py	76	preflight report includes targets rules and warnings; preflight report shows film series subpath; write preflight report creates file	dragontools/core/preflight_report.py
dragontools/tests/test_quality_tester.py	48	parse quality segments supports percent and timestamp ranges; automatic quality segments are bounded to video duration; quality run from dict clamps quality and fills defaults	dragontools/core/quality_tester.py
dragontools/tests/test_release_validation.py	380	release validation checks required files and schema; release validation warns about private paths; app bundle validation checks exe not source files; format release checks summarizes errors and warnings	dragontools/core/release_validation.py, dragontools/core/settings.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_review_stdlib.py	243	Test-/QA-Modul des Projekts.	–
dragontools/tests/test_run_summary.py	96	run summary counts ok errors and skipped; run summary maps emoji statuses; run summary preserves failure report details; run summary formats postprocess and sidecar details	dragontools/core/run_summary.py
dragontools/tests/test_settings_backup.py	308	backup exports and restores settings rules and profiles; settings to dict can mask sensitive online metadata values	dragontools/core/settings.py, dragontools/core/settings_backup.py
dragontools/tests/test_source_visual_check.py	106	source visual check blocks when enough probes are suspicious; source visual check does not block single black scene; source visual settings reads qsettings values	dragontools/core/settings.py, dragontools/worker/source_visual_check.py
dragontools/tests/test_subtitle_converter.py	135	srt to txt preserves timestamps and removes numbers; srt to txt writes timestamped output file; txt to srt recreates numbered blocks; txt to srt writes output file	dragontools/subtitle/converter.py
dragontools/tests/test_subtitle_sidecar_service.py	371	Tests fuer SubtitleSidecarService und Hilfsfunktionen.	dragontools/core/lang_codes.py, dragontools/worker/subtitle_sidecar_service.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_system_shutdown.py	54	schedule system shutdown uses short windows buffer; schedule system shutdown returns false on failure	dragontools/core/__init__.py
dragontools/tests/test_tool_diagnostics.py	100	probe ffmpeg reports version and features; format tool diagnostics marks missing tool; handbrake gui is not started for version probe; extended system test runs mini tool chain	dragontools/core/tool_diagnostics.py
dragontools/tests/test_tool_paths.py	102	find tool searches third party product dirs; extend path adds third party product dirs; find tool searches pyinstaller contents programme dirs; extend path adds pyinstaller contents programme dirs	dragontools/core/__init__.py, dragontools/core/settings.py
dragontools/tests/test_tool_runner.py	64	run tool collects stdout and stderr; run tool reports timeout	dragontools/worker/tool_runner.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_trickplay_service.py	371	trickplay uses jellyfin default folder and ffmpeg tile filter; trickplay existing folder is returned when only missing; trickplay existing root allows missing sprite variant; trickplay existing root can be backed up	dragontools/worker/__init__.py, dragontools/worker/trickplay_service.py
dragontools/tests/test_verbose_cleanup.py	65	verbose logger discard removes file; converter discards verbose after clean run; converter keeps verbose after error or abort	dragontools/core/logger.py
dragontools/tests/test_changelog.py	55	parse legacy changelog keeps overview and sections; json changelog is preferred structured source; paginate blocks splits long sections without losing order	dragontools/core/changelog.py
dragontools/tests/test_main_window_tab_visibility.py	112	closing tab persists hidden state; reopening last tab persists visible state	dragontools/gui/main_window.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_media_library.py	1364	path mapping translates jellyfin prefix; import jellyfin database builds dragon library; jellyfin import normalizes stream types to dragon schema; jellyfin import preserves explicit hdr10plus flags; storage scan builds library from real folder structure; storage scan keeps file when analysis fails	dragontools/core/media_library.py, dragontools/core/models.py
dragontools/tests/test_project_info.py	38	collect project statistics counts sources and tests; about html contains current stability summary	dragontools/core/project_info.py
dragontools/tests/test_replacement_reminders.py	95	replacement reminder persists with unique ids; replacement reminder confirm deletes defer and close keep; replacement reminder survives shutdown until next start	dragontools/core/replacement_reminders.py

Testdatei	Zeilen	Prüfschwerpunkt (aus Testnamen)	Primär importierte Projektmodule
dragontools/tests/test_settings_helpers.py	186	settings bool parses string values without python bool trap; ui section expanded key is stable and namespaced; collapsible groupbox persists user click; settings int float and text clamp or fallback; postprocess config uses centralized settings limits; output size policy accepts injected settings	dragontools/core/settings.py, dragontools/gui/convert_widget_layout.py, dragontools/worker/postprocess_service.py, dragontools/worker/output_size_policy.py
dragontools/tests/test_ui_helpers_geometry.py	81	window geometry key is stable and namespaced; restore and save window geometry use same key; install persistent window geometry saves when dialog finishes	dragontools/gui/ui_helpers.py

15.9 Modulreferenz – Renamer und Mediathek

15.9.1 dragontools/core/media_library.py

Merkmal	Wert
Kategorie	Core
Umfang	1.303 Quellzeilen
Aufgabe	Verwaltet Pfade und Schema der eingebetteten SQLite-Mediathek, importiert Jellyfin-Daten read-only, aktualisiert Medien nach lokalen Analysen/Moves und stellt Suche, Abweichungslogik sowie Serienwurzel-Auflösung bereit.

Wichtige Klassen/Funktionen: initialize_database() / _create_schema(), import_jellyfin_database(), record_media_file() / record_moved_file(), search_library() / _find_deviations(), find_series_root() / find_series_dir_from_settings(), execute_sql() / backup_database()

15.9.2 dragontools/gui/media_library_dialog.py

Merkmal	Wert
Kategorie	GUI
Umfang	646 Quellzeilen
Aufgabe	PyQt6-Dialog für Status/Import, Pfad-Mapping, strukturierte Suche und manuelle SQL-Bearbeitung. Die Suche trennt Kategorie und Kriterium und zeigt Abweichungsgründe in einer eigenen Spalte.

Wichtige Klassen/Funktionen: MediaLibraryDialog._build_import_tab(), MediaLibraryDialog._build_mapping_tab(), MediaLibraryDialog._build_search_tab(), MediaLibraryDialog._update_search_options(), MediaLibraryDialog._run_search(), MediaLibraryDialog._run_sql()

15.9.3 dragontools/gui/rules_renamer_tab.py

Merkmal	Wert
Kategorie	GUI
Umfang	188 Quellzeilen
Aufgabe	Regel-Editor für Dateizeichen-Ersetzungen, Titel-/Suchalias-Regeln und Matching-Grenzen. Derselbe Tab ist als eigener Menüpunkt und im globalen Regelfenster verwendbar.

Wichtige Klassen/Funktionen: _RenamerTab, load/save der Renamer-Regeln, Bearbeitung von character_replacements, Bearbeitung von title_exceptions, Matching-Schwellen/Fuzzy-Optionen

15.9.4 dragontools/rules/renamer_rules.py

Merkmal	Wert
Kategorie	Rules
Umfang	228 Quellzeilen
Aufgabe	Definiert Defaultschema und Migration der Renamer-Regeln. Bietet zentrale Laufzeitfunktionen für Ersetzungen, Aliasauflösung, Score-Grenzen und Fuzzy-Suchvarianten.

Wichtige Klassen/Funktionen: migrate_renamer_rules(), apply_character_replacements(), apply_title_exception(), minimum_candidate_score(), manual_review_below() / auto_accept_from(), series_search_queries()

15.9.5 dragontools/tests/test_media_library.py

Merkmal	Wert
Kategorie	Tests
Umfang	339 Quellzeilen

Merkmal	Wert
Aufgabe	Regressionstests für die neue Mediathek. Erzeugt minimale Jellyfin-/DragonTools-Testdatenbanken und prüft Import, Suche, Qualitätsklassen sowie Abweichungsentscheidungen.

Wichtige Klassen/Funktionen: test_path_mapping_translates_jellyfin_prefix, test_import_jellyfin_database_builds_dragon_library, test_search_supports_resolution_buckets_and_video_codecs, test_search_distinguishes_sdr_from_unknown_dynamic_range, test_deviation_search_returns_only_minority_episodes, test_deviation_search_marks_ties_as_uneven_without_guessing

15.10 Modulreferenz - Mediathek-Scan, Move-Konflikte und Erinnerungen

15.10.1 dragontools/core/media_library.py

Aufgabe des Moduls. Öffentliche Fassade der modularen SQLite-Schicht der DragonTools-Mediathek. Die Fachmodule legen Tabellen und Indizes an, migrieren kompatibel auf Schema 6, importieren Jellyfin-Daten, scannen Speicherpfade mit MediaInfo, führen den leichten NFO-Bestandsscan aus, schreiben Move-Ergebnisse fort und stellen Such-/SQL-Abfragen bereit.

Erwartete Informationen / Eingaben. Datenbankpfad, optionale Jellyfin-Quelle, Pfad-Mappings, MediaInfo-Ergebnisse, Verschiebeinformationen, Suchkriterien und Filter für Bereich/Typ.

Rueckgabe / Ausgabe. Normalisierte `media_items` und `media_streams`, Suchtreffer, Statistikdaten, CSV-Exports, aktive/inaktive Zustände und Wartungsergebnisse.

Datenverarbeitung und Kommunikation. Das Modul verbindet GUI, Preflight, MoveThread und Scanner über ein einheitliches Schema. `record_moved_file()` deaktiviert alte Pfade und dieselbe aktive EpisodeIdentity, bevor die neue Datei aktiv geschrieben wird.

Fehlerbehandlung und Diagnose. SQLite-Operationen laufen in Transaktionen. Schreibende Wartungsschritte sichern die Datenbank; Suchabfragen initialisieren das Schema vor dem Zugriff, damit alte Datenbanken kompatibel bleiben.

15.10.2 dragontools/core/move_conflicts.py

Aufgabe des Moduls. Erkennt Zielkonflikte beim Verschieben. Neben klassischen gleicher-Stem-Konflikten prüft das Modul Serienfolgen über eine fachliche EpisodeIdentity.

Erwartete Informationen / Eingaben. Quellpfad, Zielpfad, vorhandene Zieldateien und Dateinamen, aus denen Serienname, Staffel und Episoden extrahiert werden.

Rueckgabe / Ausgabe. Listen konkreter Konfliktpfade und EpisodeIdentity-Objekte, die vom MoveThread für Ersetzung, Log und Datenbankfortschreibung genutzt werden.

Datenverarbeitung und Kommunikation. Die Serienerkennung nutzt die zentrale Parse-Logik aus den Move-Regeln. Dadurch verwenden Renamer, Preflight und Verschieben möglichst denselben Begriff von `Serie - SxxExx`.

Fehlerbehandlung und Diagnose. Uneindeutige oder nicht erkannte Dateien fallen auf klassische Dateikonflikte zurück. Multi-Episoden werden vollständig verglichen, um Teiltreffer zu vermeiden.

15.10.3 dragontools/core/replacement_reminders.py

Aufgabe des Moduls. Persistente Erinnerungsschicht für automatische SxxExx-Ersetzungen. Das Modul verwaltet IDs, JSON-Speicherung, Lesen, Anlegen und Bestätigen.

Erwartete Informationen / Eingaben. Serienkennung, alter Pfad, neuer Pfad, Grund, Zeitstempel und optionaler Speicherpfad für Tests.

Rueckgabe / Ausgabe. Sortierte Erinnerungslisten, neue oder wiederverwendete Erinnerungs-IDs und bestätigte Löschungen.

Datenverarbeitung und Kommunikation. MoveThread legt Erinnerungen an; ConversionResultService und MainWindow lesen sie und zeigen den GUI-Dialog zum passenden Zeitpunkt.

Fehlerbehandlung und Diagnose. Die JSON-Datei wird atomar über temporäre Datei und `os.replace` geschrieben. Doppelte Erinnerungen mit gleichem alten/neuen Pfad werden nicht mehrfach angelegt.

15.10.4 dragontools/gui/media_library_dialog.py

Aufgabe des Moduls. Oberfläche für Import, Pfad-Mapping, Suche, CSV-Export, SQL-Bearbeitung, Speicherpfad-Scan und Bereinigung inaktiver Einträge.

Erwartete Informationen / Eingaben. Benutzerauswahl, Speicherpfade, Suchkriterien, Bereichs-/Typfilter, Datenbankpfad und Wartungsbefehle.

Rueckgabe / Ausgabe. Tabellarische Treffer, CSV-Dateien, Import-/Scan-Fortschritt, Statusmeldungen und Sicherheitsabfragen.

Datenverarbeitung und Kommunikation. Der Dialog kapselt lange Scans in Worker-Schritte und ruft die Core-Schicht für alle Datenbankoperationen auf. Damit bleibt die GUI fachlich dünn und die eigentliche Datenlogik testbar.

Fehlerbehandlung und Diagnose. Schreibende Aktionen haben Dialogbestätigung oder Backup. Fehler werden als verständliche Meldungen angezeigt, ohne die Jellyfin-Quelle zu verändern.

15.10.5 dragontools/gui/replacement_reminder_dialog.py

Aufgabe des Moduls. Zeigt offene Ersetzungs-Erinnerungen auffällig vor dem Abschlussbericht oder beim nächsten Start an.

Erwartete Informationen / Eingaben. Liste der Reminder-Objekte aus replacement_reminders.

Rueckgabe / Ausgabe. Benutzerentscheidung `Bestätigen` oder `Erneut vorlegen`; beim Schließen bleibt die Erinnerung erhalten.

Datenverarbeitung und Kommunikation. Der Dialog löscht selbst keine Dateien. Er meldet nur die Entscheidung zurück, danach entfernt die Core-Schicht bestätigte Reminder aus der JSON-Datei.

Fehlerbehandlung und Diagnose. Sicheres Standardverhalten ist Behalten. Dadurch kann ein versehentlich geschlossenes Fenster keine wichtige Ersetzungsinformation verlieren.

15.10.6 dragontools/worker/move_thread.py

Aufgabe des Moduls. Führt das Verschieben, Ersetzen, Sidecar-Mitnehmen, Move-Logging und die Datenbankfortschreibung aus.

Erwartete Informationen / Eingaben. Konvertierte Dateien, Zielregeln, Konfliktregeln, Postprocessing-Dateien, Mediathek-Einstellungen und erkannte EpisodeIdentity-Konflikte.

Rueckgabe / Ausgabe. Move-Ergebnisse, Logzeilen, Ersetzungsberichte, optionale Reminder-ID und aktualisierte Mediathek-Einträge.

Datenverarbeitung und Kommunikation. Der Worker ruft move_conflicts für Zielprüfungen auf, replacement_reminders für persistente Hinweise und media_library für `active`-/`exists_flag`-Fortschreibung.

Fehlerbehandlung und Diagnose. Bei Fehlern bleibt der eigentliche Konvertierungserfolg getrennt vom Verschiebefehler auswertbar. Datenbankschreibungen erfolgen nur nach erfolgreichen physischen Move-Schritten.

15.10.7 dragontools/gui/conversion_result_service.py

Aufgabe des Moduls. Koordiniert Abschlussbericht, Shutdown-Verhalten und nicht blockierende Ersetzungs-Erinnerungen nach einem Lauf.

Erwartete Informationen / Eingaben. Worker-Ergebnisse, Shutdown-Wunsch, Abschlussstatus und offene Reminder.

Rueckgabe / Ausgabe. Abschlussdialog, ausgelöste Shutdown-Logik oder vertagte Erinnerung für den nächsten Programmstart.

Datenverarbeitung und Kommunikation. Vor einem normalen Abschlussbericht wird der Reminder-Dialog geöffnet. Wenn Shutdown aktiv ist, wird kein blockierender Dialog erzeugt.

Fehlerbehandlung und Diagnose. Fehler im Reminder-Anzeigepfad dürfen den abgeschlossenen Encode nicht zerstören; die persistente JSON-Datei bleibt die zuverlässige Quelle.

15.10.8 dragontools/gui/main_window.py

Aufgabe des Moduls. Startpunkt für die Anzeige offener Ersetzungs-Erinnerungen beim Programmstart.

Erwartete Informationen / Eingaben. Persistente Reminder-Datei unter Documents\DragonTools und das initialisierte Hauptfenster.

Rueckgabe / Ausgabe. Zeitverzögerter Reminder-Dialog nach dem Start, damit das Hauptfenster zuerst stabil aufgebaut ist.

Datenverarbeitung und Kommunikation. MainWindow importiert die Reminder-Anzeige nur bei Bedarf und hält den Startpfad dadurch schlank.

Fehlerbehandlung und Diagnose. Anzeigefehler werden abgefangen, weil ein Erinnerungsdialog niemals den Programmstart blockieren darf.

15.11 Historische Modulreferenz zentraler Fachmodule

15.11.1 dragontools/core/changelog.py

Qt-freie Daten- und Paginierungsschicht für CHANGELOG.json und Legacy-TXT. Erwartet Dateipfad/Text, liefert ChangelogDocument/ChangelogSection und feste Seitenblöcke. Fehlerhafte Formatversionen oder leere JSON-Dokumente werden als ValueError gemeldet.

15.11.2 dragontools/gui/changelog_dialog.py

PyQt6-Dialog für Bereichsauswahl und Seiten-Navigation. Bevorzugt V9-JSON, fällt auf TXT zurück und verwendet für V8/V7 denselben Paginator. Der Inhaltsbereich wird als kompaktes HTML gerendert; Scrollbars sind deaktiviert.

15.11.3 dragontools/core/project_info.py

Berechnet die Programmstatistik live aus vorhandenen Python-Quellen und erzeugt den HTML-Inhalt des Über-Dialogs. Nutzt AST zum Zählen statischer Tests und feste Release-Fallbackwerte, wenn Quellen im Frozen-Build fehlen.

15.11.4 dragontools/gui/encoder_settings_controller.py

Speichert QSV nur noch mit lookahead_depth, entfernt qsv_la und persistiert Reset-Endzustände explizit. H.265-CPU-Werkreset verwendet CRF 22.

15.11.5 dragontools/core/config_migration.py

Zentrale Schemaversionen und Migrationen: Audio 4, Untertitel 6, Move 1, Renamer 3 und Profile 3. Entfernt alte QSV-lookahead-Bools, ergänzt Encoderoptionen bei Legacy-Profilen und stellt den crash-robusten atomaren JSON-Writer bereit. H.265-Fallback-CRF ist 22.

15.11.6 dragontools/core/release_validation.py

Prüft seit V9.7 zentrale Versionskonsistenz, Release-Manifest und reale PyInstaller-CLI-Buildstrategie sowie Runtime-/Optional-/Test-/Build-Requirements, pytest-/CI-/DV-HDR-Integrationsverträge, Regelschemata, Python-Paket, Bytecode-Freiheit und App-Bundle-Artefakte.

15.11.7 dragontools/worker/dv_crop_reconcile.py

Synchronisiert FFmpeg-AutoCrop mit Dolby-Vision-RPU-Level-5. Kleine Abweichungen werden automatisch konservativ aufgelöst; größere Konflikte werden über die GUI entschieden, dynamische Active Areas werden nicht global plattgemacht.

15.11.8 dragontools/gui/dv_crop_dialog.py

Benutzeroberfläche für den DV-Crop-Konflikt. Gibt AutoCrop, RPU oder DV deaktivieren an den wartenden Worker zurück.

15.11.9 dragontools/worker/dv_mkv_muxer.py

Baut die finale Dolby-Vision-MKV ausschließlich mit mkvmerge aus dem bereits injizierten HEVC-Bitstream und vorbereiteten Audiotracks.

15.11.10 dragontools/worker/hdrplus_conversion.py

HDR10+-Coordinator für den direkten 5-Schritt-Pfad. Prozessausführung liegt im HDRPlusToolRunner, Containeraufbau im HDRPlusMuxService und Extract/Inject/semantischer Vergleich im HDR10PlusBitstreamService. Der Coordinator behält schmale Kompatibilitätswrapper. MP4-Audio ist fail-closed: Ist ein Audio-Donor vorhanden, aber nicht zuverlässig analysierbar, wird der Mux abgebrochen statt eine scheinbar erfolgreiche Datei ohne Audio zu erzeugen.

15.11.11 dragontools/core/release_validation.py - PyInstaller-Pfadvertrag

Im Quellprojekt wird dragontools direkt über den Release-Smoke geprüft. Im fertigen Onedir-Build liegen Runtime-Konfigurationen unter Daten\dragontools\config; ein separater Daten\Python-Quellbaum wird nicht mehr ausgeliefert. Der App-Validator prüft deshalb den Frozen-Bundle-Vertrag und meldet einen vorhandenen lesbaren Quellbaum als Releasefehler.

15.12 Ergänzungen vom 06.09.2026

15.12.1 dragontools/core/audio_titles.py

Zentrale Erzeugung konsistenter Audio-Tracktitel. Aus Sprach-Tag, Codec, Kanalzahl und bekannter Bitrate entstehen Titel wie „Deutsch EAC3 5.1 640kbps“. Fehlt bei einer kopierten Spur die Bitrate, wird absichtlich nur der Sprachname gesetzt, damit keine unbestätigten technischen Daten behauptet werden.

15.12.2 dragontools/worker/av1_metadata_pipeline.py

Getrennte fail-closed Beta-Runner für AV1 Dolby Vision Profile 10 und AV1 HDR10+. AV1-DV10 nutzt CPU/libsvtav1, prüft die FFmpeg-Dolby-Vision-Capability und verifiziert den finalen Output erneut. AV1-HDR10+ nutzt CPU/libaom-av1 mit 10-

Bit und HDR10+-T.35/OBU-Metadaten und finaler Verifikation. Hardwareencoder werden in diesen Spezialpfaden derzeit abgelehnt, statt Metadaten still zu verlieren.

15.12.3 dragontools/tests/test_log_short_and_audio_titles.py

Regressionstests für die Trennung von sichtbarem Kurzlog und vollständigem *_lang.txt sowie für die kanonischen Audio-Titel einschließlich Sprachfallback bei unbekannter Bitrate.

15.12.4 dragontools/tests/test_av1_metadata_pipeline.py

Regressionstests für AV1-DV10- und AV1-HDR10+-Kommandos, Encoder-Capabilities, fail-closed Grenzen, Dolby-Vision-Profilprüfung sowie CPU-/Hardware-Abgrenzung.

15.12.5 dragontools/core/output_replace.py

Qt-unabhängiger zentraler Output-Commit. Kapselt Same-Path-Replace und Containerwechsel über PathSwapTransaction und ReplaceJournal, führt Backup/Rollback aus und markiert nicht löschbare Quellen als cleanup_pending für Recovery.

15.12.6 dragontools/worker/dv_subtitle_mux_service.py

Zentrale DV-Untertitelablage. MP4 folgt der globalen Sidecar-Policy; bei deaktivierten Sidecars werden kompatible Textspuren als mov_text intern gemuxt, während PGS/SUP und VobSub extern bleiben. MKV übernimmt ausgewählte Spuren intern.

15.12.7 dragontools/worker/hdrplus_tool_runner.py

Zentrale HDR10+-Toolausführung mit Timeout, Worker-Abbruch, Fehlerdiagnose und expliziten accepted_returncodes. Nichtzero-Warncodes können gezielt als zulässig behandelt werden, ohne echte Fehler zu verschlucken.

15.12.8 dragontools/worker/hdrplus_mux_service.py

Container-Service des HDR10+-Pfads. Baut MKV mit mkvmerge und MP4 mit MP4Box -inter 500, analysiert/extrahiert Audiotracks und bricht bei nicht verifizierbarem vorhandenen Audio-Donor fail-closed ab.

15.12.9 dragontools/worker/hdrplus_conversion.py - Koordinator mit getrennten Fachservices

Koordiniert weiterhin den stabilen 5-Schritt-Ablauf und behält schmale Wrapper für bestehende interne Aufrufer/Tests. Tool-, Mux- und Bitstream-Verantwortung liegen jedoch in den spezialisierten Services; dadurch sinkt Kopplung im zentralen HDR10+-Helper.

AV1-Pipeline-Priorität: Wenn eine AV1-Quelle sowohl Dolby Vision als auch HDR10+ besitzt und beide Erhaltungsoptionen aktiv sind, setzt PipelineSelector den effektiven HDR10+-Erhalt bewusst auf false, wählt AV1-DV10/SVT-AV1 und erzeugt eine sichtbare INFO. Diese Policy ist von den HEVC-Kombipfaden getrennt und löst weder Capability-Warnung noch Archivierung aus.

15.13 Aktuelle Fachmodule für Timeline, Jellyfin und Sprachreparatur

Die folgenden Fachmodule ergänzen die bestehende Modulreferenz und sind für den aktuellen Reparatur-/Integrationsstand besonders relevant. Die Liste ist eine fachliche Ergänzung, kein versionsbezogener Sammelnachtrag.

- dragontools/worker/duration_original_timeline_service.py – liest und validiert framegenaue Original-PTS und erzeugt die Timestamp-v2-Zeitbasis für sichere VFR-MKV-Reparaturen.
- dragontools/core/jellyfin_api.py – kapselt Jellyfin-Systeminformationen, PhysicalPaths, gezielte Media-Updates und Full-Library-Refresh.

- dragontools/core/jellyfin_refresh_service.py – führt Pfadmappings zusammen, validiert Zielpfade gegen Jellyfin-Roots, erzeugt Parent-Hinweise und steuert den Full-Scan-Fallback.
- dragontools/core/whisper_runtime.py – lädt faster-whisper/CTranslate2 lazy und entscheidet zwischen lokalem Modellordner und normalem Modellnamen/-cache.
- dragontools/core/nfo_stream_metadata.py – parst und staged NFO-Streamdetails für transaktionale Sprachsynchronisierung.
- dragontools/core/language_detection.py / media_library_fix_queue.py – bewerten Spracherkennung, Konfidenz und Reparaturplanung der Mediathek-Fix-Queue.
- dragontools/gui/tool_path_live_check.py – prüft die aktuell sichtbaren Werkzeugpfade direkt aus dem Einstellungsdialog, ohne sie vorzeitig zu persistieren.
- dragontools/worker/media_stream_language_service.py – koordiniert Trackadressierung, Whisper-Samples und sichere Metadatenkorrektur mit echtem Containerstreamindex.
- dragontools/gui/convert_widget_parallel_controls.py koppelt die Worker-Auswahl an den aktiven Koordinator, zeigt die aktive Zahl und speichert CPU-/GPU-Vorgaben. dragontools/gui/convert_widget_quick_settings.py beschreibt die zehn Schnellschalter; dragontools/subtitle/sidecar_matcher.py übernimmt die konservative Sprach- und Episodenzuordnung.

16 Vollständige GUI- und Bedienreferenz

Dieses Kapitel ergänzt die anwendungsorientierten Kapitel 1–3 um eine Referenz der aktuell im Code angelegten Hauptregister, Schalter und Steuerpfade. Maßgeblich sind main_window.py, main_window_menus.py, convert_widget_layout.py und die jeweiligen Spezial-Widgets.

16.1 Hauptregister und Lazy Loading

Register	Implementierung	Zweck
H.265 / DV / HDR10+	ConvertWidget("h265")	Standard-H.265-Konvertierung sowie DV-/HDR10+-Erhaltung und Sonderpipelines.
H.264	ConvertWidget("h264")	H.264-Konvertierung mit denselben Queue-, Regel- und Profilmechanismen.
AV1	ConvertWidget("av1")	AV1-Konvertierung mit CPU/GPU-Encoderwahl im Standardpfad sowie getrennten Beta-Pipelines für AV1-DV10 (CPU/SVT-AV1) und AV1-HDR10+ (CPU/libaom-av1).
ISO	ISOWidget	ISO/IMG, Disc-Root, BDMV und VIDEO_TS analysieren/extrahieren; optional an den Converter übergeben.
Merge	MergeWidget	Videodateien konservativ auf Kompatibilität prüfen und zusammenführen.

Register	Implementierung	Zweck
MP4-Remux	MP4RemuxWidget	Video ohne Re-Encode in MP4 remuxen; Audio-/Untertiteloptionen anwenden.
Audio Muxer	AudioMuxerWidget	Audio nach Regeln neu muxen/verarbeiten; Video, Untertitel und Attachments kopieren.
Audio-Video-Matcher	AudioVideoMatcherWidget	Zwei Videoquellen vergleichen, Offset/Drift/Schnittunterschiede bestimmen und deutsches Audio anpassen.
Untertitel	SubtitleWidget	Untertitel extrahieren, injizieren, konvertieren und Flags/Sprachen setzen.
Renamer	MovieRenamerWidget	Film- und Seriennamen analysieren, Online-Metadaten auflösen und sicher umbenennen.
Qualitätstester	QualityTesterWidget	Kurze Segmente mit Varianten encoden und technisch sowie per SSIM/VMAF vergleichen.

Die Register werden als Platzhalter angelegt und erst beim ersten Öffnen instanziiert. Sichtbarkeit ist pro Tab in QSettings gespeichert. Der Starttab wird über `defaults/codec` gewählt; fehlt er, startet die erste verfügbare Registerkarte.

16.2 Converter – sichtbare Bedienelemente

Bedienelement	Werte/Default	Technische Wirkung
Encoder	CPU, Auto, NVIDIA NVENC, Intel QSV, AMD AMF	Wählt die konkrete FFmpeg-Encoderfamilie; Auto verwendet die GPU-Erkennung.
Skalierung	original, 4K, 1080p, 720p, 480p	Erzeugt bei Auswahl einen Scale-Filter; original lässt die Auflösung unverändert.
Encoder-Optionen (erweitert)	einklappbar	Blendet die encoderabhängigen Qualitäts-, AQ-, Lookahead- und Presetwerte ein.
Strip-Only	aus	Kein Video-Re-Encode; Streams werden entsprechend den Regeln kopiert. Burn-In ist technisch nicht möglich und wird nicht ausgeführt.

Bedienelement	Werte/Default	Technische Wirkung
Original überschreiben	aus	Ersetzt nach erfolgreicher Validierung die Quelle durch die Ausgabe; Fehlerpfade verhindern den Replace.
Nach Abschluss verschieben	aus	Startet nach erfolgreicher Verarbeitung die Zielordner-/Move-Logik.
Herunterfahren	aus	Plant nach abgeschlossenem Batch das Herunterfahren unter Beachtung des Countdown-/Abschlussstatus.
Auto-Crop	QSettings, typischer Default aktiv	Analysiert schwarze Balken und baut bei sicherem Ergebnis einen Crop-Filter.
IMAX Auto-Erkennung	Default aus	Sucht wechselnde aktive Bildflächen; kann Auto-Crop dateibezogen deaktivieren.
Dolby Vision erhalten	Default an	Bei erkannter DV-Quelle wird in H.265 die DV-Pipeline und in AV1 der AV1-DV10-Betapfad gewählt. Bei AV1 DV+HDR10+ und beiden aktivierten Erhalt-Haken hat DV Priorität.
HDR10+ erhalten	Default an	Aktiviert bei HDR10+ die Metadatenpipeline. H.265 nutzt den HEVC-HDR10+-Pfad; AV1 nutzt den CPU/libaom-av1-Betapfad, sofern DV nicht priorisiert wird.
Profil-Assistent / Speichern / Laden	Buttons	Verwaltet Startprofile; per-Datei-Overrides können globale Werte überschreiben.
Dateien / Ordner / Entfernen / Alle	Queue	Befüllt oder bearbeitet die Queue; wartende Einträge können während eines Laufs weiter verwaltet werden.
Konvertierung starten	Button	Startet Preflight, Worker und Workflow-Engine.
DV-Remux → MP4/MKV	Button	Sonderpfad: DV-Video bleibt unverändert, Audio wird kompatibel verarbeitet, Untertitel werden extern ausgegeben.

Bedienelement	Werte/Default	Technische Wirkung
Nur verschieben	Button	Umgeht Konvertierung und führt Preflight + Move direkt aus.
Fertige verschieben	Button	Verschiebt nur bereits erfolgreich abgeschlossene Dateien während ein Batch noch läuft.
Pause	Button	Hält das Nachrücken/Weiterarbeiten des Workers kontrolliert an; laufende externe Prozesse werden nicht pauschal gekillt.
Abbruchmodus	Sofort / Nach Datei	Sofort beendet den aktiven Prozessbaum; Nach Datei setzt nur die Abbruchvormerkung.
Fortschritt	Datei + Gesamt	Bei Parallelbetrieb kann kombinierter Fortschritt oder eine konkrete aktive Datei fokussiert werden.
Log	eingebettetes Textfeld	Zeigt Laufmeldungen und bietet Sprung zum Logordner bzw. aktuellen Log.

16.3 Queue-Kontextmenü

Aktion	Verhalten
Medieninfo	Analysiert Streams/HDR/Dauer und zeigt Plan-/Override-Kontext.
Regel-/Profil-Simulator	Zeigt geplante Audio-, Untertitel- und Pipelineentscheidungen ohne Encoding.
FFmpeg für diese laufende Datei beenden	Nur bei aktivem Queue-Eintrag sichtbar. Beendet ausschließlich den aktuellen FFmpeg-Prozess dieser Datei. Es wird kein globales Abort-Flag gesetzt; die Datei läuft in Fehler/Cleanup und die restliche Queue läuft weiter.
Nur diese Datei Strip-Only AN/AUS	Setzt `processing_mode=strip_only` als dateibezogenen Override.
Datei-Einstellungen …	Öffnet die vollständigen Per-Datei-Overrides.
Encoder-Profil …	Weist nur dieser Datei ein gespeichertes Profil zu.
IMAX AN/AUS	Setzt dateibezogene IMAX-Steuerung.
Quellbildprüfung ausführen	Startet die visuelle Quellprüfung gezielt.
Quellbildprüfung übergehen	Erlaubt eine als verdächtig erkannte Quelle bewusst.

Aktion	Verhalten
Entfernen	Entfernt den Queue-Eintrag; die Quelldatei wird nicht gelöscht.

16.4 Spezialregister – Bedienelemente

Register	Wesentliche sichtbare Steuerungen
ISO	ISO-Dateien; Disc-Ordner; Titel analysieren; Titel-Auswahl; Zielpfad; Haupttitel automatisch vorschlagen; FFmpeg-Fallback bei MakeMKV-Fehler; Nach Extraktion an Konverter übergeben; Extraktion starten; Abbrechen
Merge	Dateien hinzufügen; Reihenfolge bearbeiten; Kompatibilität prüfen; Modus; Zieldatei; Merge starten; Abbrechen
MP4-Remux	Dateien/Ordner; Zielordner; Audio-Regeln aktiv; Untertitel extern speichern; Untertitel ignorieren; faststart; MP4-Remux starten; Abbrechen
Audio Muxer	Dateien/Ordner; Original überschreiben; Start; Abbrechen; Datei-/Gesamtfortschritt; Log
Audio-Video-Matcher	Deutsche Quelle; Zielvideo; Ausgabe; Analyse; erkannter Fall/Vertrauen/Offset/Drift; Fall-C-Zeitbereiche; Schnittbereiche analysieren; Audio anpassen und Datei erstellen; Abbrechen
Untertitel	Quell-Videos; Ausgabeordner; Extrahieren; Ziel-Videos; Untertiteldatei; Sprache; Forced; Einfügen; Konvertier-/Flag-Aktionen
Renamer	Dateien/Ordner; Vorschläge suchen; Auswahl akzeptieren; Sichere akzeptieren; Auswahl ablehnen; Umbenennen ausführen; Entfernen/Alle
Qualitätstester	Dateien/Ordner; automatische Segmente; Segmentdauer; manuelle Bereiche; Ausgabeordner; Testläufe; Codec/Encoder/Qualität/Preset/Bit/Skalierung/Extra-Args; Qualitätstest starten; Abbrechen

16.5 Menüs und Tastenkürzel

Menü	Inhalt
Einstellungen	Zielpfade, Logging, Log-Bereinigung, Tool-Pfade, Quellbildprüfung, Speichern, Auto-Crop, Output-Validierung/Reparatur, Move-Konflikte, IMAX, Timeouts, globales Fenster, Backup/Restore

Menü	Inhalt
Regeln	Serien-Erkennung, Audio-Regeln, Untertitel-Regeln, Untertitel-Flag, globales Regelfenster
Online-Metadaten	Renamer, Zugang einrichten, Verbindungen testen, Titel suchen, Cache leeren, TMDB/TheTVDB im Browser
Profile	Profilverwaltung
Ansicht	Tabs ein-/ausblenden, Tab-Manager, Dark Mode, externe Programme
Werkzeuge	Systemtest, erweiterter Test, Regelsimulator, laufenden Job diagnostizieren, Diagnosepaket, Qualitätstester, Matcher, Releaseprüfung, externe Tools
Zoom-Fenster	Warteschlange, Logging-Fenster
Hilfe	F1 Hilfe, F2 Handbuch, Shortcuts, Changelogs, Über
Shortcut	Aktion
F1	Hilfe
F2	Handbuch
F9	System/Werkzeuge prüfen
F11	Legacy-Changelog V8
F12	Changelog V9
Ctrl+Q	Beenden
Ctrl+W	Aktuellen Tab schließen
Ctrl+Shift+Q	Warteschlange
Ctrl+Shift+T	Letzten Tab wieder öffnen
Ctrl+Shift+A	Alle Tabs wieder öffnen
Ctrl+R	Standardwerte
Delete	Markierten Queue-Eintrag entfernen

16.6 Persistenz von GUI-Bereichen, Tabs und Fenstern

- Einklappbare Converter-Bereiche speichern ihren offenen/geschlossenen Zustand. Ein bewusst geschlossenes Encoder-/Optionen-/Fortschritt-/Log-Panel bleibt nach Neustart geschlossen.
- Geschlossene Haupt-Tabs wie AV1 speichern tabs/visible/<tab>=false und bleiben nach Neustart ausgeblendet, bis sie über Ansicht/Shortcut wieder geöffnet werden.
- Wichtige Dialoge und Zusatzfenster speichern Geometrie zentral, darunter Einstellungen, Regeln, Timeouts, Preflight, Mediathek, Medieninfo, Changelog, Hilfe, Shortcuts, Abschlussbericht, Profilverwaltung, Qualitätstest, Log-Zoom und Queue.

- Die Haupt-Tab-Leiste begrenzt ihre minimumSizeHint-Breite und verwendet Scrollbuttons sowie Text-Elide. Lange Tabnamen dürfen dadurch nicht mehr die Mindestbreite des Hauptfensters bestimmen.
- Der Renamer speichert zusätzlich den Tabellenkopfzustand (Spaltenbreiten, Reihenfolge und Sichtbarkeit) in QSettings. Die Tabelle darf horizontal scrollen, ohne ihre gesamte Inhaltsbreite an das QMainWindow weiterzugeben.

16.6.1 Live Worker Auswahl und Schnellschalter

Die Auswahl Worker neben Pause zeigt das gewünschte Limit; die zusätzliche Anzeige nennt die aktuell aktiven Konvertierungen. Nach einer Reduktion können vorübergehend mehr Dateien aktiv sein als das neue Limit erlaubt. Das ist erwartetes Verhalten: Die Dateien werden fertig verarbeitet. Bedienung und Beispiele stehen in Kapitel 3.3.1; die Schalterübersicht steht in Kapitel 3.2.1.

16.7 Über-Dialog als aktuelle Programmzusammenfassung

- Die bisher komplett hart codierte About-Zusammenfassung wurde in `dragontools/core/project_info.py` ausgelagert.
- Im Entwicklungs-/Onedir-Betrieb zählt Dragon Tools vorhandene Python-Dateien, Gesamtzeilen, nichtleere/nicht reine Kommentarzeilen, Testdateien und statisch erkennbare Tests live. Fehlen Quell-.py-Dateien im Frozen-Build, werden verifizierte Release-Fallbackwerte genutzt.
- Aktueller Quellstand 9.9.1 vom 07.10.2026: 1.297 Python-Dateien/Programme, 208.072 Gesamtzeilen und 175.787 Codezeilen (nichtleer, keine reinen Kommentarzeilen). Testpakete: 375 Python-Dateien, davon 369 test_*.py mit 3.287 statisch erkannten Testfunktionen. Produktivcode ohne Tests: 922 Dateien, 129.255 Gesamtzeilen und 111.314 Codezeilen. Patch-29-Abnahme: 5.175 bestanden (5.152 Gesamtsuite + 23 native DV/HDR-Tests). Parametrisierte Testfälle werden getrennt von Testfunktionen gezählt.
- Die kompakte Funktionsübersicht im Über-Dialog nennt Video und HDR, automatische Untertitelzuordnung, Deutsch/Englisch und Spurtitel, die zehn Schnellschalter einschließlich NFO/Trickplay sowie die Live-Worker-Steuerung von 1 bis 8. Die aktuelle APP_VERSION bleibt V9.9.1.
- Der aktuelle Stabilitätsstand umfasst zusätzlich transaktionalen Output-Commit, atomare Regel- und Profilpersistenz, HDR10+-Prüfung, MP4-Untertitelablage und Diagnose. Untertitelregeln verwenden Schema 7. Diese technischen Funktionen sind in den jeweiligen Fachkapiteln beschrieben.

17 Einstellungen, Defaultwerte und technische Auswirkungen

Dieses Kapitel ist als Referenz gedacht: Welche Stellgröße existiert, welchen Bereich akzeptiert die GUI, welcher Wert ist voreingestellt und was ändert sich technisch. Bei „Auto“ wird der betreffende FFmpeg-Parameter bewusst nicht gesetzt; damit bleibt die Entscheidung beim Encoder/FFmpeg.

17.1 Encoder-Grundlagen

Bereich	Option	Werte	Default	Auswirkung
CPU H.264/H.265/AV1	CRF	0–63	H.264 22 / H.265 23 / AV1 28	Niedriger = höhere Qualität und größere Datei; höher = kleinere Datei und mehr Verlust. Für H.265 wird main10/10-bit erzwungen.

Bereich	Option	Werte	Default	Auswirkung
CPU x264/x265	Preset	ultrafast … veryslow	medium	Langsameres Preset erhöht Analyseaufwand und meist Kompressionseffizienz; CRF-Qualitätsziel bleibt separat.
SVT-AV1 CPU	Preset	4–12	6	Normaler CPU-AV1-Encoder und Basis des AV1-DV10-Betapfads; kleinere Preset-Zahlen sind langsamer/effizienter.
Skalierung	Ziel	original, 2160p, 1080p, 720p, 480p	original	Reduktion spart Bitrate/Encodeaufwand; Hochskalierung kann Pixelzahl erhöhen, erzeugt aber keine neuen Quelldetails.

17.2 NVIDIA NVENC

Option	Werte	Default	Technische Wirkung
Preset	p1–p7	p6	p1 priorisiert Tempo; p7 priorisiert Encoderanalyse/Kompression. p6 ist im Projekt der Qualitäts-Default.
CQ	0–63	23	Niedriger = höhere Qualität/größere Datei. Der Code nutzt VBR + `-cq` + `-b:v 0`.
B-Frames	0–8	4	Mehr B-Frames können Kompression verbessern; 0 deaktiviert zugleich den rc-lookahead-Zweig im Command Builder.
B-Ref-Mode	disabled / each / middle	middle	Wird nur bei B-Frames > 0 und H.264/H.265 gesetzt. disabled ist konservativer; Referenz-B-Frames können Effizienz erhöhen.

Option	Werte	Default	Technische Wirkung
RC Lookahead	0–64	32	Anzahl voraus analysierter Frames. 0 deaktiviert die Lookahead-Argumente; höhere Werte benötigen mehr Analyse-/Pufferressourcen.
Lookahead-Level	Auto / 0 / 1 / 2 / 3	Auto	Auto setzt keinen Parameter. 0–3 werden als `-lookahead_level` übergeben; höhere Stufen können Qualität verbessern, kosten Performance. Der FFmpeg-Code definiert genau 0–3 plus Auto.
Multipass	Auto / disabled / qres / fullres	Auto	Auto setzt keinen Parameter. disabled=Single-Pass, qres=erster Pass in Viertelauflösung, fullres=erster Pass in voller Auflösung; fullres ist ressourcenintensiver.
AQ-Stärke	1–15	8	Nur zusammen mit Spatial AQ gesetzt. Höher verschiebt Bitrate aggressiver nach räumlicher Komplexität.
Spatial AQ	an/aus	an	Setzt `-spatial_aq 1` und AQ-Stärke; soll komplexen Bildbereichen mehr Bits geben.
Temporal AQ	an/aus	an	Setzt `-temporal_aq 1`; beeinflusst Bitverteilung über die Zeit.

Wichtig: `lookahead_level` und `rc-lookahead` sind verschiedene Optionen. `rc-lookahead` ist die Anzahl der Frames; `lookahead_level` ist die NVENC-Qualitätsstufe 0–3. DragonTools setzt `lookahead_level` nur, wenn B-Frames > 0 und `rc_lookahead` > 0 sind. Multipass wird unabhängig davon gesetzt, sobald nicht Auto gewählt ist.

17.3 Intel QSV, AMD AMF und CPU-x265

Seit V9.7 gültiger Werkzustand (auch in V9.8.3): CPU/x265 verwendet für H.265 CRF 22. Das alte QSV-An/Aus-Feld `qsv_la` ist entfernt; die echte Lookahead-Tiefe bleibt als `qsv_la_depth` beziehungsweise Profilfeld `lookahead_depth` erhalten. Das Profil-Schema ist Version 3. Beim globalen Reset speichert Dragon Tools den finalen Encoderzustand nach UI-Reset und Standardprofil explizit in QSettings, damit keine alten Werte nach Neustart oder Profilwechsel zurückkehren.

Encoder/Option	Werte	Default	Auswirkung
QSV Preset	veryfast … veryslow	erster Eintrag/UI; Profile können setzen	Geschwindigkeit vs. Kompression.
QSV Q	0–63	23	Qualitätsziel; niedriger = besser/größer.
QSV Lookahead-Tiefe	1–100	40	DragonTools setzt QSV Lookahead immer aktiv und übergibt die Tiefe.
AMF QP	0–63	23	Wird für I/P und bei Nicht-AV1 auch B verwendet.
AMF Qualität	speed / balanced / quality	balanced	Steuert AMF-Qualitäts-/Geschwindigkeitsmodus.
x265 AQ-Mode	0–3	2	0 aus; höhere Modi aktivieren komplexere adaptive Quantisierung.
x265 AQ-Stärke	0–3	1.0	Stärke der adaptiven Quantisierung.
x265 psy-rd	0–5	2.0	Psychovisuelle Gewichtung; höhere Werte erhalten Struktur stärker, können Bitrate erhöhen.
x265 psy-rdoq	0–50	1.0	Psychovisuelle RDOQ-Gewichtung.
x265 B-Frames	0–16	8	Mehr Referenzmöglichkeiten, höhere Analysekomplexität.
x265 Lookahead	0–250	40	Nur bei B-Frames > 0 als `rc-lookahead` in x265-params gesetzt.
x265 Tune	none/grain/animation/ssim/psnr	none	Spezialoptimierung; verändert Encoderentscheidungen und sollte nur bewusst gewählt werden.

17.4 Audio-Regeln – Standardkonfiguration

Regel	Default	Auswirkung

Regel	Default	Auswirkung
Sprachpriorität	de, en	de zuerst; max_languages=1 und tracks_per_language=1.
Kein Prioritätstreffer	keep_all	Verhindert stummes Wegfiltern, wenn keine gewünschte Sprache erkannt wird.
Kommentarspuren ignorieren	true	Kommentartracks werden nicht als Hauptaudio ausgewählt.
Descriptive Audio ignorieren	true	Audiodeskription wird nicht als normale Hauptspur priorisiert.
Passthrough-Codecs	aac, ac3, eac3	Diese Codecs dürfen innerhalb der Kanal-/Bitratengrenzen kopiert werden.
Mono	AAC 128k	Bis 1 Kanal; Ziel AAC, max. 128 kbit/s.
Stereo	AAC 192k; Copy-Fenster 192–256k	Akzeptierter Codec im Fenster wird kopiert, sonst AAC 192k.
5.1	E-AC3 640k; Copy-Fenster 428–640k	Bis 6 Kanäle, Downmix standardmäßig never.
7.1	Ziel 5.1 E-AC3 640k; Downmix always	7–8 Kanäle werden standardmäßig auf 5.1 geführt; 8ch E-AC3/AC3 wird nicht neu erzeugt.
Extra Stereo	false; AAC 256k	Optional zusätzliche Stereo-Spur neben Surround.
DRC	aus; Scale 1.0	Nur aktiv bei expliziter Aktivierung.
Loudnorm	aus; I=-18 LUFS, LRA=11, TP=-1.5	EBU-artige Lautheitsnormalisierung erst bei Aktivierung.

17.5 Untertitel-Regeln – Standardkonfiguration

Regel	Default	Technische Bedeutung
Sprachpriorität	de, en; max_languages=1; tracks_per_language=1	Deutsch wird bevorzugt; Standard-Fallback bei keinem Treffer ist keep_none.
Forced-Priorität	true	Forced-Untertitel werden vor regulären Spuren behandelt.
Auto Burn Forced	true	Geeigneter Forced-Track kann eingebrannt werden; Burn-In erzwingt Video-Re-Encode.
Burn-Sprache	de	Zielt auf deutsche Forced-Untertitel.

Regel	Default	Technische Bedeutung
Bei Mehrdeutigkeit fragen	true	Unsichere Auswahl wird nicht still festgelegt.
Forced-Plausibilität	aktiv; Warnung 3 Events/min; Block 5 Events/min	Zu dichte angebliche Forced-Spuren werden gewarnt oder blockiert.
Forced behalten	true	Forced-Spuren bleiben als Stream erhalten, soweit die konkrete Pipeline dies erlaubt.
Alle deutschen behalten	true	Deutsch wird vollständig konserviert; weitere Keep-Regeln ergänzen dies.
Englischer Fallback behalten	false	Englisch wird nicht automatisch zusätzlich konserviert.
MP4-Sidecars aktivieren	true	Globale MP4-Policy: aktiv = ausgewählte Untertitel extern; deaktiviert = Text intern als mov_text/tx3g, PGS/SUP und VobSub/DVD-Sub extern. MKV speichert intern.

17.6 Auto-Crop, IMAX, Validierung, Parallelisierung und Postprocessing

Bereich	Default	Auswirkung
Auto-Crop Modus	single	Ein Analysefenster; Probe ab 30 s, 45 s Dauer. Alternativ können andere Modi/Intervalle in Settings existieren.
Auto-Crop Probe-Intervall	600 s	Bei wiederholter Analyse größere Werte = weniger Analyseaufwand, geringere Chance kurze Wechsel zu sehen.
IMAX Probe-Intervall	90 s	Kürzer = mehr Prüfpunkte und mehr Analysezeit.
IMAX Probe-Dauer	3 s	Längere Fenster sind robuster, aber teurer.
IMAX Mindestvarianz	15 %	Schwellwert für wechselnde Bildhöhe/-fläche.
IMAX Mindesttreffer	2	Verhindert Aktivierung durch einen Einzelmessfehler.
Output Mindestgröße	1 KiB	Technischer Null-/Leerausgabe-Schutz vor weiteren Prüfungen.

Bereich	Default	Auswirkung
Dauer Minimum	90 %	Zu kurze Ausgabe löst Reparatur/Fehlerpfad aus.
Dauer Maximum	125 % + 60 s Toleranz	Begrenzt unrealistisch lange Ausgabe; zusätzliche absolute Toleranz reduziert Fehlalarme.
Duration Remux	an	Erster Reparaturpfad bei plausibler MKV-Dauerproblematik.
Timestamp-Reparatur	an	Zweiter, gezielter Reparaturpfad bei erkanntem Timestamp-Muster.
CPU Paralleljobs	1	Konservativ wegen hoher CPU-Last pro Softwareencode.
GPU Paralleljobs	2	Nutzt GPU-Parallelität ohne aggressiven Default; max. 8 einstellbar.
NFO	aus	Postprocessing ist opt-in; nur eindeutige Treffer standardmäßig true, fileinfo true, movie.nfo.
Trickplay	aus	Opt-in; 320 px, 10x10 Tiles, 10 s Intervall, JPEG 90/qscale 4, CUDA, 1 Job, Output als Quelle, Konflikt skip.
Quellbildprüfung	an	10-%-Intervalle, 2 s Sample, 2 fps; blockiert bei 80 % auffälligen Frames und mindestens 4 Treffern.

17.7 Timeouts

Kategorie	Schritt	Default	Bedeutung
DV-Pipeline	HEVC-Extraktion	240 min	Gesamtlaufzeit-Limit für HEVC-Extraktion.
DV-Pipeline	Dolby-Vision-Profilkonvertierung	240 min	dovi_tool Profilkonvertierung.
DV-Pipeline	RPU-Extraktion	10 min	Metadatenextraktion.
DV-Pipeline	HEVC-Rückextraktion	60 min	Stream aus fertigem MKV nach Encode.
DV-Pipeline	Dolby-Vision-Editor Level 5	2 min	Metadateneditor.
DV-Pipeline	RPU-Injektion	30 min	dovi_tool inject-rpu.
DV-Pipeline	Audio-Extraktion	60 min	Audio-Extraktion.

Kategorie	Schritt	Default	Bedeutung
DV-Pipeline	MP4Box-Muxing	60 min	Finaler MP4-Mux.
DV-Pipeline	ffmpeg-Encode	5 min Inaktivität	Nicht Gesamtdauer: nur Stillstand ohne Fortschritts-/stderr-Aktivität.
Encoder	Allgemeiner Encoder	5 min Inaktivität	Standard-Encode darf beliebig lange laufen, solange Aktivität kommt.
HDR10+	Toolschritte	60 min	Extraktion/Injection/Mux; Encode nutzt Encoder-Inaktivität.
Reparatur	MKV-Dauerreparatur	60 min	Remux/Timestamp-Reparatur.
Untertitel	Extrahieren	5 min	mkvextract/ffmpeg.
Untertitel	Einbetten	10 min	mkvmerge/ffmpeg.
Untertitel	Flags setzen	1 min	mkvpropedit.
Analyse	Medienanalyse	1 min	ffprobe/MediaInfo.
Analyse	Format-/Codec-Erkennung	1,5 min	Pipeline-Voranalyse.

Deaktivierte Timeouts liefern intern `None`; der gespeicherte Minutenwert bleibt erhalten. Encoder-Timeouts sind bewusst Inaktivitäts-Timeouts und keine maximale Filmlaufzeit.

17.8 Zentrale Settings-Helfer

- Die fachliche Rules-Preview ist Qt-unabhängig. Preserve-DV/HDR10+-Werte und andere QSettings werden im GUI-/Adapter-Layer gelesen und explizit an dragontools/core/rules_preview.py übergeben; der Core importiert dafür kein PyQt6/QSettings mehr.
- Robuste zentrale Lesehelfer für bool, int, float und Text reduzieren verteilte Sonderbehandlung von QSettings-Werten.
- Logging, Validierung/Reparatur, Quellbildprüfung, NFO/Trickplay und Größen-/Output-Regeln verwenden diese gemeinsame Settings-Schicht. Ungültige Werte fallen auf definierte Defaults zurück.

17.9 QSV-Altlast und Profil-Schema 3

- Das alte QSV-Bool-Feld `qsv_la` beziehungsweise `encoder_options.lookahead` wird nicht mehr geschrieben. Beim Speichern wird ein vorhandener QSettings-Key entfernt; Profilmigrationen entfernen das alte Feld ebenfalls.
- Die Lookahead-Tiefe bleibt unverändert als `qsv_la_depth` in QSettings und `lookahead_depth` im Profil erhalten.
- Das Profilmigrationsschema wurde auf Version 3 angehoben. `default_profiles.json` wird bereits in Schema 3 mit vollständigen Encoderoptionen ausgeliefert.
- Projektwurzel-, Test- und Build-Artefakte werden nur über die dafür vorgesehenen Release-/Cleanup-Funktionen bereinigt; produktive Quelldateien werden nicht automatisch gelöscht.

17.10 Zuverlässiger Standardwerte-Reset und CPU-CRF 22

- Der H.265-CPU/x265-Werkstandard ist CRF 22. Der Wert ist in UI-Grundzustand, Encoder-Controller, Standardprofil und Fallbackprofil konsistent.
- Nach `reset_to_defaults()` wird der Endzustand explizit gespeichert. Zusätzlich speichert der globale Reset nach dem anschließenden Standardprofil noch einmal den finalen Zustand jedes aktiven Converter-Tabs.
- Damit hängt Persistenz nicht mehr davon ab, ob Qt bei einem identischen Zielwert ein valueChanged/currentIndexChanged-Signal auslöst.

18 Worker-System von DragonTools

Worker kapseln langlaufende oder blockierende Arbeit außerhalb des GUI-Eventloops. Die GUI startet/steuert sie über Controller, Qt-Signale oder Serviceadapter. Der normale Converter ist zusätzlich in Workflow-Services zerlegt, damit Analyse, Pipelineauswahl, Encoding, Validierung, Reparatur, Replace und Postprocessing getrennt diagnostizierbar sind.

Modul	Worker/Service	Aufgabe	Externe Tools
archive_service.py	Service/Runner	dragontools/worker/archive_service.py Technischer Kontext aus dem Quellcode: dragontools/worker/archive_service.py Zentrale Archivierungsfunktion für Quelldateien, bevor Metadaten	intern
audio_mux_thread.py	AudioMuxThread	Worker für Audio-Muxer. Technischer Kontext aus dem Quellcode: dragontools/worker/audio_mux_thread.py Audio-Mux-Thread: Audio Mux für MKV Remux Dateien Audio: konvertieren (nach Au	ffmpeg
audio_video_match_thread.py	AudioVideoMatchThread	Führt den Audio-Video-Matcher im Hintergrund aus, damit die GUI bedienbar bleibt. Der Worker steuert Analyse, Synchronentscheidung, Schnitt-/Offset-Plan und die abschließende Datei	ffmpeg
base_worker.py	BaseWorker	dragontools/worker/base_worker.py Technischer Kontext aus dem Quellcode: dragontools/worker/base_worker.py Konvention für das Worker-Interface zwischen GUI und Worker-Threads. Kein	intern
cleanup_service.py	Service/Runner	Räumt temporäre Dateien und Sidecars auf.	intern
converter_thread.py	ConverterThread	Hauptworker für Batch-Konvertierung. Technischer Kontext aus dem Quellcode: dragontools/worker/converter_thread.py Schlanker Konvertierungs-Worker. Delegiert Spezialaufgaben an Hel	dovi_tool, ffmpeg, ffprobe, mediainfo

Modul	Worker/Service	Aufgabe	Externe Tools
duration_repair_service.py	Service/Runner	Führt MKV-Remux und Timestamp-Reparatur aus.	ffmpeg, ffprobe, mkvmerge
dv_audio_mux_service.py	Service/Runner	Bereitet Audio für DV-MP4 vor.	ffprobe, mp4box
dv_remux_thread.py	DVRemuxThread	dragontools/worker/dv_remux_thread.py Technischer Kontext aus dem Quellcode: dragontools/worker/dv_remux_thread.py DV-Remux-Thread: containerabhängiger Remux für Dolby-Vision-Dateien (MP4/MKV). Audio: norma	ffmpeg, mp4box, mkvmerge
dv_rpu_service.py	Service/Runner	Kapselt dovi_tool für RPU-Extraktion und -Injection.	intern
encode_plan_service.py	Service/Runner	Enthält EncodePlanService.	mediainfo
hdr10plus_bitstream_service.py	Service/Runner	Kapselt hdr10plus_tool Extract/Inject/Verify.	hdr10plus_tool
interfaces.py	WorkerProtocol, PausableWorkerProtocol	Enthält WorkerProtocol, PausableWorkerProtocol, AnalysisService.	intern
iso_thread.py	ISOThread	MakeMKV-Worker für ISO und Disc-Ordner.	ffmpeg, makemkvcon
media_analysis_service.py	Service/Runner	Enthält MediaAnalysisService.	intern
merge_thread.py	MergeThread	Kompatibilitätsprüfung und verlustfreier Merge.	ffprobe, mkvmerge
move_thread.py	MoveThread	Verschiebt Dateien nach Zielregeln und fragt fehlende Ziele ab. Technischer Kontext aus dem Quellcode: dragontools/worker/move_thread.py – vollständig neu, robuste Film/Serien-Logi	ffmpeg
mp4_remux_thread.py	MP4RemuxThread	Einfacher MP4-Remux mit Schutz vor DV/HDR10+.	ffmpeg
output_path_service.py	Service/Runner	Enthält OutputPathService.	intern
parallel_converter_thread.py	ParallelConverterThread	Koordiniert mehrere normale ConverterThread-Instanzen für parallele Bearbeitung.	ffmpeg
pipeline_decision_service.py	Service/Runner	Enthält PipelineDecisionService.	intern
postprocess_service.py	Service/Runner	Erzeugt NFO und Trickplay nach erfolgreicher Konvertierung; seit V9.3 auch als Hintergrundauftrag.	ffmpeg, ffprobe

Modul	Worker/Service	Aufgabe	Externe Tools
quality_test_thread.py	QualityTestThread	Führt die kurzen Qualitätstest-Encodes und technischen Messungen im Hintergrund aus.	ffprobe
replace_service.py	Service/Runner	Ersetzt Originale mit Rollback und Archivschutz.	intern
standard_pipeline_runner.py	Service/Runner	Führt Standard-FFmpeg-Encoding aus.	ffmpeg
subtitle_sidecar_service.py	Service/Runner	Exportiert Untertitel-Sidecars für DV/MP4-Sonderpfade. Technischer Kontext aus dem Quellcode: dragontools/worker/subtitle_sidecar_service.py Zentraler Export-Dienst für externe Sid	mediainfo
tool_runner.py	Service/Runner	Führt kleine Tool-Selbsttests und externe Kommandos kontrolliert aus.	intern
trickplay_service.py	Service/Runner	Erstellt Jellyfin-Trickplay-Spritebilder mit FFmpeg.	ffmpeg
worker_events.py	WorkerEvent	Enthält WorkerEvent.	intern
worker_result_service.py	WorkerConversionResultService	Enthält WorkerConversionResultService.	intern
worker_runtime_state.py	WorkerRuntimeState	Enthält WorkerRuntimeState.	intern
workflow_engine.py	Service/Runner	Orchestriert Analyse, Plan, Prozess, Verify, Replace, Cleanup. Technischer Kontext aus dem Quellcode: Gemeinsame Workflow-Engine für Konvertierungsjobs. Ablauf: 1) analyze 2) build	intern
workflow_services.py	Service/Runner	Bündelt Workflow-Fachdienste und schreibt Fehlerberichte.	ffmpeg

18.1 Normaler Converter

ConverterThread ist im aktuellen Stand eine schlanke QThread-Fassade für Signale, Queue-/Override-Kompatibilität und die bestehende öffentliche Worker-API. Runtime-Serviceaufbau, Queue-/Session-Schleife, Run-Fehler/Cleanup, Per-Datei-Workflow/Quellbildprüfung sowie Pause-/Abort-/Prozesskontrolle liegen in ConverterRuntimeBuilder, ConverterRunLoop, ConverterLifecycleService, ConverterFileExecutor und ConverterControlService. Die eigentliche Fachlogik bleibt in PipelineDecisionService, EncodePlanService, StandardPipelineRunner, DVProcessingPipeline/HDRPlusConversionHelper, OutputVerifier und den bereits bestehenden Workflow-Services.

- Startbedingungen: gültige Queue, auflösbare Toolpfade und erfolgreiche Preflight-/Speicherplatzprüfung.
- Pause: setzt den internen Pausenzustand; ein bereits laufender Toolprozess wird nicht automatisch beendet.
- Sofortabbruch: setzt globales Abort-Flag und beendet den aktuellen Prozessbaum.
- Abbruch nach Datei: merkt den Abbruch vor; die aktuelle Datei darf fertig werden.

- „FFmpeg für diese laufende Datei beenden“: beendet nur einen als ffmpeg erkannten aktuellen Prozess der ausgewählten aktiven Datei. Kein globales Abort-Flag; Queue läuft weiter.
- Auch die langen FFmpeg-Hilfsprozesse für Untertitel-Sidecar-Export und Burn-In-SRT-Extraktion laufen über den zentralen Runner. Damit greifen dieselben Timeout-, Abort- und current_process-Verträge wie bei den übrigen langlaufenden Worker-Prozessen.
- Fortschritt: Worker meldet Datei-/Gesamtfortschritt, ETA, Logs und finale Ergebnisobjekte; ParallelConverterThread aggregiert Child-Worker.
- Wiederaufnahme: JobJournal hält aktive Batchzustände für Crash-/Restart-Szenarien.

18.2 Spezialworker

Beim ISO-Worker verarbeitet ein eigener ISOInputProcessor Disc-/Ordner-Eingaben über einen öffentlichen Hostvertrag. Der Thread behält Benutzerkonfiguration und Qt-Lifecycle; Erkennung und Vorbereitung greifen nicht objektübergreifend auf private Threadmethoden zu.

Worker	Kernaufgabe
ISOThread	ISO/IMG oder Disc-Struktur analysieren/extrahieren; MakeMKV und optional FFmpeg-Fallback.
MergeThread	Eingaben prüfen und kompatible Streams/Segmente zusammenführen.
MP4RemuxThread	MP4-Remux ohne Video-Re-Encode, Audio-/Subtitle-Policy anwenden.
AudioMuxThread	Audio neu planen/muxen, andere Inhalte kopieren.
AudioVideoMatchThread	Matcher-Core ausführen und Audio-Synchron-/Schnittplan materialisieren.
QualityTestThread	Testsegmente encoden, ffprobe analysieren, SSIM/VMAF best effort messen.
MoveThread	Zielpfade, Konflikte, Sidecars und Reports verarbeiten.
DVRemuxThread	Dolby-Vision-Remux-Sonderworkflow.
ParallelConverterThread	Mehrere ConverterThread-Instanzen koordinieren und Ergebnisse zusammenführen.

18.3 Kritische Worker-Fehler und Parallelbetrieb

- Unbehandelte Laufzeitfehler im Converter-Worker werden als Crashbericht sichtbar gemacht; noch offene Dateien werden als fehlgeschlagen abgeschlossen.
- Im Parallelbetrieb wird ein Child-Worker, der ohne finales Dateiergebnis endet, als Fehler in Queue, Log und Abschluss übernommen. Ein solcher Lauf bleibt nicht mehr still hängen.

19 Online-Metadaten-System

DragonTools unterstützt TMDB und TheTVDB getrennt für Filme und Serien. Für jeden Medientyp kann `TMDB`, `TheTVDB` oder `Beide` gewählt werden. Bei normalen Einzelauflösungen (`resolve_movie`/`resolve_series`) arbeitet die Providerkette weiterhin priorisiert. Der Serien-Renamer verwendet dagegen Kandidatenlisten: bei `Beide` können bis zu

sechs gültige Episodenkandidaten je Provider sichtbar bleiben; sortiert wird primär nach Match-Qualität und erst bei Gleichstand nach Provider-Präferenz.

19.1 Providerwahl und Priorität

Einstellung	Werte	Default/Verhalten
Filme – Provider	TMDB / TheTVDB / Beide	Default TMDB
Filme – bevorzugt	TMDB / TheTVDB	Default TMDB; nur bei Beide aktiv
Serien – Provider	TMDB / TheTVDB / Beide	Default TMDB
Serien – bevorzugt	TMDB / TheTVDB	Default TMDB; nur bei Beide aktiv
TMDB verwenden	an/aus	Default aus, Zugangsdaten erforderlich
TheTVDB verwenden	an/aus	Default aus, Zugangsdaten erforderlich
Sprache	freie Combo, Vorschläge de-DE/en-US/ja-JP/fr-FR	Default de-DE
Fallback-Sprache	freie Combo	Default en-US
Cache	an/aus	Default an
Cache-Alter	1–365 Tage	Default 30 Tage

Konkrete Auflösung bei „Beide“: `provider_chain_for()` liefert die konfigurierte Reihenfolge. Direkte Einzelauflösungen verwenden diese Kette als bevorzugt/Fallback. `CompositeMetadataClient.resolve_episode_candidates()` sammelt dagegen bis zu sechs Kandidaten je aktivem Provider. `movie_renamer._sort_candidates()` sortiert score-first; die Provider-Reihenfolge ist nur Tie-Breaker. Vor Aufnahme in die Liste wird geprüft, ob die erkannte Staffel/Episode beim Kandidaten tatsächlich existiert.

19.2 Zugangsdaten und Sichtbarkeit

Feld	Verhalten
TMDB Read-Access-Token	Passwortfeld; im Dialog als empfohlen beschrieben.
TMDB API-Key	Passwortfeld; als zusätzlicher/fallbackfähiger Zugang gespeichert.
TheTVDB API-Key	Passwortfeld; Grundlage für automatischen Login.
TheTVDB PIN	Passwortfeld; optionaler Subscriber-PIN.
TheTVDB Bearer-Token	Passwortfeld; kann direkt verwendet werden.
Zugangsdaten anzeigen (TMDB + TheTVDB)	Eine gemeinsame Checkbox im separaten Bereich „Zugangsdaten“. Sie schaltet EchoMode für alle fünf geheimen Felder gleichzeitig um. Damit ist ausdrücklich erkennbar, dass sie für beide Provider gilt.

Sensible Schlüssel sind in `SENSITIVE_SETTINGS_KEYS` zusammengefasst und werden von Backup-/Diagnosefunktionen gesondert behandelt. TheTVDB kann bei Bedarf aus API-Key und PIN ein temporäres Bearer-Token erzeugen; ein manuell hinterlegtes Token wird direkt verwendet.

19.3 Datenfluss

Dateiname/Serienname -> Parser -> OnlineMetadataConfig aus QSettings -> Providerkette -> persistenter Such-/Preflight-Cache oder frische Batch-Session -> Providerclient -> HTTP/JSON -> Normalisierung/lokalisierter Titel -> Movie-/Series-/Episode-Suggestion -> Renamer, NFO oder GUI-Auswahl. Finale Verarbeitung kann Providerdaten einmal frisch pro Serie/Batch laden und anschließend innerhalb desselben Laufs wiederverwenden; ein gültiger Cache wird vor einer unnötigen TheTVDB-Authentifizierung geprüft.

TheTVDB besitzt zusätzliche Logik für lokalisierte Titel und verschiedene v4-Antwortformen. german_title_variants.py erzeugt vorsichtige Suchvarianten für deutsche Umlaute/ASCII-Schreibweisen. Eine vorhandene Episode mit leerem oder generischem Titel blockiert den Sprachfallback nicht mehr; die konfigurierte Fallback-Sprache kann den fehlenden Episodentitel ergänzen.

20 Konfiguration, Datenhaltung und Dateistrukturen

DragonTools verwendet neben QSettings, JSON-Regeldateien, Caches, Logs und Reports seit V9.5 zusätzlich eine eigene eingebettete SQLite-Mediathek. Sie ist ausdrücklich von Jellyfins internem Schema entkoppelt: eine Jellyfin-Datenbank wird nur gelesen und in `media_items`/`media_streams` importiert. DragonTools schreibt nicht in Jellyfin zurück. Pfad-Mappings, Bestandsfortschreibung, strukturierte Suche und manuelle SQL-Bearbeitung betreffen ausschließlich die eigene SQLite-Datei.

20.1 QSettings und zentrale Schlüssel

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_PATH_TV	paths/tv	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_ANIME	paths/anime	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_FILME	paths/filme	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_H264_TV	paths/h264/tv	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_H264_ANIME	paths/h264/anime	– (Default am Ladeort/Legacypfad)

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_PATH_H264_FILME	paths/h264/filme	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_H265_TV	paths/h265/tv	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_H265_ANIME	paths/h265/anime	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_H265_FILME	paths/h265/filme	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_AV1_TV	paths/av1/tv	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_AV1_ANIME	paths/av1/anime	– (Default am Ladeort/Legacypfad)
SET_KEY_PATH_AV1_FILME	paths/av1/filme	– (Default am Ladeort/Legacypfad)
SET_KEY_H264_LOG_ROOT	paths/h264_log_root	– (Default am Ladeort/Legacypfad)
SET_KEY_H264_LOG_ENABLED	logging/h264_enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_H265_LOG_ROOT	paths/h265_log_root	– (Default am Ladeort/Legacypfad)
SET_KEY_H265_LOG_ENABLED	logging/h265_enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_AV1_LOG_ROOT	paths/av1_log_root	– (Default am Ladeort/Legacypfad)
SET_KEY_AV1_LOG_ENABLED	logging/av1_enabled	– (Default am Ladeort/Legacypfad)

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_ALLOW_GROWTH	storage/allow_bigger_than_source	– (Default am Ladeort/Legacypfad)
SET_KEY_SAVE_ALLOW_LARGER_OUTPUT	save/allow_larger_output	– (Default am Ladeort/Legacypfad)
SET_KEY_SAVE_ALLOW_LARGER_OUTPUT_PERCENT	save/allow_larger_output_percent	– (Default am Ladeort/Legacypfad)
SET_KEY_SAVE_MIN_OUTPUT_SIZE_ENABLED	save/min_output_size_enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_SAVE_MIN_OUTPUT_SIZE_PERCENT	save/min_output_size_percent	– (Default am Ladeort/Legacypfad)
SET_KEY_AUTOCROP_ENABLED	convert/autocrop_enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_AUTOCROP_MODE	convert/autocrop_mode	single
SET_KEY_AUTOCROP_PROBE_START	convert/autocrop_probe_start_s	– (Default am Ladeort/Legacypfad)
SET_KEY_AUTOCROP_PROBE_DURATION	convert/autocrop_probe_duration_s	– (Default am Ladeort/Legacypfad)
SET_KEY_AUTOCROP_PROBE_INTERVAL	convert/autocrop_probe_interval_s	– (Default am Ladeort/Legacypfad)
SET_KEY_IMAX_DETECT	convert/imax_auto_detect	– (Default am Ladeort/Legacypfad)
SET_KEY_IMAX_PROBE_INTERVAL	convert/imax_probe_interval_s	– (Default am Ladeort/Legacypfad)
SET_KEY_IMAX_PROBE_DURATION	convert/imax_probe_duration_s	– (Default am Ladeort/Legacypfad)
SET_KEY_IMAX_MIN_VARIANCE_PERCENT	convert/imax_min_variance_percent	15
SET_KEY_IMAX_MIN_HITS	convert/imax_min_hits	2

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_PRESERVE_DV	encoder/h265/preserve_dv	– (Default am Ladeort/Legacypfad)
SET_KEY_PRESERVE_HDRPLUS	encoder/h265/preserve_hdrplus	– (Default am Ladeort/Legacypfad)
SET_KEY_OUTPUT_MIN_SIZE_KB	validation/output_min_size_kb	1
SET_KEY_OUTPUT_DURATION_MIN_PERCENT	validation/duration_min_percent	90
SET_KEY_OUTPUT_DURATION_MAX_PERCENT	validation/duration_max_percent	125
SET_KEY_OUTPUT_DURATION_MAX_EXTRA_S	validation/duration_max_extra_s	60
SET_KEY_REPAIR_DURATION_REMUX_ENABLED	repair/duration/remux_enabled	True
SET_KEY_REPAIR_DURATION_TIMESTAMP_ENABLED	repair/duration/timestamp_enabled	True
SET_KEY_VERBOSE_LOG_ROOT	logging/verbose_log_root	– (Default am Ladeort/Legacypfad)
SET_KEY_VERBOSE_LOG_ENABLED	logging/verbose_log_enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_ACTIVE_ALL_TV	move/active_tv	– (Default am Ladeort/Legacypfad)
SET_KEY_ACTIVE_ALL_ANIME	move/active_anime	– (Default am Ladeort/Legacypfad)
SET_KEY_ACTIVE_ALL_FILME	move/active_filme	– (Default am Ladeort/Legacypfad)
SET_KEY_SERIES_DEFAULT_TYPE	move/series_default_type	– (Default am Ladeort/Legacypfad)
SET_KEY_SHUTDOWN_COUNTDOWN	move/shutdown_countdown_seconds	– (Default am Ladeort/Legacypfad)
SET_KEY_MOVE_CONFLICT	move/conflict_mode	– (Default am Ladeort/Legacypfad)

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_PREFLIGHT_SAVE_REPORT	preflight/save_report	– (Default am Ladeort/Legacypfad)
SET_KEY_PARALLEL_CPU_JOBS	parallel/cpu_jobs	1 Im Konverter auch während des Laufs über Worker 1 bis 8 veränderbar; laufende Dateien werden beim Reduzieren nicht abgebrochen.
SET_KEY_PARALLEL_GPU_JOBS	parallel/gpu_jobs	2 Im Konverter auch während des Laufs über Worker 1 bis 8 veränderbar; laufende Dateien werden beim Reduzieren nicht abgebrochen.
SET_KEY_PARALLEL_DEFAULTS_MIGRATED	parallel/defaults_cpu1_gpu2_migrated	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_MOVIE_PROVIDER	metadata/provider/movie	tmdb
SET_KEY_METADATA_SERIES_PROVIDER	metadata/provider/series	tmdb
SET_KEY_METADATA_MOVIE_PREFERRED_PROVIDER	metadata/provider/movie_preferred	tmdb
SET_KEY_METADATA_SERIES_PREFERRED_PROVIDER	metadata/provider/series_preferred	tmdb
SET_KEY_METADATA_TMDB_ENABLED	metadata/tmdb/enabled	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_TMDB_API_KEY	metadata/tmdb/api_key	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_TMDB_READ_TOKEN	metadata/tmdb/read_access_token	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_TVDB_ENABLED	metadata/thetvdb/enabled	– (Default am Ladeort/Legacypfad)

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_METADATA_TVDB_API_KEY	metadata/thetvdb/api_key	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_TVDB_PIN	metadata/thetvdb/pin	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_TVDB_BEARER_TOKEN	metadata/thetvdb/bearer_token	– (Default am Ladeort/Legacypfad)
SET_KEY_METADATA_LANGUAGE	metadata/language	de-DE
SET_KEY_METADATA_FALLBACK_LANGUAGE	metadata/fallback_language	en-US
SET_KEY_METADATA_CACHE_ENABLED	metadata/cache_enabled	True
SET_KEY_METADATA_CACHE_DAYS	metadata/cache_days	30
SET_KEY_NFO_ENABLED	postprocess/nfo/enabled	False
SET_KEY_NFO_ONLY_UNAMBIGUOUS	postprocess/nfo/only_unambiguous	True
SET_KEY_NFO_FILEINFO_ENABLED	postprocess/nfo/fileinfo_enabled	True
SET_KEY_NFO_MOVIE_TARGET_NAME	postprocess/nfo/movie_target_name	movie.nfo
SET_KEY_NFO_CONFLICT_MODE	postprocess/nfo/conflict_mode	skip
SET_KEY_TRICKPLAY_ENABLED	postprocess/trickplay/enabled	False
SET_KEY_TRICKPLAY_ONLY_MISSING	postprocess/trickplay/only_missing	True
SET_KEY_TRICKPLAY_WIDTH	postprocess/trickplay/width	320
SET_KEY_TRICKPLAY_TILE_COLUMNS	postprocess/trickplay/tile_columns	10
SET_KEY_TRICKPLAY_TILE_ROWS	postprocess/trickplay/tile_rows	10
SET_KEY_TRICKPLAY_INTERVAL_S	postprocess/trickplay/interval_s	10
SET_KEY_TRICKPLAY_JPEG_QUALITY	postprocess/trickplay/jpeg_quality	90
SET_KEY_TRICKPLAY_QSCALE	postprocess/trickplay/qscale	4
SET_KEY_TRICKPLAY_HWACCEL	postprocess/trickplay/hwaccel	cuda
SET_KEY_TRICKPLAY_MAX_JOBS	postprocess/trickplay/max_jobs	1
SET_KEY_TRICKPLAY_SOURCE_MODE	postprocess/trickplay/source_mode	output
SET_KEY_TRICKPLAY_CONFLICT_MODE	postprocess/trickplay/conflict_mode	skip
SET_KEY_SOURCE_VISUAL_CHECK_ENABLED	source_visual_check/enabled	True
SET_KEY_SOURCE_VISUAL_CHECK_INTERVAL_PERCENT	source_visual_check/interval_percent	10
SET_KEY_SOURCE_VISUAL_CHECK_SAMPLE_DURATION_S	source_visual_check/sample_duration_s	2

Konstante	QSettings-Schlüssel	zentral definierter Default
SET_KEY_SOURCE_VISUAL_CHECK_FPS	source_visual_check/fps	2
SET_KEY_SOURCE_VISUAL_CHECK_BLOCK_PERCENT	source_visual_check/block_percent	80
SET_KEY_SOURCE_VISUAL_CHECK_MIN_HITS	source_visual_check/min_hits	4

20.2 Regeldateien

Datei	Schema	Inhalt
default_audio_rules.json	Schema 4	Sprachpriorität, Trackzahl, Passthrough/Transcode, Kanalregeln, Downmix, DRC/Loudnorm.
default_subtitle_rules.json	Schema 4	Sprachen, Forced-Priorität, Burn-In, Plausibilität, Keep-Regeln und globale MP4-Sidecar-/mov_text-Policy.
default_move_rules.json	Schema 1	Serienregex, Anime-Schlüsselwörter, Collection-Regeln und Zielordnerstruktur.
default_profiles.json	Schema 3	Einfache Codec/CRF/Preset/Scale-Defaultprofile; zusätzlich existieren erweiterte interne Profildefaults.

`config_migration.py`, `audio_rules.py`, `subtitle_rules.py` und `renamer_rules.py` normalisieren bzw. migrieren ihre jeweiligen Konfigurationsstrukturen. `renamer_rules.py` kapselt Zeichenersetzungen, Suchalias-Regeln und Matching-Grenzen. Schreibvorgänge sollen normalisiert und atomar erfolgen, damit veraltete oder teilweise geschriebene Konfigurationen nicht ungeprüft übernommen werden.

20.3 Relevante Ordner-/Dateistrukturen

Serienziel (Defaultregel):

```
{tv_root}/{series_name}/Staffel {season:02d}/
{anime_root}/{series_name}/Staffel {season:02d}/
```

Filmziel (Defaultregel):

```
{film_root}/{title} ({year})/
```

Serienerkennung akzeptiert unter anderem `S01E01`, Mehrfachfolgenformen wie `S01E01E02`/`S01E01-E02` sowie `1x01`. Die konkrete Zielpfadbildung prüft außerdem Konflikte und kann je nach Einstellung skip, delete_first, overwrite oder rename anwenden.

ISO-Tab: akzeptiert ISO/IMG-Dateien, Disc-Root-Ordner und direkt ausgewählte `BDMV`- bzw. `VIDEO_TS`-Ordner. Dadurch muss nicht zwingend eine Image-Datei vorliegen, wenn bereits eine passende Disc-Verzeichnisstruktur vorhanden ist.

20.4 Eigene SQLite-Mediathek und Jellyfin-Leseimport

Die Mediathek ist eine lokale, eingebettete SQLite-Datei und benötigt keinen dauerhaft laufenden Datenbankserver. Jellyfin ist Importquelle, nicht Betriebsdatenbank. `import_jellyfin_database()` liest passende Tabellen/Spalten defensiv, normalisiert die Daten und schreibt anschließend ausschließlich in die DragonTools-Datenbank.

Tabelle	Inhalt
meta	Schema-/Statusinformationen
path_mappings	Jellyfin-/NAS-Präfix → lokaler Pfad
media_items	Medientyp, Titel/Originaltitel, Serie/Staffel/Episode, Pfad, Dateigröße, Laufzeit, Container, Auflösung, Video-/Gesamtbitrate, HDR-Flags, Analysezustand sowie NFO-Bestandsstatus/-pfad/-typ/-zeitstempel.
media_streams	Video-/Audio-/Untertitelstreams mit Sprache, Codec, Profil, Bitrate, Dauer, Frame-/Farbdaten, Kanälen, Forced-Status und Zusatzfeldern
media_provider_ids	Normalisierte Provider-IDs je Medium, z. B. TMDB, TheTVDB und IMDb; getrennt von der Jellyfin-Quell-ID.
metadata_values / media_item_values	Normalisierte Genres, Tags und Studios sowie deren Verknüpfung mit Medien.
people / media_people	Personen einmalig per Name/ID; Relation zu Medien mit Rolle/Typ, ohne Biografien oder Bilder.
collections / collection_members	Filmreihen/Collections und deren Mitgliedschaftsbeziehungen.
nfo_metadata / nfo_provider_ids / nfo_issues	Separat gelesene NFO-Kontrolldaten, NFO-Provider-IDs und auswertbare DB↔NFO-Abweichungen; überschreiben keine Mediathek-Metadaten.

- Die DB-Initialisierung aktiviert Foreign Keys und WAL. Normale Read-Verbindungen führen keine unnötigen Schema-Writes oder wiederholte journal_mode-Umschaltungen aus.
- `record_media_file()` kann lokale Medienanalysen in die Mediathek schreiben; `record_moved_file()` hält den Pfad nach einem DragonTools-Move nach.
- `apply_path_mappings()` arbeitet präfixbasiert und toleriert unterschiedliche Slash-Schreibweisen.
- Manuelle schreibende SQL-Kommandos erzeugen vorher ein Backup der eigenen Datenbank.
- Schema 6 erweitert bestehende Datenbanken kompatibel. Fehlende Spalten werden vor davon abhängigen Indizes ergänzt. Neben technischen Stream- und Videoeigenschaften speichert DragonTools Originaltitel, normalisierte Provider-IDs, Genres/Tags/Studios, Collections/Filmreihen und schlanke Personenbeziehungen. Beim Jellyfin-Import wird normalized_title aus dem tatsächlich aufgelösten Seriennamen gebildet, damit Episoden ohne direktes SeriesName-Feld den schnellen Serien-Lookup weiterhin nutzen.

20.5 Mediathek-Suche und Qualitätsklassen

Der Mediathek-Dialog trennt GUI und Datenbankarbeit. Suchanfragen laufen in einem QThread; nur die fertigen Ergebnisse werden an die Oberfläche zurückgegeben. Der Serien-Lookup nutzt zuerst den indexierten normalized_title und fällt nur bei alten oder inkonsistenten Beständen auf langsamere Legacy-Vergleiche über series_title/title zurück. Trefferlisten bleiben für die GUI begrenzt, während CSV-Exporte dieselben Filter ohne Anzeigegrenze erneut ausführen.

Kategorie	Kriterien
Audio	Deutsch vorhanden/fehlend; mehrere Audiospuren; Stereo; 5.1; 7.1; AAC; AC3; EAC3; TrueHD; DTS; FLAC; unbekannt
Dynamikumfang	SDR; HDR; HDR10+; Dolby Vision; unbekannt
Auflösung	kleiner als HD; HD/720p; Full HD/1080p; QHD/1440p; 4K/UHD; unbekannt
Video-Codec	H.264/AVC; H.265/HEVC; AV1; unbekannt
Laufzeit	unbekannt; unter einer Minute; über fünf Stunden
Metadaten	unvollständige/fehlende Metadaten; Videoeigenschaften unbekannt
Abweichungen	Deutsch; Auflösung; Video-Codec; HDR/SDR; Audio-Codec; Audiokanäle; Anzahl Audiospuren

Auflösungsbucket: Die Klassifizierung nutzt Breite und Höhe. Dadurch wird z. B. ein 1920×800-Cinemascope-Encode weiterhin der Full-HD-Klasse zugeordnet. Die Grenzen sind grob: UHD ab etwa 3000 px Breite oder 1800 px Höhe, QHD ab 2300/1300, Full HD ab 1600/900, HD ab 1100/650; darunter SD. Sind Breite und Höhe unbekannt, wird kein erfundener Bucket gesetzt.

SDR-Schutz: `is_hdr = 0` reicht allein nicht. Eine Datei gilt nur dann als SDR, wenn weder HDR/HDR10+/DV erkannt wurde und zusätzlich der Analysezustand bzw. der Videostream ausreichend belegt, dass Videoeigenschaften bekannt sind. Unanalysierte Einträge bleiben „HDR/SDR unbekannt“.

20.6 Abweichungssuche: Mehrheits- und Gleichstandslogik

Abweichungen werden innerhalb eines gemeinsamen `parent_path` bewertet. Das entspricht in einer typischen Serienstruktur dem jeweiligen Staffel-/Medienordner und verhindert, dass unterschiedliche Staffeln oder völlig getrennte Filme gegenseitig als Abweichung wirken.

```
Beispiel: 12 Folgen im selben Staffelordner
10 × deutsche Audiospur
 2 × keine deutsche Audiospur
→ strikte Mehrheit: Deutsch (10/12)
→ Ergebnis: nur die 2 Folgen ohne Deutsch
```

Situation	Ausgabe
Alle Werte identisch	keine Abweichung
Strikte Mehrheit > 50 %	nur Minderheitsdateien; Abweichungsgrund nennt Mehrheitswert und Verhältnis
Gleichstand / keine strikte Mehrheit	Gruppe als „Uneinheitlich – keine klare Mehrheit“; DragonTools rät keine „richtige“ Seite
Gruppe < 2 Dateien	keine vergleichende Auswertung

Unterstützte Vergleichssignaturen sind deutsche Audiospur, Auflösungsbucket, Video-Codec-Bucket, Dynamikumfang, Audio-Codec-Signatur, Audiokanal-Signatur und Anzahl der Audiospuren. Die Ergebnistabelle besitzt dafür eine eigene Spalte „Abweichung“.

20.7 SQLite-Indizes, Performance und SQL-Sicherheit

Index-/Bereich	Nutzen
media_items(parent_path)	Gruppierung und Abweichungssuche
media_items(video_codec, width, height)	Codec-/Auflösungsfilter
media_streams(media_id)	schneller Join je Medium
media_streams(media_type)	Streamtyp-Filter
media_streams(media_type, language)	Sprach-/Deutschsuche
media_streams(media_type, codec)	Audio-/Video-Codec-Suche

Manuelle SQL-Abfragen bleiben ein Expertenwerkzeug. Nur reine SELECT-Abfragen werden ohne Sicherheitsbackup ausgeführt; PRAGMA, WITH und schreibende Statements werden vorsichtshalber wie potenziell verändernde Befehle behandelt und sichern vorher die DragonTools-SQLite-Datei. Sicherheitsbackup und binärer Datenbankexport verwenden die SQLite Online Backup API, sodass commitete WAL-Seiten im konsistenten Snapshot enthalten sind. Gespeicherte SQL- und GUI-Suchen liegen außerhalb der Mediathek-DB und überstehen deshalb einen Neuimport. Die eingebaute SQL-Hilfe liest das aktuelle Schema dynamisch aus und zeigt Tabellen, Spalten, Datentypen, typische Werte und Beispielabfragen. Jellyfins DB bleibt im DragonTools-Workflow read-only.

20.8 Ziel des Mediathek-Speicherpfad-Scans

Der Jellyfin-Import bleibt sinnvoll, wenn bereits eine gepflegte Jellyfin-Datenbank existiert. Er enthält aber nicht in jedem Bestand zuverlässig alle technischen Merkmale, insbesondere nicht immer HDR10+, DV-Details, Untertitel-Codec oder vollständige Streaminformationen. Der Speicherpfad-Scan löst dieses Problem direkt aus Dragon Tools: Die in den Speicherpfaden hinterlegten Film-, Anime- und TV-Ordner werden rekursiv gelesen, jede erkannte Videodatei wird mit MediaInfo analysiert, und Dragon Tools schreibt daraus eine eigene, einheitliche SQLite-Mediathek.

- Der Scan verändert weder Jellyfin noch die Videodateien auf der NAS.
- MediaInfo wird bevorzugt, weil es Streamtypen, HDR-Hinweise, Bitraten und Sprachfelder sehr vollständig und konsistent liefert.
- Kann MediaInfo eine Videodatei nicht analysieren, verliert der Speicherpfad-Scan sie nicht mehr. Dragon Tools schreibt einen Minimal-Eintrag mit Pfad, Dateiname, Container, Größe und – soweit aus dem Namen ableitbar – Serien-/Staffel-/Folgenkennung. `analysis_status=analysis_failed` kennzeichnet den Platzhalter eindeutig; technische Qualitätsfelder bleiben unbekannt statt erfundene Werte vorzutäuschen.
- Die Quelle der Daten bleibt nachvollziehbar: Jellyfin-Import, DragonTools-Move oder Speicherpfad-Scan schreiben in dasselbe Datenmodell, aber mit einheitlich normalisierten Feldern.

20.9 Datenbankzustand: exists_flag und active

Dragon Tools unterscheidet bewusst zwei Zustände. `exists_flag` beschreibt, ob ein Eintrag aus Sicht der letzten Aktualisierung physisch existiert. `active` beschreibt, ob dieser Eintrag fachlich die aktuell gültige Datei einer Medienidentität ist. Das ist vor allem bei Serienfolgen wichtig: Wenn eine Folge ersetzt wurde, kann der alte Pfad historisch noch bekannt sein, darf aber nicht mehr als gültiger Treffer in normalen Suchen auftauchen.

Feld	Bedeutung	Typische Wirkung
exists_flag=1	Pfad wurde als vorhanden importiert, gescannt oder nach einem Move geschrieben.	Eintrag kann in normalen Suchen erscheinen, sofern er auch aktiv ist.

Feld	Bedeutung	Typische Wirkung
exists_flag=0	Pfad ist Quelle einer verschobenen Datei oder wurde bewusst als nicht mehr vorhanden markiert.	Eintrag bleibt nachvollziehbar, wird aber nicht als aktueller Bestand gezählt.
active=1	Fachlich gültiger aktueller Bestandseintrag.	Normale Suchen, Preflight-Hilfen und Bestandsauswertungen arbeiten mit diesen Einträgen.
active=0	Historischer oder ersetzter Eintrag.	Kann bereinigt werden; wird nicht als aktuelle Folge gezählt.

Hinweis: Das Schema wird kompatibel auf Schema 6 migriert. Bestehende Datenbanken erhalten neue Metadaten-, Collection-, Personen- und NFO-Tabellen sowie die benötigten Bestandsfelder; Einträge mit `exists_flag=0` werden dabei nicht wieder als aktiv behandelt.

20.10 Mediathek-Suche, Duplikate und Bereinigung

Normale Mediathek-Suchen filtern aktive, vorhandene Einträge. Dadurch tauchen ersetzte Dateien nicht mehr als vermeintlich aktuelle Treffer auf. Für Wartung und Kontrolle gibt es zusätzlich die Abfrage nach doppelten aktiven SxxExx-Einträgen. Sie findet Serienfolgen, bei denen trotz Move-Logik mehr als eine aktive Datei mit gleicher Serien-/Staffel-/Folgenkennung existiert.

- Die Duplikatsuche dient als Sicherungsnetz nach manuellen Datenbankänderungen, externen Kopieraktionen oder älteren Imports.
- Inaktive Einträge können im Mediathek-Dialog mit Sicherheitsabfrage bereinigt werden.
- Vor manuellen schreibenden SQL-Aktionen wird die DragonTools-Datenbank gesichert.
- Jellyfin-Datenbanken bleiben reine Lesequellen und werden niemals von Dragon Tools beschrieben.

20.11 Fehlertoleranter Mediathek-Scan

- Der leichte NFO-Scan arbeitet ausschließlich auf bereits bekannten Mediathek-Einträgen. Er startet weder MediaInfo noch ffprobe und läuft nicht rekursiv über die gesamte NAS. Stattdessen prüft er gezielt die zu den bekannten Medienpfaden erwarteten NFO-Dateien und erfasst Status, Pfad, Typ, Änderungszeit und Scanzeit.
- NFO-Zustände sind `unknown`, `present`, `missing`, `unreachable`, `invalid` und `unreadable`. `missing` wird nur gesetzt, wenn der konkrete Medienordner erreichbar ist. Ist ein NAS-/Share-Pfad nicht erreichbar, bleibt der letzte bekannte NFO-Pfad samt geparsten NFO-Daten und Prüfergebnissen erhalten und der Status wird `unreachable`. Ordnernamen mit Punkten wie `Mr. Robot` werden anhand des Medientyps bzw. des Dateisystems als Ordner behandelt und nicht über `Path.suffix` fehlklassifiziert.
- Aus der NFO gelesene Titel, Originaltitel, Staffel/Folge, Jahr und Provider-IDs werden separat in NFO-Kontrolltabellen gespeichert und überschreiben die eigentlichen Mediathek-Metadaten nicht. DragonTools kann dadurch DB und NFO auf Abweichungen prüfen. Die NFO-Erstellung selbst bleibt davon getrennt und verwendet weiterhin die aktuelle Online-Metadatenabfrage statt potenziell veralteter Mediathek-Daten.

21 Logging, Fehlerberichte, Recovery und Diagnose

Logging ist global organisiert, obwohl Legacy-Schlüssel codecbezogen existieren. Normal-Logs, Verbose-Logs, ErrorReports, CrashReports, Audit-Log, Preflight-/Move-/Run-Reports und Diagnosepakete erfüllen unterschiedliche Aufgaben.

Artefakt/Modul	Zweck
DragonLogger / normaler Log	Laufentscheidungen, Toolaufrufe, Warnungen, Fehler und Status; Monats-/Datumsstruktur.
VerboseLogger	Zusätzliche technische Detailspur für tiefe Fehlersuche.
ErrorReports	Kontext eines fehlgeschlagenen Konvertierungsworkflows, Befehle/Schritte und bekannte Zustände.
Crash Guard	Aktivitätsmarker und CrashReports für unerwartete Prozessabbrüche.
JobJournal	Persistiert unvollständige Batches und erzeugt Resume-Pläne.
Audit Log	Nachvollziehbare Einstellungs-/Profiländerungen.
Run Summary	Zusammenfassung eines Batches inklusive Ergebnis-/Dateigrößenstatus.
Preflight Report	Optionaler Report der geplanten Ziel-/Vorabentscheidungen.
Diagnostic Package	Sammelt Diagnoseinformationen und relevante Dateien, wobei sensible Einstellungen geschützt werden sollen.

Log-Bereinigung kann normale Logs, ErrorReports, CrashReports und VerboseLogs bis zu einem Datum oder vollständig löschen. Der aktive Crash-Marker ist explizit ausgenommen, damit ein laufender Prozess nicht durch Cleanup seinen Recovery-Indikator verliert.

21.1 Änderungshistorie: JSON-Struktur und Seitennavigation

- V9 nutzt bevorzugt `Aenderungshistorie/CHANGELOG.json` als strukturierte Quelle. Die bisherige `CHANGELOG.txt` bleibt als Fallback und zur manuellen Lesbarkeit erhalten; Legacy V8/V7 bleiben TXT.
- Die frühere Button-Matrix mit vielen abgeschnittenen Bereichsnamen wurde durch eine breite Bereichsauswahl ersetzt. Die vollständige Überschrift ist jederzeit sichtbar.
- Jeder Funktionsbereich wird anhand eines festen Zeilenbudgets in Seiten geteilt. Lange Abschnitte wie „04 Konverter, Encoder & Profile“ ergeben dadurch mehrere blätterbare Seiten statt eines langen Scrolltextes.
- Die Navigation trennt Seite zurück/weiter von Bereich zurück/weiter und zeigt „Bereich x von y · Seite a von b“. Die Inhaltsansicht blendet die vertikale Scrollleiste aus.

21.2 Eindeutige Job- und Move-Journale

- Job- und Move-Läufe schreiben nicht mehr in eine gemeinsame active_*.json-Datei.
- Jeder Lauf erhält eine eigene Journaldatei mit eindeutiger ID. Parallele und überlappende Läufe überschreiben sich dadurch nicht.
- Legacy-active-Dateien bleiben lesbar, damit ältere Restzustände weiterhin erkannt werden können.

21.3 Später entscheiden und vollständige Move-Wiederaufnahme

- Das Journal trennt den fachlichen Move in video_pending, sidecars_pending, cleanup_pending und completed. Ein bereits erfolgreich verschobenes Video wird nicht als vollständig abgeschlossen markiert, solange Companion-Dateien noch offen sind.
- Offene Journale bleiben erhalten, wenn beim Programmstart „Später entscheiden“ gewählt wird.
- Beim Restore werden nicht nur Quelldateien zurückgeladen, sondern auch Zielpfade, planned_targets, Sidecars, Konfliktmodus und der Move-Kontext.
- Ein bewusst gestarteter Retry archiviert erst dann das alte Journal, wenn der neue Move-Journalzustand erfolgreich angelegt wurde.

21.4 Crash-Recovery beim Verschieben

- Nach einem Crash zwischen Video-Commit und Sidecar-/NFO-/Trickplay-Verschiebung kann der Resume mit dem bereits vorhandenen Zielvideo als Anchor fortsetzen. Bereits vorhandene Ziel-Sidecars werden idempotent als erledigt erkannt. Mediathek/DB werden erst nach erfolgreichem Companion-Commit aktualisiert.
- Running-Journale werden beim Start geprüft. Existiert das Ziel vollständig und ist die Quelle bereits weg, wird der Move als abgeschlossen erkannt.
- Temporäre Altbestand-Backups werden nur dann bereinigt, wenn ein erfolgreicher Commit eindeutig belegt ist: Ziel existiert und Quelle existiert nicht mehr. Existieren Quelle und Ziel gleichzeitig, bleibt das Backup erhalten und der Zustand wird als mehrdeutig markiert.
- Schreibfehler im Journal werden nicht mehr still verschluckt, sondern geloggt und als Recovery-Warnung sichtbar gemacht.

22 Externe Bibliotheken, Programme, Build und Veröffentlichung

22.1 Python-Abhängigkeiten

Bibliothek	Verwendung
PyQt6	GUI, QThread/Signals, QSettings; zwingend für die Desktop-App. In der Analyseumgebung nicht installiert, daher konnten PyQt-abhängige Tests nicht gesammelt werden.
NumPy	Optional/gezielt im Audio-Video-Matcher für numerische Bildsignaturen.
OpenCV (`cv2`)	Optionales Analysebackend des Audio-Video-Matchers; bei fehlendem OpenCV existiert ein Fallback.
pytest	Nur Tests/Entwicklung, nicht Laufzeit der ausgelieferten App.
pyi_splash	Optional im Frozen/PyInstaller-Startpfad für Splash-Steuerung.

22.2 Externe Programme

Programm	Rolle
FFmpeg	Encoding/Remux/Filter/Frames/Audio/Untertitel/Trickplay; zentral und in vielen Workflows erforderlich.
FFprobe	Stream-/Format-/Daueranalyse; eng mit FFmpeg gekoppelt.
MediaInfo	Erweiterte Medien-/HDR-/DV-Analyse; je nach Workflow ergänzend/fallback.
MKVToolNix (mkvmerge/mkvextract/mkvpropedit)	MKV-Remux, Extraktion, Untertitel/Flags, Dauerreparatur.
dovi_tool	Dolby-Vision-RPU und Profil-/Metadatenbearbeitung; nur DV-Erhaltungsworkflow.
hdr10plus_tool	HDR10+-Metadaten extrahieren/injizieren; nur HDR10+-Erhaltung.
MP4Box/GPAC	Finaler Dolby-Vision-MP4-Mux bzw. DV-Sonderpfad.
MakeMKV CLI (`makemkvcon`)	ISO/Disc-Extraktion, wenn verfügbar; ISO-Tab kann optional FFmpeg-Fallback verwenden.
HandBrake	Externes Programm, aus Menü startbar; keine Kernabhängigkeit des Converters.
RenameMyTVSeries	Extern startbar; Renamer-Funktionalität existiert zusätzlich nativ in DragonTools.

Aktuell geprüfter Toolstand am 05.09.2026: FFmpeg/ffprobe 2026-09-02-git-9fc8c785e2 (Gyan Full Build), MediaInfo 26.05, MKVToolNix 101.0, MakeMKV 1.18.4, hdr10plus_tool 1.7.2 und GPAC/MP4Box 26.07. dovi_tool ist kein unverändertes offizielles Release, sondern ein eigener Build auf Basis des quietvoid/dovi_tool-Branch-Stands vom 03.09.2026.

MediaInfo ist für HDR/Dolby Vision die primäre Analysequelle; ffprobe bleibt Fallback und Ergänzung. Für finale Dolby-Vision-MKV wird mkvmerge verwendet, für Dolby-Vision-/HDR10+-MP4 MP4Box. Für Subtitle-Tagging in MKV verwendet DragonTools den konfigurierten mkvpropedit-Pfad. Die GUI übergibt einen ordinalen Trackindex als track:n; Returncode 0 gilt als Erfolg, Returncode 1 als zulässige MKVToolNix-Warnung und >=2 als Fehler.

22.3 Build-Dateien im bereitgestellten Paket

Der aktuelle V9.8.3-Quellstand enthält den kanonischen Builder build_v9.bat, den Kompatibilitäts-Wrapper build_v9_angepasst.bat, release_manifest.json, die vier Requirements-Dateien, pytest.ini und INTEGRATION_TESTS.md. build_v9.bat liest APP_VERSION direkt aus dragontools\core\version.py, prüft benötigte lokale externe Tools fail-fast und baut per PyInstaller-CLI ohne zwingende handgepflegte .spec-Datei.

Der Builder legt Runtime-Konfigurationen im fertigen Onedir-Build unter Daten\dragontools\config ab, liefert aber keinen separat lesbaren Daten\Python-Quellbaum mehr aus. release_validation.py prüft Quellprojekt und Frozen-Bundle mit getrennten Verträgen. Beim Doppelklick bleibt das Buildfenster bei Erfolg oder Fehler offen; automatisierte Aufrufe können die Pause über DRAGONTOOLS_BUILD_NO_PAUSE unterdrücken.

22.4 Aktuelle Renamer-Module und Simulator-Erweiterung

Die neue Renamer-Suchlogik ist auf mehrere kleine Fachmodule verteilt: `movie_renamer_episode_refresh.py` erneuert generische Cache-Episodentitel, `movie_renamer_matching.py` wählt die konfigurierten Score-Stufen und klassifiziert Fallback-Treffer; `movie_renamer_search_actions.py` enthält die manuellen Film-/Serien-/Suchbegriff-Aktionen; `movie_renamer_resolve_search.py` bereitet diese Suchjobs vor; `movie_renamer_table_search.py` kapselt tabellenbezogene Suchzustände. `rules_preview.py` berechnet die Zielauflösung für den Regel-/Profil-Simulator; GUI und TXT-Bericht verwenden denselben Wert.

23 Qualitätssicherung und Testabdeckung

Historische Abnahme V9.7 vom 09.09.2026: Der damalige Quellbestand wurde vollständig geprüft. Alle 579 Python-Dateien sind syntaktisch lesbar; 427 Produktivmodule wurden ohne Importfehler geladen. Der vollständige Lauf aller 147 Testdateien endete mit 1.100 bestandenen Tests, 2 gezielt übersprungenen realen DV/HDR-Integrationsprüfungen und 0 Fehlern. Release-, Versions-, Container-, Persistenz-, Sidecar-, Recovery-, GUI-, Prozess- und Fassadenverträge sind durch Regressionstests abgesichert.

Testmodul	Testfunktionen	Schwerpunkte
__init__.py	0	Hilfs-/Paketdatei
test_audio_processing.py	4	drc for ac3 source forces transcode and input arg, drc for aac source is documented but does not force transcode, loudnorm forces transcode and adds filter, file override can disable global audio processing
test_audio_video_matcher.py	7	parse cut regions accepts comma decimal and timecodes, opencv signature backend when available, extra before and after source is case a not c, linear drift is case b, offset jump inside common content is case c, target extra end blocks linear audio plan
test_audit_log.py	2	audit log writes event to configured log root, audit summary masks sensitive values
test_auxiliary_workers.py	15	iso makemkv source uses iso and file prefixes, iso makemkv source uses disc root for direct bdmv folder, iso detects direct bdmv and video ts folders, iso extracts multiple titles as single makemkv calls, iso scan reports outdated makemkv, iso ffmpeg fallback detects largest bluray stream

Testmodul	Testfunktionen	Schwerpunkte
test_batch_preflight.py	12	batch preflight marks clean file as ok, batch preflight warns for ignored dv and old container, batch preflight turns analysis exception into error row, batch preflight errors when no video stream is detected, batch preflight filesystem checks show planned output, batch preflight detects overwrite container conflict
test_cleanup_rollback.py	5	cleanup removes failed output and sidecars, cleanup overwrite removes file but keeps temp dir until batch end, cleanup empty overwrite dirs keeps nonempty parallel dir silent, replace same path rolls original back on failure, replace size policy block marks file as not replaced
test_codec_profile_assistant.py	4	h265 assistant profiles cover cpu and nvenc, av1 assistant profiles use numeric svt presets, profile for assistant key returns copy, anime series cpu profile uses animation and more lookahead
test_config_migration.py	3	profile manager migrates legacy profile file, audio rules migration adds schema and stereo copy range, move rules migration adds missing defaults
test_conversion_controller_start.py	3	start worker ui state uses initial progress and starts worker once, parallel file progress defaults to combined display and can focus file, single initial file still uses parallel thread when limit is above one

Testmodul	Testfunktionen	Schwerpunkte
test_conversion_result_service.py	9	sofort abbruch fragt und startet move fuer erfolgreiche dateien, abbruch nach datei fragt und startet move fuer erfolgreiche dateien, abbruch ohne move wunsch finalisiert ohne shutdown, abbruch ohne fertige dateien bleibt altes cleanup verhalten, warning result is terminal but not successful, postprocess pending result is not terminal
test_converter_detection.py	4	imax probe offsets nutzen einstellbares intervall, imax auto erkennt wechselnde aktive bildflaeche, imax auto respektiert mindestanzahl verwertbarer punkte, imax auto bleibt aus bei konstanter aktiver bildflaeche
test_crash_guard_cleanup.py	2	clear activity removes empty fatal runtime log, unclean report removes empty previous fatal runtime log
test_current_ffmpeg_abort.py	2	terminate current ffmpeg only kills matching ffmpeg, terminate current ffmpeg rejects other current file
test_diagnostic_package.py	1	create diagnostic package collects logs and settings
test_disk_space.py	2	disk space check reports warning when reserve is low, disk space check reports critical when temp outputs do not fit
test_duration_repair_service.py	14	duration repair remuxes mkv and accepts fixed duration, duration repair archiviert wenn remux dauer falsch bleibt, duration repair greift nur bei reinem mkv laufzeitfehler, duration repair kann deaktiviert werden, workflow verify akzeptiert erfolgreiche laufzeitreparatur, detect timestamp problem repariert korrektes 23976 video nicht

Testmodul	Testfunktionen	Schwerpunkte
test_dv_regressions.py	7	dv auto exportiert keep subs als sidecars, dv custom export respektiert keep auswahl, dv sidecars werden nach replace zum finalen stem verschoben, dv level5 editor nutzt keinen autocrop fallback, hdr10plus bitstream service extract inject und verify, workflow reicht hdr10plus erhalt an dv runner weiter
test_dv_remux_abort_fix.py	0	Hilfs-/Paketdatei
test_encoder_args.py	8	h265 encoder forciert echtes 10bit, h264 bleibt 8bit kompatibel ohne 10bit zwang, nvenc lookahead wird ohne bframes nicht gesetzt, nvenc auto lookahead level und multipass werden nicht gesetzt, nvenc lookahead level und multipass werden bei auswahl gesetzt, x265 lookahead wird ohne bframes nicht gesetzt
test_encoder_inactivity_timeout.py	3	ffmpeg inactivity timeout allows active progress, ffmpeg inactivity timeout aborts silent process, explicit disabled timeout does not fall back to general
test_encoder_profile_override.py	4	profile to override rejects wrong codec, profile to override accepts codec enum defaults, normalize override preserves encoder profile, effective encoder settings uses file profile values
test_encoder_settings_shutdown.py	5	load encoder settings does not reset shutdown checkbox, cpu encoder panel is visible for h264 and expanded, reset defaults resets cpu encoder fields, assistant profile applies cpu profile and saves, av1 assistant profile applies numeric cpu preset

Testmodul	Testfunktionen	Schwerpunkte
test_error_report.py	3	error report writes context and tool output, worker result service stores failure details, worker result service blocked size policy is not success
test_extended.py	0	Hilfs-/Paketdatei
test_gui_dialog_services.py	5	timeout dialog saves minutes and enabled state, encoder profile service returns selected assistant profile, encoder profile service allows same label for different encoders, profile manager same label same encoder reuses existing key, profile manager contains extended h265 start profiles
test_hdr10_color_standard.py	7	dv7 standard encode keeps hdr10 base color tags, dv7 string profile keeps hdr10 base color tags, dv5 standard encode uses libplacebo conversion then hdr10 tags, dv cpu encode args use x265 placebo and cpu 10bit format, standard runner sets hdr10 output flags for dv7 h265, standard runner leaves sdr output untagged
test_incremental_move_queue_edit.py	6	incremental move does not lock queue edit, incremental move finish keeps queue edit enabled, incremental move finish uses finished thread reference, incremental move does not start without running conversion, preflight report save uses rows builder and logs, preflight dialog report checkbox controls save flag
test_jellyfin_nfo.py	3	movie nfo contains clean tmdb metadata without user state, episode nfo uses episode root and numbers, episode nfo writes tvdb ids for thetvdb provider

Testmodul	Testfunktionen	Schwerpunkte
test_job_journal.py	6	job journal records and archives completed run, unfinished job summary uses active journal, resume plan requeues open and running files, job journal tracks multiple parallel current files, resume plan can include failed files, archive active job journal removes active file
test_language_priority_rules.py	18	audio prioritaet nimmt naechste vorhandene sprache, audio fallback uebernimmt alle wenn keine prioritaet passt, audio kommentarspur wird uebersprungen, audio max sprachen begrenzt sprachgruppen, untertitel burn nutzt feste burn sprache, forced burn waehlt bestes format trotz mehrerer kandidaten
test_log_cleanup.py	2	log cleanup deletes old logs but keeps crash state, log cleanup respects selected categories
test_media_analyzer_pix_fmt.py	3	build video streams speichert mediainfo pix fmt im model, build video streams speichert frame infos im model, audio streams nutzen ffprobe globalindex bei bluray reihenfolge
test_mediainfo_details.py	2	mediainfo details zeigt bitraten und spuren, mediainfo details schaetzt videobitrate wenn video bitrate fehlt
test_models.py	0	Hilfs-/Paketdatei
test_move_conflicts.py	5	find target conflicts matches same video stem with other extension, find target conflicts includes exact and other video extension, find target conflicts does not delete non video by same stem, find target conflicts uses exact name for non video files, resolve rename path uses free video stem

Testmodul	Testfunktionen	Schwerpunkte
test_move_report.py	4	move report groups by target and counts conflict actions, move report format mentions deleted and replaced files, move report counts multiple removed target versions, move report groups sidecars by type and conflict action
test_move_rules_series_detection.py	4	series detection removes separator before episode code, series detection keeps internal hyphenated names, series detection normalizes missing space after separator, existing series dir matches tmdb colon and year suffix
test_movie_renamer.py	12	parse movie release name strips release tags and keeps search core, build movie rename proposal prefers matching title and year, build movie rename proposal retries german umlaut variant, movie rename proposal skips probable series episode, build series rename proposal uses episode metadata without trailing dash, parse series release name preserves episode number dot
test_online_metadata.py	15	parse movie query extracts title and year from release name, parse movie query keeps manual dotted title without video extension, clean tmdb collection name removes provider suffixes, tmdb client resolves collection with read token, tmdb client retries movie search with german umlaut variant, tmdb client resolves series year with read token
test_output_path_service.py	2	output paths are reserved for parallel non overwrite jobs, output paths are reserved for parallel overwrite temp jobs

Testmodul	Testfunktionen	Schwerpunkte
test_output_verifier.py	5	output verifier accepts video audio and plausible duration, output verifier rejects missing audio when source had audio, output verifier rejects implausibly short duration, output verifier uses configurable duration tolerance, output verifier reports missing output file
test_parallel_converter_thread.py	4	parallel thread starts limited workers and aggregates progress, parallel thread delegates pause resume and abort, parallel thread add remove and reorder queue, parallel thread frees slot while postprocessing waits
test_parallel_settings.py	6	parallel jobs use new safe defaults when not configured, parallel defaults migration updates old saved defaults, parallel defaults migration keeps custom values, parallel jobs use cpu and gpu limits, parallel jobs are limited by file count and maximum, split files for parallel workers uses round robin order
test_path_safety.py	0	Hilfs-/Paketdatei
test_per_file_strip_only.py	5	workflow effective strip only uses per file override, standard pipeline uses strip runner for per file strip only, queue label shows processing badge from override, queue label shows pipeline and postprocess badges from cache, queue label shows dv hdr10plus and named postprocess badges
test_pipeline_and_singleton.py	0	Hilfs-/Paketdatei
test_postprocess_move_sidecars.py	4	movie nfo sidecar is renamed to movie nfo in film target, episode nfo sidecar keeps filename in series target, trickplay sidecar conflict can be backed up on move, trickplay sidecar conflict can keep existing target

Testmodul	Testfunktionen	Schwerpunkte
test_postprocess_service.py	3	prepare nfo path skip keeps existing file, prepare nfo path backup marks existing file as saved, async postprocess coordinator collects outputs
test_preflight_dialog_performance.py	6	series preview path does not scan existing folders, series metadata job is payload for background lookup, series metadata job searches selected base before fallback, existing series dir can switch to fallback base, background lookup uses fallback before tmdb, background lookup searches local folders without tmdb
test_preflight_report.py	3	preflight report includes targets rules and warnings, preflight report shows film series subpath, write preflight report creates file
test_quality_tester.py	3	parse quality segments supports percent and timestamp ranges, automatic quality segments are bounded to video duration, quality run from dict clamps quality and fills defaults
test_release_validation.py	4	release validation checks required files and schema, release validation warns about private paths, app bundle validation checks exe not source files, format release checks summarizes errors and warnings
test_review_stdlib.py	0	Hilfs-/Paketdatei
test_run_summary.py	4	run summary counts ok errors and skipped, run summary maps emoji statuses, run summary preserves failure report details, run summary formats postprocess and sidecar details
test_settings_backup.py	2	backup exports and restores settings rules and profiles, settings to dict can mask sensitive online metadata values

Testmodul	Testfunktionen	Schwerpunkte
test_source_visual_check.py	3	source visual check blocks when enough probes are suspicious, source visual check does not block single black scene, source visual settings reads qsettings values
test_subtitle_converter.py	7	srt to txt preserves timestamps and removes numbers, srt to txt writes timestamped output file, txt to srt recreates numbered blocks, txt to srt writes output file, txt to ass converts timestamped txt, txt to srt rejects txt without timestamps
test_subtitle_sidecar_service.py	0	Hilfs-/Paketdatei
test_system_shutdown.py	2	schedule system shutdown uses short windows buffer, schedule system shutdown returns false on failure
test_tool_diagnostics.py	4	probe ffmpeg reports version and features, format tool diagnostics marks missing tool, handbrake gui is not started for version probe, extended system test runs mini tool chain
test_tool_paths.py	7	find tool searches third party product dirs, extend path adds third party product dirs, find tool searches pyinstaller contents programme dirs, extend path adds pyinstaller contents programme dirs, default storage dirs are created, default target path for settings key maps codec and media type
test_tool_runner.py	2	run tool collects stdout and stderr, run tool reports timeout

Testmodul	Testfunktionen	Schwerpunkte
test_trickplay_service.py	7	trickplay uses jellyfin default folder and ffmpeg tile filter, trickplay existing folder is returned when only missing, trickplay existing root allows missing sprite variant, trickplay existing root can be backed up, trickplay can read source but write for output video, trickplay logs hardware mode
test_verbose_cleanup.py	3	verbose logger discard removes file, converter discards verbose after clean run, converter keeps verbose after error or abort
test_media_library.py	8	path mapping; Jellyfin import; series root; manual SQL; resolution/video-codec filters; SDR vs unknown; deviation minority; deviation tie

Besonders relevant für die damals aktuellen V9.7-Funktionen sind unter anderem Tests für `media_library`, `movie_renamer`, `online_metadata`, `current_ffmpeg_abort`, `audio_video_matcher`, `language_priority_rules`, `duration_repair_service`, DV-Regressionen, Parallelbetrieb, Postprocessing, Toolpfade und Releasevalidierung.

23.1 Qualitätssicherung und Abnahmestand

Prüfung	Ergebnis
Statische Python-Parseprüfung	917 Python-Dateien im vollständigen 9.8.5-Stand; Syntaxprüfung des A–M-Pakets ohne neue Syntaxfehler
compileall	Komplettes dragontools-Paket des Patchstands A–M erfolgreich geprüft; Einstiegspunkt gegenüber dem 9.8.4-Basisstand unverändert
Vollständige Batch-Abnahme	Host-kompatibler 9.8.5-Abnahmelauf: 1.548 bestanden, 18 übersprungen, 0 fehlgeschlagen; Skips nur für optionale Host-/Integrationsvoraussetzungen.
Testinventar	206 Python-Dateien im Testpaket, 203 test_*.py, 1.531 statisch erkennbare Testfunktionen

Die aktuelle Datenbankprüfung deckt Migration, Jellyfin-Laufzeiten und -Größen, lokale DragonTools-Einträge, Streamdetails, CSV-Ausgabe und Laufzeitfilter ab. Timestamp-Tests erzwingen fail-closed Verhalten und belegen, dass ein Verifikationsfehler niemals Ersetzen oder Verschieben auslöst.

23.2 Regressionsschutz

Architekturtests begrenzen die Größe zentraler Fassaden, sichern Qt-freie Fachservices und prüfen öffentliche Import- und Monkeypatch-Verträge. So bleiben bestehende Aufrufe kompatibel, ohne dass Businesslogik oder Tool-I/O wieder in die Fassaden zurückwandern.

Die neue Logik wurde mit gezielten Tests abgesichert, weil sie Daten löscht beziehungsweise ersetzt. Der Testfokus liegt auf den Fällen, die in echten Serienbeständen gefährlich sind.

- Gleiche SxxExx mit unterschiedlichem Episodentitel wird als Ersetzung erkannt.
- MKV nach MP4 und MP4 nach MKV werden als derselbe fachliche Konflikt behandelt.
- Multi-Episode-Dateien ersetzen nur exakt dieselbe Episodenmenge.
- record_moved_file setzt alte aktive Einträge auf inaktiv und hält die neue Datei aktiv.
- Die Suche `Doppelte aktive SxxExx` findet übriggebliebene Fehlerzustände.
- Reminder werden angelegt, bestätigt gelöscht oder bei `Erneut vorlegen` behalten.
- Shutdown öffnet keinen blockierenden Reminder-Dialog; die Erinnerung wird beim nächsten Start gezeigt.

Die vollständige Abnahme aller 147 Testdateien besteht mit 1.100 konkreten Testfällen. Zwei reale DV/HDR-Integrationsprüfungen werden mangels konfigurierter Werkzeuge und Referenzmedien gezielt übersprungen. PyQt6-/pytest-qt-Tests liefen unter Windows und Python 3.14.6. Vor einer endgültigen Buildfreigabe bleibt die reale DV/HDR-Roundtrip-Abnahme auf dem Zielsystem erforderlich.

23.3 Release-Prüfung und Regressionstests

- `release_validation.py` erwartet für V9 `CHANGELOG.json`; `CHANGELOG.txt` bleibt optionaler Fallback. Die Regelschemata werden gegen die zentrale Schemaversion geprüft: Audio 4, Untertitel 6, Move 1, Renamer 3, Profile 3. Bei vorhandenem dist-Build werden Help, PDF-Handbuch sowie JSON-/TXT-Changelog zusätzlich bytegenau mit den Quelldateien verglichen; ein veralteter Build wird dadurch als Fehler erkannt.
- Neue Tests prüfen Legacy-QSV-Migration, Reset-Persistenz/CRF 22, Changelog-Parsing/Pagination und dynamische Projektstatistik.
- Kontrollstand vom 12.09.2026: Datenbank, Metadaten, Trickplay, Async-Postprocessing, Quellbildprüfung, Renamer, Audio-Video-Matcher und Online-Metadaten wurden mit gezielten Regressionen, compileall und Paket-Smoke abgesichert. Die zugehörigen Zieltests sowie Paket-/Shutdown-/Matcher-Prüfungen liefen ohne Fehler.

23.4 Dokumentations- und Teststand

- Historischer Quellstand 9.8.5 (19.09.2026): 924 Python-Dateien, 140.276 Gesamtzeilen, 118.657 Codezeilen, 208 Python-Dateien im Testpaket, 205 test_*.py und 1.565 statisch erkannte Testfunktionen. Die Patches N–P wurden mit fokussierten Duration-/Workflow-, Jellyfin-/Mapping- sowie Tool-/Whisper-/Mediathek-Regressionen und compileall geprüft. Der frühere vollständige A–M-Abnahmelauf (1.548 bestanden, 18 übersprungen) bleibt eine historische Basis; ein neuer vollständiger Windows-/Qt-/Hardware-Gesamtlauf ist separat erforderlich.
- Neben Release-/Persistenz-/Move-/DV-Prüfungen sichern inzwischen eigene Architekturtests die schlanken Fassaden für Mediathek, Preflight, Postprocessing, DV-Remux, Backup, Journal, ISO, Parallel-Converter, Move, Duration-Repair, HDR10+, MediaAnalyzer, Renamer, Audio-Video-Matcher und Online-Metadaten-Dialog ab.
- Der V9-Build nimmt neben den Legacy-TXT-Dateien auch CHANGELOG.json als aktuelle Änderungshistorie mit.

23.5 Aktueller technischer Reviewstand

Aktuelle Nachprüfung vom 07.10.2026: Alle 29 Patch-/Review-Schritte wurden in Reihenfolge mit Bericht und geprüftem Projekt-ZIP abgeschlossen. Die zusätzlichen Vertragsprüfungen sichern tiefe Dateioptionen, Spurtitel/Flags, sichere Ausgabeinstallation und Recovery. Die abschließende Gesamtsuite besteht mit 5.152 Tests; 14 sind übersprungen und 24 native DV/HDR-Fälle separat abgewählt. Separat bestehen 23 native DV/HDR-Tests; zwei Skips betreffen eine optionale Generator-EXE und die beim Einsammeln fehlende optionale PyTorch-Komponente. Das ist eine Source-Abnahme, keine vollständige neue Release-EXE- oder Hardware-/Live-Online-Abnahme. Die datierten früheren Reviews bleiben historische Referenzen; die ausführlichen Grenzen stehen in PATCH_01_REPORT.md bis PATCH_29_REPORT.md.

Stand 9.9.1: Vor einem Dolby-Vision-Encode wird die vollständig gelesene Quell-RPU gegen die exakt gezählten Frames der gewählten Videospur geprüft; der AV1-DV-Pfad prüft die tatsächlich vorhandenen und gelesenen RPU-Daten. Offensichtlich unbrauchbare RPUs können ausschließlich für diese Datei einen erneuten Plan ohne Dolby Vision auslösen. Die Option steht unter Einstellungen → Quellbildprüfung → Dolby Vision – Quell-RPU und ist standardmäßig eingeschaltet. HDR10+, Audio-, Untertitel-, Qualitäts- und Dateioptionen bleiben erhalten. Bei ausgeschalteter Option, kleinen unklaren Frameabweichungen, Spurfehlern oder Abbruch wird kein DV-Encode freigegeben. Vor und nach dem Encode werden getrennte Prüfergebnisse protokolliert. MKV-Muxer übernehmen aus jeder Audio-/Untertitelquelle nur ausdrücklich ausgewählte Spuren; nicht ausgewählte gleiche Untertitel blockieren eine eindeutig ausgewählte PGS-Spur nicht. Der fertige Medienvertrag wird vor dem Ersetzen geprüft. Filmersetzung und Matching verwenden dieselbe Normalisierung für Unicode, Bindestriche, Apostrophe und Dateinamen. Jahr, Edition, vorhandene Metadata-IDs und Mehrdeutigkeit schützen verschiedene Filme. Vorbereitete Identität und Ziel bleiben verbindlich; gescheiterte Dateitausche bewahren den guten Altbestand und Recovery-Dateien. PATCH_29_REPORT.md erläutert Prüfungen und Grenzen.

Stand 9.9.1: Bei der dokumentierten Abnahme nach Patch 28 wurden alle damaligen Patch-/Review-Schritte nacheinander mit Bericht und geprüftem Projekt-ZIP abgeschlossen. Die finale Prüfung umfasst die eingeschränkte Pairwise-Vertragsmatrix, reale FFmpeg-Dateien, native DV/HDR10+-Rückleseprüfungen und längere parallele Queue-Abläufe mit Umordnen, einzelner Pause und verspäteten Rückmeldungen. AV1-DV/HDR10+-Beta-Pfade sind gesonderte Planungsnachweise; nicht jede Matrixkombination wurde nativ kodiert. Test-Skips und fehlende optionale Hardware-/Online-/Generator-EXE-Abnahmen sind in PATCH_28_REPORT.md ausdrücklich aufgeführt.

Stand 9.9.1: Die asynchrone Nachbearbeitung gilt erst als abgeschlossen, wenn auch ihre registrierten terminalen Rückmeldungen abgearbeitet sind. Gleichzeitig Wartende sehen denselben Abschluss beziehungsweise Fehler. Defaults und Dateioptionen werden vor externen Erkennungen tief aufgenommen. Metadatenanwendung, Geometrieplanung und Move-Ergebnis besitzen klare gemeinsame Grenzen; die dokumentierten Architekturgrenzen wurden nicht zur Umgehung von Prüfungen erhöht.

Technischer Review vom 12.09.2026: Die Mediathek arbeitet auf Schema 6 mit migrationssicherer Initialisierung, indexiertem Serien-Fast-Path, DB-first Preflight und asynchroner Suche. Renamer und NFO trennen persistente Suchcaches von frischen Batch-Metadaten; generische Episodentitel lösen bei Bedarf einen Refresh aus, TheTVDB kann fehlende Titel über die Fallback-Sprache ergänzen. Trickplay/Postprocessing behandelt native Qt-Signale korrekt und verhindert den Doppelstart bereits geplanter Async-Jobs. Die Quellbildprüfung bündelt mehrere Prüfpunkte pro FFmpeg-Prozess.

Stabilitätskorrekturen des Abschlussreviews und der Release-Fixes A–F: HDR10+-Returncodes und Mux-Warncodes korrigiert; Untertitel-Schema 6 zentralisiert; gemeinsamer transaktionaler Output-Commit für Converter/DV-Remux/MP4-Remux/Audio-Mux; atomare Regel-/Profilpersistenz mit Corrupt-Backup; HDR10+-ToolRunner/MuxService aus dem Coordinator ausgelagert; Audio-Donor-Analyse im HDR10+-MP4-Pfad fail-closed; globale MP4-Untertitelpolicy vereinheitlicht; Failure-Cleanup auf explizite Artefakt-Ownership begrenzt; DV-Sidecar-Run-State abgesichert; Move-Journal um Companion-Phasen und Resume erweitert; mkvpropedit-Vertrag korrigiert; lange Subtitle-FFmpeg-Hilfsprozesse an zentrale Abort-/Timeout-Logik gebunden; Rules-Preview aus PyQt6/QSettings entkoppelt.

Queue-/Logging-Nacharbeit vom 05.09.2026: Der DV-Hauptlog ist auf sieben reale Pipeline-Schritte bereinigt; technische RPU-/HDR10+-Routineausgaben liegen im VerboseLog. Bestätigte Ersetzungs-Erinnerungen protokollieren die konkret gelöschte ID. Per-Datei-Overrides können Encoder-Backend, Qualität, Preset, Skalierung und backend-spezifische Optionen manuell setzen und per Mehrfachauswahl auf mehrere wartende Dateien angewendet werden; die Worker-Priorität lautet global < Profil < manueller Datei-Override.

Sicherheitsprüfung: In produktiven Python-Dateien wurden keine eval-/exec-Aufrufe, kein shell=True, kein deaktiviertes TLS-verify=False und keine ungesicherte Archivextraktion gefunden. Externe Tool-Aufrufe werden überwiegend als Argumentlisten ohne Shell ausgeführt; TMDB-/TheTVDB-Netzwerkzugriffe verwenden Timeouts. Destruktive Dateioperationen besitzen Pfad- und Transaktionsschutz, und sensible QSettings-Werte werden im Audit-Logging maskiert.

Aktueller Architekturstand: Große Orchestratoren sind entlang fachlicher Grenzen getrennt. Datenbank-Schema/Migration/Query/Repository, NFO-Inventar/Parser/Store, Trickplay-Prozess/Commit/Pfade, DV-Remux-Prozess/Mux/Output/Job, Backup-Krypto/Restore, Journal-Storage/Resume, Preflight Film/Serie/Session, Merge Analyse/Plan/Execute, ISO View/Input/Runtime, Parallel-Converter Queue/Control/Lifecycle, Move Runtime/Commit/Batch, Encoder-Override UI/State/Apply, Duration-Repair Policy/Archive/Orchestrator, HDR10+ Services, MediaAnalyzer Video/Audio/Subtitle, Renamer Resolver/Mapping/Scoring/Order sowie Matcher-/Metadaten-GUIs liegen in getrennten Modulen. Schlanke Fassaden erhalten die bestehenden Importverträge.

Logging/Errorhandling: Die Anwendung besitzt zentrale Logger, Fehlerberichte, Diagnosepakete, Worker-Exception-Boundaries und Recovery-Journale. Gleichzeitig existieren weiterhin bewusst defensive breite Exception-Fallbacks in Parsern, GUI-Cleanup und optionalen Integrationen. Diese sind nicht pauschal fehlerhaft, sollten aber bei künftigen Änderungen weiter nach Risiko bewertet werden; operative Fehler an persistenz- oder datenverlustrelevanten Grenzen dürfen nicht still geschluckt werden.

Nachreview 19.09.2026 – Laufzeitreparatur: Echte VFR-Ausgaben können bei exakt gleicher Frameanzahl die Original-PTS als MKVToolNix-Timestamp-v2-Zeitbasis übernehmen. Originaltimeline, Audio-/Videostart und Synchronität werden vor dem Commit geprüft; unsichere Fälle bleiben fail-closed.

Nachreview 19.09.2026 – Integrationen und Mediathek: Jellyfin-Refreshes validieren den Serverpfad gegen /Library/PhysicalPaths und melden Parent-Ordner; Werkzeugpfade sind vor dem Speichern live prüfbar; faster-whisper

kann ein lokales CTranslate2-Modell verwenden. Sichere Sprachfixes synchronisieren MKV, vorhandene NFO und anschließend die SQLite-Mediathek und verwenden echte Containerstreamindizes.

23.6 Architekturgrenzen und Modulzuständigkeiten

Zentrale Orchestratoren bleiben als überschaubare Fassaden erhalten. I/O, Persistenz, Toolaufrufe, reine Fachlogik und GUI-Zustand liegen in getrennten Modulen. Bedienung, Dateiformate und öffentliche Importpfade bleiben kompatibel; intern sind Zuständigkeiten klar voneinander abgegrenzt.

- Core: Mediathek, Jellyfin-Import, Query-Aufbau, NFO-Scan, Backup/Restore, Journal, Renamer-Kandidaten, MediaAnalyzer-Streams, Online-Metadaten, Duration-Timing und weitere Fachlogik besitzen fokussierte Parser-, Mapping-, Storage-, Scoring- und Service-Module.
- Worker: Postprocessing, Trickplay, DV-Remux, Merge, ISO, Parallel-Converter, Move, Duration-Repair, HDR10+ und Quellbildprüfung trennen Prozesssteuerung, Tool-I/O, Commit/Rollback und Fachentscheidung. Die exactly-once-Grenze des Async-Postprocessings verhindert doppelte NFO-/Trickplay-Läufe.
- GUI: Mediathek-Dialog, Preflight-Serie, Encoder-Override, ISO, Audio-Video-Matcher und Online-Metadaten-Dialog sind nach View, State, Pfaden, Settings und Worker-Lifecycle gegliedert. Lange SQLite-Suchen laufen außerhalb des GUI-Threads.
- Absicherung: Architekturtests definieren Größen-/Verantwortungsgrenzen und schützen wichtige Kompatibilitäts-Re-Exports. Release-Smoke verlangt die ausgelagerten Fachmodule; gezielte Regressionen prüfen Datenintegrität, Rollback, Async-Abschluss, Qt-Signal-Logging, Metadaten-Refresh und Paketierbarkeit.

24 Erweiterte Fehlerdiagnose – symptomorientierte Referenz

Symptom	Benutzermaßnahme	Technischer Hintergrund
Encoding einer einzelnen Datei soll beendet werden, Restqueue weiter	Aktiver Queue-Eintrag → Kontextmenü → „FFmpeg für diese laufende Datei beenden“.	DragonTools prüft, ob genau diese Datei aktiv ist und ob der aktuelle Prozess wirklich ffmpeg ist. Dann wird nur dessen Prozessbaum beendet; kein globales Abort-Flag.
TheTVDB liefert unerwünschte/anderssprachige Treffer	Sprache/Fallback und Providerpriorität prüfen.	Bei „Beide“ zuerst den gewünschten Provider als „Bevorzugt“ setzen. TheTVDB-Titel werden aus mehreren v4-Lokalisierungsformen gelesen; Fallbacksprache kann greifen.
TMDB/TheTVDB-Zugangsdaten nicht sichtbar	Gemeinsame Checkbox „Zugangsdaten anzeigen (TMDB + TheTVDB)“ aktivieren.	Sie schaltet alle Provider-Geheimfelder gemeinsam von Password auf Normal.
NVENC `lookahead_level` ungültig	Nur Auto oder 0–3 verwenden.	Der Command Builder akzeptiert ausschließlich 0,1,2,3 und setzt bei Auto gar keinen Parameter.
NVENC Multipass wird nicht erwartet	Auto wählen, wenn FFmpeg/NVENC-Default gelten soll.	qres/fullres/disabled werden explizit als `-multipass` gesetzt; Auto unterdrückt die Option.

Symptom	Benutzermaßnahme	Technischer Hintergrund
Output-Dauer unplausibel	Logs/ErrorReport auf Verifier und DurationRepair prüfen.	Zuerst normaler Remux, danach nur bei passendem Muster Timestamp-Reparatur; anschließend erneute Validierung.
ISO-Ordner statt Image	Disc-Ordner hinzufügen; BDMV/VIDEO_TS oder Root akzeptiert.	ISOThread analysiert Struktur; MakeMKV-Pfad und optionaler FFmpeg-Fallback entscheiden die Extraktion.
Postprocessing-NFO/Trickplay fehlt	Finalen ✅-Status und Postprocess-Log prüfen.	🧩 bedeutet: Kernkonvertierung ersetzt, Nacharbeit läuft noch. Batchabschluss wartet auf offene Nacharbeit.
Tool wird nicht gefunden	F9/Systemtest, Tool-Pfadsettings, PATH und gebündelte Ordner prüfen.	Zentrale ToolPaths-Logik soll verhindern, dass GUI und Worker unterschiedliche Binaries verwenden.
Testlauf startet in dieser Dokumentationsumgebung nicht	PyQt6 fehlt.	Projektquellcode kompiliert statisch; Runtime-/GUI-Tests benötigen die Projektabhängigkeiten.

25 Historische Modulübersicht vom 06 September 2026

Die folgende Einzeldateiliste ist eine historische Referenzaufnahme vom 06.09.2026. Ihre Zeilenzahlen und damaligen Datei-Grenzen werden bewusst nicht als aktuelle Live-Inventur ausgegeben. Der aktuelle Projektumfang und die nachfolgenden Fassaden/Fachmodule sind in Abschnitt 23.6 dokumentiert; für exakte Dateinamen und Inhalte ist der aktuelle Quellbaum maßgeblich.

25.1 Einstiegspunkt - 1 Datei(en)

DragonToolsV9.py - 502 Zeilen - DragonToolsV9.py – Einstiegspunkt Dragon Tools V9 | Klassen: DragonSplashScreen | Funktionen: _close_pyinstaller_boot_splash, _find_icon, _find_splash_image, _run_frozen_smoke_test, main

25.2 Core - 78 Datei(en)

dragontools/core/__init__.py - 1 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/core/audio_titles.py - 43 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _channel_label, build_audio_title

dragontools/core/audio_video_matcher.py - 66 Zeilen - Compatibility facade for the audio/video matching subsystem.

dragontools/core/audit_log.py - 122 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: settings_change_log_dir, append_audit_event, snapshot_qsettings, summarize_changes, log_qsettings_changes, _display_value, _log_base_from_settings

dragontools/core/batch_preflight.py - 34 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/core/changelog.py - 228 Zeilen - Strukturierte Änderungshistorie und deterministische Seiteneinteilung. | Klassen: ChangelogSection, ChangelogDocument | Funktionen: _blocks_from_lines, parse_legacy_changelog, _section_from_json, load_changelog, estimate_block_lines, _split_oversized_block, paginate_blocks

dragontools/core/codec_profile_assistant.py - 368 Zeilen - Empfohlene Startprofile für den Codec-Profil-Assistenten. | Klassen: CodecProfileSuggestion | Funktionen: _cpu_h265, _nvenc, _cpu_simple, _h265_start_profile_set, assistant_profiles_for_codec, profile_for_assistant_key

dragontools/core/codec_utils.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _enum_or_value, normalize_target_codec, encoder_10bit_filter_pixel_format, value_or_default

dragontools/core/config_migration.py - 391 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: UnsupportedConfigSchemaError, MigrationResult | Funktionen: schema_version, current_schema_version, ensure_config_write_compatible, sanitize_config_for_persistence, write_json_atomic, log_config_migration, finish_migration, _safe_int, _clamp_int, _codec_from_profile_name, _default_encoder_options, migrate_encoder_profile, migrate_profile_collection

dragontools/core/crash_guard.py - 372 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _fallback_diagnostic, install_crash_guard, mark_activity, clear_activity, write_manual_crash_note, _handle_unhandled_exception, _handle_thread_exception, _report_unclean_previous_run, _report_unclean_previous_runs, _report_unclean_state, _build_report_text, _ensure_report_dir, _cleanup_empty_fatal_log, _remove_empty_fatal_log_from_state, _remove_empty_fatal_log_path, _read_state, _normalize_command, _safe_int, _pid_is_running

dragontools/core/diagnostic_package.py - 280 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: default_diagnostic_package_path, create_diagnostic_package, _diagnostic_archive, _has_unowned_link, _diagnose_info, _diagnostic_tool_paths, _tool_diagnostics_text, _extended_systemtest_text, _selected_log_files, _latest_files, _latest_job_journals, _latest_from_list, _write_existing_file, _safe_arcname

dragontools/core/disk_space.py - 156 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DiskSpaceIssue, DiskSpaceCheckResult | Funktionen: check_conversion_disk_space, format_disk_space_issues, _existing_probe_path, _volume_key, _format_bytes

dragontools/core/encoder_profile_override.py - 283 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _merge_encoder_options, _codec_text, _same_codec, _scale_mode, scoped_encoder_options, file_override_may_enable_encoder_option, runtime_encoder_capabilities, profile_to_override, normalize_profile_override, normalize_encoder_override, _apply_profile_settings, _apply_manual_settings, effective_encoder_settings

dragontools/core/error_report.py - 357 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: write_conversion_error_report, build_error_report_preview, _section, _workflow_lines, _pipeline_diagnostic_lines, _source_lines, _path_lines, _encoder_lines, _override_lines, _verify_lines, _text_lines, _stream_summary, _hdr_summary, _encoder_option, _json, _plain, _format_size, _format_duration_s, _format_duration_ms, _yes_no, _value, _safe_filename, current_traceback_text

dragontools/core/formatting.py - 22 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: format_binary_size

dragontools/core/german_title_variants.py - 58 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: german_umlaut_search_variants, fold_german_umlauts, _preserve_case

dragontools/core/gpu_detection.py - 175 Zeilen - dragontools/core/gpu_detection.py | Klassen: GpuInfo | Funktionen: _is_igpu, detect_gpus, _detect_windows, _detect_linux, best_encoder, gpu_summary

dragontools/core/jellyfin_nfo.py - 421 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: write_movie_nfo, write_episode_nfo, build_fileinfo, build_planned_fileinfo, _append_video_stream, _append_audio_stream,

_append_subtitle_stream, _append_fileinfo, _planned_fileinfo_element, _add_text, _add_many, _add_actors, _add_uniqueid, _format_float, _duration_seconds, _frame_rate, _write_xml, _fsync_parent_best_effort, _indent

dragontools/core/job_journal.py - 276 Zeilen - Öffentliche Job-Journal-Fassade und zustandsbehafteter Writer. | Klassen: JobJournalWriteError, JobJournal

dragontools/core/json_io.py - 84 Zeilen - Robuste atomare JSON-Schreiboperationen für persistente Zustandsdateien. | Funktionen: _replace_with_retry, quarantine_corrupt_file, atomic_write_json

dragontools/core/lang_codes.py - 327 Zeilen - dragontools/core/lang_codes.py | Funktionen: lang_display, lang_iso_tag, _clean_language_token, canonical_lang, mkv_language_tags, language_aliases, language_matches, normalize_language_priority, sub_codec_to_ext_and_args

dragontools/core/log_cleanup.py - 167 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: LogCleanupResult | Funktionen: cleanup_logs, _cleanup_folder, _file_category, _is_link_or_junction

dragontools/core/logger.py - 219 Zeilen - DragonTools logging facade and core writer implementation. | Klassen: DragonLogger | Funktionen: create_worker_logger

dragontools/core/media_analyzer.py - 138 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MediaInfoHdrInspection | Funktionen: inspect_dynamic_hdr_with_mediainfo, _analysis_payloads, analyze_media

dragontools/core/media_hdr_detection.py - 411 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _default_dv_info, _text, _join_values, _major_profile, contains_hdr10plus_marker, _extract_dv_tag, _extract_profile_from_text, parse_dolby_vision_profile, parse_dolby_vision_from_ffprobe_stream, merge_dolby_vision_info, choose_dovi_convert_mode, detect_hdr_from_mediainfo_track, detect_hdr_from_ffprobe_stream, detect_hdr10plus_from_ffprobe_frames, validate_hdr_flags

dragontools/core/media_library.py - 93 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/core/media_library_db.py - 259 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _online_backup_database, _snapshot_database, initialize_database, initialize_database_once, backup_database, get_stats, normalize_database_stream_types, cleanup_inactive_media_items, sql_is_read_only, _split_sql_script, _first_sql_keyword, _execute_mutating_script_atomically, execute_sql

dragontools/core/media_library_export.py - 125 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: export_database, export_database_to_csv, export_media_overview_to_csv, export_search_results_to_csv

dragontools/core/media_library_jellyfin.py - 194 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _load_source_snapshot, _import_rows, import_jellyfin_database

dragontools/core/media_library_paths.py - 24 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/core/media_library_repository.py - 29 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/core/media_library_scan.py - 348 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _scan_video_files, _year_from_text, _strip_year_suffix, _season_number_from_folder, _is_season_folder, _folder_item, _apply_storage_scan_context, _insert_storage_scan_hierarchy, _commit_storage_scan, scan_storage_paths_to_database

dragontools/core/media_library_search.py - 50 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: search_library

dragontools/core/media_library_types.py - 125 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: PathMapping, LibraryStats, LibraryImportResult, LibraryLightScanResult, LibraryScanResult | Funktionen: describe_series_path_resolution, default_media_library_dir, default_media_library_db_path, _now

dragontools/core/media_library_utils.py - 131 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _bool, _int_or_none, _float_or_none, _normalize_title, _infer_item_type, _safe_parent, _normalize_stream_type

dragontools/core/media_metadata.py - 265 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: normalize_color_range_for_zscale, normalize_color_range_label, is_german, _normalize_lang, normalize_video_codec, _track_value, _parse_mediainfo_bit_depth, _parse_seconds_value, _parse_mediainfo_duration_s, _parse_frame_rate_fraction, _nearest_common_frame_rate, _frame_rate_label, _frame_rate_mode_label, infer_ffprobe_frame_rate_mode, clean_metadata_text, first_metadata_text, build_pix_fmt_from_mediainfo, parse_bit_depth

dragontools/core/mediainfo_details.py - 399 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MediaInfoDisplayDetails | Funktionen: load_mediainfo_json, build_mediainfo_display_details, _fallback_details, _build_video_rows, _build_audio_row, _build_subtitle_row, _tracks, _tracks_of_type, _first_track, _first_number, _duration, _value, _join_values, _format_int, _format_size, _format_bitrate, _format_duration, _fps_label, _bit_depth_label, _channels_label, _sampling_label, _language_label, _flags_label, _stream_label

dragontools/core/models.py - 209 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: JobStatus, TargetCodec, Pipeline, SubtitleOverride, AudioStream, SubtitleStream, VideoStream, MediaInfo, ConversionJob

dragontools/core/move_conflicts.py - 246 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EpisodeIdentity | Funktionen: _episode_label, norm_path, same_path, _episode_identity_for_named_path, episode_identity_for_path, find_episode_identity_conflicts, find_episode_replacement_artifacts, find_target_conflicts, resolve_rename_path, format_conflict_names

dragontools/core/move_file_service.py - 230 Zeilen - Qt-independent facade for transactional file and directory moves. | Klassen: JournalLike, MoveFileService

dragontools/core/move_journal.py - 330 Zeilen - Persistentes Journal für nicht abgeschlossene Verschiebevorgänge. | Klassen: MoveJournal | Funktionen: recover_active_move_backups

dragontools/core/move_postprocess.py - 72 Zeilen - Postprocessing nach erfolgreichem Media-Move. | Funktionen: record_replacement_reminder, record_media_library_move

dragontools/core/move_report.py - 240 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: build_move_target_summary, format_move_target_summary_lines, _normalize_entry, _entry_count, _sidecar_type_label

dragontools/core/move_routing.py - 167 Zeilen - Zielermittlung fuer Move-Vorgaenge, ohne Qt-Abhaengigkeit. | Klassen: MoveRouter | Funktionen: _fmt, _similar

dragontools/core/move_sidecars.py - 508 Zeilen - Sidecar- und Trickplay-Verschiebung als eigener Service. | Klassen: MoveSidecarService

dragontools/core/move_transaction.py - 306 Zeilen - Transaktionale Dateisystem-Bausteine fuer Move-/Replace-Workflows. | Klassen: PathTransactionRollbackError, PathSwapTransaction | Funktionen: remove_path, unique_staging_path, publish_staged_no_replace, copy_file_to_staging, copy_path_to_staging

dragontools/core/movie_renamer.py - 325 Zeilen - Compatibility facade and orchestration for file renaming. | Funktionen: _series_candidate_from_result, build_movie_rename_proposal, build_series_rename_proposal, build_rename_proposal, _parse_series_source

dragontools/core/online_metadata.py - 85 Zeilen - Stabile Kompatibilitaets-Fassade fuer die Online-Metadaten-API.

dragontools/core/online_metadata_cache.py - 85 Zeilen - Gemeinsamer JSON-Dateicache für Online-Metadatenprovider. | Funktionen: read_metadata_cache, write_metadata_cache, reset_metadata_session

dragontools/core/online_metadata_common.py - 29 Zeilen - Kompatibilitätsfassade für gemeinsame Online-Metadaten-Bausteine.

dragontools/core/online_metadata_service.py - 342 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: CompositeMetadataClient | Funktionen: _client_provider_label, _raise_if_all_providers_failed, client_from_settings, _client_for_provider, client_from_config, client_from_settings_for, suggest_movie_metadata_for_file, suggest_series_metadata_for_name, suggest_episode_metadata_for_file, format_movie_suggestion, format_series_suggestion, _provider_label

dragontools/core/online_metadata_tmdb.py - 53 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TmdbClient

dragontools/core/online_metadata_tvdb.py - 66 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TheTvdbClient

dragontools/core/online_metadata_tvdb_helpers.py - 133 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _records_from_data, _single_record_from_data, _episodes_from_tvdb_response, _tvdb_record_id, _tvdb_remote_id, _year_from_tvdb_record, _episode_title_is_fallback, _merge_episode_language_fallback

dragontools/core/output_replace.py - 311 Zeilen - Crash-robuster Commit bereits erzeugter Ausgabedateien. | Klassen: OutputCommitResult | Funktionen: unique_backup_path, commit_staged_output, _commit_same_path, _commit_container_change, _check_commit_abort, _require_expected_receipt

dragontools/core/parallel_settings.py - 87 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: clamp_parallel_jobs, is_gpu_encoder, _contains, _value_as_int, migrate_parallel_defaults, parallel_jobs_for_encoder, split_files_for_parallel_workers

dragontools/core/path_safety.py - 129 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _resolved, _unresolved_local, _is_link_or_junction, _is_root, _has_link_ancestor, is_safe_subpath, safe_unlink, safe_rmtree

dragontools/core/paths.py - 39 Zeilen - Backward-compatible path facade. | Funktionen: resource_path, _known_tool_dirs, extend_path, find_tool

dragontools/core/preflight_report.py - 245 Zeilen - Speicherbarer Preflight-Bericht für Start- und Verschiebeprüfungen. | Funktionen: default_preflight_report_dir, default_preflight_report_path, _text, _short_path, _planned_target_text, _clean_series_name, _classification, _rows_by_path, _summary_line, _add_row_details, _add_warnings, _add_decision_reasons, build_preflight_report, write_preflight_report

dragontools/core/process_runner.py - 103 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: tool_available, subprocess_no_window_kwargs, run_analysis_tool

dragontools/core/profile_manager.py - 307 Zeilen - dragontools/core/profile_manager.py – Default + User Profile | Klassen: ProfileManager | Funktionen: _extend_defaults_with_assistant_profiles, _normalise_profile_label, _slugify_profile_label, _profile_encoder, _profile_codec, _codec_display

dragontools/core/project_info.py - 169 Zeilen - Projektstatistik und Kurzbeschreibung für den Über-Dialog. | Klassen: ProjectStatistics | Funktionen: _source_files, _count_static_tests, _fmt_int, collect_project_statistics, build_about_html

dragontools/core/quality_tester.py - 255 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: QualitySegment, QualityTestRun, QualityMetricResult, QualityFileInfo, QualityComparisonResult, QualityComparisonSummary | Funktionen: comparison_duration_limit, comparison_segment_starts, quality_comparison_assessment, _safe_float, seconds_to_label, parse_time_value, parse_quality_segments, automatic_quality_segments, quality_output_name, parse_extra_args, quality_run_from_dict

dragontools/core/release_packaging.py - 398 Zeilen - Sichere Erstellung von DragonTools-Quellrelease-ZIPs. | Funktionen: sanitize_public_text, _write_public_member, _is_dragontools_project, _public_source_path_allowed, _validate_public_source_contract, iter_public_source_files, _is_ignored_release_dir, _is_excluded_local_tree, is_forbidden_release_path, find_forbidden_release_artifacts, clean_forbidden_release_artifacts, _iter_release_files, create_source_release_zip

dragontools/core/release_validation.py - 63 Zeilen - Stable public facade for release validation. | Funktionen: validate_release, _stdout_supports_status_icons, format_release_checks

dragontools/core/replace_journal.py - 141 Zeilen - Durables Journal fuer destruktive Converter-Replace-Transaktionen. | Klassen: ReplaceJournalWriteError, ReplaceJournal | Funktionen: replace_journal_dir, list_replace_journals, recover_active_replace_journals, _remove_path, _now

dragontools/core/replacement_reminders.py - 161 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: default_replacement_reminder_path, _now, _timestamp, _id_prefix, _load, _save, list_replacement_reminders, _next_id, add_replacement_reminder, _add_locked, dismiss_replacement_reminders, has_replacement_reminders

dragontools/core/rules_preview.py - 134 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: build_rules_preview

dragontools/core/run_summary.py - 195 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _status_key, _path_name, _size_of, _fmt_size, _sidecar_kind, _sidecar_label, _postprocess_label, _postprocess_totals, _postprocess_totals_label, normalize_result_row, build_run_summary

dragontools/core/settings.py - 23 Zeilen - Backward-compatible settings facade.

dragontools/core/settings_backup.py - 34 Zeilen - Öffentliche Fassade für DragonTools-Einstellungsbackups.

dragontools/core/sidecar_journal.py - 197 Zeilen - Durables Journal fuer Sidecar-Commit-Transaktionen. | Klassen: SidecarJournalWriteError, SidecarJournal | Funktionen: sidecar_journal_dir, list_sidecar_journals, recover_active_sidecar_journals, _path_exists, _now

dragontools/core/sidecar_transaction.py - 249 Zeilen - Transaktionale Finalisierung erzeugter Sidecar-Dateien. | Klassen: SidecarCommitError, _CommittedSidecar, SidecarCommitTransaction

dragontools/core/system_shutdown.py - 82 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _log_shutdown, _shutdown_delay, schedule_system_shutdown, _shutdown_error_detail

dragontools/core/timeout_settings.py - 522 Zeilen - dragontools/core/timeout_settings.py | Klassen: TimeoutDef | Funktionen: _read_qsettings_or_none, _write_qsettings_transaction, _read_timeout_value, get_timeout_value, is_timeout_enabled, get_timeout, get_all_timeouts, get_all_timeout_enabled, save_timeout, save_timeout_enabled, save_all_timeouts, reset_all_timeouts, get_default, migrate_v91_timeout_defaults

dragontools/core/tool_diagnostics.py - 254 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _exists_or_which, _run_tool, _run_command, _first_useful_line, _probe_ffmpeg_features, _successful_probe_output, probe_tool, build_tool_diagnostics, _path_for, _mini_row, _file_ok, _duration_ok, run_extended_system_test, format_tool_diagnostics, format_extended_system_test

dragontools/core/type_utils.py - 79 Zeilen - Gemeinsame, zustandslose Typkonvertierungs-Hilfsfunktionen für DragonTools. | Funktionen: _safe_int, _safe_float, _safe_bool

dragontools/core/version.py - 13 Zeilen - Zentrale Versionsdefinition fuer Dragon Tools.

25.3 GUI - 120 Datei(en)

dragontools/gui/__init__.py - 1 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/gui/application_shutdown.py - 134 Zeilen - Koordinierter Shutdown aller von geladenen Haupt-Tabs gehaltenen Worker. | Klassen: ApplicationShutdownResult | Funktionen: _worker_name, _is_running, _request_stop, _wait, collect_shutdown_workers, shutdown_workers, shutdown_loaded_widgets

dragontools/gui/audio_muxer_widget.py - 221 Zeilen - Created on Thu Apr 16 01:49:23 2026 | Klassen: AudioMuxerWidget

dragontools/gui/audio_video_matcher_widget.py - 32 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: AudioVideoMatcherWidget

dragontools/gui/batch_preflight_dialog.py - 258 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: BatchPreflightDialog

dragontools/gui/changelog_dialog.py - 295 Zeilen - Seitenbasierte Änderungshistorie ohne überladenes Button-Raster. | Klassen: ChangelogDialog | Funktionen: _find_changelog, _changelog_title, _icon_for_section, _block_to_html, _page_html

dragontools/gui/conversion_controller.py - 167 Zeilen - Schmale GUI-Fassade für den Conversion-Workflow. | Klassen: ConversionController

dragontools/gui/conversion_diagnostics.py - 121 Zeilen - Qt-unabhängige Formatierung des laufenden Conversion-Worker-Zustands. | Klassen: ConversionDiagnosticsService

dragontools/gui/conversion_progress_presenter.py - 138 Zeilen - Progress event coordinator; selector and display rendering live in dedicated helpers. | Klassen: ConversionProgressPresenter

dragontools/gui/conversion_result_service.py - 67 Zeilen - Fassade für Datei-Ergebnisse und Abschluss eines Konvertierungslaufs. | Klassen: ConversionResultService

dragontools/gui/conversion_session_state.py - 240 Zeilen - dragontools/gui/conversion_session_state.py | Klassen: ConversionSessionState

dragontools/gui/conversion_start_coordinator.py - 293 Zeilen - Start-Workflows für Convert, Move-Only und DV-Remux. | Klassen: ConversionStartCoordinator

dragontools/gui/conversion_worker_factory.py - 181 Zeilen - Worker- und Config-Erzeugung für die Conversion-GUI. | Klassen: ConversionConfigBuilder, ConversionWorkerFactory

dragontools/gui/conversion_worker_lifecycle.py - 288 Zeilen - Lifecycle und Bedienaktionen laufender Conversion-Worker. | Klassen: ConversionWorkerLifecycle

dragontools/gui/convert_queue_window.py - 289 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: QueueWindowListWidget, ConvertQueueWindow

dragontools/gui/convert_widget.py - 299 Zeilen - Converter tab composition root. | Klassen: ConvertWidget

dragontools/gui/convert_widget_composition.py - 222 Zeilen - Composition root and signal wiring for :class:`ConvertWidget`. | Klassen: ConvertWidgetComposition

dragontools/gui/convert_widget_custom_widgets.py - 361 Zeilen - dragontools/gui/convert_widget_custom_widgets.py | Klassen: BannerLabel, DragonProgressBar, PathRow | Funktionen: _eta, _find_banner_single

dragontools/gui/convert_widget_encoder_override.py - 137 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderOverrideDialogHelper

dragontools/gui/convert_widget_file_queue.py - 321 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: FileListWidget, ConvertWidgetFileQueueHelper

dragontools/gui/convert_widget_host_actions.py - 66 Zeilen - Desktop/log/shutdown actions used by the converter widget. | Klassen: ConvertWidgetHostActions

dragontools/gui/convert_widget_layout.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetLayoutBuilder

dragontools/gui/convert_widget_override_dialog.py - 323 Zeilen - File-override dialog orchestration facade used by ConvertWidget. | Klassen: ConvertWidgetOverrideDialogHelper

dragontools/gui/convert_widget_paths.py - 67 Zeilen - Target-path settings used by :mod:`dragontools.gui.convert_widget`. | Klassen: ConvertWidgetTargetPathService

dragontools/gui/convert_widget_queue_actions.py - 35 Zeilen - Aggregation facade for ConvertWidget queue-related GUI actions. | Klassen: ConvertWidgetQueueActionsMixin

dragontools/gui/convert_widget_queue_badges.py - 128 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueBadgesMixin

dragontools/gui/convert_widget_queue_context_actions.py - 246 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueContextActionsMixin

dragontools/gui/convert_widget_queue_dragdrop.py - 79 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueDragDropMixin

dragontools/gui/convert_widget_queue_management.py - 61 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueManagementMixin

dragontools/gui/convert_widget_queue_override_actions.py - 184 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueOverrideActionsMixin

dragontools/gui/convert_widget_queue_window_actions.py - 63 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetQueueWindowActionsMixin

dragontools/gui/convert_widget_recovery.py - 188 Zeilen - Job restoration and batch-preflight preparation for ConvertWidget. | Klassen: ConvertWidgetRecoveryService

dragontools/gui/convert_widget_runtime_ui.py - 82 Zeilen - Runtime UI state for the converter widget. | Klassen: ConvertWidgetRuntimeUI

dragontools/gui/convert_widget_source_visual_actions.py - 128 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConvertWidgetSourceVisualActionsMixin

dragontools/gui/drop_path_extractor.py - 321 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _log_drop_message, _warn_non_video_file, _mime_has_file_payload, _debug_mime_data, _resolve_drop_logger, _reconstruct_local_path_from_url, _extract_dropped_local_path, _dedupe_drop_paths, _extract_paths_from_mime_data, _resolve_shell_idlist_to_paths, _extract_paths_from_shell_idlist, _warn_drop_once, _log_drop_rejection, _is_long_path_dragdrop_failure, _log_long_path_dragdrop_warning

dragontools/gui/dv_crop_dialog.py - 37 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: show_dv_crop_decision

dragontools/gui/encoder_profile_service.py - 136 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderProfileService

dragontools/gui/encoder_settings_controller.py - 41 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsController

dragontools/gui/encoder_settings_options.py - 189 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsOptionsMixin | Funktionen: encoder_quality_range

dragontools/gui/encoder_settings_panels.py - 106 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsPanelMixin

dragontools/gui/encoder_settings_persistence.py - 179 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsPersistenceMixin | Funktionen: _apply_combo_value

dragontools/gui/encoder_settings_profiles.py - 126 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsProfilesMixin

dragontools/gui/encoder_settings_state.py - 24 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsState

dragontools/gui/encoder_settings_ui.py - 347 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncoderSettingsWidgets, EncoderSettingsUI

dragontools/gui/file_drop_widgets.py - 41 Zeilen - Wiederverwendbare Qt-Widgets für DragonTools-Datei-Dropflächen. | Klassen: FileDropTable

dragontools/gui/help_dialog.py - 108 Zeilen - dragontools/gui/help_dialog.py | Klassen: HelpDialog | Funktionen: _find_help

dragontools/gui/info_button.py - 42 Zeilen - dragontools/gui/info_button.py | Klassen: InfoButton | Funktionen: labeled_row

dragontools/gui/iso_widget.py - 48 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISOWidget

dragontools/gui/job_resume_dialog.py - 86 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: JobResumeDialog

dragontools/gui/journal_resume_base.py - 54 Zeilen - Gemeinsame Aktions-/Button-Logik für Journal-Recovery-Dialoge. | Klassen: JournalResumeDialogBase

dragontools/gui/main_window.py - 111 Zeilen - DragonTools-Hauptfenster: schlanke Composition-Root für Menüs, Aktionen, Tabs und Recovery. | Klassen: MainWindow | Funktionen: bring_to_front

dragontools/gui/main_window_actions.py - 30 Zeilen - Aggregations-Fassade für die fachlich getrennten MainWindow-Aktionen. | Klassen: MainWindowActionsMixin

dragontools/gui/main_window_backup_actions.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowBackupActionsMixin

dragontools/gui/main_window_convert_actions.py - 101 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowConvertActionsMixin

dragontools/gui/main_window_help_actions.py - 168 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowHelpActionsMixin

dragontools/gui/main_window_menus.py - 266 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowMenuMixin

dragontools/gui/main_window_metadata_actions.py - 173 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowMetadataActionsMixin | Funktionen: _metadata_action_running, _metadata_result_is_current, _start_metadata_action_thread, stop_metadata_action_thread

dragontools/gui/main_window_profile_actions.py - 68 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowProfileActionsMixin

dragontools/gui/main_window_recovery.py - 282 Zeilen - Recovery-/Resume-Orchestrierung des MainWindow. | Klassen: MainWindowRecoveryMixin | Funktionen: _recover_replace_transactions, _recover_sidecar_transactions

dragontools/gui/main_window_settings_actions.py - 148 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowSettingsActionsMixin

dragontools/gui/main_window_system_actions.py - 104 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MainWindowSystemActionsMixin

dragontools/gui/main_window_tabs.py - 419 Zeilen - Tab-, Lazy-Loading- und Converter-Handoff-Logik des MainWindow. | Klassen: _TabBar, MainWindowTabsMixin | Funktionen: ensure_one_visible_tab

dragontools/gui/media_info_dialog.py - 311 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _MediaInfoLoadThread, MediaInfoDialog | Funktionen: active_media_info_workers, _make_pair_table, _make_track_table, _fill_pair_table, _fill_track_table, _item

dragontools/gui/media_info_text_builder.py - 390 Zeilen - Textaufbereitung für den Medieninfo-Dialog ohne Qt-Abhängigkeit. | Funktionen: _build_rules_preview, _yes_no, _safe, _channels_to_label, _bitrate_label, _duration_label, _int_label, _fps_label, _lang_label, _dolby_vision_label, _video_lines, _audio_lines, _subtitle_lines, _audio_preview_lines, _subtitle_preview_lines, _override_and_move_lines, _rules_preview_lines, build_media_info_text

dragontools/gui/media_library_dialog.py - 363 Zeilen - Schlanke Orchestrierungs-Fassade für die DragonTools-Mediathek-GUI. | Klassen: MediaLibraryDialog

dragontools/gui/media_library_dialog_presenter.py - 141 Zeilen - Darstellungs-/Formatierungslogik für die Mediathek-GUI. | Klassen: MediaLibraryDialogPresenter

dragontools/gui/media_library_dialog_service.py - 264 Zeilen - Qt-unabhängige Anwendungsservices für den Mediathek-Dialog. | Klassen: MediaLibraryDialogState, MediaLibraryDialogService

dragontools/gui/media_library_dialog_view.py - 107 Zeilen - Thin widget owner for the media-library dialog. | Klassen: MediaLibraryDialogView

dragontools/gui/media_library_maintenance_controller.py - 150 Zeilen - GUI-Controller für Mediathek-Wartung, Import und Export. | Klassen: MediaLibraryMaintenanceController

dragontools/gui/media_library_mapping_controller.py - 74 Zeilen - GUI-Controller für Mediathek-Pfad-Mappings. | Klassen: MediaLibraryMappingController

dragontools/gui/media_library_scan_controller.py - 110 Zeilen - Thread-Lifecycle des Speicherpfad-Scans für die Mediathek-GUI. | Klassen: MediaLibraryStorageScanWorker, MediaLibraryScanCoordinator

dragontools/gui/media_library_search_controller.py - 323 Zeilen - GUI-Controller für Mediathek-Suche, gespeicherte Abfragen und SQL-Konsole. | Klassen: MediaLibrarySearchController

dragontools/gui/merge_widget.py - 355 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MergeWidget

dragontools/gui/move_lifecycle_coordinator.py - 72 Zeilen - Schmale Fassade für regulären und inkrementellen Move-Lifecycle. | Klassen: MoveLifecycleCoordinator

dragontools/gui/move_preflight_controller.py - 131 Zeilen - Schlanker Orchestrator für Preflight und Move-GUI. | Klassen: MovePreflightController

dragontools/gui/move_preflight_workflow.py - 156 Zeilen - Preflight-Workflow: Dialogstart, Nachstart-Preflight und Bericht. | Klassen: MovePreflightWorkflow

dragontools/gui/move_request_dialogs.py - 278 Zeilen - Benutzerentscheidungs-Dialoge für MoveThread-Anfragen. | Klassen: MoveRequestDialogHandler

dragontools/gui/move_resume_dialog.py - 56 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MoveResumeDialog

dragontools/gui/movie_renamer_actions.py - 248 Zeilen - User actions and filesystem commit for the renamer GUI. | Klassen: MovieRenamerActionController

dragontools/gui/movie_renamer_resolver.py - 143 Zeilen - Metadata worker and lifecycle coordinator for the renamer GUI. | Klassen: MovieRenameResolveThread, MovieRenamerResolveCoordinator

dragontools/gui/movie_renamer_table_controller.py - 311 Zeilen - Table state and proposal presentation for the movie/series renamer. | Klassen: RenamerColumns, MovieRenamerTableController

dragontools/gui/movie_renamer_view.py - 225 Zeilen - Qt view primitives for the movie/series renamer. | Klassen: RenamerStatusLabel, RenameTable, MovieRenamerView

dragontools/gui/movie_renamer_widget.py - 143 Zeilen - Movie/series renamer widget. | Klassen: MovieRenamerWidget

dragontools/gui/mp4_remux_widget.py - 273 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MP4RemuxWidget

dragontools/gui/online_metadata_dialog.py - 28 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: OnlineMetadataDialog

dragontools/gui/preflight_dialog.py - 283 Zeilen - Preflight-Dialog als schlanker Orchestrator. | Klassen: PreFlightDialog

dragontools/gui/preflight_metadata.py - 104 Zeilen - Orchestrierung der Hintergrund-Metadatensuche für den Preflight-Dialog. | Funktionen: online_metadata_enabled, media_library_preflight_enabled, run_metadata_lookup

dragontools/gui/preflight_view.py - 127 Zeilen - Reiner View-Aufbau des Preflight-Dialogs. | Funktionen: build_preflight_view

dragontools/gui/preflight_widgets.py - 15 Zeilen - Kompatibilitätsfassade für die Preflight-Zielwidgets.

dragontools/gui/profile_manager_dialog.py - 174 Zeilen - dragontools/gui/profile_manager_dialog.py | Klassen: ProfileManagerDialog | Funktionen: open_profile_manager

dragontools/gui/quality_tester_widget.py - 172 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _QualityFileTable, QualityTesterWidget

dragontools/gui/replacement_reminder_dialog.py - 129 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ReplacementReminderDialog | Funktionen: show_pending_replacement_reminders

dragontools/gui/rules_audio_tab.py - 29 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _AudioTab

dragontools/gui/rules_dialog.py - 110 Zeilen - dragontools/gui/rules_dialog.py | Klassen: RulesDialog

dragontools/gui/rules_dialog_common.py - 13 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _hr

dragontools/gui/rules_dialog_storage.py - 61 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _rules_dir, _assert_writable, _load, _save

dragontools/gui/rules_flags_tab.py - 58 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _FlagsTab

dragontools/gui/rules_language_editor.py - 156 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _LanguagePriorityEditor

dragontools/gui/rules_regex_help.py - 93 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _show_regex_help

dragontools/gui/rules_renamer_tab.py - 254 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _RenamerTab

dragontools/gui/rules_series_tab.py - 156 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _SeriesTab

dragontools/gui/rules_subtitle_tab.py - 286 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _SubtitleTab

dragontools/gui/run_summary_dialog.py - 162 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: RunSummaryDialog

dragontools/gui/save_settings_dialog.py - 14 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: SaveSettingsDialog

dragontools/gui/settings_dialog.py - 198 Zeilen - Globaler Settings-Dialog als schlanke Orchestrierungs-Fassade. | Klassen: SettingsDialog

dragontools/gui/settings_sections/__init__.py - 17 Zeilen - Fachlich getrennte Bereiche des globalen Einstellungsdialogs.

dragontools/gui/settings_sections/base.py - 33 Zeilen - Gemeinsame Infrastruktur für SettingsDialog-Teilbereiche. | Klassen: SettingsSection

dragontools/gui/settings_sections/media.py - 380 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MediaPostprocessSection

dragontools/gui/settings_sections/runtime.py - 388 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: RuntimeToolsSection

dragontools/gui/settings_sections/safety.py - 217 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: SafetyValidationSection

dragontools/gui/settings_sections/storage.py - 277 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: StorageLoggingSection

dragontools/gui/settings_sections/video.py - 383 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: VideoAnalysisSection

dragontools/gui/shortcut_dialog.py - 47 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ShortcutDialog

dragontools/gui/styles.py - 64 Zeilen - dragontools/gui/styles.py

dragontools/gui/subtitle_widget.py - 390 Zeilen - Untertitel-Widget: UI-Orchestrierung für Extraktion, Injection und Konvertierung. | Klassen: SubtitleWidget

dragontools/gui/tab_manager.py - 195 Zeilen - dragontools/gui/tab_manager.py | Klassen: TabManagerDialog | Funktionen: _open_external, _find_bundled_exe, get_visible_tabs

dragontools/gui/timeout_settings_dialog.py - 318 Zeilen - dragontools/gui/timeout_settings_dialog.py | Klassen: TimeoutSettingsDialog | Funktionen: _s_to_min, _min_to_s, _format_default

dragontools/gui/tool_path_settings.py - 103 Zeilen - Qt-Implementierung des ToolPathSettingsProvider. | Klassen: QtToolPathSettingsProvider

dragontools/gui/ui_helpers.py - 98 Zeilen - dragontools/gui/ui_helpers.py | Funktionen: schedule_window_callback, set_file_list_item_text, window_geometry_key, restore_window_geometry, save_window_geometry, install_persistent_window_geometry

dragontools/gui/video_file_input.py - 136 Zeilen - Gemeinsame Datei-/Drag&Drop-Helfer für einfache Video-Worker-Widgets. | Klassen: VideoDropListWidget | Funktionen: iter_video_files_from_dir, add_video_paths_to_list, choose_video_files, choose_directory

25.4 Rules - 8 Datei(en)

dragontools/rules/__init__.py - 1 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/rules/audio_plan.py - 343 Zeilen - dragontools/rules/audio_plan.py | Klassen: AudioTrackDecision | Funktionen: needs_stereo_downmix_filter, _fmt_filter_float, _mode_enabled_from_override, _scale_from_override, _loudnorm_i_from_override, effective_audio_processing, _loudnorm_filter, _processing_for_decision, audio_filter_chain, audio_input_args_for_plan, output_default_for_decision, _build_track_decision, _extra_stereo_decisions, compute_audio_track_plan

dragontools/rules/audio_rules.py - 44 Zeilen - Compatibility facade for DragonTools audio rules.

dragontools/rules/move_rules.py - 186 Zeilen - Move-Regeln: schlanke Fassade plus zentrale Zielpfad-Orchestrierung. | Funktionen: resolve_target_path, resolve_film_target_for_path, planned_target_dir, valid_move_bases, is_planned_move_target_valid

dragontools/rules/pipeline_selector.py - 224 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: PipelineCapabilityError, PipelineSelector | Funktionen: _parse_pipeline_override, _requested_preservation, _resolve_pipeline, resolve_pipeline_context

dragontools/rules/renamer_rules.py - 393 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _clamp_float, _clamp_int, migrate_renamer_rules, get_rules, reload_rules, character_replacements, apply_character_replacements, sanitize_renamer_text, _title_key, apply_title_exception, release_groups, strip_configured_release_groups, matching_rules, minimum_candidate_score, fallback_candidate_scores, candidate_score_stages,

candidate_discovery_floor, manual_review_below, auto_accept_from, fuzzy_fallback_enabled, retry_without_year_enabled, series_search_queries

dragontools/rules/rule_loader.py - 262 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _report_visible_warning, _quarantine_user_rules, _rules_dir, _default_rules_path, _default_copy, _load_json_rules_result, load_json_rules, _apply_migrator, load_named_rules, load_subtitle_rules

dragontools/rules/subtitle_rules.py - 108 Zeilen - Compatibility facade and orchestration for subtitle policy decisions. | Funktionen: compute_subtitle_plan, choose_burn_subtitle, choose_keep_subtitles

25.5 Subtitle - 6 Datei(en)

dragontools/subtitle/__init__.py - 1 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/subtitle/converter.py - 265 Zeilen - dragontools/subtitle/converter.py | Funktionen: _read_text, _srt_matches, _validate_time_span, _normalize_srt_timestamp, _srt_ts_to_ass, _ass_ts_to_srt, build_ass_style, _ass_header, _srt_text_to_ass, _parse_timestamped_txt, srt_to_txt, srt_to_ass, txt_to_srt, txt_to_ass, ass_to_txt, subtitle_to_txt

dragontools/subtitle/extractor.py - 83 Zeilen - dragontools/subtitle/extractor.py | Funktionen: _staging_output, extract_with_ffmpeg

dragontools/subtitle/injector.py - 352 Zeilen - Subtitle injection: plan against inventories, mux, verify, publish. | Funktionen: _staging_output, _commit_staging, _ffprobe_for_ffmpeg, _probe_subtitle_count, _allowed_output, _mux_and_publish, _identify, inject_with_mkvmerge, _source_dispositions, inject_with_ffmpeg

dragontools/subtitle/tagger.py - 86 Zeilen - Sprach-Tags und Flags in MKV-Containern setzen (via mkvpropedit). | Funktionen: _run_mkvpropedit, set_track_language, set_forced_flag

dragontools/subtitle/tool_logging.py - 32 Zeilen - Gemeinsame Fehlerausgabe für externe Subtitle-Tools. | Funktionen: log_tool_error

25.6 Worker - 111 Datei(en)

dragontools/worker/__init__.py - 1 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/worker/archive_service.py - 158 Zeilen - dragontools/worker/archive_service.py | Klassen: ArchiveService

dragontools/worker/audio_mux_thread.py - 116 Zeilen - Qt lifecycle facade for AudioMux jobs. | Klassen: AudioMuxThread

dragontools/worker/audio_video_match_thread.py - 111 Zeilen - Thin Qt adapter for the audio/video match service. | Klassen: AudioVideoMatchThread

dragontools/worker/av1_metadata_pipeline.py - 393 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _AV1MetadataPipelineBase, AV1DolbyVisionPipeline, AV1HDR10PlusPipeline

dragontools/worker/base_worker.py - 151 Zeilen - dragontools/worker/base_worker.py | Klassen: BaseWorker

dragontools/worker/cleanup_service.py - 152 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: CleanupService | Funktionen: cleanup_empty_overwrite_dirs

dragontools/worker/command_formatting.py - 20 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: is_ffmpeg_command, command_to_log_string

dragontools/worker/converter_config.py - 79 Zeilen - ConverterConfig – Datenhaltungsobjekt für ConverterThread. | Klassen: ConverterConfig

dragontools/worker/converter_control.py - 171 Zeilen - Pause-/Abort-/Prozess- und Diagnosekontrolle für ConverterThread. | Klassen: ConverterControlService

dragontools/worker/converter_detection.py - 273 Zeilen - dragontools/worker/converter_detection.py | Klassen: ConverterDetectionHelper | Funktionen: _tools, _safe_int, _sanitize_imax_interval, _sanitize_autocrop_mode, _sanitize_autocrop_start, _sanitize_autocrop_duration, _sanitize_autocrop_interval, _imax_probe_offsets, _autocrop_probe_offsets, _best_crop_from_output

dragontools/worker/converter_file_executor.py - 113 Zeilen - Per-Datei-Fehlergrenze und Quellbild-Preflight für ConverterThread. | Klassen: ConverterFileExecutor

dragontools/worker/converter_lifecycle.py - 138 Zeilen - Fehlergrenze und Abschlussarbeiten für ConverterThread.run(). | Klassen: ConverterLifecycleService

dragontools/worker/converter_progress.py - 61 Zeilen - Facade for converter probing, progress parsing and subprocess execution. | Klassen: ConverterProgressHelper

dragontools/worker/converter_queue_state.py - 135 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ConverterQueueState

dragontools/worker/converter_run_loop.py - 189 Zeilen - Queue- und Session-Orchestrierung für den Converter-Worker. | Klassen: ConverterRunLoop | Funktionen: quick_video_resolution

dragontools/worker/converter_runtime_builder.py - 274 Zeilen - Runtime-Service-Aufbau für :class:`ConverterThread`. | Klassen: ConverterRuntimeBuilder

dragontools/worker/converter_static_composition.py - 87 Zeilen - Startzeit-unabhängige Composition Root für ``ConverterThread``. | Funktionen: build_static_converter_services

dragontools/worker/converter_stream_args.py - 121 Zeilen - Compatibility facade for ffmpeg stream argument construction. | Klassen: BurnSubtitlePreparationError, ConverterStreamArgsHelper | Funktionen: _subtitle_rules, _tools, _set_burn_sub_tmp, _esc

dragontools/worker/converter_strip.py - 71 Zeilen - Strip-only converter facade: video copy plus rule-driven audio/subtitle handling. | Klassen: ConverterStripHelper

dragontools/worker/converter_thread.py - 314 Zeilen - dragontools/worker/converter_thread.py | Klassen: ConverterThread

dragontools/worker/converter_thread_state.py - 131 Zeilen - Gebündelter Runtime-/Session-State für :class:`ConverterThread`. | Klassen: ConverterJobState, ConverterControlState, ConverterSessionState, ConverterServiceRegistry

dragontools/worker/converter_utils.py - 106 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _fs, _fd, _parse_crop, _physical_crop_level5_json, _level5_json

dragontools/worker/duration_remux_service.py - 138 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DurationRemuxService | Funktionen: _fmt_duration

dragontools/worker/duration_repair_commands.py - 119 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: build_timestamp_repair_command, build_genpts_repair_command, setts_filter_for_fps, command_arg_after

dragontools/worker/duration_repair_models.py - 182 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MediaTimingInfo, TimestampProblem, TimestampRepairResult, DurationRepairOutcome | Funktionen: calculate_expected_duration, duration_close, is_extreme_mismatch, detect_timestamp_problem

dragontools/worker/duration_repair_runtime.py - 67 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DurationRepairRuntime

dragontools/worker/duration_repair_service.py - 211 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DurationRepairService

dragontools/worker/duration_repair_validation.py - 216 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _repair_duration_close, _sane_reference, source_video_reference_s, source_container_reference_s, _verify_message_is_stream_presence, validate_timestamp_repair

dragontools/worker/duration_timestamp_service.py - 386 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TimestampRepairService | Funktionen: _load_source_timing_reference, _timing_summary, _ffmpeg_supports_setts, _try_original_timeline_fallback, _fmt_duration

dragontools/worker/duration_timing_analyzer.py - 86 Zeilen - Public façade for media timing analysis. | Klassen: MediaTimingAnalyzer

dragontools/worker/dv_audio_mux_service.py - 260 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVExtractedAudioTrack, DVMuxAudioTrack, DVAudioMuxService

dragontools/worker/dv_command_runner.py - 187 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVCommandRunner

dragontools/worker/dv_crop_reconcile.py - 322 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVCropComparison, DVCropOutcome | Funktionen: crop_from_level5_offsets, _collect_offset_dicts, read_level5_offsets, parse_level5_export, compare_crops, automatic_choice, _normalize_candidate, reconcile_dv_crop, _split_filter_chain, replace_crop_in_vf_args

dragontools/worker/dv_dynamic_metadata_service.py - 415 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVDynamicMetadataService

dragontools/worker/dv_encode_command.py - 108 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVEncodeCommand | Funktionen: dv_video_encode_args, build_dv_encode_command

dragontools/worker/dv_failure_recovery.py - 392 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVFailureRecovery | Funktionen: preserve_completed_dv_work, _nonempty_file, _reserve_dynamic_archive, _copy_diagnostic_artifact, _select_dynamic_video_candidate, _valid_json_object, _copy_rpu_diagnostics, _write_dynamic_archive_status, _persistent_diagnostic_paths

dragontools/worker/dv_final_mux_service.py - 43 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVFinalMuxService

dragontools/worker/dv_level5_editor.py - 189 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVLevel5Editor

dragontools/worker/dv_mkv_muxer.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVMKVMuxer

dragontools/worker/dv_mp4box_muxer.py - 71 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVMP4BoxMuxer

dragontools/worker/dv_pipeline_context.py - 209 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVRunRequest, DVWorkFiles, DVPipelineState, DVPipelineResult | Funktionen: normalize_dv_profile_major

dragontools/worker/dv_pipeline_stages.py - 435 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVPipelineStages | Funktionen: _video_service, _metadata_service, _mux_service

dragontools/worker/dv_pipeline_timeouts.py - 40 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: timeout_hevc_extract, timeout_dovi_convert, timeout_rpu_extract, timeout_dovi_editor, timeout_rpu_inject, timeout_mp4box, timeout_mkvmerge, timeout_audio, timeout_encode

dragontools/worker/dv_processing_pipeline.py - 297 Zeilen - Compatibility facade and orchestrator for Dolby-Vision processing. | Klassen: DVProcessingPipeline

dragontools/worker/dv_remux_components.py - 44 Zeilen - Compatibility facade for the refactored Dolby Vision remux components. | Klassen: DVMP4BoxPipelineRunner

dragontools/worker/dv_remux_thread.py - 394 Zeilen - dragontools/worker/dv_remux_thread.py | Klassen: DVRemuxThread

dragontools/worker/dv_rpu_service.py - 73 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVRpuService

dragontools/worker/dv_runtime_models.py - 49 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVEncoderConfig, DVTempState

dragontools/worker/dv_subtitle_mux_service.py - 305 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVSubtitleJob, DVMuxSubtitleTrack, DVSubtitleMuxService | Funktionen: dv_subtitle_storage, _prepared_mkv_codec

dragontools/worker/dv_video_filters.py - 126 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _dv_vf_prefix, _dv_vf_suffix, _dv_p5_libplacebo_filter, build_dv5_libplacebo_vf, inject_dv_colorspace, remap_vf_to_input1

dragontools/worker/dv_video_stage_service.py - 502 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVVideoStageService | Funktionen: _dovi_tool_can_read_container_directly

dragontools/worker/dv_workflow_pipeline_adapter.py - 84 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DVPipelineExecutorAdapter

dragontools/worker/encode_plan.py - 15 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncodePlan

dragontools/worker/encode_plan_service.py - 94 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: EncodePlanService

dragontools/worker/encoder_args.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _int_option, _bounded_int_option, _text_option, _h265_10bit_args, _av1_10bit_args, _cpu_args, _nvenc_args, _qsv_args, _amf_args, _vid_args, _scale

dragontools/worker/hdr10_color.py - 102 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: hdr10_setparams_filter, dv_p5_libplacebo_filter, _norm, _is_pq, _is_bt2020, _profile_major, source_has_hdr10_base, should_apply_standard_hdr10_color, hdr10_output_args

dragontools/worker/hdr10plus_bitstream_service.py - 154 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDR10PlusBitstreamService

dragontools/worker/hdrplus_conversion.py - 320 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusConversionHelper

dragontools/worker/hdrplus_encode_service.py - 106 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusEncodeService

dragontools/worker/hdrplus_mux_service.py - 134 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusMuxService

dragontools/worker/hdrplus_pipeline_coordinator.py - 527 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusPipelineHooks, HDRPlusPipelinePaths, HDRPlusPipelineCoordinator

dragontools/worker/hdrplus_runtime_models.py - 103 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusEncoderConfig, HDRPlusExecutionContext, HDRPlusPipelineOutcome

dragontools/worker/hdrplus_stream_service.py - 107 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusStreamService

dragontools/worker/hdrplus_tool_runner.py - 161 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: HDRPlusToolRunner

dragontools/worker/interfaces.py - 117 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkerProtocol, PausableWorkerProtocol, AnalysisService, PipelineService, EncodeService, VerifyService, ReplaceService, CleanupService, ResultService, OutputPathService

dragontools/worker/iso_disc_inspector.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISODiscInspector | Funktionen: safe_name, parse_duration_to_seconds, parse_size_to_bytes, format_size, quote_concat_path

dragontools/worker/iso_ffmpeg_fallback_service.py - 207 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISOFFmpegFallbackService

dragontools/worker/iso_makemkv_service.py - 211 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISOMakeMKVService | Funktionen: tool_exists

dragontools/worker/iso_models.py - 24 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISOUserAbortError, ISOScanResult, ISOExtractionResult

dragontools/worker/iso_selection.py - 123 Zeilen - Qt-unabhängige Auswahlregeln für ISO-/MakeMKV-Titel. | Funktionen: _title_id, _duration, _size, choose_main_title, detect_series_episode_titles, choose_auto_titles

dragontools/worker/iso_thread.py - 239 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ISOThread

dragontools/worker/media_analysis_service.py - 42 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MediaAnalysisService

dragontools/worker/media_contract.py - 78 Zeilen - Öffentliche Fassade für den erwarteten finalen Medienvertrag. | Funktionen: _audio_codec_family, _subtitle_codec_family, build_expected_media_contract

dragontools/worker/merge_thread.py - 128 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: MergeThread

dragontools/worker/move_completion_service.py - 168 Zeilen - Abschlussphase eines bereits erfolgreich installierten Video-Moves. | Klassen: MoveCompletionOutcome, MoveCompletionService

dragontools/worker/move_thread.py - 158 Zeilen - QThread-Fassade fuer DragonTools-Verschiebevorgaenge. | Klassen: MoveThread

dragontools/worker/mp4_remux_file_service.py - 297 Zeilen - Ausführung und Commit einer einzelnen normalen MP4-Remux-Datei. | Klassen: MP4RemuxFileService

dragontools/worker/mp4_remux_plan.py - 326 Zeilen - Qt-unabhängige Planung für den normalen MP4-Remux. | Klassen: MP4RemuxPlan, MP4RemuxPlanner | Funktionen: _subtitle_disposition, resolve_mp4_output_path

dragontools/worker/mp4_remux_sidecars.py - 105 Zeilen - Sidecar-Export und transaktionaler Sidecar-Commit für MP4-Remux. | Klassen: MP4RemuxSidecarService

dragontools/worker/mp4_remux_thread.py - 217 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _UserAbortError, MP4RemuxThread

dragontools/worker/output_path_service.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: OutputPathService | Funktionen: release_output_path_reservation, _reserve_unique_output_path, _reservation_key

dragontools/worker/output_size_policy.py - 143 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _log_warn, _unique_path_in_dir, _preserve_in_archiv, validate_output_size_policy, _try_archive

dragontools/worker/output_verifier.py - 267 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: OutputVerifier

dragontools/worker/parallel_child_result_coordinator.py - 163 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ParallelChildResultCoordinator

dragontools/worker/parallel_converter_compat.py - 105 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ParallelConverterCompatibilityMixin

dragontools/worker/parallel_converter_state.py - 234 Zeilen - Qt-independent state models for parallel conversion coordination. | Klassen: ParallelQueueState, ParallelResultState, ParallelWorkerRegistry

dragontools/worker/parallel_converter_thread.py - 221 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ParallelConverterThread

dragontools/worker/parallel_worker_launcher.py - 112 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ParallelWorkerLauncher

dragontools/worker/pipeline_decision_service.py - 193 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: SettingsLike, PipelineDecisionService

dragontools/worker/postprocess_service.py - 16 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/worker/process_control.py - 404 Zeilen - dragontools/worker/process_control.py | Funktionen: _log_process_control, _windows_process_suspend_resume, suspend_process, resume_process, _taskkill_tree, terminate_process_tree, wait_while_paused

dragontools/worker/quality_test_thread.py - 98 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: QualityTestThread

dragontools/worker/replace_service.py - 173 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ReplaceService

dragontools/worker/source_visual_check.py - 233 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: SourceVisualCheckService | Funktionen: _luma

dragontools/worker/standard_pipeline_runner.py - 432 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: StandardPipelineRunner | Funktionen: _clear_reencoded_video_stat_tags

dragontools/worker/subtitle_sidecar_service.py - 260 Zeilen - Zentraler Export-Dienst fuer externe Untertitel-Sidecars. | Klassen: SubtitleSidecarService

dragontools/worker/tool_runner.py - 346 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ToolRunResult, ToolBytesResult | Funktionen: _cmd_for_log, _normalize_command, _finalize_text_capture, run_tool, _wait_binary_tool, run_tool_bytes, log_tool_failure

dragontools/worker/trickplay_service.py - 294 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TrickplayGenerator

dragontools/worker/worker_contracts.py - 50 Zeilen - Qt-unabhängige Verträge und Queue-Helfer für Worker. | Klassen: RemoveFileStatus | Funktionen: normalize_worker_path, file_override_for_path

dragontools/worker/worker_events.py - 48 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkerEvent | Funktionen: progress_event, result_event, log_event

dragontools/worker/worker_result_service.py - 247 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkerConversionResultService | Funktionen: _safe_emit, _format_size

dragontools/worker/worker_runtime_state.py - 14 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkerRuntimeState

dragontools/worker/workflow_engine.py - 234 Zeilen - Gemeinsame Workflow-Engine für Konvertierungsjobs. | Klassen: WorkflowVerifyResult, WorkflowContext, ConversionWorkflowRunner

dragontools/worker/workflow_factory.py - 84 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: build_workflow_services

dragontools/worker/workflow_models.py - 138 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowConfig, PipelineExecutionRequest, PipelineExecutionResult | Funktionen: _enum_value_text

dragontools/worker/workflow_output_commit.py - 240 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowOutputCommitCoordinator

dragontools/worker/workflow_pipeline_executor.py - 117 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowPipelineExecutor

dragontools/worker/workflow_planning_service.py - 197 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowPlanningService

dragontools/worker/workflow_services.py - 299 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowServices

dragontools/worker/workflow_verification_service.py - 73 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: WorkflowVerificationService

25.7 Tests - 144 Datei(en)

dragontools/tests/__init__.py - 0 Zeilen - Quellmodul ohne Modul-Docstring

dragontools/tests/ci_requirements.py - 87 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: ExternalMediaEnvironment | Funktionen: env_flag, missing_qt_dependencies, _resolve_tool, external_media_environment

dragontools/tests/conftest.py - 53 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: pytest_addoption, pytest_sessionstart, pytest_collection_modifyitems

dragontools/tests/test_audio_processing.py - 101 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _stream, _rules, test_drc_for_ac3_source_forces_transcode_and_input_arg, test_drc_for_aac_source_is_documented_but_does_not_force_transcode, test_loudnorm_forces_transcode_and_adds_filter, test_file_override_can_disable_global_audio_processing

dragontools/tests/test_audio_video_matcher.py - 160 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _info, _points, test_parse_cut_regions_accepts_comma_decimal_and_timecodes, test_opencv_signature_backend_when_available, test_extra_before_and_after_source_is_case_a_not_c, test_linear_drift_is_case_b, test_offset_jump_inside_common_content_is_case_c, test_target_extra_end_blocks_linear_audio_plan, test_case_c_plan_keeps_segments_and_removes_source_extra

dragontools/tests/test_audit_log.py - 45 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_audit_log_writes_event_to_configured_log_root, test_audit_summary_masks_sensitive_values, test_audit_summary_masks_jellyfin_api_key_too

dragontools/tests/test_auxiliary_workers.py - 563 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _qt_core_stub_modules, _reload_module, _fake_tools, _fake_logger, _iso_thread, _mp4_thread, _dv_remux_thread, test_iso_makemkv_source_uses_iso_and_file_prefixes, test_iso_makemkv_source_uses_disc_root_for_direct_bdmv_folder, test_iso_detects_direct_bdmv_and_video_ts_folders, test_iso_extracts_multiple_titles_as_single_makemkv_calls, test_iso_scan_reports_outdated_makemkv, test_iso_ffmpeg_fallback_detects_largest_bluray_stream, test_iso_ffmpeg_fallback_detects_largest_dvd_vob_group, test_iso_process_uses_ffmpeg_fallback_when_makemkv_scan_has_no_titles, test_mp4_remux_rejects_dolby_vision, test_mp4_remux_rejects_hdr10plus, test_mp4_remux_allows_plain_hevc, test_subtitle_injector_uses_configured_mkvmerge_path, test_subtitle_injector_mp4_text_mode_maps_only_video_audio_and_new_subtitle, test_subtitle_extract_with_ffmpeg_accepts_codec_args, test_subtitle_to_txt_supports_ass, test_mp4_overwrite_exportiert_sidecars_vor_original_replace, test_mp4_overwrite_blockiert_replace_wenn_sidecar_export_fehlschlaegt, test_dv_remux_exportiert_sidecars_vor_container_replace

dragontools/tests/test_av1_metadata_pipeline.py - 139 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Tools, _Progress | Funktionen: _media, _request, test_av1_dv_builds_native_svt_profile10_command,

test_av1_dv_rejects_profile7_and_geometry_changes, test_av1_dv_rejects_burn_in, test_av1_dv_hardware_is_fail_closed, test_av1_hdrplus_builds_libaom_t35_command, test_av1_hdrplus_hardware_is_fail_closed, test_media_contract_requires_av1_dynamic_metadata_and_10bit

dragontools/tests/test_batch_preflight.py - 417 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _base_preview, test_batch_preflight_marks_clean_file_as_ok, test_batch_preflight_warns_for_ignored_dv_and_old_container, test_batch_preflight_turns_analysis_exception_into_error_row, test_batch_preflight_errors_when_no_video_stream_is_detected, test_batch_preflight_filesystem_checks_show_planned_output, test_batch_preflight_detects_overwrite_container_conflict, test_batch_preflight_detects_move_conflict_with_other_video_extension, test_batch_preflight_pipeline_summary_shows_encoder_profile, test_batch_preflight_pipeline_summary_shows_strip_only_override, test_batch_preflight_warns_for_strip_only_dv_mp4_and_burn_in, test_batch_preflight_explains_rule_decisions, test_batch_preflight_shows_subtitle_sidecar_results, test_rules_preview_derives_sidecar_candidates_from_selected_subtitles, test_batch_preflight_report_includes_decisions_and_warnings, test_batch_preflight_pipeline_summary_shows_manual_encoder_override

dragontools/tests/test_block2_crash_durability.py - 177 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _run_child, test_move_hard_crash_after_backup_rename_is_recovered, test_replace_hard_crash_after_original_backup_is_recovered, test_replace_container_change_cleanup_pending_is_retried_on_recovery, test_move_cleanup_pending_is_recovered_and_status_becomes_ok, test_replace_journal_failure_aborts_before_original_is_renamed

dragontools/tests/test_block3_process_hardening.py - 212 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _direct_popen_calls, test_block3_aux_workers_have_no_direct_popen_calls, test_tool_runner_delegates_shared_process_lifecycle, test_worker_media_and_avmatch_timeouts_have_safe_defaults, test_run_tool_inactivity_timeout_is_reset_by_output, test_run_tool_line_callback_receives_live_output, test_run_tool_bytes_preserves_binary_stdout, test_run_tool_timeout_kills_spawned_child_on_posix, test_run_tool_merge_stderr_uses_single_drain_without_crashing, test_run_tool_marks_immediate_worker_abort_consistently

dragontools/tests/test_block6_coordinator_architecture.py - 79 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _source, _class_node, test_convert_file_queue_has_no_owner_private_backreference, test_convert_queue_window_uses_explicit_callbacks_not_owner_private_api, test_parallel_converter_delegates_qt_independent_state, test_move_thread_drops_legacy_service_forwarders

dragontools/tests/test_block7_architecture_cleanup.py - 102 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _tree, test_non_gui_layers_do_not_import_gui_modules, test_gpu_detection_lives_in_core_only, test_private_methods_are_not_called_across_object_boundaries, test_removed_compatibility_shims_stay_removed, test_iso_title_selection_is_qt_independent, test_subtitle_sidecar_service_has_only_structured_export_api

dragontools/tests/test_block_f_architecture.py - 14 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_move_preflight_uses_existing_shared_ui_helper

dragontools/tests/test_changelog.py - 90 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_parse_legacy_changelog_keeps_overview_and_sections, test_json_changelog_is_preferred_structured_source, test_paginate_blocks_splits_long_sections_without_losing_order, test_json_changelog_rejects_non_object_sections, test_repository_changelog_exposes_latest_patch_entries_through_real_loader

dragontools/tests/test_cleanup_rollback.py - 177 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_cleanup_removes_failed_output_and_sidecars,
test_cleanup_overwrite_removes_file_but_keeps_temp_dir_until_batch_end,
test_cleanup_empty_overwrite_dirs_keeps_nonempty_parallel_dir_silent,
test_replace_same_path_rolls_original_back_on_failure, test_replace_size_policy_block_marks_file_as_not_replaced

dragontools/tests/test_codec_profile_assistant.py - 58 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_h265_assistant_profiles_cover_cpu_and_nvenc, test_av1_assistant_profiles_use_numeric_svt_presets,
test_profile_for_assistant_key_returns_copy, test_anime_series_cpu_profile_uses_animation_and_more_lookahead

dragontools/tests/test_config_migration.py - 369 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_profile_manager_migrates_legacy_profile_file, test_profile_migration_removes_legacy_qsv_lookahead_flag,
test_profile_migration_clamps_qsv_lookahead_depth,
test_audio_rules_migration_adds_schema_and_stereo_copy_range,
test_move_rules_migration_adds_missing_defaults, test_profile_migration_uses_crf22_only_for_h265_cpu_fallback,
test_config_migration_atomic_writer_delegates_to_shared_crash_safe_writer,
test_profile_migration_normalizes_invalid_backend_and_legacy_types,
test_rule_migrations_normalize_string_booleans,
test_current_subtitle_rules_do_not_log_runtime_marker_as_migration,
test_subtitle_migration_renames_schema5_ass_srt_option,
test_rule_loader_does_not_rewrite_current_subtitle_rules_for_runtime_marker,
test_profile_migration_recovers_from_non_mapping_encoder_options,
test_profile_collection_migration_preserves_malformed_user_entry

dragontools/tests/test_conversion_controller_architecture.py - 77 Zeilen - Quellmodul ohne Modul-Docstring |
Funktionen: _parse, test_conversion_controller_is_only_orchestration_facade,
test_conversion_services_have_bounded_single_responsibilities,
test_conversion_controller_contains_no_direct_process_or_file_operations

dragontools/tests/test_conversion_controller_start.py - 603 Zeilen - Quellmodul ohne Modul-Docstring | Klassen:
_Button, _Bar, _Label, _Combo, _Worker, _ParallelWorker | Funktionen:
test_start_worker_ui_state_uses_initial_progress_and_starts_worker_once,
test_parallel_file_progress_defaults_to_combined_display_and_can_focus_file,
test_100_percent_progress_does_not_overwrite_postprocess_star_with_hourglass,
test_single_initial_file_uses_coordinator_for_live_worker_changes,
test_start_convert_delegates_preflight_factory_lifecycle_and_progress,
test_start_worker_ui_state_does_not_start_worker_when_job_journal_fails,
test_start_convert_blocks_reentrant_double_start_before_worker_reports_running,
test_failed_worker_launch_releases_start_reservation, test_reset_for_run_preserves_active_start_reservation

dragontools/tests/test_conversion_result_service.py - 310 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Button,
_Label, _Bar, _UI | Funktionen: _service, test_sofort_abbruch_fragt_und_startet_move_fuer_erfolgreiche_dateien,
test_abbruch_nach_datei_fragt_und_startet_move_fuer_erfolgreiche_dateien,
test_abbruch_ohne_move_wunsch_finalisiert_ohne_shutdown,
test_abbruch_ohne_fertige_dateien_bleibt_altes_cleanup_verhalten,
test_warning_result_is_terminal_but_not_successful, test_postprocess_pending_result_is_not_terminal,
test_postprocess_pending_uses_star_like_display_marker, test_finished_waits_for_pending_postprocess_before_move,
test_success_result_without_live_thread_still_records_output,
test_finalize_run_skips_summary_dialog_when_shutdown_is_enabled,
test_finalize_run_shows_summary_dialog_when_shutdown_is_disabled,
test_finalize_run_shows_replacement_reminders_before_summary_when_shutdown_disabled,

test_finalize_run_does_not_show_replacement_reminders_when_shutdown_is_enabled, test_replacement_reminder_confirmation_logs_exact_deleted_id

dragontools/tests/test_convert_widget_architecture.py - 71 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _module, test_convert_widget_has_no_desktop_or_preflight_business_dependencies, test_convert_widget_collaborators_stay_focused, test_convert_widget_paths_own_codec_specific_settings_constants, test_convert_widget_does_not_own_preflight_builder

dragontools/tests/test_convert_widget_queue_actions_architecture.py - 181 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _module, _class, test_queue_actions_facade_contains_no_business_methods, test_queue_action_collaborators_are_bounded_and_single_owner, test_queue_action_mixins_keep_unique_aggregation_surface, test_source_visual_and_profile_dependencies_have_dedicated_owners, test_convert_widget_remove_signal_targets_existing_queue_api

dragontools/tests/test_convert_widget_services.py - 149 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Settings, _FileList | Funktionen: test_target_path_service_uses_codec_specific_keys, test_restore_job_files_preserves_move_context, test_restore_job_files_rejects_missing_and_non_video, test_preflight_badge_refresh_failure_is_logged

dragontools/tests/test_converter_detection.py - 103 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_imax_probe_offsets_nutzen_einstellbares_intervall, test_imax_auto_erkennt_wechselnde_aktive_bildflaeche, test_imax_auto_respektiert_mindestanzahl_verwertbarer_punkte, test_imax_auto_bleibt_aus_bei_konstanter_aktiver_bildflaeche

dragontools/tests/test_converter_thread_refactor.py - 301 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _class_and_method, test_converter_thread_is_bounded_qthread_facade, test_converter_refactor_services_have_bounded_responsibilities, test_runtime_builder_uses_explicit_workflow_composition_root, test_run_loop_honors_abort_after_current_file, test_lifecycle_open_file_recovery_excludes_completed_entries, test_control_abort_after_file_can_be_cleared_without_killing_process, test_control_immediate_abort_uses_existing_process_tree_termination, test_control_pause_and_resume_keep_event_contract, test_converter_state_models_are_qt_independent_and_explicit, test_converter_job_state_copies_mutable_config_maps

dragontools/tests/test_crash_guard_cleanup.py - 48 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_clear_activity_removes_empty_fatal_runtime_log, test_unclean_report_removes_empty_previous_fatal_runtime_log

dragontools/tests/test_current_ffmpeg_abort.py - 115 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_terminate_current_ffmpeg_only_kills_matching_ffmpeg, test_terminate_current_ffmpeg_rejects_other_current_file, test_taskkill_tree_uses_bounded_timeout, test_taskkill_tree_timeout_returns_for_python_fallback, test_taskkill_tree_oserror_returns_false_for_fallback

dragontools/tests/test_diagnostic_package.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Settings | Funktionen: test_create_diagnostic_package_collects_logs_and_settings

dragontools/tests/test_disk_space.py - 57 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_disk_space_check_reports_warning_when_reserve_is_low, test_disk_space_check_reports_critical_when_temp_outputs_do_not_fit

dragontools/tests/test_duration_repair_architecture.py - 56 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_duration_repair_coordinator_does_not_reimplement_stage_tool_logic,

test_duration_repair_services_stay_qt_independent,
test_duration_repair_file_replace_is_owned_by_runtime_boundary

dragontools/tests/test_duration_repair_service.py - 1.502 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Verifier, _PathAwareVerifier | Funktionen: explicit_normal_remux_packet_boundary, _verify_result, test_duration_repair_remuxes_mkv_and_accepts_fixed_duration, test_duration_repair_archiviert_wenn_remux_dauer_falsch_bleibt, test_duration_repair_greift_nur_bei_passendem_mkv_oder_mp4_laufzeitfehler, test_duration_repair_remuxes_mp4_with_mp4box, test_mp4_timestamp_repair_command_uses_mp4box_not_mkvtoolnix_or_ffmpeg, test_duration_repair_kann_deaktiviert_werden, test_workflow_verify_akzeptiert_erfolgreiche_laufzeitreparatur, _timing_probe_json, _fake_tool, _packet_probe_result, _tool_runner_from_subprocess, test_detect_timestamp_problem_repariert_korrektes_23976_video_nicht, test_detect_timestamp_problem_repariert_korrektes_25fps_video_nicht, test_timestamp_repair_nutzt_setts_nach_erfolglosem_remux_23976, test_setts_filter_nutzt_25fps_dynamisch, test_vfr_datei_startet_keine_blinde_cfr_timestamp_reparatur, test_fehlgeschlagene_timestamp_reparatur_archiviert_ausgabe_und_entfernt_tmp, test_abweichende_ffprobe_framerates_werden_nicht_blind_als_cfr_behandelt, test_vfr_pts_outlier_without_independent_cfr_source_is_not_forced_to_cfr, test_timinganalyse_nutzt_mediainfo_framecount_ohne_ffprobe_count_frames, test_workflow_duration_repair_keeps_verified_dynamic_metadata_evidence, test_workflow_verify_rejected_repair_is_fail_closed_and_never_switches_to_archive, test_genpts_fallback_accepts_valid_candidate_even_with_ffmpeg_einval_returncode, test_rejected_timestamp_candidate_is_archived_for_manual_analysis, test_genpts_candidate_with_unexpected_nonzero_returncode_is_rejected, test_workflow_runner_never_replaces_or_finalizes_after_failed_verification, _has_genpts_flag, _vfr_mediainfo_json, _source_frame_timeline_json, test_vfr_reparatur_uebernimmt_original_timeline_wenn_frameanzahl_identisch, test_vfr_original_timeline_wird_bei_abweichender_frameanzahl_nicht_angewendet, test_workflow_duration_repair_reicht_original_inputpfad_an_reparaturservice_weiter

dragontools/tests/test_dv_audio_mux_service.py - 59 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_dv_audio_probe_uses_injected_runner_and_extracts_tracks, test_dv_audio_probe_malformed_json_is_safe

dragontools/tests/test_dv_pipeline_architecture.py - 260 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _enc_cfg, test_profile_major_normalisierung_deckt_p5_p7_p8_und_string_cachewerte_ab, test_dovi_convert_mode_ist_fuer_p5_auch_bei_string_mode_3, test_p5_encode_verwendet_originalcontainer_libplacebo_und_nicht_p8_stream, test_p5_filter_complex_behält_original_input_und_setzt_libplacebo_vor_userfilter, test_p7_und_p8_encode_verwenden_direkt_die_mkv_und_keinen_p8_arbeitsstream, test_p7_filter_complex_bleibt_auf_mkv_input0, test_request_p5_string_wird_als_p5_sonderpfad_markiert, test_produktiver_p5_run_hat_keinen_legacy_remux_aufruf, test_legacy_p5_remux_kompatibilitaetspfad_ist_entfernt, _make_pipeline_for_preflight, test_pipeline_p5_string_prueft_libplacebo_vor_jeder_dateiarbeit, test_pipeline_lehnt_nicht_h265_dv_fail_fast_ab

dragontools/tests/test_dv_regressions.py - 1.366 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _make_sub, _make_mi, test_dv_auto_exportiert_keep_subs_als_sidecars, test_dv_custom_export_respektiert_keep_auswahl, test_dv_sidecars_werden_nach_replace_zum_finalen_stem_verschoben, test_dv_level5_editor_nutzt_keinen_autocrop_fallback, test_hdr10plus_bitstream_service_extract_inject_und_verify, test_workflow_reicht_hdr10plus_erhalt_ueber_typed_request_an_dv_pipeline_weiter, test_converter_hat_keine_dv_hdrplus_legacy_wrapper_mehr, test_dv_command_runner_sichert_toolfehler_fuer_errorreport,

test_dv_pipeline_uebernimmt_stufenfehler_in_oeffentliche_diagnose, test_error_report_enthaelt_pipeline_diagnose, test_dv_pipeline_stage_builder_ist_vollstaendig_verdrahtet, test_dv_pipeline_dateivalidierung_liefert_konkrete_diagnose, _make_dv_stage_test_context, test_dv_p8_profilnormalisierung_erzeugt_keinen_p8_arbeitsstream_mehr, test_dv_p8_ohne_hdr10plus_normalisiert_rpu_direkt_aus_mkv, test_dv_p5_rpu_wird_direkt_aus_mkv_mit_mode3_auf_p81_normalisiert, test_dv_p7_und_p8_rpu_extraktion_nutzt_mkv_und_mode2, test_dv_unbekannter_cmv4_block_wird_jetzt_beim_extract_rpu_gemeldet, test_dv_command_runner_kennzeichnet_unbekannten_cmv4_block_als_toolinkompatibilitaet, test_dv_combo_erkennt_has_hdr10plus_alias_ohne_has_hdrplus, test_hdr10plus_service_entfernt_stale_outputs_vor_toollauf, test_hdr10plus_finalcheck_vergleicht_metadaten_mit_quelle, test_hdr10plus_finalcheck_ignoriert_nur_toolinfo, test_workflow_sidecar_finalisierung_sichert_vorhandenes_sidecar, test_dv_pipeline_blockiert_unvollstaendigen_sidecar_export, test_dv_adapter_propagates_pipeline_verified_dolby_vision, test_dv_adapter_propagates_pipeline_verified_hdr10plus, test_dv_post_mux_prueft_hdr10plus_und_rpu_aus_finalem_mp4, test_dv_final_verify_uses_mediainfo_signalling_and_also_proves_final_rpu, test_dv_final_verify_checks_dv_and_hdr10plus_when_hdr10plus_signalling_is_missing, test_final_mediainfo_hdr_inspection_reads_dv_and_hdr10plus_without_ffprobe, test_dv_adapter_drops_stale_sidecars_on_failed_run, test_dv_pipeline_reset_clears_previous_run_sidecars, test_cropped_dv_forces_final_rpu_integrity_check_for_mkv_and_mp4, test_cropped_dv_postmux_extraction_uses_correct_mp4_and_mkv_bitstream_path, test_final_rpu_hash_mismatch_is_nonfatal_and_recorded_for_archive

dragontools/tests/test_dv_remux_abort_fix.py - 105 Zeilen - Tests für den DVRemuxThread abort_type Bug-Fix und WorkerProtocol. | Klassen: TestDVRemuxThreadAbortType, TestWorkerProtocol

dragontools/tests/test_dv_remux_container_selection.py - 424 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _noop, test_dv_output_manager_uses_selected_mkv_suffix, test_dv_output_manager_overwrite_uses_selected_mkv_suffix, test_dv_remux_audio_plan_receives_mkv_and_uses_mka, test_dv_remux_mkv_mux_uses_mkvmerge, test_dv_remux_mp4_mux_still_uses_mp4box, test_dv_remux_subtitle_storage_is_sidecar_for_mp4_and_internal_for_mkv, test_dv_remux_mkv_builds_internal_subtitle_jobs_and_preserves_burn_candidate, test_dv_remux_mkv_mux_writes_subtitle_language_title_and_forced_flag, test_dv_remux_mp4_mux_embeds_text_track_with_forced_flag, test_standard_dv_mkv_does_not_duplicate_actual_burn_in_subtitle, test_standard_dv_mkv_muxer_embeds_selected_subtitle, test_standard_dv_subtitle_stage_routes_mp4_to_sidecar_and_mkv_internal

dragontools/tests/test_dv_stage_service_architecture.py - 61 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _class_node, test_dv_stage_coordinator_bleibt_klein_und_ohne_toolausfuehrung, test_dv_stage_services_bleiben_qt_frei, test_dv_stage_services_starten_keine_eigenen_subprocesses

dragontools/tests/test_encoder_args.py - 319 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_h265_encoder_forciert_echtes_10bit, test_h264_bleibt_8bit_kompatibel_ohne_10bit_zwang, test_nvenc_lookahead_wird_ohne_bframes_nicht_gesetzt, test_nvenc_auto_lookahead_level_und_multipass_werden_nicht_gesetzt, test_nvenc_lookahead_level_und_multipass_werden_bei_auswahl_gesetzt, test_x265_lookahead_wird_ohne_bframes_nicht_gesetzt, test_qsv_h264_uses_la_icq_without_qscale_conflict, test_qsv_hevc_av1_use_extbrc_lookahead_without_h264_only_switch,

test_x265_hdr10_vui_parameter_werden_bei_hdr10_ausgabe_gesetzt,
test_nvenc_legacy_string_options_are_normalized_at_ffmpeg_boundary,
test_qsv_and_amf_legacy_float_strings_become_integer_arguments,
test_qsv_lookahead_depth_is_clamped_before_ffmpeg,
test_nvenc_aq_uses_current_canonical_ffmpeg_option_names,
test_amf_only_emits_qp_b_for_codecs_that_expose_it,
test_hardware_quality_values_are_clamped_to_encoder_ranges, test_nvenc_aq_strength_is_integer_and_clamped

dragontools/tests/test_encoder_inactivity_timeout.py - 140 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _FakeWorker | Funktionen: _install_qt_core_stub, _load_converter_progress, test_ffmpeg_inactivity_timeout_allows_active_progress, test_ffmpeg_inactivity_timeout_aborts_silent_process, test_explicit_disabled_timeout_does_not_fall_back_to_general, test_run_capture_times_out_silent_tool_process

dragontools/tests/test_encoder_profile_override.py - 282 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_profile_to_override_rejects_wrong_codec, test_profile_to_override_accepts_codec_enum_defaults, test_normalize_override_preserves_encoder_profile, test_effective_encoder_settings_uses_file_profile_values, test_normalize_override_preserves_manual_encoder_override, test_effective_encoder_settings_manual_override_wins_over_profile_and_global, test_effective_encoder_settings_ignores_manual_override_for_wrong_codec, test_scoped_encoder_options_do_not_leak_cpu_values_into_nvenc, test_scoped_encoder_options_preserve_active_nvenc_values, test_manual_override_accepts_legacy_float_string_quality, _assert_current_nvenc_aq_spelling, test_nvenc_canonical_aq_options_apply_to_global_profile_and_manual_override

dragontools/tests/test_encoder_settings_architecture.py - 92 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _classes, _method_names, test_encoder_settings_controller_is_small_composition_facade, test_encoder_settings_methods_have_exactly_one_implementation_owner, test_encoder_settings_persistence_and_profiles_are_separated

dragontools/tests/test_encoder_settings_shutdown.py - 384 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Signal, _Combo, _Spin, _Check, _Panel, _PersistedPanel, _Settings | Funktionen: _widgets, test_load_encoder_settings_does_not_reset_shutdown_checkbox, test_cpu_encoder_panel_is_visible_for_h264_and_expanded, test_cpu_encoder_panel_does_not_override_persisted_collapsed_state, test_reset_defaults_resets_cpu_encoder_fields, test_assistant_profile_applies_cpu_profile_and_saves, test_av1_assistant_profile_applies_numeric_cpu_preset, test_qsv_encoder_settings_no_longer_write_legacy_checkbox_key

dragontools/tests/test_end_to_end_media_fixture.py - 149 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _make_tiny_mkv, test_real_ffmpeg_fixture_satisfies_expected_media_contract, test_real_ffmpeg_sidecar_export_reads_original_stream

dragontools/tests/test_episode_replacement_artifacts.py - 98 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _service, _episode_fixture, test_episode_replacement_removes_old_video_nfo_and_trickplay_together, test_episode_replacement_rolls_back_nfo_and_trickplay_when_video_copy_fails, test_episode_replacement_does_not_touch_other_episode_artifacts

dragontools/tests/test_error_report.py - 166 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_error_report_writes_context_and_tool_output, test_worker_result_service_stores_failure_details, test_worker_result_service_blocked_size_policy_is_not_success

dragontools/tests/test_exception_boundary_architecture.py - 71 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _function, test_replacement_reminder_gui_boundary_logs_broad_exception,

test_process_runner_has_no_catch_all_exception_boundary, _broad_handler, _handler_has_logger_call, test_quiet_move_and_job_recovery_still_log_failures, test_base_worker_abort_has_no_broad_silent_terminate_catch

dragontools/tests/test_extended.py - 980 Zeilen - Ergänzende Tests für dragontools – deckt Lücken auf, die im Code-Review | Klassen: TestSafeInt, TestSafeFloat, TestNormalizeOverrideDictEdgeCases, TestNormalizeAudioCodec, TestReloadRules, TestLoadJsonRules, TestResolvePipelineContext, TestComputeAudioTrackPlan, TestMediaInfoProperties | Funktionen: _make_video_stream, _make_audio_stream, _make_media_info

dragontools/tests/test_fix_20260906_iso_renamer.py - 108 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_iso_series_disc_ignores_play_all_and_short_extra, test_iso_movie_with_unrelated_extras_falls_back_to_main_title, test_iso_episode_only_disc_detects_all_similar_titles, test_iso_series_detection_can_be_disabled_for_legacy_main_title_behavior, test_iso_series_duration_tolerance_is_configurable, test_iso_thread_and_widget_keep_series_detection_as_explicit_option, test_renamer_manual_search_can_override_detected_media_type

dragontools/tests/test_formatting_helpers.py - 15 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_format_binary_size_preserves_iso_display_semantics, test_format_binary_size_clamps_negative_values_like_previous_helpers

dragontools/tests/test_gui_dialog_services.py - 257 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _QtObject, _Signal, _Spin, _Check | Funktionen: _install_pyqt_stubs, test_timeout_dialog_saves_minutes_and_enabled_state, test_encoder_profile_service_returns_selected_assistant_profile, test_encoder_profile_service_allows_same_label_for_different_encoders, test_profile_manager_same_label_same_encoder_reuses_existing_key, test_profile_manager_contains_extended_h265_start_profiles

dragontools/tests/test_hdr10_color_standard.py - 367 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Logger, _StreamArgs | Funktionen: _standard_request, _media_info, _plan_service, test_dv7_standard_encode_keeps_hdr10_base_color_tags, test_dv7_string_profile_keeps_hdr10_base_color_tags, test_dv5_standard_encode_uses_libplacebo_conversion_then_hdr10_tags, test_dv_cpu_encode_args_use_x265_placebo_and_cpu_10bit_format, test_standard_runner_sets_hdr10_output_flags_for_dv7_h265, test_standard_runner_leaves_sdr_output_untagged, test_standard_runner_entfernt_veraltete_video_statistik_tags, test_standard_runner_preserves_ffmpeg_failure_diagnostics, test_encode_plan_treats_string_false_imax_and_autocrop_flags_as_false, test_dv_nvenc_encode_args_use_current_ffmpeg_aq_option_names

dragontools/tests/test_hdrplus_architecture.py - 227 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _worker_dir, _class_node, test_hdrplus_coordinator_does_not_reimplement_mux_or_metadata_json_parsing, test_hdrplus_tool_calls_are_confined_to_injected_runner_bridge, test_hdrplus_tool_runner_records_command_and_failure_diagnostics, test_hdrplus_metadata_wrappers_use_existing_bitstream_service, test_hdrplus_execute_does_not_mutate_default_encoder_state, test_hdrplus_execution_context_defensively_freezes_job_options, test_hdrplus_new_services_stay_qt_free_and_coordinator_methods_bounded, test_hdrplus_new_job_clears_stale_tool_diagnostics_before_preflight

dragontools/tests/test_hdrplus_final_verification.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _helper, test_final_hdr10plus_verification_extracts_from_finished_container, test_hdrplus_run_fails_if_final_semantic_verification_fails, test_hdrplus_run_marks_final_semantic_verification_success,

test_hdr10plus_semantic_compare_ignores_toolinfo_and_derived_scene_summary,
test_hdr10plus_semantic_compare_still_rejects_changed_dynamic_scene_data,
test_hdrplus_tool_runner_accepts_success_and_configured_warning_rc,
test_hdrplus_tool_runner_rejects_unaccepted_returncode,
test_hdrplus_mux_runner_honors_accepted_warning_returncode,
test_subtitle_default_schema_matches_central_schema_and_is_idempotent

dragontools/tests/test_hdrplus_five_step_pipeline.py - 274 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _helper, test_hdrplus_mkv_uses_direct_flow_and_annexb_only_as_extract_fallback,
test_hdrplus_encode_builds_stream_donor_in_same_ffmpeg_run,
test_hdrplus_mkv_mux_uses_mkvmerge_not_ffmpeg, test_hdrplus_mp4_mux_is_streaming_optimized_with_mp4box,
test_hdrplus_mp4_mux_aborts_instead_of_silently_dropping_audio_on_probe_failure

dragontools/tests/test_incremental_move_queue_edit.py - 336 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _QtObject, _QSettings, _Signal, _Button, _FileList, _MoveThread | Funktionen: _install_pyqt_stubs, _controller,
test_move_status_text_uses_shared_ui_helper, test_incremental_move_does_not_lock_queue_edit,
test_incremental_move_finish_keeps_queue_edit_enabled,
test_incremental_move_finish_uses_finished_thread_reference,
test_incremental_move_does_not_start_without_running_conversion,
test_preflight_report_save_uses_rows_builder_and_logs, test_preflight_dialog_report_checkbox_controls_save_flag,
test_incremental_missing_candidate_stays_queued_with_diagnostics

dragontools/tests/test_iso_architecture.py - 46 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _class,
test_iso_thread_is_lifecycle_coordinator_not_tool_implementation, test_iso_core_services_remain_qt_independent,
test_iso_disc_inspector_has_no_tool_execution_dependency

dragontools/tests/test_jellyfin_nfo.py - 91 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_movie_nfo_contains_clean_tmdb_metadata_without_user_state,
test_episode_nfo_uses_episode_root_and_numbers, test_episode_nfo_writes_tvdb_ids_for_thetvdb_provider

dragontools/tests/test_job_journal.py - 316 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_job_journal_records_and_archives_completed_run, test_unfinished_job_summary_uses_active_journal,
test_resume_plan_requeues_open_and_running_files, test_job_journal_tracks_multiple_parallel_current_files,
test_resume_plan_can_include_failed_files, test_archive_active_job_journal_removes_active_file,
test_job_journal_uses_unique_files_for_overlapping_runs, test_job_journal_start_fails_closed_on_disk_full,
test_job_journal_start_fails_closed_on_permission_denied, test_job_journal_start_propagates_atomic_replace_failure,
test_job_journal_archive_failure_keeps_active_journal_recoverable,
test_job_journal_write_failure_preserves_last_durable_file

dragontools/tests/test_language_priority_rules.py - 512 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_audio, _sub, test_audio_prioritaet_nimmt_naechste_vorhandene_sprache,
test_audio_fallback_uebernimmt_alle_wenn_keine_prioritaet_passt, test_audio_kommentarspur_wird_uebersprungen,
test_audio_max_sprachen_begrenzt_sprachgruppen, test_untertitel_burn_nutzt_feste_burn_sprache,
test_forced_burn_waehlt_bestes_format_trotz_mehrerer_kandidaten,
test_forced_burn_bleibt_mehrdeutig_bei_gleichem_bestformat,
test_forced_burn_plausibilitaet_unter_grenze_brennt_normal,
test_forced_burn_plausibilitaet_warnt_bei_mittlerer_dichte,
test_forced_burn_plausibilitaet_blockiert_full_sub_und_rettet_spur,
test_forced_burn_plausibilitaet_rettet_sidecar_zusaetzlich_trotz_limit,
test_untertitel_keep_nimmt_nur_erste_vorhandene_prioritaetssprache,
test_alte_untertitelregel_behaelt_weiterhin_deutsch, test_alte_untertitelregel_respektiert_max_ein_sub_je_sprache,
test_dv_sidecar_begrenzt_untertitel_und_bevorzugt_srt, test_dv_sidecar_alte_formatliste_erkennt_srt_alias,

test_dv_sidecar_exportiert_keine_forced_subs_nach_forced_burn,
test_dv_sidecar_custom_override_darf_mehrere_subs_exportieren

dragontools/tests/test_legacy_cleanup_architecture.py - 118 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _imports, test_removed_legacy_dv_modules_stay_removed, test_workers_do_not_import_helper_contracts_from_base_worker, test_worker_contracts_remain_qt_independent, test_shared_cleanup_modules_are_part_of_source_tree, test_process_runner_has_no_legacy_run_cmd_alias, test_media_analyzer_has_no_obsolete_detection_wrappers, test_journals_share_atomic_json_writer, test_metadata_providers_share_cache_implementation, test_worker_contracts_have_no_private_path_alias, test_dv_remux_has_no_formatting_compatibility_wrappers, test_media_library_dialog_has_no_legacy_widget_mirror

dragontools/tests/test_log_cleanup.py - 59 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_log_cleanup_deletes_old_logs_but_keeps_crash_state, test_log_cleanup_respects_selected_categories

dragontools/tests/test_log_short_and_audio_titles.py - 35 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_audio_title_transcoded_full, test_audio_title_copy_without_bitrate_language_only, test_short_and_long_log_are_split

dragontools/tests/test_main_window_actions_architecture.py - 145 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _classes, _methods, test_actions_facade_contains_no_business_methods, test_action_mixins_preserve_complete_aggregation_surface_without_duplicates, test_facade_only_composes_focused_action_mixins

dragontools/tests/test_main_window_architecture.py - 99 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _class, _methods, test_main_window_is_small_composition_root, test_main_window_tab_and_recovery_owners_are_separate_and_bounded, test_main_window_tab_bar_can_shrink_without_forcing_window_width, test_main_window_legacy_method_surface_is_preserved_across_owners, test_renamer_view_does_not_force_a_wide_main_window_and_has_configurable_columns

dragontools/tests/test_main_window_tab_visibility.py - 116 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Settings, _Action, _TabBar, _Tabs | Funktionen: _fake_window, test_closing_tab_persists_hidden_state, test_reopening_last_tab_persists_visible_state

dragontools/tests/test_media_analyzer_pix_fmt.py - 94 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TestBuildPixFmtFromMediaInfo | Funktionen: _track, test_build_video_streams_speichert_mediainfo_pix_fmt_im_model, test_build_video_streams_speichert_frame_infos_im_model, test_audio_streams_nutzen_ffprobe_globalindex_bei_bluray_reihenfolge

dragontools/tests/test_media_contract.py - 197 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _audio, _sub, _media, test_contract_uses_audio_and_subtitle_plan, test_dv_contract_expects_no_internal_subtitles_and_dv_hdr10plus, test_strip_only_contract_copies_burn_subtitle_and_preserves_source_video, test_hdrplus_pipeline_requires_hdr10plus_even_without_selection_flag, test_dv_only_contract_never_requires_hdr10plus_from_global_preserve_flag

dragontools/tests/test_media_info_text_builder.py - 144 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _media_info, test_build_media_info_text_preserves_all_sections, test_build_media_info_text_keeps_analysis_when_rules_preview_fails, test_media_info_text_forwards_current_rule_tester_options

dragontools/tests/test_media_library.py - 2.647 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _db_connection, test_backup_database_contains_committed_wal_data,

test_database_export_contains_committed_wal_data,
test_backup_database_removes_partial_snapshot_after_backup_failure,
test_write_action_does_not_continue_when_database_backup_fails, _create_minimal_jellyfin_db, _fake_media_info,
test_video_bitrate_uses_stream_size_when_embedded_bps_tag_is_broken,
test_path_mapping_translates_jellyfin_prefix, test_import_jellyfin_database_builds_dragon_library,
test_jellyfin_import_normalizes_stream_types_to_dragon_schema,
test_jellyfin_import_preserves_explicit_hdr10plus_flags,
test_jellyfin_import_supports_current_baseitems_media_stream_infos_schema,
test_jellyfin_import_deduplicates_equivalent_windows_paths,
test_storage_scan_builds_library_from_real_folder_structure, test_storage_scan_keeps_file_when_analysis_fails,
test_storage_scan_abort_keeps_existing_database,
test_import_jellyfin_database_skips_unmapped_metadata_paths_when_mapping_exists,
test_jellyfin_import_keeps_generic_video_and_movies_namespace_out_of_movie_episode_counts,
test_search_and_series_root_use_imported_library,
test_series_root_uses_year_to_disambiguate_duplicate_series_names,
test_series_root_prefilters_large_library_before_path_resolution,
test_movie_root_prefilters_large_library_before_path_resolution,
test_series_root_reapplies_mapping_and_can_require_existing_path,
test_series_root_rebases_old_local_db_path_to_current_storage_path,
test_series_path_resolution_notices_distinguish_mapping_reasons,
test_series_root_reports_unusable_database_path_when_current_storage_does_not_match,
test_series_root_unusable_match_keeps_database_area_when_default_base_differs,
test_manual_sql_updates_are_backed_by_schema, _insert_library_video,
test_search_supports_resolution_buckets_and_video_codecs,
test_search_distinguishes_sdr_from_unknown_dynamic_range, test_search_treats_jellyfin_smpte2084_as_hdr,
test_deviation_search_returns_only_minority_episodes,
test_deviation_search_marks_ties_as_uneven_without_guessing,
test_search_supports_legacy_jellyfin_stream_types_and_scopes,
test_normalize_database_stream_types_updates_existing_library, test_media_library_csv_exports,
test_database_overview_csv_is_not_limited_to_500_rows, test_record_moved_file_replaces_same_episode_identity,
test_search_finds_duplicate_active_sxxexx_and_cleanup_removes_inactive,
test_tv_and_anime_path_mappings_use_the_same_unc_normalization,
test_backup_database_names_are_collision_safe,
test_jellyfin_import_reads_committed_wal_data_without_modifying_source,
test_jellyfin_import_stages_next_to_target_for_atomic_replace,
test_jellyfin_snapshot_failure_keeps_existing_target_unchanged,
test_database_schema_migrates_legacy_media_items_with_size_bytes,
test_storage_scan_records_real_file_size_and_size_filter,
test_jellyfin_import_exposes_file_size_from_jellyfin_database,
test_jellyfin_duration_seconds_is_not_mistaken_for_ticks, test_jellyfin_import_keeps_direct_video_stream_properties,
test_storage_scan_records_external_subtitles_nfo_and_trickplay,
test_jellyfin_import_preserves_external_subtitle_marker_when_available,
test_jellyfin_import_marks_missing_and_empty_trickplay_infos, _create_extended_jellyfin_metadata_db,
test_jellyfin_import_adds_original_title_providers_values_people_and_collections, _insert_test_media_for_nfo,
test_nfo_lightscan_records_inventory_and_detects_mismatches_without_overwriting_db,
test_nfo_lightscan_marks_unreachable_without_discarding_known_nfo_data,
test_media_library_public_facade_exports_nfo_lightscan_api,
test_nfo_lightscan_marks_missing_and_full_audit_rechecks_known_entries,

test_nfo_lightscan_dotted_series_folder_stays_unreachable_not_missing,
test_database_schema_migrates_minimal_legacy_tables_before_creating_indexes,
test_jellyfin_import_normalized_title_uses_series_id_when_series_name_is_empty

dragontools/tests/test_media_library_architecture.py - 77 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_media_library_facade_preserves_public_api, test_media_library_facade_contains_no_business_logic, test_media_library_implementation_is_split_by_responsibility

dragontools/tests/test_media_library_dialog_architecture.py - 68 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _tree, test_dialog_does_not_import_database_search_or_export_implementations, test_dialog_has_no_direct_sqlite_or_destructive_file_calls, test_view_contains_widget_construction_but_no_core_business_imports, test_service_is_qt_independent, test_dialog_does_not_mirror_view_widgets_as_legacy_attributes

dragontools/tests/test_media_library_dialog_service.py - 247 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: FakeSettings | Funktionen: test_dialog_service_loads_defaults_and_storage_mapping, test_dialog_service_roundtrips_settings_state, test_storage_path_mapping_prefers_h265_then_fallback, test_sql_mutation_detection_accepts_safe_read_only_diagnostics, test_search_csv_export_ignores_gui_500_row_limit, test_presenter_formats_stats_without_gui_dependency, test_presenter_search_row_keeps_dv_hdr_priority, test_saved_queries_and_dynamic_schema_help

dragontools/tests/test_mediainfo_details.py - 91 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_mediainfo_details_zeigt_bitraten_und_spuren, test_mediainfo_details_schaetzt_videobitrate_wenn_video_bitrate_fehlt

dragontools/tests/test_models.py - 128 Zeilen - Tests für core/models.py – normalize_override_dict | Klassen: TestNormalizeOverrideDict, TestMediaInfoHdrFlags | Funktionen: test_override_string_booleans_are_normalized_without_python_bool_trap

dragontools/tests/test_move_conflicts.py - 191 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_find_target_conflicts_matches_same_video_stem_with_other_extension, test_find_target_conflicts_includes_exact_and_other_video_extension, test_find_target_conflicts_does_not_delete_non_video_by_same_stem, test_find_target_conflicts_uses_exact_name_for_non_video_files, test_find_target_conflicts_matches_same_episode_with_different_title, test_find_target_conflicts_matches_same_episode_when_container_changes_back, test_episode_identity_conflicts_keep_multi_episode_identity_precise, test_resolve_rename_path_uses_free_video_stem, test_episode_replacement_artifacts_follow_sxxexx_for_nfo_and_trickplay, test_episode_replacement_artifacts_keep_multi_episode_identity_precise

dragontools/tests/test_move_journal.py - 544 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_move_journal_keeps_failed_files_for_restart, test_move_journal_archives_completed_run, test_interrupted_backup_is_restored_only_when_new_target_is_missing, test_interrupted_backup_is_cleaned_when_new_target_exists, test_running_move_is_completed_after_crash_when_target_exists_and_source_is_gone, test_move_journals_are_unique_and_do_not_supersede_open_runs, test_move_resume_plan_preserves_context, test_move_journal_write_failure_is_fatal, test_recovery_can_cleanup_directory_backup_after_committed_directory_move, test_recovery_keeps_backup_when_source_and_destination_both_exist, test_recovery_restores_backup_when_destination_is_missing_and_source_still_exists, test_recovery_keeps_backup_when_restore_fails,

test_recovery_reports_journal_write_failure_without_deleting_active_journal,
test_move_resume_summary_surfaces_ambiguous_recovery_state,
test_sidecars_pending_is_not_marked_complete_after_video_commit_crash,
test_resume_plan_uses_committed_video_for_companion_only_recovery,
test_move_journal_archive_failure_is_fatal_and_notifies_callback,
test_move_batch_lifecycle_counts_journal_finalization_failure_as_error,
test_planned_target_can_change_while_move_file_is_still_queued,
test_planned_target_change_is_rejected_after_move_started

dragontools/tests/test_move_progress_callback_safety.py - 59 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_service, test_progress_callback_failure_does_not_break_successful_hardlink_move,
test_progress_callback_failure_does_not_break_copy_fallback

dragontools/tests/test_move_report.py - 161 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_move_report_groups_by_target_and_counts_conflict_actions,
test_move_report_format_mentions_deleted_and_replaced_files,
test_move_report_counts_multiple_removed_target_versions,
test_move_report_groups_sidecars_by_type_and_conflict_action

dragontools/tests/test_move_rules_series_detection.py - 83 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_series_detection_removes_separator_before_episode_code,
test_series_detection_keeps_internal_hyphenated_names,
test_series_detection_normalizes_missing_space_after_separator,
test_existing_series_dir_matches_tmdb_colon_and_year_suffix,
test_series_detection_removes_release_dots_and_trailing_year_before_episode_code,
test_series_detection_removes_parenthesized_year_before_episode_code,
test_existing_series_dir_tolerates_optional_internal_hyphen,
test_existing_series_dir_uses_year_to_disambiguate_fraction_titles

dragontools/tests/test_move_service_architecture.py - 95 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_move_core_services_are_qt_independent,
test_move_thread_contains_no_direct_destructive_move_implementation,
test_file_service_moves_simple_file_without_qt, test_move_thread_orchestrates_planned_move_end_to_end

dragontools/tests/test_move_sidecar_recovery.py - 46 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_service, test_missing_source_sidecar_is_success_when_expected_target_exists,
test_missing_source_sidecar_remains_retryable_when_target_is_missing

dragontools/tests/test_move_transaction_core.py - 164 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_tree_pair, test_stage_copy_failure_keeps_existing_destination_and_cleans_partial,
test_commit_failure_restores_old_destination, test_backup_callback_failure_is_rolled_back_and_propagated,
test_successful_commit_keeps_backup_until_explicit_discard, test_failed_rollback_surfaces_backup_path,
test_external_staging_can_survive_commit_failure_for_caller_recovery

dragontools/tests/test_move_transactional.py - 341 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_bare_move_thread, _trickplay_service, test_episode_replacement_rolls_back_old_file_when_copy_fails,
test_journal_failure_after_backup_rename_rolls_back_and_propagates,
test_directory_overwrite_copy_failure_keeps_old_destination,
test_directory_overwrite_commit_failure_restores_old_destination,
test_directory_overwrite_success_commits_new_and_removes_backup, _make_trickplay_pair,
test_trickplay_overwrite_commit_failure_restores_old_target,
test_trickplay_backup_commit_failure_restores_original_name,

test_trickplay_overwrite_success_replaces_old_transactionally,
test_trickplay_backup_success_keeps_old_version_as_bak,
test_directory_overwrite_journal_failure_restores_old_destination

dragontools/tests/test_movie_renamer.py - 600 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_parse_movie_release_name_strips_release_tags_and_keeps_search_core,
test_build_movie_rename_proposal_prefers_matching_title_and_year,
test_build_movie_rename_proposal_retries_german_umlaut_variant,
test_movie_rename_proposal_skips_probable_series_episode,
test_build_series_rename_proposal_uses_episode_metadata_without_trailing_dash,
test_build_series_rename_proposal_retries_german_umlaut_variant_with_resolver,
test_build_series_rename_proposal_retries_german_umlaut_variant_with_client,
test_parse_series_release_name_preserves_episode_number_dot,
test_parse_series_release_name_removes_release_dots_and_year_from_series,
test_build_movie_rename_proposal_marks_conflict, test_target_filename_sanitizes_windows_characters,
test_series_target_filename_uses_standard_pattern_and_special_replacements,
test_series_target_filename_preserves_official_trailing_period_in_series_title,
test_movie_rename_candidate_keeps_provider_for_dropdown,
test_rename_movie_file_keeps_folder_and_changes_only_file_name,
test_series_metadata_candidates_honor_preferred_provider_before_score,
test_explicit_series_alias_is_selected_before_unaliased_exact_match,
test_series_missing_provider_episode_title_uses_folge_fallback_and_not_100_percent,
test_series_without_episode_metadata_uses_provider_series_and_folge_fallback,
test_series_no_metadata_result_never_reuses_release_filename_as_episode_title,
test_series_generic_provider_episode_title_is_normalized_and_confidence_capped,
test_series_renamer_refreshes_cached_fallback_episode_title,
test_episode_refresh_failure_logs_reason_and_keeps_fallback

dragontools/tests/test_movie_renamer_widget_architecture.py - 111 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _module, _class, test_movie_renamer_widget_is_thin_orchestrator,
test_movie_renamer_collaborators_are_bounded,
test_movie_renamer_widget_exposes_only_user_actions_and_lifecycle,
test_filesystem_rename_is_owned_by_action_controller_only, test_metadata_worker_is_not_owned_by_main_widget

dragontools/tests/test_mp4_remux_architecture.py - 33 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_class_node, test_mp4_remux_thread_stays_a_small_qt_orchestrator,
test_mp4_remux_services_remain_qt_independent

dragontools/tests/test_mp4_subtitle_policy.py - 181 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _sub,
_plan, test_legacy_dv_sidecar_setting_migrates_to_global_mp4_setting,
test_new_sidecar_flags_are_migrated_and_gate_non_mp4_exports,
test_mp4_sidecars_enabled_exports_all_selected_subtitle_types,
test_mp4_sidecars_disabled_keeps_text_internal_and_bitmap_external,
test_mp4_remux_policy_preserves_burn_candidate_when_no_burn_is_possible,
test_converter_stream_args_transcodes_text_to_mov_text_and_leaves_pgs_for_sidecar,
test_sidecar_service_exports_only_bitmap_when_global_mp4_sidecars_are_off,
test_dv_mp4_internal_job_converts_ass_and_srt_to_srt_for_mp4box

dragontools/tests/test_online_metadata.py - 830 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_parse_movie_query_extracts_title_and_year_from_release_name,
test_parse_movie_query_keeps_manual_dotted_title_without_video_extension,

test_clean_tmdb_collection_name_removes_provider_suffixes, test_tmdb_client_resolves_collection_with_read_token, test_tmdb_client_retries_movie_search_with_german_umlaut_variant, test_tmdb_client_resolves_series_year_with_read_token, test_series_folder_name_removes_brand_colon_without_underscore, test_series_folder_name_replaces_title_colon_with_separator, test_parse_series_query_removes_episode_and_extracts_year, test_tmdb_client_requires_credentials, test_thetvdb_client_resolves_episode_with_login_token, test_thetvdb_client_finds_specials_in_season_zero, test_thetvdb_client_resolves_movie_with_login_token, test_both_provider_uses_available_source_without_blocking_missing_second_source, test_both_provider_honors_series_preference, test_thetvdb_series_uses_fallback_translation_instead_of_original_japanese_name, test_tmdb_missing_episode_title_uses_folge_fallback_not_source_filename, test_tmdb_refresh_episode_candidates_replaces_cached_fallback_title, test_tvdb_missing_episode_title_uses_folge_fallback_not_source_filename, test_tvdb_refresh_episode_candidates_replaces_cached_fallback_title, test_generic_provider_episode_titles_are_normalized_to_german_fallback, test_tvdb_disk_cache_is_checked_before_login, test_tmdb_fresh_session_bypasses_disk_cache_once_then_reuses_live_response

dragontools/tests/test_online_metadata_architecture.py - 111 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_online_metadata_facade_preserves_public_api, test_online_metadata_facade_contains_no_business_logic, test_online_metadata_implementation_is_split_by_responsibility, test_provider_clients_never_fallback_to_source_stem_as_episode_title, test_tmdb_client_is_a_thin_composition_facade

dragontools/tests/test_output_path_service.py - 51 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_output_paths_are_reserved_for_parallel_non_overwrite_jobs, test_output_paths_are_reserved_for_parallel_overwrite_temp_jobs

dragontools/tests/test_output_verifier.py - 398 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _patch_probe, test_output_verifier_accepts_video_audio_and_plausible_duration, test_output_verifier_rejects_missing_audio_when_source_had_audio, test_output_verifier_rejects_implausibly_short_duration, test_output_verifier_uses_configurable_duration_tolerance, test_output_verifier_reports_missing_output_file, _contract, test_output_verifier_contract_rejects_missing_planned_audio_track, test_output_verifier_contract_rejects_audio_codec_channels_and_language, test_output_verifier_accepts_undetermined_audio_without_language_tag, test_output_verifier_contract_rejects_subtitle_forced_mismatch, test_output_verifier_contract_rejects_8bit_hevc_when_10bit_planned, test_output_verifier_contract_checks_hdr_dv_and_hdr10plus, test_output_verifier_accepts_semantic_hdr10plus_pipeline_evidence, test_output_verifier_checks_actual_container_not_only_extension, test_output_verifier_contract_allows_intentionally_audio_free_output, test_output_verifier_accepts_verified_dv_pipeline_evidence_when_ffprobe_omits_dovi

dragontools/tests/test_parallel_converter_architecture.py - 34 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _class, test_parallel_converter_thread_ist_koordination_statt_god_class, test_parallel_converter_helpers_bleiben_qt_unabhaengig

dragontools/tests/test_parallel_converter_thread.py - 393 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: FakeSignal, FakeSignalDescriptor, FakeQObject, FakeLogger, FakeWorker | Funktionen: _install_parallel_fakes, _thread,

test_parallel_thread_starts_limited_workers_and_aggregates_progress, _complete,
test_live_reduction_drains_running_files_before_replacing_one,
test_live_increase_fills_free_slots_immediately_in_queue_order, test_limit_change_during_pause_waits_for_resume,
test_limit_increase_does_not_override_abort_after_file, test_live_limit_applies_to_dv_encode_slot_release,
test_single_initial_file_can_add_jobs_after_live_increase, test_parallel_thread_delegates_pause_resume_and_abort,
test_parallel_thread_add_remove_and_reorder_queue, test_parallel_thread_frees_slot_while_postprocessing_waits,
test_parallel_thread_waits_for_terminal_postprocess_signal_before_finished,
test_parallel_thread_marks_unreported_child_finish_as_error,
test_parallel_thread_deduplicates_initial_queue_before_starting_workers

dragontools/tests/test_parallel_settings.py - 104 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DummySettings, MutableSettings | Funktionen: test_parallel_jobs_use_new_safe_defaults_when_not_configured,
test_parallel_defaults_migration_updates_old_saved_defaults, test_parallel_defaults_migration_keeps_custom_values,
test_parallel_jobs_use_cpu_and_gpu_limits, test_parallel_jobs_are_limited_by_file_count_and_maximum,
test_split_files_for_parallel_workers_uses_round_robin_order, test_parallel_jobs_accept_legacy_float_string_values

dragontools/tests/test_patch1_burnin_resolution.py - 194 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _burn_sub, _worker, test_text_burn_failure_is_fail_closed_and_removes_temp,
test_unknown_planned_burn_codec_is_fail_closed, test_media_contract_1080p_requires_target_height,
test_media_contract_original_with_crop_requires_exact_dimensions,
test_output_verifier_rejects_wrong_planned_resolution

dragontools/tests/test_patch2_sidecar_crash_recovery.py - 150 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _write, test_recovery_rolls_back_sidecar_when_video_staging_still_exists,
test_recovery_completes_sidecar_when_video_commit_is_visible,
test_video_committed_flag_completes_even_if_video_path_is_same,
test_sidecar_plan_is_prepared_before_any_mutation, test_journal_write_failure_blocks_sidecar_mutation

dragontools/tests/test_patch3_dv_metadata_and_process.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_dv_encode_forces_frame_passthrough, test_physical_crop_writes_zero_level5_offsets,
test_dv_command_runner_uses_shared_process_control, _stages_for_verification,
test_rpu_frame_mismatch_blocks_injection, test_rpu_post_injection_hash_mismatch_is_rejected,
test_physical_left_right_crop_is_zeroed_in_rpu_for_mkv_and_mp4_logic,
test_physical_crop_fails_closed_if_final_rpu_still_has_nonzero_l5

dragontools/tests/test_patch3_subtitle_process_control.py - 71 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_extract_does_not_overwrite_existing_file, test_extract_uses_shared_runner_and_worker,
test_ffmpeg_inject_does_not_overwrite_existing_file, test_ffmpeg_inject_uses_shared_runner

dragontools/tests/test_patch4_lifecycle_hardening.py - 174 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _AbortWorker, _CancelWorker, _InterruptWorker, _StubbornWorker | Funktionen:
test_crash_guard_parallel_mark_activity_is_race_free, test_crash_guard_uses_shared_atomic_json_writer,
test_application_shutdown_requests_all_workers_before_waiting,
test_application_shutdown_never_force_terminates_stubborn_thread,
test_loaded_widget_collection_uses_public_worker_provider_only, _class_methods,
test_all_lazy_main_tabs_expose_shutdown_worker_contract,
test_subtitle_widget_has_per_job_worker_registry_and_targeted_cancel,
test_main_window_shutdown_is_fail_closed_and_clears_crash_marker_after_workers_stop

dragontools/tests/test_patch5_release_ci.py - 153 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_patch5_release_contract_files_are_present, test_patch5_release_checks_accept_current_declared_environment,
test_runtime_dependency_check_rejects_missing_cryptography,

test_test_environment_must_include_runtime_requirements,
test_required_qt_mode_fails_instead_of_silently_skipping,
test_required_dv_hdr_mode_fails_when_real_tools_are_missing,
test_external_media_environment_accepts_explicit_tool_paths,
test_external_media_environment_finds_bundled_tool_dirs

dragontools/tests/test_patch_pipeline_codec_safety.py - 285 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _media, test_target_codec_normalization_handles_enum_and_aliases, test_unknown_target_codec_fails_fast, test_dv_preserve_override_cannot_force_incompatible_target, test_hdrplus_preserve_override_cannot_force_incompatible_target, test_av1_dv_preserve_selects_dedicated_profile10_pipeline, test_av1_hdrplus_preserve_selects_dedicated_pipeline, test_av1_dv_has_priority_when_dv_and_hdr10plus_preservation_are_both_active, test_explicit_av1_hdrplus_override_can_override_automatic_dv_priority, test_av1_source_per_file_dv_true_cannot_reenable_dv_pipeline, test_explicit_impossible_pipeline_override_fails_before_worker, test_hdrplus_override_rejects_av1_source_even_when_target_is_h265, test_x265_explicit_zero_tuning_values_are_preserved, test_all_encoders_reject_unknown_target_codec, test_unknown_encoder_is_not_silently_treated_as_cpu, test_encoder_accepts_targetcodec_enum_without_stringification_bug, test_explicit_standard_override_disables_dynamic_metadata_contract_flags, test_explicit_hdrplus_override_does_not_claim_dv_preservation, test_dv_hdr10plus_source_with_dv_disabled_selects_hdrplus_without_reenabling_dv, test_dv_only_source_does_not_inherit_global_hdr10plus_requirement, test_hdr10plus_only_source_does_not_inherit_global_dv_requirement, test_dv_hdr10plus_source_keeps_combined_metadata_contract

dragontools/tests/test_path_safety.py - 98 Zeilen - Tests für core/path_safety.py | Klassen: TestIsSafeSubpath, TestSafeUnlink, TestSafeRmtree | Funktionen: tmp

dragontools/tests/test_per_file_strip_only.py - 188 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _FakeSettings | Funktionen: _queue_owner, test_workflow_effective_strip_only_uses_per_file_override, test_workflow_pipeline_executor_uses_strip_runner_for_per_file_strip_only, test_queue_label_shows_processing_badge_from_override, test_queue_label_shows_pipeline_and_postprocess_badges_from_cache, test_queue_label_shows_dv_hdr10plus_and_named_postprocess_badges, test_strip_only_toggle_applies_to_all_selected_paths, test_strip_only_context_menu_uses_current_multi_selection

dragontools/tests/test_pipeline_and_singleton.py - 96 Zeilen - Tests für rules/pipeline_selector.py und core/paths.py (Singleton) | Klassen: TestPipelineSelector, TestToolPathsSingleton | Funktionen: _make_media_info

dragontools/tests/test_pipeline_decision_service_safety.py - 148 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Settings, _Logger, _Archive | Funktionen: _media, _service, test_h265_dv_routes_to_mp4_without_archive, test_incompatible_target_override_archives_and_downgrades_before_worker, test_av1_hdrplus_source_is_archived_instead_of_using_hevc_hdrplus_tool, test_invalid_configured_target_codec_fails_at_service_construction, test_string_false_encoder_option_does_not_force_dv_pipeline, test_av1_dv_priority_is_logged_as_short_info_without_archive

dragontools/tests/test_postprocess_move_sidecars.py - 184 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _make_sidecar_service, test_movie_nfo_sidecar_is_renamed_to_movie_nfo_in_film_target, test_episode_nfo_sidecar_keeps_filename_in_series_target, test_trickplay_sidecar_conflict_can_be_backed_up_on_move, test_trickplay_sidecar_conflict_can_keep_existing_target,

test_move_thread_replaces_same_sxxexx_even_when_conflict_mode_skip,
test_move_thread_records_replacement_reminder

dragontools/tests/test_postprocess_service.py - 90 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_prepare_nfo_path_skip_keeps_existing_file, test_prepare_nfo_path_backup_marks_existing_file_as_saved,
test_async_postprocess_coordinator_collects_outputs

dragontools/tests/test_preflight_dialog_performance.py - 649 Zeilen - Quellmodul ohne Modul-Docstring | Klassen:
_ComboStub, _LineEditStub, _LabelStub | Funktionen: _series_widget_stub, _series_widget_stub_with_bases,
test_series_preview_path_does_not_scan_existing_folders,
test_series_metadata_job_is_payload_for_background_lookup,
test_series_metadata_job_searches_selected_base_before_fallback,
test_existing_series_dir_can_switch_to_fallback_base,
test_unusable_library_match_switches_to_database_area_and_applies_year,
test_background_lookup_uses_fallback_before_tmdb, test_background_movie_lookup_uses_library_before_online,
test_background_lookup_caches_duplicate_series_jobs,
test_background_lookup_searches_local_folders_without_tmdb,
test_background_library_mapping_notice_is_forwarded_verbatim,
test_background_lookup_uses_online_year_to_pick_existing_series_folder,
test_background_lookup_passes_online_year_to_library_search,
test_background_lookup_returns_folder_choices_when_year_is_unknown,
test_unusable_library_series_hit_still_runs_online_year_lookup,
test_composite_series_result_applies_online_year_and_keeps_library_warning

dragontools/tests/test_preflight_gui_architecture.py - 31 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_preflight_dialog_is_orchestrator_not_god_class, test_move_preflight_controller_is_orchestrator_not_god_class

dragontools/tests/test_preflight_report.py - 76 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_preflight_report_includes_targets_rules_and_warnings, test_preflight_report_shows_film_series_subpath,
test_write_preflight_report_creates_file

dragontools/tests/test_process_runner_boundaries.py - 27 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_run_analysis_tool_wraps_oserror_as_runtimeerror, test_run_analysis_tool_does_not_hide_programming_errors

dragontools/tests/test_project_info.py - 91 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_collect_project_statistics_counts_sources_and_tests, test_collect_project_statistics_includes_hdr10plus_generator,
test_about_html_is_short_general_overview, test_collect_project_statistics_includes_generator_entry_and_extras

dragontools/tests/test_quality_tester.py - 79 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_parse_quality_segments_supports_percent_and_timestamp_ranges,
test_automatic_quality_segments_are_bounded_to_video_duration,
test_quality_run_from_dict_clamps_quality_and_fills_defaults,
test_comparison_duration_limit_respects_positive_and_negative_b_offset,
test_comparison_segment_starts_supports_b_offset_in_both_directions,
test_quality_comparison_assessment_is_relative_to_reference_and_mentions_size_delta

dragontools/tests/test_real_dv_hdr_integration.py - 374 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
_sha256, _run, _run_callback, _make_hevc, _extract_hevc_from_mp4,
test_real_dolby_vision_rpu_and_mp4box_roundtrip, test_real_hdr10plus_extract_inject_verify_roundtrip,
_make_pq_hevc, test_real_generated_hdr10plus_metadata_inject_extract_roundtrip,
test_real_built_hdr10plus_generator_exe_uses_explicit_tool_paths

dragontools/tests/test_release_artifact_e2e.py - 135 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_source_release_roundtrip_validates_cleanly, test_architecture_tests_do_not_depend_on_project_cwd,

test_source_release_omits_generated_pyinstaller_specs,
test_main_entrypoint_version_switch_is_headless_and_uses_central_version,
test_private_snapshot_manifest_matches_current_source_inventory,
test_smoke_test_initializes_real_qapplication_contract, test_pytest_default_paths_include_standalone_generator_suite

dragontools/tests/test_release_packaging.py - 226 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_source_release_zip_excludes_bytecode_and_dev_caches, test_source_release_zip_failure_keeps_existing_target,
test_forbidden_release_path_handles_windows_and_posix_paths, test_release_packaging_filters_test_and_lint_caches,
test_source_release_zip_excludes_local_and_external_trees,
test_clean_forbidden_release_artifacts_removes_bytecode_and_caches,
test_release_artifact_checks_ignore_excluded_local_environments,
test_release_artifact_scan_flags_generated_spec_in_source_tree,
test_real_project_public_inventory_excludes_private_snapshot_and_review_artifacts,
test_public_source_zip_omits_private_snapshot_artifacts,
test_privacy_scan_uses_same_public_inventory_as_source_packager,
test_public_sanitizer_normalizes_both_private_unc_spellings

dragontools/tests/test_release_validation.py - 670 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _write_json,
test_current_refactor_modules_are_release_smoke_checked, _write_package_smoke_files,
test_release_validation_checks_required_files_and_schema,
test_release_validation_treats_schema_mismatch_as_error, test_release_validation_warns_about_private_paths,
test_app_bundle_validation_checks_exe_not_source_files,
test_format_release_checks_summarizes_errors_and_warnings,
test_source_only_manifest_makes_source_archive_self_consistent,
test_release_validation_rejects_python_bytecode_artifacts,
test_release_validation_accepts_clean_tree_without_bytecode,
test_package_only_manifest_validates_code_only_release, test_release_validation_rejects_test_cache_directories,
test_release_validation_detects_stale_built_documentation, test_validate_source_release_accepts_relative_root,
test_release_validation_rejects_structurally_invalid_changelog

dragontools/tests/test_replacement_reminders.py - 95 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen:
test_replacement_reminder_persists_with_unique_ids,
test_replacement_reminder_confirm_deletes_defer_and_close_keep,
test_replacement_reminder_survives_shutdown_until_next_start

dragontools/tests/test_review_patch_block2_output_persistence.py - 176 Zeilen - Quellmodul ohne Modul-Docstring |
Funktionen: test_auxiliary_overwrite_workers_use_central_commit_service,
test_rules_dialog_storage_uses_atomic_json_writer, test_profile_manager_save_uses_atomic_json_writer,
test_profile_migration_rewrite_uses_atomic_json_writer,
test_profile_manager_failed_set_does_not_publish_unsaved_profile,
test_profile_manager_failed_delete_keeps_profile_in_memory,
test_profile_manager_migration_write_failure_does_not_break_profile_loading

dragontools/tests/test_review_patch_block3_resilience_cleanup.py - 152 Zeilen - Quellmodul ohne Modul-Docstring |
Funktionen: test_quarantine_corrupt_file_moves_original_and_preserves_payload,
test_profile_manager_quarantines_broken_json_and_reports_warning,
test_profile_manager_quarantines_non_object_json_instead_of_overwriting_it,
test_profile_manager_does_not_swallow_migration_programming_errors,
test_user_rule_loader_quarantines_broken_json_before_migration_write,
test_user_rule_loader_quarantines_non_object_json,
test_media_library_series_parse_failure_is_logged_instead_of_silently_swallowed

dragontools/tests/test_review_stdlib.py - 243 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: TestWorkerImports, TestConvertWidgetPathHooks, TestBaseWorkerContract | Funktionen: _parse_module, _imported_names, _class_def, _function_def, _qt_core_stub_modules, _reload_module

dragontools/tests/test_rules_preview_headless.py - 16 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_rules_preview_module_has_no_pyqt_runtime_import

dragontools/tests/test_run_summary.py - 96 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_run_summary_counts_ok_errors_and_skipped, test_run_summary_maps_emoji_statuses, test_run_summary_preserves_failure_report_details, test_run_summary_formats_postprocess_and_sidecar_details

dragontools/tests/test_settings_backup.py - 640 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: FakeSettings, FailingSyncSettings | Funktionen: test_backup_exports_and_restores_settings_rules_and_profiles, test_settings_to_dict_can_mask_sensitive_online_metadata_values, test_default_backup_omits_sensitive_values_and_preserves_local_secrets_on_restore, test_encrypted_backup_roundtrip_and_wrong_password_is_non_destructive, test_encrypted_backup_requires_password, test_legacy_v1_plaintext_backup_remains_restore_compatible, test_legacy_v1_restore_can_keep_local_plaintext_secrets, test_restore_backup_rolls_back_settings_and_files_on_error, test_future_backup_format_is_rejected_without_mutating_settings, test_backup_member_limit_is_enforced_before_restore_mutates_settings, test_backup_member_size_and_compression_ratio_are_bounded, test_unknown_scrypt_profile_is_rejected_before_key_derivation, test_excluded_backup_preserves_opaque_dpapi_storage_value_verbatim, test_failed_restore_rolls_back_opaque_dpapi_storage_value_verbatim, test_v2_excluded_backup_rejects_plaintext_secret_in_settings_json, test_encrypted_backup_rejects_non_secret_keys_inside_secret_block, test_restore_rejects_unexpected_or_nested_backup_file_members, test_restore_rejects_invalid_serialized_bytes_without_mutating_settings, test_atomic_restore_write_fsyncs_staged_file, test_encrypted_backup_fails_instead_of_exporting_empty_unreadable_dpapi_secret

dragontools/tests/test_settings_dialog_architecture.py - 109 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _tree, _method_span, _literal_tuple_assignment, _section_titles, test_settings_dialog_is_orchestrator_not_god_class, test_settings_sections_cover_every_visible_section_exactly_once, test_settings_dialog_uses_public_collaborator_helpers_without_private_aliases, test_output_container_has_direct_settings_menu_entry, test_jellyfin_api_has_direct_settings_menu_entry

dragontools/tests/test_settings_helpers.py - 186 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: DummySettings | Funktionen: test_settings_bool_parses_string_values_without_python_bool_trap, test_ui_section_expanded_key_is_stable_and_namespaced, test_collapsible_groupbox_persists_user_click, test_settings_int_float_and_text_clamp_or_fallback, test_postprocess_config_uses_centralized_settings_limits, test_output_size_policy_accepts_injected_settings, test_type_utils_handle_legacy_float_strings_and_unknown_bool_safely, test_settings_int_accepts_qsettings_float_string

dragontools/tests/test_sidecar_transaction.py - 55 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_sidecar_commit_preserves_existing_user_file_as_backup, test_sidecar_commit_rollback_restores_user_file_and_staging

dragontools/tests/test_source_visual_check.py - 157 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _frame, test_source_visual_check_blocks_when_enough_probes_are_suspicious,

test_source_visual_check_does_not_block_single_black_scene, test_source_visual_settings_reads_qsettings_values, test_source_visual_batch_reader_groups_probe_processes, test_source_visual_batch_reader_falls_back_per_probe

dragontools/tests/test_stability_review_fixes.py - 181 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_tool_runner_does_not_leave_resource_warnings, test_converter_progress_run_closes_pipes_without_resource_warning, test_process_control_reports_suspend_failure_and_clears_false_pause, test_postprocess_enablement_error_is_fail_closed, test_crash_guard_reports_marker_write_failure, test_quality_tester_has_a_configurable_hard_timeout, test_move_preflight_uses_public_collaborator_api, test_production_code_contains_no_runtime_assert_statements, test_converter_runtime_builder_has_no_converter_thread_import, test_process_lifecycle_crash_marker_includes_activity_file

dragontools/tests/test_subtitle_converter.py - 153 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_srt_to_txt_preserves_timestamps_and_removes_numbers, test_srt_to_txt_writes_timestamped_output_file, test_txt_to_srt_recreates_numbered_blocks, test_txt_to_srt_writes_output_file, test_txt_to_ass_converts_timestamped_txt, test_txt_to_srt_rejects_txt_without_timestamps, test_ass_to_txt_preserves_timestamps_for_roundtrip, test_ass_style_supports_configurable_font_and_size, test_ass_style_sanitizes_comma_and_clamps_font_size

dragontools/tests/test_subtitle_sidecar_service.py - 541 Zeilen - Tests fuer SubtitleSidecarService und Hilfsfunktionen. | Klassen: TestSafeLangTag, TestLangIsoTag, TestSidecarFilename, TestSubtitleSidecarService | Funktionen: _make_sub, _make_mi, test_structured_result_marks_partial_export_as_incomplete, test_ass_sidecar_can_also_export_srt_variant, test_mkv_text_to_srt_exports_only_srt_variant, test_mkv_mov_text_can_export_srt_sidecar, test_sidecar_export_never_overwrites_existing_destination, test_abort_happens_before_unsupported_codec_is_reported, test_pgs_mkvextract_fallback_maps_ffprobe_subtitle_ordinal_to_mkv_track

dragontools/tests/test_subtitle_tagger_contract.py - 40 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_mkvpropedit_uses_configured_path_ordinal_track_and_accepts_warning, test_mkvpropedit_rc2_is_failure

dragontools/tests/test_system_shutdown.py - 70 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_schedule_system_shutdown_uses_short_windows_buffer, test_schedule_system_shutdown_returns_false_on_failure

dragontools/tests/test_tool_diagnostics.py - 100 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_probe_ffmpeg_reports_version_and_features, test_format_tool_diagnostics_marks_missing_tool, test_handbrake_gui_is_not_started_for_version_probe, test_extended_system_test_runs_mini_tool_chain

dragontools/tests/test_tool_paths.py - 102 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_find_tool_searches_third_party_product_dirs, test_extend_path_adds_third_party_product_dirs, test_find_tool_searches_pyinstaller_contents_programme_dirs, test_extend_path_adds_pyinstaller_contents_programme_dirs, test_default_storage_dirs_are_created, test_default_target_path_for_settings_key_maps_codec_and_media_type, test_default_target_paths_can_be_returned_without_creating_dirs

dragontools/tests/test_tool_runner.py - 70 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_run_tool_collects_stdout_and_stderr, test_run_tool_reports_timeout, test_terminate_plain_logs_terminate_and_kill_failures, test_terminate_plain_does_not_swallow_unexpected_programming_error

dragontools/tests/test_trickplay_service.py - 371 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_trickplay_uses_jellyfin_default_folder_and_ffmpeg_tile_filter, test_trickplay_existing_folder_is_returned_when_only_missing,

test_trickplay_existing_root_allows_missing_sprite_variant, test_trickplay_existing_root_can_be_backed_up, test_trickplay_can_read_source_but_write_for_output_video, test_trickplay_logs_hardware_mode, test_trickplay_logs_cpu_fallback_mode, test_trickplay_uses_cuda_download_retry_before_cpu_fallback, test_trickplay_overwrite_commit_failure_restores_previous_root, test_trickplay_rollback_failure_keeps_old_backup, test_trickplay_overwrite_keeps_backup_if_cleanup_fails, test_trickplay_runner_has_runtime_timeout_and_drains_large_stderr, test_trickplay_abort_after_file_request_still_stops_running_process

dragontools/tests/test_ui_helpers_geometry.py - 90 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Settings, _Signal, _Widget | Funktionen: test_window_geometry_key_is_stable_and_namespaced, test_restore_and_save_window_geometry_use_same_key, test_install_persistent_window_geometry_saves_when_dialog_finishes

dragontools/tests/test_v97_four_patches.py - 205 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _mi, test_dv_crop_level5_export_wird_zu_crop_filter, test_dv_crop_kleine_abweichung_nimmt_groesseren_autocrop, test_dv_crop_rpu_darf_autocrop_auch_bei_groesserer_flaeche_nicht_ueberschreiben, test_dv_crop_grosse_abweichung_braucht_benutzerentscheidung, test_dv_crop_kein_autocrop_gegen_grossen_rpu_crop_ist_ebenfalls_konflikt, test_dv_crop_dynamic_level5_wird_nicht_als_globaler_crop_verwendet, test_dv_crop_filter_wird_in_vf_kette_korrekt_ersetzt, test_container_policy_standard_hdrplus_und_dv_sind_getrennt, test_mp4box_mux_ist_streamingoptimiert, test_manuelle_seriensuche_ueberschreibt_nur_den_suchbegriff, test_preflight_release_erkennung_markiert_punktnamen_und_technik_tags, test_dv_mkv_mux_hat_eigenen_timeout_und_keinen_veralteten_reextract_timeout, test_series_release_group_does_not_misclassify_normal_episode_title_word, test_series_release_group_still_detects_scene_group_after_technical_tags, test_preflight_and_renamer_share_series_query_and_local_title_rules

dragontools/tests/test_verbose_cleanup.py - 65 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_verbose_logger_discard_removes_file, test_converter_discards_verbose_after_clean_run, test_converter_keeps_verbose_after_error_or_abort

dragontools/tests/test_version_contract.py - 73 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: _literal_app_version_assignments, test_app_version_has_one_python_literal_source, test_public_package_version_and_settings_share_central_version, test_release_manifest_matches_central_version_and_cli_builder, test_builder_uses_dynamic_build_name_without_old_version_literals

dragontools/tests/test_windows_path_helpers.py - 68 Zeilen - Quellmodul ohne Modul-Docstring | Funktionen: test_unc_path_is_normalized_without_host_reinterpretation, test_forward_slash_unc_and_backslash_unc_compare_equal, test_drive_path_join_keeps_windows_separator, test_windows_filename_helpers_are_host_independent, test_path_child_check_is_case_insensitive_for_windows_paths, test_jellyfin_posix_path_helpers_keep_forward_slashes, test_move_rules_use_windows_filename_and_target_syntax

dragontools/tests/test_workflow_pipeline_contract.py - 272 Zeilen - Quellmodul ohne Modul-Docstring | Klassen: _Pipeline, _TempState | Funktionen: _request, test_pipeline_request_from_context_normalisiert_enum_und_kopiert_mutable_daten, test_pipeline_executor_dispatches_dv_via_typed_executor, test_pipeline_executor_strip_only_bypasses_dv_hdrplus_and_standard, test_pipeline_executor_preserves_structured_strip_failure_diagnostics, test_workflow_services_maps_structured_pipeline_failure_without_runner_introspection,

test_workflow_refactor_has_explicit_boundaries_and_no_private_pipeline_peeking,
test_pipeline_request_from_context_preserves_string_enum_value_for_h265

25.8 Sonstige Paketdateien - 1 Datei(en)

dragontools/__init__.py - 7 Zeilen - DragonTools package metadata.

26 Interne und externe Abhängigkeiten

Modul	direkt importiert von	eigene interne Imports
dragontools/core/settings.py	404	0
dragontools/core/paths.py	684	2
dragontools/core/models.py	344	1
dragontools/worker/base_worker.py	151	1
dragontools/core/media_analyzer.py	750	6
dragontools/core/logger.py	713	2
dragontools/rules/move_rules.py	594	1
dragontools/gui/info_button.py	42	0
dragontools/rules/subtitle_rules.py	978	3
dragontools/rules/audio_plan.py	406	2
dragontools/core/timeout_settings.py	475	1
dragontools/core/lang_codes.py	303	0
dragontools/core/process_runner.py	98	0
dragontools/core/online_metadata.py	85	4
dragontools/rules/rule_loader.py	152	2
dragontools/core/config_migration.py	318	1
dragontools/worker/workflow_engine.py	186	1
dragontools/rules/audio_rules.py	819	4
dragontools/core/audit_log.py	114	1
dragontools/gui/conversion_session_state.py	174	0
dragontools/worker/encoder_args.py	175	0
dragontools/worker/workflow_services.py	182	6
dragontools/worker/subtitle_sidecar_service.py	379	4
dragontools/worker/tool_runner.py	532	3
dragontools/worker/dv_runtime_models.py	45	0
dragontools/worker/process_control.py	362	0

Modul	direkt importiert von	eigene interne Imports
dragontools/worker/converter_thread.py	399	37
dragontools/gui/convert_widget_file_queue.py	690	4
dragontools/gui/convert_widget_layout.py	604	4
dragontools/gui/rules_dialog_storage.py	52	4
dragontools/gui/preflight_dialog.py	258	3
dragontools/core/batch_preflight.py	886	2
dragontools/core/profile_manager.py	211	2
dragontools/core/crash_guard.py	336	1
dragontools/core/job_journal.py	463	1
dragontools/core/parallel_settings.py	93	1
dragontools/rules/pipeline_selector.py	241	1
dragontools/core/__init__.py	1	0
dragontools/core/codec_profile_assistant.py	361	0
dragontools/core/gpu_detection.py	174	0
dragontools/gui/ui_helpers.py	88	0
dragontools/worker/dv_processing_pipeline.py	326	15
dragontools/gui/move_preflight_controller.py	131	8
dragontools/gui/conversion_result_service.py	470	5
dragontools/core/audio_video_matcher.py	1239	4
dragontools/core/preflight_report.py	245	2
dragontools/gui/convert_widget_custom_widgets.py	356	2
dragontools/gui/timeout_settings_dialog.py	290	2
dragontools/worker/cleanup_service.py	186	2
dragontools/worker/source_visual_check.py	379	2
dragontools/worker/standard_pipeline_runner.py	170	2
dragontools/core/error_report.py	354	1
dragontools/core/move_conflicts.py	174	1
dragontools/core/path_safety.py	64	1
dragontools/core/settings_backup.py	442	1
dragontools/core/tool_diagnostics.py	386	1
dragontools/gui/drop_path_extractor.py	468	1
dragontools/worker/dv_level5_editor.py	124	1

Modul	direkt importiert von	eigene interne Imports
dragontools/worker/hdr10_color.py	87	1
dragontools/core/encoder_profile_override.py	236	0
dragontools/core/log_cleanup.py	132	0
dragontools/core/media_metadata.py	218	0
dragontools/core/type_utils.py	79	0
dragontools/gui/encoder_settings_state.py	23	0
dragontools/rules/__init__.py	1	0
dragontools/subtitle/converter.py	208	0
dragontools/worker/output_path_service.py	58	0
dragontools/worker/worker_events.py	48	0
dragontools/gui/conversion_controller.py	161	16
dragontools/gui/convert_widget_queue_actions.py	31	12
dragontools/core/rules_preview.py	275	9
dragontools/worker/parallel_converter_thread.py	559	8
dragontools/gui/settings_dialog.py	154	7
dragontools/core/diagnostic_package.py	191	6
dragontools/worker/duration_repair_service.py	732	6
dragontools/worker/encode_plan_service.py	167	5
dragontools/worker/move_thread.py	463	5
dragontools/worker/postprocess_service.py	519	5
dragontools/core/mediainfo_details.py	394	4
dragontools/core/movie_renamer.py	936	3
dragontools/gui/encoder_settings_controller.py	41	3
dragontools/core/jellyfin_nfo.py	272	2
dragontools/gui/encoder_profile_service.py	116	2
dragontools/gui/tab_manager.py	214	2
dragontools/subtitle/extractor.py	47	2
dragontools/subtitle/injector.py	128	2
dragontools/worker/converter_queue_state.py	117	2
dragontools/worker/output_verifier.py	369	2
dragontools/core/release_validation.py	807	1
dragontools/gui/encoder_settings_ui.py	343	1

Modul	direkt importiert von	eigene interne Imports
dragontools/gui/rules_language_editor.py	156	1
dragontools/worker/converter_detection.py	237	1
dragontools/worker/duration_repair_models.py	176	1
dragontools/worker/output_size_policy.py	147	1
dragontools/worker/replace_service.py	237	1
dragontools/worker/trickplay_service.py	452	1
dragontools/worker/worker_result_service.py	196	1
dragontools/core/disk_space.py	146	0
dragontools/core/german_title_variants.py	58	0
dragontools/core/move_report.py	240	0
dragontools/core/quality_tester.py	176	0
dragontools/core/run_summary.py	195	0
dragontools/core/system_shutdown.py	58	0
dragontools/gui/styles.py	64	0
dragontools/worker/__init__.py	1	0
dragontools/worker/archive_service.py	158	0
dragontools/worker/converter_utils.py	89	0
dragontools/worker/hdr10plus_bitstream_service.py	143	0
dragontools/worker/interfaces.py	109	0
dragontools/gui/main_window_actions.py	30	20
dragontools/gui/convert_widget.py	250	19
dragontools/gui/main_window.py	113	18
dragontools/worker/dv_remux_thread.py	465	10
dragontools/worker/mp4_remux_thread.py	489	9
dragontools/worker/dv_remux_components.py	454	8
dragontools/gui/rules_audio_tab.py	554	7
dragontools/gui/rules_dialog.py	90	6
dragontools/gui/subtitle_widget.py	576	6
dragontools/gui/audio_video_matcher_widget.py	368	5
dragontools/gui/movie_renamer_widget.py	119	5
dragontools/gui/quality_tester_widget.py	799	5
dragontools/gui/rules_subtitle_tab.py	216	5

Modul	direkt importiert von	eigene interne Imports
dragontools/worker/audio_video_match_thread.py	382	5
dragontools/worker/hdrplus_conversion.py	727	5
dragontools/worker/quality_test_thread.py	334	5
dragontools/gui/rules_series_tab.py	156	4
dragontools/worker/audio_mux_thread.py	311	4
dragontools/worker/converter_progress.py	457	4
dragontools/worker/converter_stream_args.py	317	4
dragontools/worker/merge_thread.py	448	4
dragontools/worker/pipeline_decision_service.py	158	4
dragontools/gui/convert_widget_override_dialog.py	805	3
dragontools/gui/media_info_dialog.py	254	3
dragontools/worker/dv_audio_mux_service.py	279	3
dragontools/worker/iso_thread.py	750	3
dragontools/gui/changelog_dialog.py	302	2
dragontools/gui/help_dialog.py	107	2
dragontools/gui/mp4_remux_widget.py	267	2
dragontools/gui/save_settings_dialog.py	14	2
dragontools/gui/tool_path_settings.py	86	2
dragontools/subtitle/tagger.py	48	2
dragontools/worker/converter_strip.py	146	2
dragontools/worker/duration_timing_analyzer.py	366	2
dragontools/core/media_hdr_detection.py	296	1
dragontools/gui/audio_muxer_widget.py	211	1
dragontools/gui/batch_preflight_dialog.py	255	1
dragontools/gui/convert_queue_window.py	222	1
dragontools/gui/iso_widget.py	427	1
dragontools/gui/job_resume_dialog.py	86	1
dragontools/gui/main_window_menus.py	251	1
dragontools/gui/merge_widget.py	347	1
dragontools/gui/online_metadata_dialog.py	367	1
dragontools/gui/profile_manager_dialog.py	152	1
dragontools/gui/rules_flags_tab.py	59	1

Modul	direkt importiert von	eigene interne Imports
dragontools/worker/dv_processing_pipeline.py	326	1
dragontools/worker/dv_failure_recovery.py	96	1
dragontools/worker/dv_pipeline_timeouts.py	40	1
dragontools/worker/subtitle_sidecar_service.py	379	1
dragontools/worker/media_analysis_service.py	34	1
dragontools/gui/rules_dialog_common.py	13	0
dragontools/gui/rules_regex_help.py	93	0
dragontools/gui/run_summary_dialog.py	162	0
dragontools/gui/shortcut_dialog.py	47	0
dragontools/worker/command_formatting.py	18	0
dragontools/worker/dv_mp4box_muxer.py	59	0
dragontools/worker/dv_rpu_service.py	52	0
dragontools/worker/dv_video_filters.py	121	0
dragontools/worker/encode_plan.py	14	0
dragontools/worker/worker_runtime_state.py	14	0
DragonToolsV9.py	446	4
dragontools/__init__.py	7	0
dragontools/gui/__init__.py	1	0
dragontools/subtitle/__init__.py	1	0
dragontools/worker/converter_config.py	79	0

27 Externe Programme

Programm	Pfad-/Diagnoselogik	Notwendigkeit
FFmpeg	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Encoding/Remux/Filter/Frames/Audio/Untertitel/Trickplay; zentral und in vielen Workflows erforderlich.
FFprobe	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Stream-/Format-/Daueranalyse; eng mit FFmpeg gekoppelt.
MediaInfo	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Erweiterte Medien-/HDR-/DV-Analyse; je nach Workflow ergänzend/fallback.

Programm	Pfad-/Diagnoselogik	Notwendigkeit
MKVToolNix (mkvmerge/mkvextract/mkvpropedit)	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	MKV-Remux, Extraktion, Untertitel/Flags, Dauerreparatur.
dovi_tool	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Dolby-Vision-RPU und Profil-/Metadatenbearbeitung; nur DV-Erhaltungsworkflow.
hdr10plus_tool	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	HDR10+-Metadaten extrahieren/injizieren; nur HDR10+-Erhaltung.
MP4Box/GPAC	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Finaler Dolby-Vision-MP4-Mux bzw. DV-Sonderpfad.
MakeMKV CLI (`makemkvcon`)	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	ISO/Disc-Extraktion, wenn verfügbar; ISO-Tab kann optional FFmpeg-Fallback verwenden.
HandBrake	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Externes Programm, aus Menü startbar; keine Kernabhängigkeit des Converters.
RenameMyTVSeries	Zentral über core/paths.py bzw. Tooldiagnose/Settings aufgelöst.	Extern startbar; Renamer-Funktionalität existiert zusätzlich nativ in DragonTools.

28 Pythonbibliotheken

Bibliothek	Rolle
PyQt6	GUI, QThread/Signals, QSettings; zwingend für die Desktop-App. In der Analyseumgebung nicht installiert, daher konnten PyQt-abhängige Tests nicht gesammelt werden.
NumPy	Optional/gezielt im Audio-Video-Matcher für numerische Bildsignaturen.
OpenCV (`cv2`)	Optionales Analysebackend des Audio-Video-Matchers; bei fehlendem OpenCV existiert ein Fallback.

Bibliothek	Rolle
pytest	Nur Tests/Entwicklung, nicht Laufzeit der ausgelieferten App.
pyi_splash	Optional im Frozen/PyInstaller-Startpfad für Splash-Steuerung.

29 Workerübersicht

Modul	Worker/Service	Aufgabe	Tools
archive_service.py	Service/Runner	dragontools/worker/archive_service.py Technischer Kontext aus dem Quellcode: dragontools/worker/archive_service.py Zentrale Archivierungsfunktion für Quelldateien, bevor Metadaten	intern
audio_mux_thread.py	AudioMuxThread	Worker für Audio-Muxer. Technischer Kontext aus dem Quellcode: dragontools/worker/audio_mux_thread.py Audio-Mux-Thread: Audio Mux für MKV Remux Dateien Audio: konvertieren (nach Au	ffmpeg
audio_video_match_thread.py	AudioVideoMatchThread	Führt den Audio-Video-Matcher im Hintergrund aus, damit die GUI bedienbar bleibt. Der Worker steuert Analyse, Synchronentscheidung, Schnitt-/Offset-Plan und die abschließende Datei	ffmpeg
base_worker.py	BaseWorker	dragontools/worker/base_worker.py Technischer Kontext aus dem Quellcode: dragontools/worker/base_worker.py Konvention für das Worker-Interface zwischen GUI und Worker-Threads. Kein	intern
cleanup_service.py	Service/Runner	Räumt temporäre Dateien und Sidecars auf.	intern
converter_thread.py	ConverterThread	Hauptworker für Batch-Konvertierung. Technischer Kontext aus dem Quellcode: dragontools/worker/converter_thread.py Schlanker Konvertierungs-Worker. Delegiert Spezialaufgaben an Hel	dovi_tool, ffmpeg, ffprobe, mediainfo
duration_repair_service.py	Service/Runner	Führt MKV-Remux und Timestamp-Reparatur aus.	ffmpeg, ffprobe, mkvmerge
dv_audio_mux_service.py	Service/Runner	Bereitet Audio für DV-MP4 vor.	ffprobe, mp4box

Modul	Worker/Service	Aufgabe	Tools
dv_remux_thread.py	DVRemuxThread	dragontools/worker/dv_remux_thread.py Technischer Kontext aus dem Quellcode: dragontools/worker/dv_remux_thread.py DV-Remux-Thread: containerabhängiger Remux für Dolby-Vision-Dateien (MP4/MKV). Audio: norma	ffmpeg, mp4box, mkvmerge
dv_rpu_service.py	Service/Runner	Kapselt dovi_tool für RPU-Extraktion und -Injection.	intern
encode_plan_service.py	Service/Runner	Enthält EncodePlanService.	mediainfo
hdr10plus_bitstream_service.py	Service/Runner	Kapselt hdr10plus_tool Extract/Inject/Verify.	hdr10plus_tool
interfaces.py	WorkerProtocol, PausableWorkerProtocol	Enthält WorkerProtocol, PausableWorkerProtocol, AnalysisService.	intern
iso_thread.py	ISOThread	MakeMKV-Worker für ISO und Disc-Ordner.	ffmpeg, makemkvcon
media_analysis_service.py	Service/Runner	Enthält MediaAnalysisService.	intern
merge_thread.py	MergeThread	Kompatibilitätsprüfung und verlustfreier Merge.	ffprobe, mkvmerge
move_thread.py	MoveThread	Verschiebt Dateien nach Zielregeln und fragt fehlende Ziele ab. Technischer Kontext aus dem Quellcode: dragontools/worker/move_thread.py – vollständig neu, robuste Film/Serien-Logi	ffmpeg
mp4_remux_thread.py	MP4RemuxThread	Einfacher MP4-Remux mit Schutz vor DV/HDR10+.	ffmpeg
output_path_service.py	Service/Runner	Enthält OutputPathService.	intern
parallel_converter_thread.py	ParallelConverterThread	Koordiniert mehrere normale ConverterThread-Instanzen für parallele Bearbeitung.	ffmpeg
pipeline_decision_service.py	Service/Runner	Enthält PipelineDecisionService.	intern
postprocess_service.py	Service/Runner	Erzeugt NFO und Trickplay nach erfolgreicher Konvertierung; seit V9.3 auch als Hintergrundauftrag.	ffmpeg, ffprobe
quality_test_thread.py	QualityTestThread	Führt die kurzen Qualitätstest-Encodes und technischen Messungen im Hintergrund aus.	ffprobe
replace_service.py	Service/Runner	Ersetzt Originale mit Rollback und Archivschutz.	intern

Modul	Worker/Service	Aufgabe	Tools
standard_pipeline_runner.py	Service/Runner	Führt Standard-FFmpeg-Encoding aus.	ffmpeg
subtitle_sidecar_service.py	Service/Runner	Exportiert Untertitel-Sidecars für DV/MP4-Sonderpfade. Technischer Kontext aus dem Quellcode: dragontools/worker/subtitle_sidecar_service.py Zentraler Export-Dienst für externe Sid	mediainfo
tool_runner.py	Service/Runner	Führt kleine Tool-Selbsttests und externe Kommandos kontrolliert aus.	intern
trickplay_service.py	Service/Runner	Erstellt Jellyfin-Trickplay-Spritebilder mit FFmpeg.	ffmpeg
worker_events.py	WorkerEvent	Enthält WorkerEvent.	intern
worker_result_service.py	WorkerConversionResultService	Enthält WorkerConversionResultService.	intern
worker_runtime_state.py	WorkerRuntimeState	Enthält WorkerRuntimeState.	intern
workflow_engine.py	Service/Runner	Orchestriert Analyse, Plan, Prozess, Verify, Replace, Cleanup. Technischer Kontext aus dem Quellcode: Gemeinsame Workflow-Engine für Konvertierungsjobs. Ablauf: 1) analyze 2) build	intern
workflow_services.py	Service/Runner	Bündelt Workflow-Fachdienste und schreibt Fehlerberichte.	ffmpeg
dv_subtitle_mux_service.py	Service/Runner	Globale Untertitelablage für DV-MP4/DV-MKV; mov_text oder Sidecar nach Containerpolicy.	ffmpeg, mp4box, mkvmerge
hdrplus_tool_runner.py	HDRPlusToolRunner	Zentrale HDR10+-Toolausführung mit accepted_returncodes, Timeout und Diagnose.	hdr10plus_tool, ffmpeg, ffprobe
hdrplus_mux_service.py	HDRPlusMuxService	Finaler HDR10+-MKV/MP4-Mux; Audioanalyse fail-closed.	ffprobe, ffmpeg, mkvmerge, mp4box

30 Einstellungen und Defaultwerte

Kompaktindex. Die ausführliche Wirkung steht in Kapitel 17.

Einstellung	Default	Wertebereich	Auswirkung

Einstellung	Default	Wertebereich	Auswirkung
NVENC Preset	p6	p1–p7	p1 priorisiert Tempo; p7 priorisiert Encoderanalyse/Kompression. p6 ist im Projekt der Qualitäts-Default.
NVENC CQ	23	0–63	Niedriger = höhere Qualität/größere Datei. Der Code nutzt VBR + `-cq` + `-b:v 0`.
NVENC B-Frames	4	0–8	Mehr B-Frames können Kompression verbessern; 0 deaktiviert zugleich den rc-lookahead-Zweig im Command Builder.
NVENC B-Ref-Mode	middle	disabled / each / middle	Wird nur bei B-Frames > 0 und H.264/H.265 gesetzt. disabled ist konservativer; Referenz-B-Frames können Effizienz erhöhen.
NVENC RC Lookahead	32	0–64	Anzahl voraus analysierter Frames. 0 deaktiviert die Lookahead-Argumente; höhere Werte benötigen mehr Analyse-/Pufferressourcen.
NVENC Lookahead-Level	Auto	Auto / 0 / 1 / 2 / 3	Auto setzt keinen Parameter. 0–3 werden als `-lookahead_level` übergeben; höhere Stufen können Qualität verbessern, kosten Performance. Der FFmpeg-Code definiert genau 0–3 plus Auto.
NVENC Multipass	Auto	Auto / disabled / qres / fullres	Auto setzt keinen Parameter. disabled=Single-Pass, qres=erster Pass in Viertelauflösung, fullres=erster Pass in voller Auflösung; fullres ist ressourcenintensiver.

Einstellung	Default	Wertebereich	Auswirkung
NVENC AQ-Stärke	8	1–15	Nur zusammen mit Spatial AQ gesetzt. Höher verschiebt Bitrate aggressiver nach räumlicher Komplexität.
NVENC Spatial AQ	an	an/aus	Setzt `-spatial_aq 1` und AQ-Stärke; soll komplexen Bildbereichen mehr Bits geben.
NVENC Temporal AQ	an	an/aus	Setzt `-temporal_aq 1`; beeinflusst Bitverteilung über die Zeit.
QSV Preset	erster Eintrag/UI; Profile können setzen	veryfast … veryslow	Geschwindigkeit vs. Kompression.
QSV Q	23	0–63	Qualitätsziel; niedriger = besser/größer.
QSV Lookahead-Tiefe	40	1–100	DragonTools setzt QSV Lookahead immer aktiv und übergibt die Tiefe.
AMF QP	23	0–63	Wird für I/P und bei Nicht-AV1 auch B verwendet.
AMF Qualität	balanced	speed / balanced / quality	Steuert AMF-Qualitäts-/Geschwindigkeitsmodus.
x265 AQ-Mode	2	0–3	0 aus; höhere Modi aktivieren komplexere adaptive Quantisierung.
x265 AQ-Stärke	1.0	0–3	Stärke der adaptiven Quantisierung.
x265 psy-rd	2.0	0–5	Psychovisuelle Gewichtung; höhere Werte erhalten Struktur stärker, können Bitrate erhöhen.
x265 psy-rdoq	1.0	0–50	Psychovisuelle RDOQ-Gewichtung.
x265 B-Frames	8	0–16	Mehr Referenzmöglichkeiten, höhere Analysekomplexität.
x265 Lookahead	40	0–250	Nur bei B-Frames > 0 als `rc-lookahead` in x265-params gesetzt.

Einstellung	Default	Wertebereich	Auswirkung
x265 Tune	none	none/grain/animation/ssim/psnr	Spezialoptimierung; verändert Encoderentscheidungen und sollte nur bewusst gewählt werden.
Metadaten Filme – Provider	Default TMDB	TMDB / TheTVDB / Beide	siehe Kapitel 19
Metadaten Filme – bevorzugt	Default TMDB; nur bei Beide aktiv	TMDB / TheTVDB	siehe Kapitel 19
Metadaten Serien – Provider	Default TMDB	TMDB / TheTVDB / Beide	siehe Kapitel 19
Metadaten Serien – bevorzugt	Default TMDB; nur bei Beide aktiv	TMDB / TheTVDB	siehe Kapitel 19
Metadaten TMDB verwenden	Default aus, Zugangsdaten erforderlich	an/aus	siehe Kapitel 19
Metadaten TheTVDB verwenden	Default aus, Zugangsdaten erforderlich	an/aus	siehe Kapitel 19
Metadaten Sprache	Default de-DE	freie Combo, Vorschläge de-DE/en-US/ja-JP/fr-FR	siehe Kapitel 19
Metadaten Fallback-Sprache	Default en-US	freie Combo	siehe Kapitel 19
Metadaten Cache	Default an	an/aus	siehe Kapitel 19
Metadaten Cache-Alter	Default 30 Tage	1–365 Tage	siehe Kapitel 19
Auto-Crop Modus	single	siehe Kapitel 17	Ein Analysefenster; Probe ab 30 s, 45 s Dauer. Alternativ können andere Modi/Intervalle in Settings existieren.
Auto-Crop Probe-Intervall	600 s	siehe Kapitel 17	Bei wiederholter Analyse größere Werte = weniger Analyseaufwand, geringere Chance kurze Wechsel zu sehen.
IMAX Probe-Intervall	90 s	siehe Kapitel 17	Kürzer = mehr Prüfpunkte und mehr Analysezeit.
IMAX Probe-Dauer	3 s	siehe Kapitel 17	Längere Fenster sind robuster, aber teurer.

Einstellung	Default	Wertebereich	Auswirkung
IMAX Mindestvarianz	15 %	siehe Kapitel 17	Schwellwert für wechselnde Bildhöhe/-fläche.
IMAX Mindesttreffer	2	siehe Kapitel 17	Verhindert Aktivierung durch einen Einzelmessfehler.
Output Mindestgröße	1 KiB	siehe Kapitel 17	Technischer Null-/Leerausgabe-Schutz vor weiteren Prüfungen.
Dauer Minimum	90 %	siehe Kapitel 17	Zu kurze Ausgabe löst Reparatur/Fehlerpfad aus.
Dauer Maximum	125 % + 60 s Toleranz	siehe Kapitel 17	Begrenzt unrealistisch lange Ausgabe; zusätzliche absolute Toleranz reduziert Fehlalarme.
Duration Remux	an	siehe Kapitel 17	Erster Reparaturpfad bei plausibler MKV-Dauerproblematik.
Timestamp-Reparatur	an	siehe Kapitel 17	Zweiter, gezielter Reparaturpfad bei erkanntem Timestamp-Muster.
CPU Paralleljobs	1	siehe Kapitel 17	Konservativ wegen hoher CPU-Last pro Softwareencode.
GPU Paralleljobs	2	siehe Kapitel 17	Nutzt GPU-Parallelität ohne aggressiven Default; max. 8 einstellbar.
NFO	aus	siehe Kapitel 17	Postprocessing ist opt-in; nur eindeutige Treffer standardmäßig true, fileinfo true, movie.nfo.
Trickplay	aus	siehe Kapitel 17	Opt-in; 320 px, 10x10 Tiles, 10 s Intervall, JPEG 90/qscale 4, CUDA, 1 Job, Output als Quelle, Konflikt skip.
Quellbildprüfung	an	siehe Kapitel 17	10-%-Intervalle, 2 s Sample, 2 fps; blockiert bei 80 % auffälligen Frames und mindestens 4 Treffern.

31 Zentrale Datenflüsse

Workflow	Datenfluss

Workflow	Datenfluss
Normale Konvertierung	Datei → MediaAnalysisService/MediaAnalyzer → PipelineDecisionService → EncodePlanService → Standard/DV/HDR10+/AV1-Metadaten-Runner → OutputVerifier → ggf. DurationRepair → Replace/Cleanup → Ergebnis/Postprocessing → Move/Report.
Audio	Streammetadaten → Sprach-/Kommentarfilter → Kanalregel/Codec-/Bitratenfenster → Copy/Transcode/Downmix → FFmpeg-Args → Output.
Untertitel	Streams → Sprache/Forced/Format → Plausibilität → Burn-/Keep-Plan → Video-Filter oder Streamcopy/Sidecar → Ausgabe.
Metadaten	Dateiname → Parser → Providerkette → Cache/HTTP → lokalisierte Normalisierung → Suggestion → Renamer/NFO/GUI.
ISO	Image/Disc-Ordner → Struktur-/Titelanalyse → MakeMKV bzw. FFmpeg-Fallback → Extraktion → optional Converter-Handoff.
Audio-Video-Matcher	deutsche Quelle + Zielvideo → ffprobe/Frame-Signaturen → gemeinsame Bereiche/Offset/Drift/Schnittfall → Audio-Transformationsplan → Zieldatei.
Postprocessing	validierte/ersetzte Datei → 🧩 → Metadaten/NFO + Trickplay → Sidecarregistrierung → ✅ → Batchabschluss/Move/Report.

32 Dateistrukturen

Serienbeispiel:

```
Serie/
├── Staffel 01/
│   ├── Serie - S01E01 - Titel.mkv
│   └── Serie - S01E02 - Titel.mkv
└── Staffel 00/
    └── Serie - S00E01 - Special.mkv
```

ISO/Disc-Eingaben: `Film.iso`, `Film.img`, Disc-Root mit `BDMV/`, direkt `BDMV/`, DVD-Root mit `VIDEO_TS/` oder direkt `VIDEO_TS/`.

Log-/Reportdaten werden unter den von DragonTools ermittelten Dokument-/Logpfaden in getrennten Unterordnern bzw. Monatsstrukturen abgelegt. MetadataCache trennt TMDB und TheTVDB providerbezogen.

33 Fehler und Troubleshooting

Symptom	Maßnahme	Hintergrund

Symptom	Maßnahme	Hintergrund
Encoding einer einzelnen Datei soll beendet werden, Restqueue weiter	Aktiver Queue-Eintrag → Kontextmenü → „FFmpeg für diese laufende Datei beenden“.	DragonTools prüft, ob genau diese Datei aktiv ist und ob der aktuelle Prozess wirklich ffmpeg ist. Dann wird nur dessen Prozessbaum beendet; kein globales Abort-Flag.
TheTVDB liefert unerwünschte/anderssprachige Treffer	Sprache/Fallback und Providerpriorität prüfen.	Bei „Beide“ zuerst den gewünschten Provider als „Bevorzugt“ setzen. TheTVDB-Titel werden aus mehreren v4-Lokalisierungsformen gelesen; Fallbacksprache kann greifen.
TMDB/TheTVDB-Zugangsdaten nicht sichtbar	Gemeinsame Checkbox „Zugangsdaten anzeigen (TMDB + TheTVDB)“ aktivieren.	Sie schaltet alle Provider-Geheimfelder gemeinsam von Password auf Normal.
NVENC `lookahead_level` ungültig	Nur Auto oder 0–3 verwenden.	Der Command Builder akzeptiert ausschließlich 0,1,2,3 und setzt bei Auto gar keinen Parameter.
NVENC Multipass wird nicht erwartet	Auto wählen, wenn FFmpeg/NVENC-Default gelten soll.	qres/fullres/disabled werden explizit als `-multipass` gesetzt; Auto unterdrückt die Option.
Output-Dauer unplausibel	Logs/ErrorReport auf Verifier und DurationRepair prüfen.	Zuerst normaler Remux, danach nur bei passendem Muster Timestamp-Reparatur; anschließend erneute Validierung.
ISO-Ordner statt Image	Disc-Ordner hinzufügen; BDMV/VIDEO_TS oder Root akzeptiert.	ISOThread analysiert Struktur; MakeMKV-Pfad und optionaler FFmpeg-Fallback entscheiden die Extraktion.
Postprocessing-NFO/Trickplay fehlt	Finalen ✅-Status und Postprocess-Log prüfen.	🧩 bedeutet: Kernkonvertierung ersetzt, Nacharbeit läuft noch. Batchabschluss wartet auf offene Nacharbeit.
Tool wird nicht gefunden	F9/Systemtest, Tool-Pfadsettings, PATH und gebündelte Ordner prüfen.	Zentrale ToolPaths-Logik soll verhindern, dass GUI und Worker unterschiedliche Binaries verwenden.
Testlauf startet in dieser Dokumentationsumgebung nicht	PyQt6 fehlt.	Projektquellcode kompiliert statisch; Runtime-/GUI-Tests benötigen die Projektabhängigkeiten.

34 Glossar

Begriff	Erklärung
CRF	Constant Rate Factor; Qualitätssteuerung von Softwareencodern. Niedriger = höhere Qualität/größere Datei.
CQ	NVENC-Qualitätsziel in DragonTools; niedriger = höhere Qualität/größere Datei.
NVENC	NVIDIA Hardwareencoder in FFmpeg.
RC Lookahead	Anzahl voraus betrachteter Frames für Rate-Control-Entscheidungen.
Lookahead-Level	Separate NVENC-Qualitätsstufe 0–3 plus Auto; nicht mit der Frameanzahl verwechseln.
Multipass	NVENC Mehrpassmodus: disabled, qres oder fullres.
Remux	Streams ohne Video-Neuencoding in einen anderen/neuen Container verpacken.
Transcoding	Dekodieren und erneutes Kodieren eines Streams.
Codec	Kodierverfahren eines Streams, z. B. H.265/E-AC3.
Container	Dateiformat, das Streams kapselt, z. B. MKV/MP4.
Stream	Einzelner Video-, Audio-, Untertitel- oder Datenstrom innerhalb eines Containers.
Forced Subtitle	Untertitel für Passagen, die auch bei ansonsten passender Audiosprache eingeblendet werden sollen.
PGS	Blu-ray-Bilduntertitelformat.
VobSub	DVD-basierter Bilduntertitel.
Worker	Hintergrundkomponente für langlaufende Aufgaben.
Thread	Ausführungspfad; in Qt häufig QThread für Hintergrundarbeit.
Signal/Slot	Qt-Kommunikationsmechanismus zwischen Objekten/Threads.
Sidecar	Begleitdatei neben dem Video, z. B. .srt oder .nfo.
NFO	XML-artige Metadatendatei für Medienserver wie Jellyfin/Kodi.
Trickplay	Vorschaubilder/Sprites für schnelles Scrubbing in Medienservern.
DV/RPU	Dolby Vision; RPU enthält dynamische Dolby-Vision-Metadaten.
HDR10+	HDR-Format mit dynamischen Metadaten.

Begriff	Erklärung
Preflight	Vorabprüfung und Planungsdialog vor eigentlicher Verarbeitung/Verschiebung.
Fallback	Alternativer Pfad/Dienst, wenn der bevorzugte Weg nicht erfolgreich ist.

35 Aktuelles Quellinventar

Inventur 07.10.2026 für Version 9.9.1. 1.297 Python-Dateien/Programme, 208.072 Gesamtzeilen und 175.787 Codezeilen (nichtleer, keine reinen Kommentarzeilen). Testpakete: 375 Python-Dateien, davon 369 test_*.py mit 3.287 statisch erkannten Testfunktionen. Patch-29-Abnahme: 5.175 bestanden (5.152 Gesamtsuite + 23 native DV/HDR-Tests).

Die folgende Referenz nennt alle gepflegten Python-Dateien mit aktuellen Gesamt- und Codezeilen. Enthalten sind DragonTools, der eigenständige HDR10+-Generator und gepflegte Extras. Externe Programme, virtuelle Umgebungen, Build-Ausgaben und Cache-Dateien sind ausgeschlossen. Nichtleere Zeilen ohne reine Kommentare zählen als Codezeilen; Docstrings bleiben enthalten. Die Modulbeschreibungen mit Zeilenangaben sind auf dieses Inventar aktualisiert; ausdrücklich datierte ältere Abnahmen bleiben historische Referenzen.

35.1 Einstiegspunkt

Datei	Gesamtzeilen	Codezeilen
DragonToolsV9.py	502	379

35.2 GUI

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/__init__.py	1	0
dragontools/gui/application_shutdown.py	134	104
dragontools/gui/application_worker_sources.py	25	22
dragontools/gui/audio_muxer_widget.py	221	175
dragontools/gui/audio_video_matcher_paths.py	63	51
dragontools/gui/audio_video_matcher_results.py	108	94
dragontools/gui/audio_video_matcher_runtime.py	104	90
dragontools/gui/audio_video_matcher_view.py	162	146
dragontools/gui/audio_video_matcher_widget.py	32	25
dragontools/gui/batch_preflight_dialog.py	258	227
dragontools/gui/bitmap_subtitle_ocr_review.py	110	95
dragontools/gui/changelog_dialog.py	295	252
dragontools/gui/conversion_controller.py	167	138
dragontools/gui/conversion_diagnostics.py	121	110
dragontools/gui/conversion_job_journal_tracker.py	49	42
dragontools/gui/conversion_progress_display.py	107	94
dragontools/gui/conversion_progress_focus.py	112	102
dragontools/gui/conversion_progress_presenter.py	138	122
dragontools/gui/conversion_queue_admission.py	60	49
dragontools/gui/conversion_result_file_events.py	199	171
dragontools/gui/conversion_result_finish.py	133	117
dragontools/gui/conversion_result_service.py	67	56
dragontools/gui/conversion_run_finalizer.py	209	191
dragontools/gui/conversion_run_reporting.py	53	45
dragontools/gui/conversion_session_state.py	264	210
dragontools/gui/conversion_start_coordinator.py	293	258
dragontools/gui/conversion_worker_factory.py	181	158
dragontools/gui/conversion_worker_lifecycle.py	288	260
dragontools/gui/convert_override_groups.py	244	226
dragontools/gui/convert_override_lifecycle.py	81	59
dragontools/gui/convert_override_state.py	92	78

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/convert_override_tracks.py	192	173
dragontools/gui/convert_queue_window.py	289	252
dragontools/gui/convert_widget.py	327	260
dragontools/gui/convert_widget_composition.py	222	205
dragontools/gui/convert_widget_custom_widgets.py	361	270
dragontools/gui/convert_widget_encoder_apply.py	36	27
dragontools/gui/convert_widget_encoder_controls.py	126	109
dragontools/gui/convert_widget_encoder_override.py	137	120
dragontools/gui/convert_widget_encoder_panels.py	138	123
dragontools/gui/convert_widget_encoder_state.py	85	72
dragontools/gui/convert_widget_file_queue.py	321	284
dragontools/gui/convert_widget_host_actions.py	66	55
dragontools/gui/convert_widget_layout.py	65	61
dragontools/gui/convert_widget_layout_components.py	157	133
dragontools/gui/convert_widget_layout_options.py	237	221
dragontools/gui/convert_widget_layout_runtime.py	256	224
dragontools/gui/convert_widget_override_apply.py	53	42
dragontools/gui/convert_widget_override_dialog.py	323	286
dragontools/gui/convert_widget_parallel_controls.py	83	72
dragontools/gui/convert_widget_paths.py	67	57
dragontools/gui/convert_widget_pipeline_preview.py	154	139
dragontools/gui/convert_widget_queue_actions.py	35	29
dragontools/gui/convert_widget_queue_add.py	170	149
dragontools/gui/convert_widget_queue_badges.py	137	123
dragontools/gui/convert_widget_queue_context_actions.py	250	222
dragontools/gui/convert_widget_queue_dragdrop.py	79	64
dragontools/gui/convert_widget_queue_management.py	61	47
dragontools/gui/convert_widget_queue_override_actions.py	184	159
dragontools/gui/convert_widget_queue_remove.py	175	155
dragontools/gui/convert_widget_queue_reorder_actions.py	50	41
dragontools/gui/convert_widget_queue_target_actions.py	143	121
dragontools/gui/convert_widget_queue_window_actions.py	63	55
dragontools/gui/convert_widget_quick_settings.py	146	133
dragontools/gui/convert_widget_recovery.py	188	171
dragontools/gui/convert_widget_runtime_ui.py	89	74
dragontools/gui/convert_widget_source_visual_actions.py	128	109
dragontools/gui/convert_widget_watch_intake.py	193	170
dragontools/gui/dialog_ownership.py	12	10
dragontools/gui/drop_path_decode.py	58	49
dragontools/gui/drop_path_extractor.py	321	250
dragontools/gui/drop_path_files.py	25	20
dragontools/gui/drop_path_windows.py	91	80
dragontools/gui/dv_crop_dialog.py	37	33
dragontools/gui/encoder_profile_application.py	69	64
dragontools/gui/encoder_profile_options.py	82	67
dragontools/gui/encoder_profile_service.py	136	123
dragontools/gui/encoder_settings_controller.py	41	35
dragontools/gui/encoder_settings_options.py	189	181

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/encoder_settings_panels.py	106	92
dragontools/gui/encoder_settings_persistence.py	179	167
dragontools/gui/encoder_settings_profiles.py	126	107
dragontools/gui/encoder_settings_state.py	24	20
dragontools/gui/encoder_settings_ui.py	347	330
dragontools/gui/external_program_launch.py	33	29
dragontools/gui/file_drop_widgets.py	41	29
dragontools/gui/file_list_queue_index.py	94	80
dragontools/gui/file_worker_pause.py	71	60
dragontools/gui/help_dialog.py	108	86
dragontools/gui/info_button.py	42	35
dragontools/gui/iso_widget.py	48	41
dragontools/gui/iso_widget_inputs.py	102	89
dragontools/gui/iso_widget_runtime.py	214	184
dragontools/gui/iso_widget_view.py	132	115
dragontools/gui/jellyfin_connection_test.py	33	26
dragontools/gui/jellyfin_refresh_dispatch.py	216	178
dragontools/gui/job_resume_dialog.py	86	71
dragontools/gui/journal_resume_base.py	54	44
dragontools/gui/log_zoom_window.py	71	63
dragontools/gui/main_window.py	111	90
dragontools/gui/main_window_actions.py	30	24
dragontools/gui/main_window_backup_actions.py	243	224
dragontools/gui/main_window_convert_actions.py	101	85
dragontools/gui/main_window_help_actions.py	168	141
dragontools/gui/main_window_menus.py	266	247
dragontools/gui/main_window_metadata_actions.py	173	142
dragontools/gui/main_window_profile_actions.py	68	57
dragontools/gui/main_window_recovery.py	296	262
dragontools/gui/main_window_settings_actions.py	148	108
dragontools/gui/main_window_shutdown.py	87	72
dragontools/gui/main_window_system_actions.py	104	91
dragontools/gui/main_window_tabs.py	426	356
dragontools/gui/media_info_dialog.py	311	266
dragontools/gui/media_info_text_builder.py	390	348
dragontools/gui/media_library_dialog.py	363	288
dragontools/gui/media_library_dialog_contracts.py	46	43
dragontools/gui/media_library_dialog_options.py	114	109
dragontools/gui/media_library_dialog_presenter.py	141	128
dragontools/gui/media_library_dialog_service.py	264	230
dragontools/gui/media_library_dialog_view.py	107	89
dragontools/gui/media_library_fix_actions.py	45	31
dragontools/gui/media_library_fix_controller.py	329	296
dragontools/gui/media_library_fix_tab.py	200	180
dragontools/gui/media_library_fix_worker.py	78	62
dragontools/gui/media_library_maintenance_controller.py	150	138
dragontools/gui/media_library_mapping_controller.py	74	61
dragontools/gui/media_library_mapping_tab.py	51	45

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/media_library_nfo_controller.py	111	92
dragontools/gui/media_library_nfo_scan_controller.py	87	72
dragontools/gui/media_library_scan_controller.py	110	92
dragontools/gui/media_library_search_controller.py	323	292
dragontools/gui/media_library_search_tab.py	120	105
dragontools/gui/media_library_search_worker.py	34	26
dragontools/gui/media_library_sql_tab.py	78	68
dragontools/gui/media_library_status_tab.py	148	131
dragontools/gui/merge_widget.py	355	313
dragontools/gui/move_incremental_lifecycle.py	248	227
dragontools/gui/move_journal_recovery_thread.py	65	51
dragontools/gui/move_lifecycle_coordinator.py	72	59
dragontools/gui/move_lifecycle_helpers.py	87	70
dragontools/gui/move_preflight_controller.py	131	103
dragontools/gui/move_preflight_workflow.py	156	143
dragontools/gui/move_regular_lifecycle.py	250	226
dragontools/gui/move_request_dialogs.py	278	248
dragontools/gui/move_resume_dialog.py	61	48
dragontools/gui/movie_renamer_actions.py	226	197
dragontools/gui/movie_renamer_browser_mapping.py	267	243
dragontools/gui/movie_renamer_browser_runtime.py	102	93
dragontools/gui/movie_renamer_candidate_combo.py	80	63
dragontools/gui/movie_renamer_commit_runtime.py	150	130
dragontools/gui/movie_renamer_completeness_dialog.py	110	101
dragontools/gui/movie_renamer_completeness_runtime.py	150	132
dragontools/gui/movie_renamer_job_queue.py	67	56
dragontools/gui/movie_renamer_metadata_browser.py	541	476
dragontools/gui/movie_renamer_resolve_runtime.py	128	119
dragontools/gui/movie_renamer_resolve_search.py	103	88
dragontools/gui/movie_renamer_resolver.py	143	128
dragontools/gui/movie_renamer_search_actions.py	67	58
dragontools/gui/movie_renamer_season_prompt.py	179	154
dragontools/gui/movie_renamer_series_override_state.py	87	74
dragontools/gui/movie_renamer_table_controller.py	311	275
dragontools/gui/movie_renamer_table_search.py	65	54
dragontools/gui/movie_renamer_view.py	231	180
dragontools/gui/movie_renamer_view_state.py	147	128
dragontools/gui/movie_renamer_widget.py	166	139
dragontools/gui/movie_renamer_year_edit.py	58	52
dragontools/gui/mp4_remux_widget.py	273	232
dragontools/gui/online_metadata_action_workers.py	88	71
dragontools/gui/online_metadata_dialog.py	28	22
dragontools/gui/online_metadata_dialog_state.py	95	84
dragontools/gui/online_metadata_dialog_view.py	193	173
dragontools/gui/online_metadata_settings_state.py	119	103
dragontools/gui/preflight_dialog.py	283	254
dragontools/gui/preflight_film_hints.py	62	45
dragontools/gui/preflight_film_widget.py	220	191

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/preflight_metadata.py	104	92
dragontools/gui/preflight_metadata_apply.py	13	10
dragontools/gui/preflight_metadata_commands.py	51	43
dragontools/gui/preflight_metadata_common.py	120	96
dragontools/gui/preflight_metadata_movie.py	73	62
dragontools/gui/preflight_metadata_series.py	59	51
dragontools/gui/preflight_metadata_series_resolver.py	148	125
dragontools/gui/preflight_metadata_series_sources.py	85	69
dragontools/gui/preflight_series_choices.py	89	73
dragontools/gui/preflight_series_metadata.py	190	169
dragontools/gui/preflight_series_paths.py	119	101
dragontools/gui/preflight_series_view.py	163	140
dragontools/gui/preflight_series_widget.py	41	35
dragontools/gui/preflight_view.py	127	100
dragontools/gui/preflight_widget_common.py	56	44
dragontools/gui/preflight_widgets.py	15	12
dragontools/gui/profile_manager_dialog.py	174	144
dragontools/gui/qt_receiver_state.py	7	5
dragontools/gui/quality_file_compare_dialog.py	301	267
dragontools/gui/quality_tester_execution.py	90	78
dragontools/gui/quality_tester_files.py	77	65
dragontools/gui/quality_tester_run_config.py	187	173
dragontools/gui/quality_tester_run_dialog.py	316	281
dragontools/gui/quality_tester_widget.py	172	151
dragontools/gui/quality_worker_lifecycle.py	114	90
dragontools/gui/queue_ordering.py	77	64
dragontools/gui/release_frozen_widgets.py	20	20
dragontools/gui/replacement_reminder_dialog.py	129	111
dragontools/gui/rules_audio_tab.py	29	23
dragontools/gui/rules_audio_tab_channels.py	202	190
dragontools/gui/rules_audio_tab_sections.py	215	195
dragontools/gui/rules_audio_tab_state.py	118	109
dragontools/gui/rules_dialog.py	110	99
dragontools/gui/rules_dialog_common.py	13	5
dragontools/gui/rules_dialog_storage.py	61	49
dragontools/gui/rules_flags_tab.py	58	45
dragontools/gui/rules_language_editor.py	156	137
dragontools/gui/rules_regex_help.py	93	77
dragontools/gui/rules_renamer_tab.py	254	223
dragontools/gui/rules_series_tab.py	156	134
dragontools/gui/rules_subtitle_tab.py	286	258
dragontools/gui/run_summary_dialog.py	162	139
dragontools/gui/save_settings_dialog.py	14	10
dragontools/gui/settings_dialog.py	198	170
dragontools/gui/settings_sections/__init__.py	17	15
dragontools/gui/settings_sections/automation.py	157	138
dragontools/gui/settings_sections/base.py	33	22
dragontools/gui/settings_sections/comfyui_fields.py	115	98

Datei	Gesamtzeilen	Codezeilen
dragontools/gui/settings_sections/jellyfin.py	239	212
dragontools/gui/settings_sections/media.py	382	345
dragontools/gui/settings_sections/notifications.py	90	75
dragontools/gui/settings_sections/runtime.py	388	341
dragontools/gui/settings_sections/safety.py	217	199
dragontools/gui/settings_sections/storage.py	277	247
dragontools/gui/settings_sections/video.py	383	361
dragontools/gui/shortcut_dialog.py	47	38
dragontools/gui/shutdown_worker_state.py	48	37
dragontools/gui/styles.py	64	58
dragontools/gui/subtitle_widget.py	391	326
dragontools/gui/subtitle_widget_files.py	54	45
dragontools/gui/subtitle_widget_workers.py	350	303
dragontools/gui/tab_lazy_loading.py	130	106
dragontools/gui/tab_manager.py	195	150
dragontools/gui/timeout_settings_dialog.py	318	239
dragontools/gui/tool_diagnostics_dialog.py	28	20
dragontools/gui/tool_path_live_check.py	110	94
dragontools/gui/tool_path_settings.py	103	86
dragontools/gui/ui_helpers.py	98	80
dragontools/gui/update_checker.py	68	55
dragontools/gui/utility_worker_start.py	95	75
dragontools/gui/video_file_input.py	136	111
dragontools/gui/watch_folder_controller.py	248	209
dragontools/gui/watch_folder_intake.py	51	45
dragontools/gui/watch_folder_main_window_bridge.py	94	75
dragontools/gui/watch_folder_queue_ownership.py	13	11
dragontools/gui/watch_folder_rule_dialog.py	113	95
dragontools/gui/windows_notification_backend.py	56	47
dragontools/gui/windows_restart_guard.py	246	214

35.3 Core

Datei	Gesamtzeilen	Codezeilen
dragontools/core/__init__.py	1	0
dragontools/core/analysis_process.py	76	63
dragontools/core/audio_sync_planner.py	343	313
dragontools/core/audio_sync_validation.py	63	53
dragontools/core/audio_titles.py	43	36
dragontools/core/audio_video_frame_analysis.py	362	324
dragontools/core/audio_video_match_identity.py	12	10
dragontools/core/audio_video_match_models.py	154	123
dragontools/core/audio_video_match_services.py	260	238
dragontools/core/audio_video_match_utils.py	74	61
dragontools/core/audio_video_matcher.py	66	61
dragontools/core/audio_video_sampling_plan.py	19	17
dragontools/core/audio_video_stream_timing.py	48	38
dragontools/core/audio_video_time_mapping.py	98	89

Datei	Gesamtzeilen	Codezeilen
dragontools/core/audio_video_time_mapping_edges.py	69	58
dragontools/core/audio_video_time_mapping_fit.py	116	98
dragontools/core/audit_log.py	122	103
dragontools/core/batch_preflight.py	34	30
dragontools/core/batch_preflight_decisions.py	303	260
dragontools/core/batch_preflight_formatting.py	222	189
dragontools/core/batch_preflight_report.py	66	55
dragontools/core/batch_preflight_rows.py	224	191
dragontools/core/batch_preflight_storage.py	224	188
dragontools/core/bitmap_subtitle_ocr.py	387	338
dragontools/core/callback_dispatch.py	50	35
dragontools/core/changelog.py	228	195
dragontools/core/codec_profile_assistant.py	368	349
dragontools/core/codec_utils.py	65	51
dragontools/core/comfyui_hdr_models.py	251	207
dragontools/core/comfyui_timing.py	26	21
dragontools/core/comfyui_workflow.py	212	176
dragontools/core/config_migration.py	391	332
dragontools/core/conversion_artifacts.py	111	92
dragontools/core/conversion_notifications.py	99	81
dragontools/core/crash_guard.py	372	310
dragontools/core/crash_state_files.py	6	4
dragontools/core/diagnostic_package.py	280	227
dragontools/core/diagnostic_privacy.py	44	37
dragontools/core/diagnostic_redaction.py	193	162
dragontools/core/disk_space.py	156	127
dragontools/core/encoder_profile_override.py	283	231
dragontools/core/episode_replacement_policy.py	52	41
dragontools/core/error_report.py	357	303
dragontools/core/exclusive_text.py	24	21
dragontools/core/file_override_normalization.py	267	232
dragontools/core/file_update_lock.py	54	49
dragontools/core/formatting.py	22	16
dragontools/core/german_title_variants.py	58	46
dragontools/core/gpu_detection.py	175	136
dragontools/core/gui_error_report.py	67	55
dragontools/core/hdr10plus_generation.py	229	184
dragontools/core/hdr10plus_json_validation.py	126	111
dragontools/core/jellyfin_api.py	250	217
dragontools/core/jellyfin_full_scan_guard.py	69	51
dragontools/core/jellyfin_nfo.py	421	367
dragontools/core/jellyfin_paths.py	16	12
dragontools/core/jellyfin_refresh_service.py	304	262
dragontools/core/job_journal.py	276	246
dragontools/core/job_journal_resume.py	78	69
dragontools/core/job_journal_storage.py	134	106
dragontools/core/job_resume_selection.py	62	51
dragontools/core/journal_archive.py	54	49

Datei	Gesamtzeilen	Codezeilen
dragontools/core/journal_runtime.py	90	77
dragontools/core/json_io.py	84	67
dragontools/core/lang_codes.py	334	275
dragontools/core/language_detection.py	274	231
dragontools/core/log_cleanup.py	167	142
dragontools/core/logger.py	219	172
dragontools/core/logger_messages.py	343	305
dragontools/core/logger_paths.py	92	71
dragontools/core/logger_verbose.py	170	141
dragontools/core/matroska_writer.py	46	40
dragontools/core/media_analyzer.py	138	121
dragontools/core/media_analyzer_audio_streams.py	94	87
dragontools/core/media_analyzer_dynamic_hdr.py	66	54
dragontools/core/media_analyzer_io.py	241	202
dragontools/core/media_analyzer_metadata.py	177	155
dragontools/core/media_analyzer_result.py	71	67
dragontools/core/media_analyzer_streams.py	21	18
dragontools/core/media_analyzer_subtitle_streams.py	169	152
dragontools/core/media_analyzer_video_streams.py	218	190
dragontools/core/media_duration.py	127	99
dragontools/core/media_hdr_detection.py	411	351
dragontools/core/media_library.py	93	89
dragontools/core/media_library_analysis_merge.py	21	15
dragontools/core/media_library_classification.py	143	121
dragontools/core/media_library_db.py	259	217
dragontools/core/media_library_episode_identity.py	91	71
dragontools/core/media_library_export.py	125	114
dragontools/core/media_library_fix_queue.py	425	387
dragontools/core/media_library_item_sql.py	178	155
dragontools/core/media_library_jellyfin.py	194	176
dragontools/core/media_library_jellyfin_items.py	122	110
dragontools/core/media_library_jellyfin_metadata.py	21	19
dragontools/core/media_library_jellyfin_metadata_loader.py	197	177
dragontools/core/media_library_jellyfin_metadata_types.py	28	19
dragontools/core/media_library_jellyfin_metadata_writer.py	47	37
dragontools/core/media_library_jellyfin_source.py	108	85
dragontools/core/media_library_jellyfin_streams.py	152	137
dragontools/core/media_library_media_info_mapper.py	283	254
dragontools/core/media_library_migrations.py	147	130
dragontools/core/media_library_movie_paths.py	200	184
dragontools/core/media_library_movie_resolution.py	89	79
dragontools/core/media_library_nfo_inventory.py	76	67
dragontools/core/media_library_nfo_parser.py	112	90
dragontools/core/media_library_nfo_paths.py	65	52
dragontools/core/media_library_nfo_scan.py	127	107
dragontools/core/media_library_nfo_store.py	243	222
dragontools/core/media_library_path_mappings.py	313	267
dragontools/core/media_library_paths.py	24	19

Datei	Gesamtzeilen	Codezeilen
dragontools/core/media_library_publication.py	20	16
dragontools/core/media_library_query.py	53	46
dragontools/core/media_library_query_filters.py	57	48
dragontools/core/media_library_query_fragments.py	141	119
dragontools/core/media_library_query_presets.py	221	202
dragontools/core/media_library_query_scope_filters.py	61	55
dragontools/core/media_library_repository.py	29	26
dragontools/core/media_library_repository_items.py	49	42
dragontools/core/media_library_repository_moves.py	149	137
dragontools/core/media_library_saved_queries.py	71	58
dragontools/core/media_library_scan.py	348	308
dragontools/core/media_library_scan_analysis.py	40	36
dragontools/core/media_library_scan_plan.py	62	54
dragontools/core/media_library_schema.py	294	266
dragontools/core/media_library_scope.py	78	68
dragontools/core/media_library_search.py	50	42
dragontools/core/media_library_search_enrichment.py	22	16
dragontools/core/media_library_search_fields.py	67	50
dragontools/core/media_library_search_service.py	187	168
dragontools/core/media_library_search_streams.py	57	47
dragontools/core/media_library_series_lookup.py	69	59
dragontools/core/media_library_series_paths.py	256	220
dragontools/core/media_library_series_resolution.py	123	107
dragontools/core/media_library_sidecars.py	92	82
dragontools/core/media_library_sql_help.py	189	158
dragontools/core/media_library_sql_safety.py	170	146
dragontools/core/media_library_sqlite.py	36	27
dragontools/core/media_library_types.py	125	102
dragontools/core/media_library_utils.py	131	104
dragontools/core/media_library_video_flags.py	25	18
dragontools/core/media_metadata.py	265	214
dragontools/core/media_stream_selection.py	58	47
dragontools/core/media_track_pairing.py	66	54
dragontools/core/mediainfo_details.py	399	334
dragontools/core/mkv_track_metadata.py	64	55
dragontools/core/models.py	209	168
dragontools/core/move_companion_discovery.py	78	61
dragontools/core/move_conflict_transactions.py	180	165
dragontools/core/move_conflicts.py	246	203
dragontools/core/move_copy_verification.py	72	63
dragontools/core/move_directory_transfer.py	114	101
dragontools/core/move_file_service.py	231	195
dragontools/core/move_journal.py	335	309
dragontools/core/move_journal_adapter.py	48	38
dragontools/core/move_journal_contracts.py	18	14
dragontools/core/move_journal_recovery.py	294	262
dragontools/core/move_journal_resume.py	124	109
dragontools/core/move_journal_storage.py	94	82

Datei	Gesamtzeilen	Codezeilen
dragontools/core/move_journal_utils.py	82	68
dragontools/core/move_postprocess.py	72	63
dragontools/core/move_preparation.py	205	188
dragontools/core/move_report.py	240	216
dragontools/core/move_result.py	22	20
dragontools/core/move_routing.py	167	153
dragontools/core/move_sidecars.py	508	464
dragontools/core/move_source_probe.py	121	105
dragontools/core/move_transaction.py	306	259
dragontools/core/move_transfer_executor.py	227	204
dragontools/core/move_trickplay_transfer.py	110	104
dragontools/core/movie_identity.py	114	106
dragontools/core/movie_renamer.py	325	304
dragontools/core/movie_renamer_candidate_mapping.py	245	212
dragontools/core/movie_renamer_candidate_order.py	46	39
dragontools/core/movie_renamer_candidate_resolvers.py	91	78
dragontools/core/movie_renamer_candidate_scoring.py	108	83
dragontools/core/movie_renamer_candidates.py	33	30
dragontools/core/movie_renamer_episode_refresh.py	54	45
dragontools/core/movie_renamer_matching.py	85	71
dragontools/core/movie_renamer_models.py	169	139
dragontools/core/movie_renamer_parsing.py	480	398
dragontools/core/movie_renamer_release_warnings.py	32	26
dragontools/core/movie_renamer_season_override.py	77	65
dragontools/core/movie_renamer_year_override.py	21	17
dragontools/core/movie_replacement_preparation.py	40	37
dragontools/core/nfo_stream_metadata.py	131	106
dragontools/core/online_metadata.py	85	81
dragontools/core/online_metadata_cache.py	85	75
dragontools/core/online_metadata_cache_paths.py	30	21
dragontools/core/online_metadata_common.py	29	27
dragontools/core/online_metadata_config.py	169	148
dragontools/core/online_metadata_http.py	93	83
dragontools/core/online_metadata_identity.py	32	25
dragontools/core/online_metadata_parsing.py	192	135
dragontools/core/online_metadata_payload.py	133	114
dragontools/core/online_metadata_retry.py	76	61
dragontools/core/online_metadata_service.py	342	296
dragontools/core/online_metadata_tmdb.py	53	45
dragontools/core/online_metadata_tmdb_candidate_cache.py	101	83
dragontools/core/online_metadata_tmdb_renamer.py	274	249
dragontools/core/online_metadata_tmdb_resolver.py	472	440
dragontools/core/online_metadata_tmdb_suggestions.py	239	219
dragontools/core/online_metadata_tmdb_transport.py	131	115
dragontools/core/online_metadata_tvdb.py	66	52
dragontools/core/online_metadata_tvdb_candidate_cache.py	89	73
dragontools/core/online_metadata_tvdb_candidates.py	181	164
dragontools/core/online_metadata_tvdb_episode_data.py	170	153

Datei	Gesamtzeilen	Codezeilen
dragontools/core/online_metadata_tvdb_helpers.py	133	104
dragontools/core/online_metadata_tvdb_localization.py	106	84
dragontools/core/online_metadata_tvdb_pagination.py	40	35
dragontools/core/online_metadata_tvdb_resolver.py	166	145
dragontools/core/online_metadata_tvdb_suggestions.py	216	204
dragontools/core/online_metadata_tvdb_transport.py	261	227
dragontools/core/online_metadata_types.py	274	222
dragontools/core/output_replace.py	311	265
dragontools/core/output_timestamps.py	20	16
dragontools/core/owned_process.py	119	102
dragontools/core/owned_process_pause.py	152	134
dragontools/core/parallel_settings.py	87	66
dragontools/core/path_defaults.py	125	106
dragontools/core/path_safety.py	129	91
dragontools/core/path_syntax.py	225	181
dragontools/core/paths.py	39	25
dragontools/core/pgs_display_set.py	431	360
dragontools/core/planned_target_edit.py	72	56
dragontools/core/preflight_metadata_identity.py	70	61
dragontools/core/preflight_report.py	245	205
dragontools/core/process_runner.py	103	90
dragontools/core/process_status.py	23	19
dragontools/core/profile_manager.py	307	249
dragontools/core/project_info.py	169	138
dragontools/core/quality_extra_args.py	27	25
dragontools/core/quality_target.py	113	94
dragontools/core/quality_tester.py	255	216
dragontools/core/recovery_file.py	42	37
dragontools/core/release_archive_commit.py	12	10
dragontools/core/release_document_privacy.py	45	37
dragontools/core/release_frozen_runtime.py	22	17
dragontools/core/release_packaging.py	398	324
dragontools/core/release_private_paths.py	20	15
dragontools/core/release_qt_icu.py	52	44
dragontools/core/release_requirement_bounds.py	19	17
dragontools/core/release_source_identity.py	14	10
dragontools/core/release_validation.py	64	53
dragontools/core/release_validation_app.py	44	37
dragontools/core/release_validation_build.py	88	78
dragontools/core/release_validation_common.py	112	87
dragontools/core/release_validation_environment.py	416	383
dragontools/core/release_validation_package.py	354	340
dragontools/core/release_validation_privacy.py	295	244
dragontools/core/release_validation_smoke_modules.py	178	164
dragontools/core/release_validation_source.py	259	234
dragontools/core/renamer_candidate_decision.py	120	104
dragontools/core/renamer_completeness.py	96	78
dragontools/core/renamer_completeness_export.py	28	20

Datei	Gesamtzeilen	Codezeilen
dragontools/core/renamer_completeness_provider.py	115	105
dragontools/core/renamer_file_commit.py	50	44
dragontools/core/renamer_identity_review.py	30	26
dragontools/core/renamer_metadata_browser.py	532	476
dragontools/core/renamer_metadata_handoff.py	72	59
dragontools/core/renamer_year_safety.py	20	16
dragontools/core/replace_journal.py	141	119
dragontools/core/replace_recovery.py	96	85
dragontools/core/replacement_reminders.py	161	131
dragontools/core/resource_paths.py	77	67
dragontools/core/result_status.py	16	8
dragontools/core/rules_preview.py	134	129
dragontools/core/rules_preview_audio.py	44	38
dragontools/core/rules_preview_common.py	12	7
dragontools/core/rules_preview_context.py	94	85
dragontools/core/rules_preview_subtitles.py	111	97
dragontools/core/rules_preview_video.py	86	79
dragontools/core/run_summary.py	195	166
dragontools/core/sdr_hdr_enhancement.py	196	169
dragontools/core/secret_settings.py	179	142
dragontools/core/settings.py	23	17
dragontools/core/settings_access.py	219	179
dragontools/core/settings_app.py	10	6
dragontools/core/settings_backup.py	34	31
dragontools/core/settings_backup_common.py	194	158
dragontools/core/settings_backup_crypto.py	100	89
dragontools/core/settings_backup_export.py	102	91
dragontools/core/settings_backup_limits.py	89	71
dragontools/core/settings_backup_payload.py	44	38
dragontools/core/settings_backup_restore.py	219	186
dragontools/core/settings_conversion.py	136	116
dragontools/core/settings_jellyfin.py	27	22
dragontools/core/settings_media_library.py	28	24
dragontools/core/settings_metadata.py	66	57
dragontools/core/settings_notifications.py	51	41
dragontools/core/settings_postprocess.py	46	39
dragontools/core/settings_storage.py	69	60
dragontools/core/settings_watch.py	90	69
dragontools/core/sidecar_journal.py	210	177
dragontools/core/sidecar_recovery.py	63	54
dragontools/core/sidecar_transaction.py	249	213
dragontools/core/strict_numbers.py	17	12
dragontools/core/system_shutdown.py	82	66
dragontools/core/timeout_settings.py	522	449
dragontools/core/tool_diagnostics.py	254	214
dragontools/core/tool_diagnostics_extended.py	207	182
dragontools/core/tool_paths.py	322	254
dragontools/core/track_titles.py	45	36

Datei	Gesamtzeilen	Codezeilen
dragontools/core/transaction_identity.py	96	77
dragontools/core/type_utils.py	79	66
dragontools/core/update_check.py	115	91
dragontools/core/version.py	13	10
dragontools/core/watch_folder.py	213	182
dragontools/core/watch_folder_observations.py	39	31
dragontools/core/whisper_runtime.py	269	227
dragontools/core/windows_restart_policy.py	126	105

35.4 Worker

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/__init__.py	1	0
dragontools/worker/archive_service.py	158	126
dragontools/worker/audio_metadata_args.py	37	30
dragontools/worker/audio_mux_job.py	128	116
dragontools/worker/audio_mux_output_verifier.py	44	36
dragontools/worker/audio_mux_plan_service.py	133	123
dragontools/worker/audio_mux_thread.py	116	98
dragontools/worker/audio_video_match_analysis_service.py	76	65
dragontools/worker/audio_video_match_contracts.py	64	53
dragontools/worker/audio_video_match_create_service.py	133	117
dragontools/worker/audio_video_match_render.py	171	155
dragontools/worker/audio_video_match_reporting.py	56	46
dragontools/worker/audio_video_match_runtime.py	76	61
dragontools/worker/audio_video_match_service.py	54	46
dragontools/worker/audio_video_match_thread.py	111	95
dragontools/worker/audio_video_output_contract.py	54	45
dragontools/worker/av1_metadata_pipeline.py	393	347
dragontools/worker/base_worker.py	151	116
dragontools/worker/bitmap_subtitle_ocr_service.py	391	342
dragontools/worker/bitmap_subtitle_packet_probe.py	228	212
dragontools/worker/cleanup_service.py	152	128
dragontools/worker/comfyui_client.py	179	156
dragontools/worker/comfyui_job_monitor.py	115	95
dragontools/worker/comfyui_mux_plan.py	45	37
dragontools/worker/comfyui_render_plan.py	77	71
dragontools/worker/comfyui_runtime.py	301	259
dragontools/worker/comfyui_video_contract.py	139	115
dragontools/worker/comfyui_video_worker.py	107	89
dragontools/worker/command_formatting.py	20	14
dragontools/worker/converter_audio_args.py	51	44
dragontools/worker/converter_config.py	79	52
dragontools/worker/converter_control.py	171	153
dragontools/worker/converter_detection.py	273	239
dragontools/worker/converter_file_executor.py	113	100
dragontools/worker/converter_lifecycle.py	138	121
dragontools/worker/converter_media_probe.py	55	46

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/converter_optional_runtime.py	162	138
dragontools/worker/converter_process_executor.py	206	186
dragontools/worker/converter_progress.py	61	47
dragontools/worker/converter_progress_eta.py	25	19
dragontools/worker/converter_progress_parser.py	102	87
dragontools/worker/converter_queue_state.py	143	119
dragontools/worker/converter_run_loop.py	189	172
dragontools/worker/converter_runtime_builder.py	274	257
dragontools/worker/converter_static_composition.py	87	81
dragontools/worker/converter_stream_args.py	121	95
dragontools/worker/converter_strip.py	71	64
dragontools/worker/converter_strip_audio.py	34	30
dragontools/worker/converter_strip_movtext_fallback.py	119	107
dragontools/worker/converter_strip_runtime.py	22	14
dragontools/worker/converter_strip_sidecars.py	32	28
dragontools/worker/converter_strip_subtitles.py	88	78
dragontools/worker/converter_subtitle_args.py	95	83
dragontools/worker/converter_thread.py	314	230
dragontools/worker/converter_thread_bootstrap.py	73	62
dragontools/worker/converter_thread_compat.py	13	8
dragontools/worker/converter_thread_state.py	131	107
dragontools/worker/converter_utils.py	106	85
dragontools/worker/converter_video_filter_args.py	32	20
dragontools/worker/crop_geometry.py	138	112
dragontools/worker/duration_original_timeline_service.py	450	415
dragontools/worker/duration_packet_integrity.py	124	107
dragontools/worker/duration_remux_service.py	138	118
dragontools/worker/duration_repair_archive.py	58	48
dragontools/worker/duration_repair_commands.py	119	101
dragontools/worker/duration_repair_diagnostics.py	61	49
dragontools/worker/duration_repair_models.py	183	155
dragontools/worker/duration_repair_orchestrator.py	201	185
dragontools/worker/duration_repair_policy.py	49	42
dragontools/worker/duration_repair_runtime.py	68	56
dragontools/worker/duration_repair_service.py	217	188
dragontools/worker/duration_repair_stream_guard.py	227	199
dragontools/worker/duration_repair_validation.py	216	175
dragontools/worker/duration_timestamp_candidate_archive.py	51	43
dragontools/worker/duration_timestamp_candidate_service.py	164	141
dragontools/worker/duration_timestamp_candidate_validation.py	128	110
dragontools/worker/duration_timestamp_helpers.py	97	81
dragontools/worker/duration_timestamp_plan.py	36	34
dragontools/worker/duration_timestamp_service.py	412	372
dragontools/worker/duration_timing_analyzer.py	86	70
dragontools/worker/duration_timing_inference.py	62	49
dragontools/worker/duration_timing_mapping.py	88	80
dragontools/worker/duration_timing_parsing.py	114	89
dragontools/worker/duration_timing_sources.py	38	29

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/dv5_encode_fallback.py	147	114
dragontools/worker/dv_audio_mux_service.py	260	223
dragontools/worker/dv_command_runner.py	187	166
dragontools/worker/dv_crop_reconcile.py	322	267
dragontools/worker/dv_dynamic_metadata_service.py	415	399
dragontools/worker/dv_edge_frame_repair.py	339	286
dragontools/worker/dv_embedded_source_check.py	48	45
dragontools/worker/dv_encode_command.py	108	88
dragontools/worker/dv_failure_recovery.py	392	349
dragontools/worker/dv_filter_graph.py	39	31
dragontools/worker/dv_final_metadata_verifier.py	247	220
dragontools/worker/dv_final_mux_service.py	43	32
dragontools/worker/dv_final_output_service.py	44	38
dragontools/worker/dv_level5_editor.py	189	158
dragontools/worker/dv_level5_values.py	24	20
dragontools/worker/dv_matroska_track_selection.py	47	40
dragontools/worker/dv_mkv_muxer.py	65	58
dragontools/worker/dv_mkv_source_subtitles.py	171	148
dragontools/worker/dv_mp4box_muxer.py	71	61
dragontools/worker/dv_mux_input_validation.py	9	7
dragontools/worker/dv_output_install.py	39	29
dragontools/worker/dv_partial_frame_repair.py	698	592
dragontools/worker/dv_pipeline_compat.py	118	86
dragontools/worker/dv_pipeline_context.py	209	187
dragontools/worker/dv_pipeline_runtime.py	369	337
dragontools/worker/dv_pipeline_stage_factory.py	59	52
dragontools/worker/dv_pipeline_stages.py	435	380
dragontools/worker/dv_pipeline_timeouts.py	40	20
dragontools/worker/dv_postprocess_gate.py	46	37
dragontools/worker/dv_processing_components.py	101	91
dragontools/worker/dv_processing_pipeline.py	297	266
dragontools/worker/dv_remux_audio.py	87	77
dragontools/worker/dv_remux_components.py	44	35
dragontools/worker/dv_remux_file_dispatcher.py	98	85
dragontools/worker/dv_remux_job.py	503	459
dragontools/worker/dv_remux_muxers.py	149	128
dragontools/worker/dv_remux_output.py	239	216
dragontools/worker/dv_remux_output_paths.py	33	26
dragontools/worker/dv_remux_output_verifier.py	123	103
dragontools/worker/dv_remux_pipeline.py	390	362
dragontools/worker/dv_remux_policy.py	105	90
dragontools/worker/dv_remux_process.py	165	148
dragontools/worker/dv_remux_profile_service.py	135	122
dragontools/worker/dv_remux_thread.py	394	307
dragontools/worker/dv_remux_transaction_state.py	23	18
dragontools/worker/dv_result_contract.py	116	98
dragontools/worker/dv_rpu_service.py	73	63
dragontools/worker/dv_runtime_models.py	49	40

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/dv_source_rpu_check.py	124	109
dragontools/worker/dv_source_rpu_fallback.py	28	23
dragontools/worker/dv_subtitle_mux_service.py	305	273
dragontools/worker/dv_track_preparation_service.py	59	54
dragontools/worker/dv_video_filters.py	126	96
dragontools/worker/dv_video_stage_service.py	502	461
dragontools/worker/dv_workflow_pipeline_adapter.py	84	74
dragontools/worker/encode_color_plan.py	90	77
dragontools/worker/encode_geometry_plan.py	79	72
dragontools/worker/encode_plan.py	15	11
dragontools/worker/encode_plan_service.py	94	80
dragontools/worker/encoder_args.py	243	196
dragontools/worker/frame_count_evidence.py	160	139
dragontools/worker/hdr10_color.py	102	73
dragontools/worker/hdr10plus_bitstream_service.py	154	135
dragontools/worker/hdr10plus_generator_client.py	315	278
dragontools/worker/hdr10plus_generator_frame_contract.py	33	30
dragontools/worker/hdr10plus_workflow_policy.py	29	22
dragontools/worker/hdr_metadata_file_ownership.py	19	16
dragontools/worker/hdr_metadata_picture_policy.py	17	15
dragontools/worker/hdrplus_conversion.py	320	295
dragontools/worker/hdrplus_encode_service.py	106	92
dragontools/worker/hdrplus_helper_compat.py	167	148
dragontools/worker/hdrplus_helper_services.py	113	103
dragontools/worker/hdrplus_mp4_audio_service.py	69	64
dragontools/worker/hdrplus_mux_service.py	134	119
dragontools/worker/hdrplus_pipeline_coordinator.py	527	456
dragontools/worker/hdrplus_recovery_archive.py	132	105
dragontools/worker/hdrplus_runtime_models.py	103	88
dragontools/worker/hdrplus_source_selection.py	27	22
dragontools/worker/hdrplus_stream_service.py	107	97
dragontools/worker/hdrplus_tool_runner.py	161	146
dragontools/worker/hdrplus_workspace.py	35	25
dragontools/worker/interfaces.py	117	87
dragontools/worker/iso_disc_inspector.py	243	212
dragontools/worker/iso_ffmpeg_fallback_service.py	207	185
dragontools/worker/iso_input_processor.py	135	118
dragontools/worker/iso_makemkv_service.py	211	183
dragontools/worker/iso_models.py	24	14
dragontools/worker/iso_output_publication.py	44	37
dragontools/worker/iso_processor_host_mixin.py	54	40
dragontools/worker/iso_selection.py	123	98
dragontools/worker/iso_thread.py	239	204
dragontools/worker/iso_title_parser.py	29	26
dragontools/worker/job_process_owner.py	38	27
dragontools/worker/live_queue_overrides.py	20	14
dragontools/worker/log_dispatch.py	104	78
dragontools/worker/media_analysis_service.py	48	40

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/media_contract.py	78	68
dragontools/worker/media_contract_builder.py	284	260
dragontools/worker/media_contract_types.py	49	40
dragontools/worker/media_library_fix_service.py	183	158
dragontools/worker/media_stream_language_service.py	280	254
dragontools/worker/media_stream_metadata_guard.py	182	152
dragontools/worker/merge_analysis.py	178	157
dragontools/worker/merge_common.py	136	112
dragontools/worker/merge_executor.py	218	202
dragontools/worker/merge_output_verifier.py	78	68
dragontools/worker/merge_plan.py	76	65
dragontools/worker/merge_thread.py	128	109
dragontools/worker/mkv_audio_track_args.py	13	11
dragontools/worker/move_batch_executor.py	318	278
dragontools/worker/move_batch_lifecycle.py	118	101
dragontools/worker/move_companion_adapter.py	127	115
dragontools/worker/move_completion_service.py	168	153
dragontools/worker/move_result_commit.py	130	113
dragontools/worker/move_runtime_control.py	85	70
dragontools/worker/move_thread.py	158	140
dragontools/worker/mp4_default_flags.py	108	95
dragontools/worker/mp4_remux_file_service.py	317	284
dragontools/worker/mp4_remux_output_verifier.py	65	54
dragontools/worker/mp4_remux_plan.py	327	302
dragontools/worker/mp4_remux_sidecars.py	106	98
dragontools/worker/mp4_remux_thread.py	217	193
dragontools/worker/mp4box_track_args.py	35	24
dragontools/worker/nfo_commit.py	219	181
dragontools/worker/output_contract_metadata.py	19	15
dragontools/worker/output_contract_tracks.py	125	113
dragontools/worker/output_contract_verifier.py	73	61
dragontools/worker/output_contract_video.py	150	126
dragontools/worker/output_duration_policy.py	36	30
dragontools/worker/output_path_service.py	65	50
dragontools/worker/output_probe.py	100	86
dragontools/worker/output_size_policy.py	143	122
dragontools/worker/output_verification_archive.py	429	394
dragontools/worker/output_verifier.py	271	246
dragontools/worker/owned_probe.py	12	10
dragontools/worker/packet_json_stream.py	79	72
dragontools/worker/packet_snapshot.py	91	81
dragontools/worker/parallel_child_controls.py	16	14
dragontools/worker/parallel_child_result_coordinator.py	163	150
dragontools/worker/parallel_converter_compat.py	105	78
dragontools/worker/parallel_converter_control.py	118	98
dragontools/worker/parallel_converter_lifecycle.py	147	129
dragontools/worker/parallel_converter_queue.py	200	176
dragontools/worker/parallel_converter_shutdown.py	28	24

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/parallel_converter_state.py	234	210
dragontools/worker/parallel_converter_thread.py	221	201
dragontools/worker/parallel_file_control.py	56	48
dragontools/worker/parallel_launch_ownership.py	23	19
dragontools/worker/parallel_queue_coordination.py	28	24
dragontools/worker/parallel_worker_launcher.py	112	103
dragontools/worker/pipeline_decision_service.py	193	162
dragontools/worker/postprocess_async.py	257	206
dragontools/worker/postprocess_config.py	147	138
dragontools/worker/postprocess_lifecycle.py	52	45
dragontools/worker/postprocess_metadata.py	81	68
dragontools/worker/postprocess_metadata_identity.py	15	13
dragontools/worker/postprocess_metadata_resolution.py	229	183
dragontools/worker/postprocess_models.py	77	58
dragontools/worker/postprocess_nfo_ownership.py	44	36
dragontools/worker/postprocess_runner.py	503	457
dragontools/worker/postprocess_service.py	16	12
dragontools/worker/postprocess_source_trickplay.py	81	70
dragontools/worker/process_control.py	404	342
dragontools/worker/quality_compare_service.py	152	139
dragontools/worker/quality_compare_thread.py	116	97
dragontools/worker/quality_metrics_service.py	75	66
dragontools/worker/quality_output_validation.py	35	29
dragontools/worker/quality_process_runner.py	36	31
dragontools/worker/quality_target_integration.py	29	26
dragontools/worker/quality_target_service.py	241	222
dragontools/worker/quality_test_service.py	185	165
dragontools/worker/quality_test_thread.py	98	79
dragontools/worker/queue_pipeline_preview.py	62	55
dragontools/worker/replace_service.py	173	148
dragontools/worker/required_sidecar_step.py	24	21
dragontools/worker/sdr_hdr_runtime.py	50	43
dragontools/worker/source_visual_analysis.py	66	53
dragontools/worker/source_visual_check.py	233	200
dragontools/worker/source_visual_models.py	90	73
dragontools/worker/source_visual_sampling.py	175	155
dragontools/worker/source_visual_settings.py	54	48
dragontools/worker/source_visual_thread.py	91	69
dragontools/worker/standard_pipeline_runner.py	432	404
dragontools/worker/subtitle_export_models.py	122	103
dragontools/worker/subtitle_movtext_backup.py	48	45
dragontools/worker/subtitle_movtext_plan.py	62	58
dragontools/worker/subtitle_ocr_sidecars.py	98	93
dragontools/worker/subtitle_sidecar_exporter.py	171	163
dragontools/worker/subtitle_sidecar_plan.py	8	6
dragontools/worker/subtitle_sidecar_service.py	262	210
dragontools/worker/subtitle_sidecar_targets.py	174	156
dragontools/worker/timestamp_diagnostics.py	83	73

Datei	Gesamtzeilen	Codezeilen
dragontools/worker/tool_binary_output.py	36	31
dragontools/worker/tool_output_buffer.py	43	37
dragontools/worker/tool_process_lifecycle.py	309	254
dragontools/worker/tool_runner.py	346	310
dragontools/worker/tool_text_output.py	102	89
dragontools/worker/trickplay_commit.py	121	107
dragontools/worker/trickplay_concurrency.py	106	86
dragontools/worker/trickplay_ffmpeg.py	125	109
dragontools/worker/trickplay_models.py	33	27
dragontools/worker/trickplay_paths.py	57	43
dragontools/worker/trickplay_service.py	294	266
dragontools/worker/trickplay_sprite_verification.py	29	27
dragontools/worker/utility_copy_contract.py	61	52
dragontools/worker/utility_media_analysis.py	36	26
dragontools/worker/utility_output_workspace.py	44	38
dragontools/worker/verification_control.py	14	10
dragontools/worker/worker_contracts.py	50	36
dragontools/worker/worker_events.py	48	37
dragontools/worker/worker_result_accounting.py	42	36
dragontools/worker/worker_result_service.py	247	214
dragontools/worker/worker_runtime_state.py	14	10
dragontools/worker/workflow_duration_repair.py	63	57
dragontools/worker/workflow_engine.py	234	207
dragontools/worker/workflow_factory.py	84	80
dragontools/worker/workflow_geometry_validation.py	284	243
dragontools/worker/workflow_models.py	138	119
dragontools/worker/workflow_output_commit.py	240	192
dragontools/worker/workflow_override_summary.py	45	36
dragontools/worker/workflow_pipeline_executor.py	117	104
dragontools/worker/workflow_planning_service.py	197	176
dragontools/worker/workflow_postprocess_commit.py	211	185
dragontools/worker/workflow_services.py	303	253
dragontools/worker/workflow_sidecar_commit.py	103	92
dragontools/worker/workflow_verification_service.py	73	60

35.5 Rules

Datei	Gesamtzeilen	Codezeilen
dragontools/rules/__init__.py	1	0
dragontools/rules/audio_plan.py	348	292
dragontools/rules/audio_plan_policy.py	198	167
dragontools/rules/audio_rule_basics.py	212	182
dragontools/rules/audio_rule_migration.py	192	170
dragontools/rules/audio_rule_repository.py	36	27
dragontools/rules/audio_rules.py	44	40
dragontools/rules/audio_selection.py	164	130
dragontools/rules/audio_transcode_policy.py	221	174
dragontools/rules/move_path_helpers.py	154	113

Datei	Gesamtzeilen	Codezeilen
dragontools/rules/move_rule_config.py	63	55
dragontools/rules/move_rules.py	186	160
dragontools/rules/move_series_detection.py	112	83
dragontools/rules/move_series_directories.py	104	78
dragontools/rules/pipeline_capabilities.py	135	123
dragontools/rules/pipeline_policy.py	111	93
dragontools/rules/pipeline_selector.py	224	210
dragontools/rules/renamer_rules.py	393	320
dragontools/rules/rule_loader.py	262	226
dragontools/rules/subtitle_burn_policy.py	150	132
dragontools/rules/subtitle_default_policy.py	64	56
dragontools/rules/subtitle_keep_policy.py	171	145
dragontools/rules/subtitle_output_plan.py	145	128
dragontools/rules/subtitle_plan_models.py	30	23
dragontools/rules/subtitle_plan_service.py	161	143
dragontools/rules/subtitle_rule_config.py	247	224
dragontools/rules/subtitle_rules.py	112	102
dragontools/rules/subtitle_selection.py	30	27
dragontools/rules/subtitle_selection_common.py	185	149
dragontools/rules/subtitle_selection_priority.py	42	33
dragontools/rules/subtitle_selection_sidecar.py	58	47
dragontools/rules/subtitle_storage.py	126	97

35.6 Subtitle

Datei	Gesamtzeilen	Codezeilen
dragontools/subtitle/__init__.py	1	0
dragontools/subtitle/converter.py	265	229
dragontools/subtitle/extractor.py	178	169
dragontools/subtitle/formats.py	131	98
dragontools/subtitle/injector.py	352	331
dragontools/subtitle/matroska_tracks.py	55	48
dragontools/subtitle/media_verification.py	223	196
dragontools/subtitle/output_safety.py	31	23
dragontools/subtitle/sidecar_matcher.py	65	53
dragontools/subtitle/tag_edit_verification.py	21	18
dragontools/subtitle/tagger.py	86	74
dragontools/subtitle/tool_logging.py	32	26

35.7 Dragon HDR10+ Generator (Source)

Datei	Gesamtzeilen	Codezeilen
dragon_hdr10plus_generator/hdrplusgenerator_entry.py	5	3
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/__init__.py	3	2
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/__init__.py	1	1
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/decoder.py	128	113

Datei	Gesamtzeilen	Codezeilen
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/luminance.py	33	24
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/scanner.py	369	309
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/scene_detection.py	45	36
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/analyzer/statistics.py	46	37
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/cli.py	224	201
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/config.py	13	9
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/metadata/__init__.py	1	1
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/metadata/json_writer.py	29	24
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/metadata/st2094_40.py	159	122
dragon_hdr10plus_generator/src/dragon_hdr10plus_generator/metadata/tone_mapping.py	19	12

35.8 Tests (DragonTools + Generator)

Datei	Gesamtzeilen	Codezeilen
dragon_hdr10plus_generator/tests/conftest.py	8	6
dragon_hdr10plus_generator/tests/test_foundation.py	362	289
dragon_hdr10plus_generator/tests/test_frame_count_progress_contract.py	99	78
dragon_hdr10plus_generator/tests/test_hardening.py	45	33
dragon_hdr10plus_generator/tests/test_review09_regressions.py	81	71
dragontools/tests/__init__.py	0	0
dragontools/tests/ci_requirements.py	87	71
dragontools/tests/conftest.py	53	42
dragontools/tests/responsibility_checks.py	61	51
dragontools/tests/subtitle_command_fixtures.py	59	53
dragontools/tests/test_audio_processing.py	101	77
dragontools/tests/test_audio_tail_and_repair_diagnostics.py	146	116
dragontools/tests/test_audio_video_matcher.py	160	126
dragontools/tests/test_audit_log.py	45	33
dragontools/tests/test_autocrop_authority_and_target_races.py	185	147
dragontools/tests/test_auxiliary_workers.py	563	433
dragontools/tests/test_av1_metadata_pipeline.py	139	114
dragontools/tests/test_batch_preflight.py	417	349
dragontools/tests/test_bitmap_burn_timestamps.py	115	99
dragontools/tests/test_block2_crash_durability.py	177	146
dragontools/tests/test_block3_process_hardening.py	212	170
dragontools/tests/test_block6_coordinator_architecture.py	79	62
dragontools/tests/test_block7_architecture_cleanup.py	102	82
dragontools/tests/test_block_f_architecture.py	14	9
dragontools/tests/test_changelog.py	90	69
dragontools/tests/test_cleanup_rollback.py	177	134
dragontools/tests/test_codec_profile_assistant.py	58	45
dragontools/tests/test_config_migration.py	369	304

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_conversion_controller_architecture.py	77	64
dragontools/tests/test_conversion_controller_start.py	603	491
dragontools/tests/test_conversion_result_service.py	310	230
dragontools/tests/test_convert_queue_window_runtime.py	97	77
dragontools/tests/test_convert_widget_architecture.py	71	57
dragontools/tests/test_convert_widget_queue_actions_architecture.py	182	159
dragontools/tests/test_convert_widget_services.py	149	126
dragontools/tests/test_converter_detection.py	103	79
dragontools/tests/test_converter_thread_refactor.py	301	243
dragontools/tests/test_crash_guard_cleanup.py	48	35
dragontools/tests/test_crop_geometry_normalization.py	124	101
dragontools/tests/test_current_ffmpeg_abort.py	115	80
dragontools/tests/test_diagnostic_package.py	65	50
dragontools/tests/test_disk_space.py	57	46
dragontools/tests/test_downscale_only_policy.py	209	171
dragontools/tests/test_duration_repair_architecture.py	56	45
dragontools/tests/test_duration_repair_service.py	1502	1271
dragontools/tests/test_duration_repair_stream_guard.py	134	108
dragontools/tests/test_dv_audio_mux_service.py	59	47
dragontools/tests/test_dv_corrupt_rpu_fallback.py	225	187
dragontools/tests/test_dv_direct_mkv_rpu.py	128	105
dragontools/tests/test_dv_edge_frame_repair.py	347	277
dragontools/tests/test_dv_mapping_forced_timeout.py	56	46
dragontools/tests/test_dv_mkv_pgs_mux_regressions.py	112	100
dragontools/tests/test_dv_partial_frame_repair.py	144	122
dragontools/tests/test_dv_pipeline_architecture.py	260	222
dragontools/tests/test_dv_regressions.py	1366	1132
dragontools/tests/test_dv_remux_abort_fix.py	105	76
dragontools/tests/test_dv_remux_audio_job_regressions.py	75	63
dragontools/tests/test_dv_remux_container_selection.py	424	351
dragontools/tests/test_dv_stage_service_architecture.py	61	48
dragontools/tests/test_encoder_args.py	319	268
dragontools/tests/test_encoder_inactivity_timeout.py	140	106
dragontools/tests/test_encoder_profile_override.py	282	237
dragontools/tests/test_encoder_settings_architecture.py	92	74
dragontools/tests/test_encoder_settings_shutdown.py	384	309
dragontools/tests/test_end_to_end_media_fixture.py	149	133
dragontools/tests/test_episode_replacement_artifacts.py	98	77
dragontools/tests/test_error_report.py	166	149
dragontools/tests/test_exception_boundary_architecture.py	71	54
dragontools/tests/test_exception_logging_contract.py	22	17
dragontools/tests/test_extended.py	980	793
dragontools/tests/test_facade_split_architecture.py	57	45
dragontools/tests/test_final_rpu_presence_guard.py	111	95
dragontools/tests/test_fix_20260906_iso_renamer.py	108	81
dragontools/tests/test_formatting_helpers.py	15	10
dragontools/tests/test_gui_dialog_services.py	257	204
dragontools/tests/test_hdr10_color_standard.py	367	309

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_hdrplus_architecture.py	227	186
dragontools/tests/test_hdrplus_final_verification.py	243	188
dragontools/tests/test_hdrplus_five_step_pipeline.py	274	229
dragontools/tests/test_incremental_move_queue_edit.py	336	257
dragontools/tests/test_iso_architecture.py	46	34
dragontools/tests/test_jellyfin_nfo.py	91	76
dragontools/tests/test_job_journal.py	316	246
dragontools/tests/test_language_priority_rules.py	512	416
dragontools/tests/test_legacy_cleanup_architecture.py	118	91
dragontools/tests/test_live_parallel_controls.py	179	157
dragontools/tests/test_log_cleanup.py	59	47
dragontools/tests/test_log_short_and_audio_titles.py	35	28
dragontools/tests/test_main_window_actions_architecture.py	145	131
dragontools/tests/test_main_window_architecture.py	99	66
dragontools/tests/test_main_window_tab_visibility.py	116	80
dragontools/tests/test_media_analyzer_pix_fmt.py	94	68
dragontools/tests/test_media_contract.py	197	163
dragontools/tests/test_media_info_text_builder.py	144	127
dragontools/tests/test_media_library.py	2647	2332
dragontools/tests/test_media_library_architecture.py	77	69
dragontools/tests/test_media_library_dialog_architecture.py	68	53
dragontools/tests/test_media_library_dialog_service.py	247	210
dragontools/tests/test_media_library_nfo_parser_security.py	70	50
dragontools/tests/test_media_library_series_lookup_performance.py	188	159
dragontools/tests/test_mediainfo_details.py	91	80
dragontools/tests/test_memory_shutdown_boundaries.py	270	227
dragontools/tests/test_models.py	128	105
dragontools/tests/test_move_completion_queue_cleanup.py	143	128
dragontools/tests/test_move_conflicts.py	191	141
dragontools/tests/test_move_journal.py	544	425
dragontools/tests/test_move_only_companion_discovery.py	45	36
dragontools/tests/test_move_progress_callback_safety.py	59	46
dragontools/tests/test_move_report.py	161	144
dragontools/tests/test_move_rules_series_detection.py	83	55
dragontools/tests/test_move_service_architecture.py	95	75
dragontools/tests/test_move_sidecar_recovery.py	46	35
dragontools/tests/test_move_transaction_core.py	164	120
dragontools/tests/test_move_transactional.py	341	257
dragontools/tests/test_move_verified_copy_reuse.py	111	83
dragontools/tests/test_movie_renamer.py	600	468
dragontools/tests/test_movie_renamer_background_commit.py	256	216
dragontools/tests/test_movie_renamer_table_controller_runtime.py	85	58
dragontools/tests/test_movie_renamer_widget_architecture.py	111	91
dragontools/tests/test_movie_trickplay_replacement.py	212	169
dragontools/tests/test_mp4_remux_architecture.py	33	25
dragontools/tests/test_mp4_subtitle_policy.py	181	143
dragontools/tests/test_nfo_timing_during.py	225	182
dragontools/tests/test_online_metadata.py	830	692

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_online_metadata_architecture.py	111	97
dragontools/tests/test_output_path_service.py	51	41
dragontools/tests/test_output_verifier.py	398	316
dragontools/tests/test_override_analysis_safety.py	78	66
dragontools/tests/test_override_dialog_analysis_ui.py	91	86
dragontools/tests/test_packet_integrity_safety.py	76	62
dragontools/tests/test_parallel_converter_architecture.py	34	25
dragontools/tests/test_parallel_converter_thread.py	393	302
dragontools/tests/test_parallel_settings.py	104	69
dragontools/tests/test_patch01_second_review.py	120	104
dragontools/tests/test_patch02_second_review.py	443	339
dragontools/tests/test_patch03_second_review.py	209	163
dragontools/tests/test_patch04_second_review.py	262	209
dragontools/tests/test_patch05_second_review.py	190	151
dragontools/tests/test_patch06_second_review.py	623	548
dragontools/tests/test_patch07_real_boundaries.py	107	89
dragontools/tests/test_patch07_second_review.py	417	352
dragontools/tests/test_patch08_real_mux.py	61	54
dragontools/tests/test_patch08_second_review.py	235	201
dragontools/tests/test_patch09_real_generator.py	131	119
dragontools/tests/test_patch09_second_review.py	367	289
dragontools/tests/test_patch10_real_media.py	107	96
dragontools/tests/test_patch10_second_review.py	416	350
dragontools/tests/test_patch11_real_media.py	104	91
dragontools/tests/test_patch11_second_review.py	205	166
dragontools/tests/test_patch12_real_media.py	708	662
dragontools/tests/test_patch12_second_review.py	520	437
dragontools/tests/test_patch13_real_media.py	118	101
dragontools/tests/test_patch13_second_review.py	362	306
dragontools/tests/test_patch14_real_media.py	108	95
dragontools/tests/test_patch14_second_review.py	351	295
dragontools/tests/test_patch15_real_media.py	69	64
dragontools/tests/test_patch15_second_review.py	730	632
dragontools/tests/test_patch16_individual_pause.py	381	340
dragontools/tests/test_patch16_native_shutdown.py	269	235
dragontools/tests/test_patch16_second_review.py	404	344
dragontools/tests/test_patch17_media_plans.py	109	86
dragontools/tests/test_patch17_merge_iso.py	150	130
dragontools/tests/test_patch17_native_widgets.py	169	152
dragontools/tests/test_patch17_real_media.py	151	135
dragontools/tests/test_patch17_second_review.py	285	245
dragontools/tests/test_patch18_identity_safety.py	171	138
dragontools/tests/test_patch18_native_renamer.py	224	194
dragontools/tests/test_patch19_metadata_contracts.py	322	265
dragontools/tests/test_patch1_burnin_resolution.py	194	160
dragontools/tests/test_patch20_library_contracts.py	440	386
dragontools/tests/test_patch21_gui_contracts.py	678	589
dragontools/tests/test_patch22_convert_contracts.py	603	530

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_patch23_quality_contracts.py	526	444
dragontools/tests/test_patch23_real_media.py	123	108
dragontools/tests/test_patch24_release_contracts.py	289	229
dragontools/tests/test_patch24_release_icu.py	75	60
dragontools/tests/test_patch24_release_junctions.py	27	25
dragontools/tests/test_patch24_release_opencv.py	35	27
dragontools/tests/test_patch25_architecture_ownership.py	160	132
dragontools/tests/test_patch26_contract_matrix.py	231	211
dragontools/tests/test_patch26_mp4_finalization.py	111	91
dragontools/tests/test_patch26_native_chain.py	83	73
dragontools/tests/test_patch26_pipeline_boundaries.py	228	194
dragontools/tests/test_patch27_artifact_snapshot.py	32	28
dragontools/tests/test_patch27_move_recovery.py	65	57
dragontools/tests/test_patch27_native_recovery.py	44	42
dragontools/tests/test_patch27_nfo_lifecycle.py	41	39
dragontools/tests/test_patch27_sidecar_recovery.py	146	128
dragontools/tests/test_patch27_stage_faults.py	50	47
dragontools/tests/test_patch28_final_stress.py	118	109
dragontools/tests/test_patch29_movie_replace.py	75	69
dragontools/tests/test_patch29_native_mux.py	206	199
dragontools/tests/test_patch29_native_source.py	59	57
dragontools/tests/test_patch29_source_rpu.py	187	171
dragontools/tests/test_patch2_sidecar_crash_recovery.py	150	118
dragontools/tests/test_patch3_dv_metadata_and_process.py	243	207
dragontools/tests/test_patch3_subtitle_process_control.py	71	51
dragontools/tests/test_patch4_lifecycle_hardening.py	174	125
dragontools/tests/test_patch5_release_ci.py	153	119
dragontools/tests/test_patch8_move_signal_ci_hardening.py	265	208
dragontools/tests/test_patch_a_subtitle_injection.py	144	109
dragontools/tests/test_patch_ac_renamer_status_geometry.py	44	31
dragontools/tests/test_patch_ad_move_jellyfin_hardening.py	89	61
dragontools/tests/test_patch_ae_renamer_watch_jellyfin.py	197	157
dragontools/tests/test_patch_ag_external_hdr10plus_generator.py	626	524
dragontools/tests/test_patch_ah_comfyui_backend.py	370	305
dragontools/tests/test_patch_ai_davinci_launcher.py	36	25
dragontools/tests/test_patch_ai_hdrtvdm_bridge_optional.py	176	155
dragontools/tests/test_patch_aj_comfyui_layout.py	43	32
dragontools/tests/test_patch_al_comfyui_video_worker.py	315	267
dragontools/tests/test_patch_an_comfyui_media_input_fallback.py	129	107
dragontools/tests/test_patch_ap_sdr_hdr_settings_autostart_override.py	160	131
dragontools/tests/test_patch_as_media_hdr_detection.py	266	229
dragontools/tests/test_patch_av_movtext_timestamp_repair.py	237	200
dragontools/tests/test_patch_aw_movtext_internal_fallback.py	112	86
dragontools/tests/test_patch_ax_hdr10plus_import_closure.py	24	18
dragontools/tests/test_patch_ay_workflow_import_closure.py	34	28
dragontools/tests/test_patch_az_renamer_bare_e_manual_season.py	80	59
dragontools/tests/test_patch_b_lazy_tab_loading.py	87	68

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_patch_ba_source_timing_reference.py	176	154
dragontools/tests/test_patch_bb_renamer_episode_layout.py	76	57
dragontools/tests/test_patch_bb_renamer_episode_runtime.py	85	59
dragontools/tests/test_patch_bc_multi_file_settings.py	57	44
dragontools/tests/test_patch_bc_multi_file_settings_runtime.py	139	117
dragontools/tests/test_patch_bh_dv_overlap_hdr_progress.py	155	124
dragontools/tests/test_patch_bi_frame_evidence_and_dv_postprocess.py	473	401
dragontools/tests/test_patch_bj_hdr10plus_failure_archive.py	263	231
dragontools/tests/test_patch_bl_output_timestamps.py	166	131
dragontools/tests/test_patch_bl_timestamp_media.py	175	145
dragontools/tests/test_patch_bm_release_privacy.py	82	62
dragontools/tests/test_patch_bo_cpu_x265_filter_path.py	219	181
dragontools/tests/test_patch_bp_sdr_hdr_pixel_format_and_logs.py	77	65
dragontools/tests/test_patch_bq_queue_ownership.py	76	63
dragontools/tests/test_patch_br_release_safety.py	167	143
dragontools/tests/test_patch_bt_preview_output_container.py	145	126
dragontools/tests/test_patch_bt_renamer_metadata_browser.py	266	212
dragontools/tests/test_patch_bu_metadata_browser_hardening.py	120	95
dragontools/tests/test_patch_bw_dv_recovery.py	121	107
dragontools/tests/test_patch_bx_watch_shutdown.py	59	48
dragontools/tests/test_patch_c_job_journal_queue_order.py	43	34
dragontools/tests/test_patch_c_queue_ordering.py	73	60
dragontools/tests/test_patch_d_online_metadata_retry.py	204	150
dragontools/tests/test_patch_dv_frame_mismatch_recovery.py	160	128
dragontools/tests/test_patch_e_episode_replacement_policy.py	164	121
dragontools/tests/test_patch_f_jellyfin_api.py	448	335
dragontools/tests/test_patch_g_watch_folder.py	209	165
dragontools/tests/test_patch_h_media_library_fix_queue.py	294	242
dragontools/tests/test_patch_i_language_detection.py	308	261
dragontools/tests/test_patch_j_bitmap_subtitle_ocr.py	232	200
dragontools/tests/test_patch_k_quality_target.py	210	173
dragontools/tests/test_patch_l_windows_notifications.py	145	115
dragontools/tests/test_patch_m_sdr_hdr_enhancement.py	228	189
dragontools/tests/test_patch_p_live_tools_whisper_nfo_sync.py	273	222
dragontools/tests/test_patch_pgs_watch_hardening.py	173	150
dragontools/tests/test_patch_pipeline_codec_safety.py	285	238
dragontools/tests/test_patch_q_companion_first_move.py	292	242
dragontools/tests/test_patch_s_handbrake_whisper_runtime.py	93	64
dragontools/tests/test_patch_t_renamer_live_search.py	109	77
dragontools/tests/test_patch_v_startup_signal_geometry.py	46	29
dragontools/tests/test_patch_w_qobject_event_signal_collision.py	60	49
dragontools/tests/test_patch_y_review_hardening.py	98	67
dragontools/tests/test_patch_z_renamer_popup_geometry.py	79	64
dragontools/tests/test_path_safety.py	98	74
dragontools/tests/test_per_file_strip_only.py	188	149
dragontools/tests/test_pgs_display_set_parser.py	305	242
dragontools/tests/test_pipeline_and_singleton.py	96	75
dragontools/tests/test_pipeline_decision_service_safety.py	148	110

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_pixel_verification_tolerance.py	437	350
dragontools/tests/test_planned_target_edit.py	46	36
dragontools/tests/test_postprocess_move_sidecars.py	184	140
dragontools/tests/test_postprocess_service.py	90	71
dragontools/tests/test_postprocess_signal_logging.py	210	158
dragontools/tests/test_power_module_refactor.py	286	244
dragontools/tests/test_preflight_dialog_performance.py	649	502
dragontools/tests/test_preflight_gui_architecture.py	31	25
dragontools/tests/test_preflight_report.py	76	62
dragontools/tests/test_process_runner_boundaries.py	27	16
dragontools/tests/test_project_info.py	91	72
dragontools/tests/test_quality_tester.py	79	56
dragontools/tests/test_queue_remove_readd.py	199	169
dragontools/tests/test_quick_toggle_settings.py	99	83
dragontools/tests/test_real_dv_hdr_integration.py	374	319
dragontools/tests/test_refactor_block10_architecture.py	19	13
dragontools/tests/test_refactor_block11_analyzer_renamer_architectur e.py	81	70
dragontools/tests/test_refactor_block12_matcher_metadata_architectur e.py	102	78
dragontools/tests/test_refactor_block2_architecture.py	47	39
dragontools/tests/test_refactor_block3_dv_media_view_architecture.py	57	45
dragontools/tests/test_refactor_block4_architecture.py	48	40
dragontools/tests/test_refactor_block5_architecture.py	7	4
dragontools/tests/test_refactor_block5_runtime_architecture.py	77	68
dragontools/tests/test_refactor_block6_iso_parallel_architecture.py	55	42
dragontools/tests/test_refactor_block7_move_architecture.py	75	54
dragontools/tests/test_refactor_block8_encoder_duration_architecture .py	70	50
dragontools/tests/test_refactor_block9_12_architecture.py	39	29
dragontools/tests/test_refactor_block9_hdr_move_library_architecture .py	165	140
dragontools/tests/test_refactor_runtime_connections.py	130	116
dragontools/tests/test_release_990_startup.py	35	27
dragontools/tests/test_release_artifact_e2e.py	135	107
dragontools/tests/test_release_hardening_985.py	195	144
dragontools/tests/test_release_packaging.py	226	168
dragontools/tests/test_release_validation.py	670	594
dragontools/tests/test_remux_startup_and_queue_pipeline.py	284	236
dragontools/tests/test_renamer_completeness.py	176	138
dragontools/tests/test_renamer_completeness_gui.py	144	118
dragontools/tests/test_renamer_fallback_and_simulator.py	107	86
dragontools/tests/test_renamer_filename_only.py	96	82
dragontools/tests/test_renamer_preflight_handoff.py	119	95
dragontools/tests/test_renamer_release_groups_episode_formats.py	141	108
dragontools/tests/test_renamer_year_override.py	163	138
dragontools/tests/test_replacement_reminders.py	95	77
dragontools/tests/test_responsibility_contracts.py	62	45
dragontools/tests/test_review01_settings_transaction.py	198	145

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_review02_process_safety_regressions.py	639	457
dragontools/tests/test_review03_media_analysis.py	840	702
dragontools/tests/test_review04_rules_preflight.py	579	476
dragontools/tests/test_review05_converter_standard.py	238	180
dragontools/tests/test_review06_conversion_runtime_gui.py	901	726
dragontools/tests/test_review07_dolby_vision_core.py	262	220
dragontools/tests/test_review08_dv_mux_recovery.py	487	399
dragontools/tests/test_review09_hdr10plus.py	571	480
dragontools/tests/test_review10_sdr_hdr.py	98	82
dragontools/tests/test_review11_audio.py	397	324
dragontools/tests/test_review12_subtitle_safety.py	647	526
dragontools/tests/test_review13_output_verification.py	522	417
dragontools/tests/test_review14_postprocess_jellyfin.py	431	354
dragontools/tests/test_review15_move_safety.py	383	309
dragontools/tests/test_review16_watch_queue_parallel.py	447	355
dragontools/tests/test_review17_iso_merge_remux.py	655	540
dragontools/tests/test_review18_renamer_regressions.py	243	187
dragontools/tests/test_review19_online_metadata_regressions.py	834	643
dragontools/tests/test_review20_media_library_safety.py	448	370
dragontools/tests/test_review21_main_gui_regressions.py	106	83
dragontools/tests/test_review22_quality_matcher.py	382	303
dragontools/tests/test_review23_release_build.py	192	135
dragontools/tests/test_review24_cross_pipeline_contracts.py	516	434
dragontools/tests/test_review_985_safety.py	288	245
dragontools/tests/test_review_986_additional_behavior.py	175	129
dragontools/tests/test_review_dependency_ownership.py	111	85
dragontools/tests/test_review_followup_safety.py	194	169
dragontools/tests/test_review_patch_block2_output_persistence.py	176	134
dragontools/tests/test_review_patch_block3_resilience_cleanup.py	152	108
dragontools/tests/test_review_patch_safety_regressions.py	232	195
dragontools/tests/test_review_patch_v10_artifact_contract.py	122	96
dragontools/tests/test_review_patch_v11_release_blockers.py	355	301
dragontools/tests/test_review_patch_v12_dv_transaction_safety.py	333	264
dragontools/tests/test_review_patch_v3_architecture.py	72	55
dragontools/tests/test_review_patch_v4_architecture.py	51	42
dragontools/tests/test_review_patch_v4_quality_services.py	98	78
dragontools/tests/test_review_patch_v5_safety.py	112	90
dragontools/tests/test_review_patch_v6_architecture.py	48	33
dragontools/tests/test_review_patch_v6_safety.py	155	126
dragontools/tests/test_review_patch_v7_safety.py	198	161
dragontools/tests/test_review_patch_v8_safety.py	271	224
dragontools/tests/test_review_patch_v9_integration.py	355	302
dragontools/tests/test_review_regressions_year_and_lifecycle.py	169	146
dragontools/tests/test_review_stdlib.py	243	200
dragontools/tests/test_rules_preview_headless.py	16	13
dragontools/tests/test_rules_refactor_architecture.py	23	14
dragontools/tests/test_run_summary.py	96	81
dragontools/tests/test_secret_settings.py	70	49

Datei	Gesamtzeilen	Codezeilen
dragontools/tests/test_settings_backup.py	640	506
dragontools/tests/test_settings_dialog_architecture.py	109	85
dragontools/tests/test_settings_helpers.py	186	145
dragontools/tests/test_sidecar_transaction.py	55	43
dragontools/tests/test_source_visual_check.py	157	123
dragontools/tests/test_source_zip_bat_contract.py	36	27
dragontools/tests/test_stability_983.py	268	236
dragontools/tests/test_stability_review_fixes.py	181	133
dragontools/tests/test_startup_move_recovery.py	160	139
dragontools/tests/test_strip_audio_and_webvtt.py	120	107
dragontools/tests/test_subtitle_attachment_inventory.py	96	81
dragontools/tests/test_subtitle_converter.py	153	119
dragontools/tests/test_subtitle_default_policy.py	240	204
dragontools/tests/test_subtitle_injection_options.py	104	78
dragontools/tests/test_subtitle_sidecar_service.py	541	427
dragontools/tests/test_subtitle_tab_all_formats_and_middle_close.py	216	165
dragontools/tests/test_subtitle_tagger_contract.py	40	28
dragontools/tests/test_system_shutdown.py	70	57
dragontools/tests/test_timestamp_commit_safety.py	38	34
dragontools/tests/test_timestamp_original_reference.py	193	165
dragontools/tests/test_tool_diagnostics.py	100	75
dragontools/tests/test_tool_paths.py	102	70
dragontools/tests/test_tool_runner.py	70	51
dragontools/tests/test_trickplay_service.py	371	293
dragontools/tests/test_tvdb_bearer_auto_refresh_persistence.py	276	221
dragontools/tests/test_ui_helpers_geometry.py	90	61
dragontools/tests/test_update_check.py	72	54
dragontools/tests/test_v97_four_patches.py	205	163
dragontools/tests/test_verbose_cleanup.py	65	49
dragontools/tests/test_version_contract.py	73	54
dragontools/tests/test_windows_path_helpers.py	68	46
dragontools/tests/test_windows_restart_policy.py	64	49
dragontools/tests/test_workflow_pipeline_contract.py	272	225

35.9 Sonstige

Datei	Gesamtzeilen	Codezeilen
dragontools/__init__.py	7	4
extras/comfyui/DragonTools_HDRTVDM/__init__.py	3	2
extras/comfyui/DragonTools_HDRTVDM/nodes.py	693	601
extras/release_pyinstaller.py	37	29
extras/snapshot_manifest.py	149	130